

Unit 11

PROGRAMMING

198 items · 1315 marks

Syllabus sections in this unit

- 11.1 Programming Basics
- 11.2 Constructs
- 11.3 Structured Programming

Unit 11: Programming

1. M/J 26/P21/Q1/a,b

1 Study this pseudocode:

```
FOR i ← 3 TO 12 STEP 3
    OUTPUT i, i * i
NEXT i
```

(a) Identify the type of construct used in this pseudocode.

[1]

(b) State how many iterations are performed by this pseudocode.

[1]

2. M/J 26/P21/Q5/a,b

5 Electric light bulbs are advertised online with the price showing:

\$1.75 each or 20 bulbs for \$30.00

A pseudocode function `BulbCost()`

- takes the quantity of bulbs purchased as its single parameter
- returns the total cost in dollars.

Pseudocode for the main program:

- inputs the number of bulbs purchased
- returns the total cost from the function `BulbCost()`
- outputs the total cost.

(a) Write pseudocode for the function `BulbCost()`

[4]

(b) The main program accepts a user input for the quantity of bulbs purchased and calls function `BulbCost()`

Complete the pseudocode for the main program.

```
// Main program
DECLARE NumberPurchased : INTEGER
DECLARE TotalCost : REAL

OUTPUT "How many bulbs? "
INPUT NumberPurchased

TotalCost ←

OUTPUT '$' , TotalCost
// end main program
```

[1]

3. M/J 26/P21/Q6/a,b

- 6 Fence parts are manufactured by a number of similar machines. The length of the part must fall within a certain range. Parts **not** within the length range are rejected.

The data for each rejected part forms a data packet made up of three data items, each separated by the '/' character delimiter. The format of a data packet is:

<length>/<machine ID>/<description of the fault>

- The length is 3 digits (in millimetres).
- The machine ID is a single character ('A', 'B', or 'C').
- The description of the fault is a variable length string.

A pseudocode program will process the data for several data packets.

The program contains the statement:

DataPacket ← "563/B/the fence length is too long"

- (a) Complete the **two** pseudocode expressions which extract the appropriate data item.

PartLength ←

MachineID ←

[2]

- (b) The program will create a module `RejectDescription()` to return the fault description.

Write the pseudocode for the module `RejectDescription()`

[4]

4. M/J 26/P22/Q1/a(ii,iii)

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (ii) Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

IF (Pass) < 6 OR (Pass) > THEN

OUTPUT "Error"

OUTPUT "The password has to contain between 6 and 8 characters."

ENDIF

[2]

- (iii) Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```
DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ....., .....)
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF
```

[7]

5. M/J 26/P22/Q5/c

- 5 A programmer has defined a global 1D array `Numbers` of type `INTEGER`, which can store 1000 integers. The lower bound of the array `Numbers` is set to 1. Each element of the array has been initialised with a random integer in the range 200 to 99999.

The programmer has written a bubble sort algorithm to sort the array `Numbers` into ascending order.

Study algorithm A.

Assume all the variables have been correctly declared.

algorithm A:

```
FOR LoopCounter ← 1 TO 999                                //Outer Loop
  FOR Index ← 1 TO 999                                    //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    ENDIF
  NEXT Index
NEXT LoopCounter
```

- (c) The programmer has defined a new procedure:

```
PROCEDURE Swap (Num1 : INTEGER, Num2 : INTEGER)
  DECLARE Temp : INTEGER
  Temp ← Num1
  Num1 ← Num2
  Num2 ← Temp
ENDPROCEDURE
```

The programmer replaces the block of pseudocode in the algorithm that swaps the values in two neighbouring elements in the array with a call to this procedure.

The block of code:

```
Temp ← Numbers[Index]
Numbers[Index] ← Numbers[Index + 1]
Numbers[Index + 1] ← Temp
```

Is replaced by:

```
CALL Swap (Numbers[Index], Numbers[Index + 1])
```

The `Swap()` procedure will **not** work as expected when used in the bubble sort algorithm.

- (i) Explain why the procedure `Swap()` will **not** work as expected.

[2]

- (ii) Write pseudocode to amend the version of the `Swap()` procedure given so that it will work as expected.

The amended version of the `Swap()` procedure must **not** use global variables.

[2]

6. M/J 26/P22/Q6/a

6 A program is being developed to encrypt a message.

The following terms are used in the encryption technique to be used:

term	definition
plain text	The original message before it is encrypted.
cipher text	The encrypted version of the plain text.

The encryption method to be used is based on the algorithm for the Caesar Box Cipher. All plain text messages to be encrypted must be 25 characters in length or less. If a message is less than 25 characters in length, then spaces are added to the end to make the padded plain text the required length.

A square grid of five columns and five rows is used. The padded plain text message is put into the grid row by row. The cipher text is obtained by reading the grid column by column.

For example:

plain text: This is a message

The number of characters in the original plain text, including spaces, is 17.

plain text with added spaces at the end: This△is△a△message△△△△△△△△

The character △ is used to indicate a space in the padded plain text.

The grid filled, row by row, with the padded plain text.

T	h	i	s	△
i	s	△	a	△
m	e	s	s	a
g	e	△	△	△
△	△	△	△	△

The cipher text is obtained by reading the grid column by column.

cipher text: Timg△hsee△i△s△△sas△△△△a△△

The programmer has defined the first program module as:

module	description
AddPadding()	• user prompt for and input of plain text • check plain text contains between 1 and 25 characters inclusive • re-input of plain text until it contains between 1 and 25 characters • if required, additional space characters are added to the end of the plain text to ensure it contains exactly 25 characters • return of amended plain text

(a) Write pseudocode for module AddPadding()

7. M/J 26/P23/Q1/a,b

1 Study the pseudocode.

```
i ← 1
REPEAT
    OUTPUT i*i
    i ← i + 1
UNTIL i = 6
```

(a) Identify the type of programming construct used in the pseudocode.

[1]

(b) State how many iterations are carried out by the pseudocode.

[1]

8. M/J 26/P23/Q5/a

5 Data is stored about people who have attended a job interview. Each interview is held at one of three office locations.

Each person's data is stored as a data packet made up of four data items:

<initial><shortened surname><office code><summary of the interview>

- The initial is a single character.
- The shortened surname is the first five characters of the surname.
 - You can assume all surnames are at least five characters.
- The office code is a single character 'A', 'B' or 'C'.
- The interview summary text is of variable length.

Example:

Peter Watkins was interviewed at office 'B'. After the interview it was decided to invite him for a second interview.

His data packet is stored by variable `Applicant`:

```
Applicant ← "PWatkiBSecond interview"
```

(a) A pseudocode function `InterviewSummary()` will:

- take the data packet as a single parameter
- return the interview summary text.

Write the pseudocode for the function `InterviewSummary()`

[4]

9. O/N 25/P21/Q1/a

Uses the insert (Appendix)

1 A program is being developed to meet a particular customer requirement.**(a)** The program contains these variables:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER
MyDOB	DATE

Complete the table by filling in the gaps using functions and/or operators from the **insert**.

Each expression must be valid.

Expression
MyString ← 'X' MyString
MyChar ← ("ABCD", ,)
MyString ← (..... (MyDOB))
MyInt ← (..... (MyString) / 2)

[4]

10. O/N 25/P21/Q5/b

- 5** A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return `-1` if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
  DECLARE Index, FoundAt : INTEGER
  FoundAt ← -1
  FOR Index ← 1 TO 10000
    IF MyData[Index] = SearchString THEN // outer conditional clause
      IF FoundAt = -1 THEN               // inner conditional clause
        FoundAt ← Index
      ENDIF
    ENDIF
  NEXT Index
  RETURN FoundAt
ENDFUNCTION
```

- (b)** The pseudocode does **not** use the most appropriate loop construct.

- (i)** Explain why this is **not** the most appropriate loop construct.

[1]

- (ii)** Suggest and justify a more appropriate loop construct that could be used.

Construct

Justification

[2]

11. O/N 25/P21/Q6/b

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The method described can be used to detect single-digit errors. For example, if the new number is incorrectly input as 41633, then the check digit does not match and the input will be rejected.

- (b) A module `Generate()` will take an integer value representing an original number and return an integer value representing a new number which includes the check digit.

The original number is always at least three digits in length.

Write pseudocode for the module `Generate()`

Assume that the parameter is valid.

[7]

12. O/N 25/P22/Q1/c

- (b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
Result	5.42	
MonthLetter	"JFMAMJJASOND"	
Birthday	15/11/2009	

[3]

- (c) Evaluate each expression in the table by using the data values shown in (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
<code>INT(Result) + 1 > 6</code>	
<code>NUM_TO_STR(LENGTH(MonthLetter))</code>	
<code>NUM_TO_STR(Result + "3.2")</code>	
<code>MID(MonthLetter, MONTH(Birthday) - 2, 1)</code>	

[4]

13. O/N 25/P22/Q6

- 6** A string function `Compare()` will compare two strings.

The function will:

- take **four** parameters:
 - two strings, `String1` and `String2`
 - a character, `Position`, to indicate whether `String1` will be compared with the start ('s'), or the end ('e') of `String2`
 - a Boolean, `CaseMatters`, to indicate whether an upper case character and lower case character (for example, 'A' and 'a') are regarded as different (TRUE), or as the same (FALSE)
- return FALSE if `String2` has fewer characters than `String1`
- return TRUE if the comparison is true, otherwise return FALSE

For example:

Parameter				Return value
String1	String2	CaseMatters	Position	
"Cat"	"Catalogue"	TRUE	's'	TRUE
"CAT"	"Catalogue"	TRUE	's'	FALSE
"CAT"	"Catalogue"	FALSE	's'	TRUE
"Cat"	"Catalogue"	TRUE	'e'	FALSE
"GUE"	"Catalogue"	FALSE	'e'	TRUE
"Catalogue"	"Cat"	TRUE	's'	FALSE

Write pseudocode for the function `Compare()`

[7]

14. O/N 25/P23/Q1/a,b,c

- 1** A user requires a program that implements complex data encryption to a recognised international standard. A programmer has started the design of the program.

- (a) The programmer decides to use a library routine to provide part of the solution. One reason for this is that library routines may perform functions that the programmer is unable to program themselves.

State **three** other benefits of using library routines in the development of the program.

- 1
- 2
- 3

[3]

- (b) The programmer has identified a different part of the algorithm that would be appropriate to implement as a subroutine (a procedure or a function); no suitable library routine exists for this part.

- (i) State **two** reasons why the programmer may decide to use a subroutine.

- 1

2

[2]

- (ii) A function header in pseudocode is defined as:

```
FUNCTION Pass2 (Count : INTEGER) RETURNS BOOLEAN
```

Complete the table by describing the terms used in the function header.

The first row has been completed.

Term	Description
Pass2	the name of the function
Count	
BOOLEAN	

[2]

- (c) Variables in the program have example values:

Variable	Example value
DoB	23/6/2011
Multiplier	2.5
AddressLine[1]	"35 Lincoln Avenue"

Complete the table by evaluating each expression using the example values:

Expression	Evaluates to
LENGTH(NUM_TO_STR(Multiplier))	
MONTH(DoB) > 4	
15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	

[3]

15. O/N 25/P23/Q5/b

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
```

```
Num[2] ← 2
```

```
Num[3] ← 5
```

```
Num[4] ← 3
```


A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count <= 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
        OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE
```

(b) As a reminder, the CASE structure in the pseudocode is:

```
CASE OF CaseVar
  1 : Num[Index] ← Num[Index] + Index //clause one
    Index ← Index + 1
    Count ← Count + 1

  2 : Num[Index] ← Num[Index] + Index //clause two
    Index ← Index + 2
    Count ← Count + 2

  3 : Num[Index] ← Num[Index] * 2      //clause three
    Index ← Index + 3
    Count ← Count - 1

  4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
ENDCASE
```

The CASE structure could be optimised by combining two existing clauses.

(i) Identify the `CaseVar` values for these **two** existing clauses.

[1]

(ii) Write pseudocode for a single clause to replace the **two** clauses identified in **(b) (i)**.

[2]

16. O/N 25/P23/Q6/b

- 6** Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (b)** A function `CheckNumber()` will take an integer value and return the Boolean value `TRUE` if the check digit is correct, otherwise return `FALSE`

Write pseudocode for the function `CheckNumber()`

Assume that the parameter is valid.

[7]

17. M/J 25/P21/Q4/a**4** Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I ≤ 4                                //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J ≥ 0 AND Chars[J] > Key          //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(a) The outer loop structure used in the algorithm is **not** the most appropriate one to use.

State the type of loop structure that would be the most appropriate to use and justify why it is the most appropriate.

Loop structure

Justification

[2]

18. M/J 25/P21/Q7/a

- 7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	• called with two integer parameters: ◦ the number of coffees ordered ◦ the current number of points • output a suitable message giving the number of free coffees • return a value for the new points total.

- (a) Write pseudocode for module `CustomerOrder()`

19. M/J 25/P23/Q1/a,b

Uses the insert (Appendix)

1 A program is being developed for the management of a sports centre.**(a)** The programmer developing the software decides to use modules (procedures or functions) and local variables. They also decide not to use global variables.**(i)** State **two** reasons why the programmer decides to use modules.

1

2

[2]

(ii) State **one** difference between local and global variables.

[1]

(iii) State **two** benefits of using local variables.

1

2

[2]

(b) The pseudocode design contains a number of expressions.

Complete each pseudocode expression so that it evaluates to the value shown.

Refer to the **insert** for the list of pseudocode functions and operators.

Expression	Evaluates to
(68)	' D '
..... (04/02/2025)	2
.....TRUE	FALSE
..... (..... ("Court1.4Upper" , ,))	1 . 4

[5]

20. M/J 25/P23/Q5

- 5 A module `Parity()` takes a string as a parameter. The parameter has the identifier `BitString` and it represents a binary value.

The module will concatenate a single character to the end of `BitString` by applying one of the two rules:

- '0' if `BitString` contains an even number of 1s
- '1' if `BitString` contains an odd number of 1s.

The modified value of `BitString` is then returned.

For example:

Parameter	String returned	Explanation
"0010010110"	"00100101100"	there are an even number of 1s in the string passed to <code>Parity()</code> so a '0' is concatenated to the end of <code>BitString</code>
"101010"	"1010101"	there are an odd number of 1s in the string passed to <code>Parity()</code> so a '1' is concatenated to the end of <code>BitString</code>

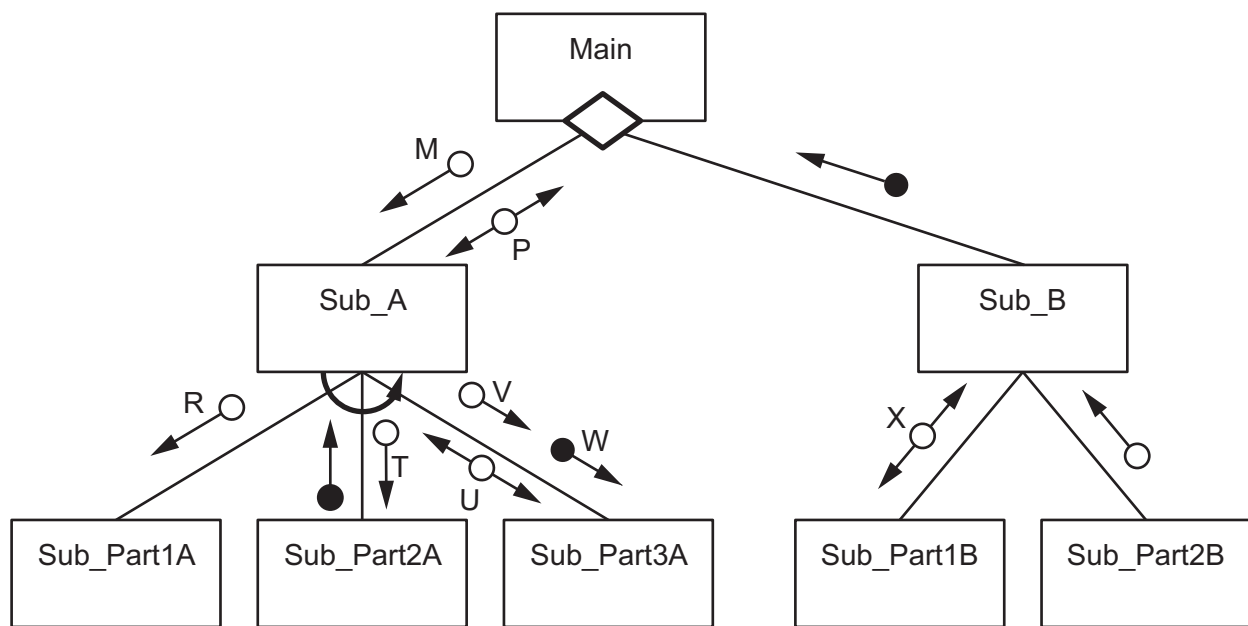
Write pseudocode for the module `Parity()`

Assume the parameter `BitString` can only contain the characters '0' and '1'

[8]

21. M/J 25/P23/Q6/a

6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(a) Explain **one** difference between functions and procedures.

[1]

22. M/J 25/P23/Q7/a,b(ii)

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
UpdateVisit()	- called with two parameters: - loyalty points of type <code>INTEGER</code> - last visit date of type <code>DATE</code> - change the loyalty points: - increase by four if the last visit date and the current date are in the same month - otherwise increase by one - change the last visit date to the current date - the changed values must be available to the code that follows the call to <code>UpdateVisit()</code>

(a) (i) Write pseudocode for module `UpdateVisit()`

Assume the customer has made at least **one** previous visit.

[5]

(ii) The programmer decides to amend the `UpdateVisit()` module so that loyalty points are further increased if the customer visits the coffee shop the same day of the week as their last visit.

Write the pseudocode for this condition.

[1]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: `DDMMYYYY`.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

`"100123,132,05032025"`

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
<code>FindCustomer()</code> (is already written)	- called with a parameter of type <code>STRING</code> that represents a customer ID - returns the line of data read from the text file <code>Loyalty.txt</code> containing the data items for that customer
<code>MondayCheck()</code>	- called with a parameter of type <code>STRING</code> that represents the customer ID of a customer - calls <code>FindCustomer()</code> - extracts the loyalty points from the string returned by <code>FindCustomer()</code> - increases the loyalty points for the current customer by 10 if the day visited is a Monday - returns an integer value representing the loyalty points

(ii) A date is stored in `LastVisitDateString` in the format:

DDMMYYYY

DD is a 2-digit string

MM is a 2-digit string

YYYY is a 4-digit string.

For example, the date 03/09/2024 is stored as "03092024"

An algorithm is needed to convert the string stored in `LastVisitDateString` to data type `DATE` which is then stored in the variable `LastVisitDate`.

Complete the pseudocode for this algorithm:

Assume that `LastVisitDateString` has been declared and contains valid data.

```
DECLARE DayInt, MonthInt, YearInt : INTEGER
```

```
DECLARE LastVisitDate : DATE
```

[4]

23. O/N 24/P21/Q1/b,d

- 1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25    // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

- (b) During the design of the program, tax rate values have been used wherever they are needed as shown in the pseudocode example above. Tax rates do not change while the program runs.

(i) Identify a more appropriate way of representing the tax rate values in the final program.

[1]

(ii) Describe the benefits of your answer to part (b)(i) with reference to this program.

[3]

- (d) The final CASE condition (> 200) in the pseudocode example could be replaced with a keyword.

Give the keyword.

[1]

24. O/N 24/P21/Q2

- 2 A program uses three global integer variables HH, MM and SS to represent the current time in hours, minutes and seconds using the 24-hour clock notation.

Midnight would be represented as 00:00:00 (HH:MM:SS). If the variables HH, MM and SS contained the values 16, 30 and 10 respectively, then the time would be 16:30:10 or just after 4.30 in the afternoon.

A procedure Tick() will be called every second.

The procedure Tick() will:

- update the value in SS each time it is called
- update the values in HH and MM as appropriate
- call a procedure CheckAlarm() at the start of each minute
- call a procedure NewDay() whenever the time reaches midnight.

Complete the pseudocode for procedure Tick().

```
PROCEDURE Tick()
ENDPROCEDURE
```

[6]

25. O/N 24/P21/Q4/b,c

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (b) The designer considers implementing the **checks and calculation** as a module (a procedure or a function). One reason for this is that the same checks and calculations are performed at several places in the program.

Give **another** reason why this is a suitable approach **and** state what is avoided by this approach.

Reason

Avoided

[2]

- (c) The module to perform the checks and calculation will be implemented as a function. The function will need to return both a real **and** a Boolean value. To achieve this a record type is defined in pseudocode as follows:

```
TYPE Result
    DECLARE Done : BOOLEAN
    DECLARE Value : REAL
ENDTYPE
```

The function `Evaluate()` will:

- take two parameters of type string representing the two numeric values
- return a variable of type `Result` with the `Done` field set to `FALSE` if either of the following applies:
 - at least one of the strings does **not** represent a valid numeric value
 - the numeric value of the string representing value `y` is zero
- otherwise return a variable of type `Result` with the `Done` field set to `TRUE` and the `Value` field assigned the result of the formula (based on the numeric value of the two parameters).

Write pseudocode for the function `Evaluate()`.

[6]

26. O/N 24/P21/Q6/b

- 6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

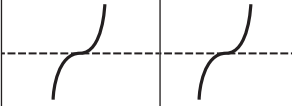
The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
	
11	30
12	31

Note: Leap years are **not** considered

(b) Complete the pseudocode for the function `GetDate()`.

Date functions from the **insert** should be used in your solution.

```
FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE
ENDFUNCTION
```

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`
 Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

```
RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`
 Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

```
MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
```

returns a string of length y starting at position x from `ThisString`
 Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of `ThisString`
 Example: `LENGTH("Happy Days")` returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

`DAY(ThisDate : DATE) RETURNS INTEGER`

returns the day number from `ThisDate`

Example: `DAY(04/10/2003)` returns 4

`MONTH(ThisDate : DATE) RETURNS INTEGER`

returns the month number from `ThisDate`

Example: `MONTH(04/10/2003)` returns 10

`YEAR(ThisDate : DATE) RETURNS INTEGER`

returns the year number from `ThisDate`

Example: `YEAR(04/10/2003)` returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes: - may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR and/or STRING - cannot be used to compare two records
<	
>=	
<=	
<>	
	Examples: "Program" = "program" evaluates to FALSE - Count = 4 evaluates to TRUE when Count contains the value 4

27. O/N 24/P21/Q8/a

- 8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
<code>GradeBoundary[1]</code>	65	The minimum mark for an A grade.
<code>GradeBoundary[2]</code>	57	The minimum mark for a B grade.
<code>GradeBoundary[3]</code>	43	The minimum mark for a C grade.
<code>GradeBoundary[4]</code>	35	The minimum mark for a D grade.
<code>GradeBoundary[5]</code>	27	The minimum mark for an E grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark	Element	ID
<code>Result[4, 1]</code>	56	<code>Result[4, 2]</code>	1074832
<code>Result[5, 1]</code>	54	<code>Result[5, 2]</code>	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
<code>CheckMark()</code>	- called with a parameter of type integer representing a candidate mark - returns <code>TRUE</code> if the mark is within 2 of any of the five grade boundaries, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckMark()`.

28. O/N 24/P22/Q2/a,b(ii)

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character ' * '.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
"1234567890"	"*****7890"
"1234567897652"	"*****7652"
"1234567890123456"	"*****3456"

(a) The function `Conceal()` will:

- take a numeric string as a parameter representing the card number
- return a string in which the asterisk character replaces all except the last four characters of the card number parameter.

Write pseudocode for the function `Conceal()`.

[6]

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(ii) The new procedure `Conceal()` will write the modified string to the corresponding element in column two.

The array `CardNumber` is passed as a parameter to the new procedure `Conceal()`.

Identify how this parameter should be specified in the new procedure header.

[1]

29. O/N 24/P22/Q4

- 4 A global integer variable `Tick` is always incremented every millisecond (1000 times per second) regardless of the other programs running.

The value of `Tick` can be read by any program but the value should not be changed.

Assume that the value of `Tick` does not overflow.

As an example, the following pseudocode algorithm would output "Goodbye" 40 seconds after outputting "Hello".

```
DECLARE Start : INTEGER
OUTPUT "Hello"
Start ← Tick
REPEAT
    //do nothing
UNTIL Tick = Start + 40000
OUTPUT "Goodbye"
```

A program is needed to help a user to time an event such as boiling an egg.

The time taken for the event is known as the elapsed time.

The program contains a procedure `Timer()` which will:

- take two integer values representing an elapsed time in minutes and seconds
- use the value of variable `Tick` to calculate the elapsed time
- output a warning message 30 seconds before the elapsed time is up
- output a final message when the total time has elapsed.

For example, to set an alarm for 5 minutes and 45 seconds the program makes the following call:

```
CALL Timer(5, 45)
```

When 5 minutes and 15 seconds have elapsed, the program will output:

```
"30 seconds to go"
```

When 5 minutes and 45 seconds have elapsed, the program will output:

```
"The time is up!"
```

Write pseudocode for the procedure `Timer()`.

30. O/N 24/P22/Q5/c

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN RetVal
ENDFUNCTION
```

- (c) The function `Process()` contains a selection construct using a `CASE` structure.

Write pseudocode using a single selection construct with the same functionality **without** using a `CASE` structure.

[2]

31. O/N 24/P23/Q1/c

- (b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Available	TRUE	
Received	"18/04/2021"	
Index	100	

[3]

- (c) Evaluate each expression in the table by using the data values shown in part (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
Available AND NOT(Index > 100)	
Index MOD 30	
NUM_TO_STR(Index + "33")	

[3]

32. O/N 24/P23/Q6

- 6 In some countries, on the third Sunday in March, daylight saving time begins when clocks move forward by one hour.

A module `AdjustClock()` will take an integer parameter representing a year. The module will return an integer value representing the number of the day in March on which the clocks move forward.

For example, the following line of pseudocode would assign `DayNumber` the value 20:

DayNumber $\leftarrow$ AdjustClock(2022)

Write pseudocode for the function AdjustClock().

Date functions from the **insert** should be used in your solution.

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type CHAR or STRING
- A CHAR may be assigned to, or concatenated with, a STRING
- A STRING of length greater than 1 cannot be assigned to a CHAR

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER

returns the day number from ThisDate

Example: DAY(04/10/2003) returns 4

MONTH(ThisDate : DATE) RETURNS INTEGER

returns the month number from ThisDate

Example: MONTH(04/10/2003) returns 10

YEAR(ThisDate : DATE) RETURNS INTEGER

returns the year number from ThisDate

Example: YEAR(04/10/2003) returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

<code>&</code>	concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> evaluates to <code>"Summer Pudding"</code> may also be used to concatenate a <code>CHAR</code> with a <code>STRING</code>
<code>AND</code>	performs a logical <code>AND</code> on two Boolean values. Example: <code>TRUE AND FALSE</code> evaluates to <code>FALSE</code>
<code>OR</code>	performs a logical <code>OR</code> on two Boolean values. Example: <code>TRUE OR FALSE</code> evaluates to <code>TRUE</code>
<code>NOT</code>	performs a logical <code>NOT</code> on a Boolean value. Example: <code>NOT TRUE</code> evaluates to <code>FALSE</code>
<code>MOD</code>	finds the remainder when one number is divided by another. Example: <code>10 MOD 3</code> evaluates to 1
<code>DIV</code>	finds the quotient when one number is divided by another. Example <code>10 DIV 3</code> evaluates to 3

Comparison operators

<code>=</code>	used to compare two items of the same type. evaluates to <code>TRUE</code> if the condition is true, otherwise evaluates to <code>FALSE</code>
<code>></code>	Notes: - may be used to compare types <code>REAL</code> and <code>INTEGER</code> - may be used to compare types <code>CHAR</code> and <code>STRING</code> - case sensitive when used to compare types <code>CHAR</code> and/or <code>STRING</code> - cannot be used to compare two records
<code><</code>	
<code>>=</code>	
<code><=</code>	
<code><></code>	
	Examples: <code>"Program" = "program"</code> evaluates to <code>FALSE</code> <code>Count = 4</code> evaluates to <code>TRUE</code> when <code>Count</code> contains the value 4

33. M/J 24/P21/Q1/b

1 An algorithm is developed in pseudocode before being coded in a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Sorted	False
Tries	9
ID	"ZGAC001"

Complete the table by evaluating each expression, using the example values.

Expression	Evaluates to
<code>Tries < 10 AND NOT Sorted</code>	
<code>Tries MOD 4</code>	
<code>TO_LOWER(MID(ID, 3, 1))</code>	
<code>LENGTH(ID & "xx") >= Tries</code>	

[4]

34. M/J 24/P21/Q2/b

2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values A and B
2. count the number of pairs that are input until A has been greater than B 10 times
3. output the number of pairs that were input.

(b) Step 1 of the algorithm is changed.

A variable `ThisSequence` is used to enter a sequence of 10 pairs of numeric values, using a single input statement.

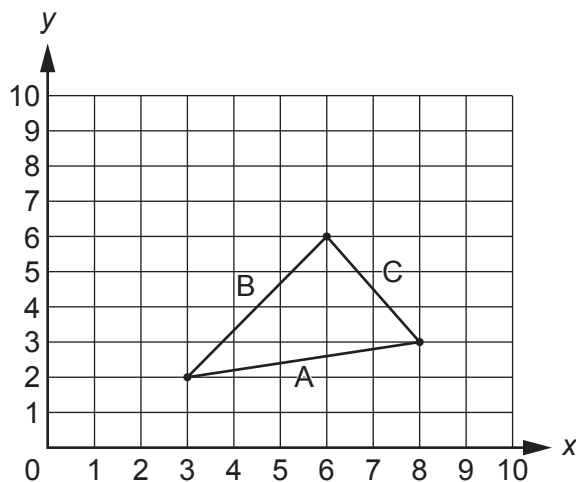
Following the input of `ThisSequence` the revised algorithm will extract the pairs of numbers.

Describe the variable `ThisSequence` and how the numbers are extracted.

[2]

35. M/J 24/P21/Q6/a

- 6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



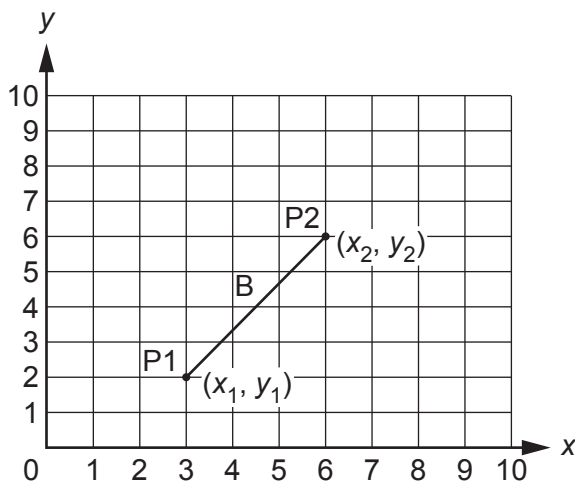
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator ‘^’ represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
    ENDFUNCTION
```

[6]

36. M/J 24/P21/Q8/a

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
MichaelAday_src.txt	student project file produced by student Michael Aday
MichaelAday_S1.txt	file produced by stage 1
MichaelAday_S2.txt	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
<code>DeleteComment()</code>	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: <code>IF X2 > 13 THEN //check if limit exceeded</code> After: <code>IF X2 > 13 THEN</code>

- (a) Complete the pseudocode for module `DeleteComment()`.

```
FUNCTION DeleteComment(Line : STRING) RETURNS STRING
    ENDFUNCTION
```

[8]

37. M/J 24/P22/Q1/b

Uses the insert (Appendix)

(b) Program variables have data types as follows:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER

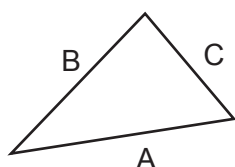
Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
MyInt ← (3.1415926)
MyChar ← ("Elwood", 3, 1)
MyString ← (..... (27.509))
MyInt ← (..... ("ABC123", 3))

[4]

38. M/J 24/P22/Q4

4 A triangle has sides of length A, B and C.



In this example, A is the length of the longest side.

This triangle is said to be right-angled if the following equation is true:

$$A \times A = (B \times B) + (C \times C)$$

A procedure will be written to check whether three lengths represent a right-angled triangle. The lengths will be input in any sequence.

The procedure `IsRA()` will:

- prompt and input three integer values representing the three lengths
- test whether the three lengths correspond to the sides of a right-angled triangle
- output a suitable message.

The length of the longest side may **not** be the first value input.

Write pseudocode for the procedure `IsRA()`.

[5]

39. M/J 24/P22/Q5/a(ii)

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(ii) The procedure contains a statement that is **not** needed.

Identify the pseudocode statement **and** explain why it is **not** needed.

Statement

Explanation

[2]

40. M/J 24/P22/Q6/a

- 6** A music player stores music in a digital form and has a display which shows the track being played.

(a) Up to 16 characters can be displayed. Track titles longer than 16 characters will need to be trimmed as follows:

- Words must be removed from the end of the track title until the resulting title is less than 14 characters.
- When a word is removed, the space in front of that word is also removed.
- Three dots are added to the end of the last word displayed when one or more words have been removed.

The table below shows some examples:

Original title	Display string															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bat out of Hull	B	a	t		o	u	t		o	f		H	u	l	l	
Bohemian Symphony	B	o	h	e	m	i	a	n	.	.	.					
Paperbook Writer	P	a	p	e	r	b	o	o	k		W	r	i	t	e	r
Chris Sings the Blues	C	h	r	i	s		S	i	n	g	s	.	.	.		
Green Home Alabama	G	r	e	e	n		H	o	m	e	.	.	.			

A function `Trim()` will:

- take a string representing the original title
- return the string to be displayed.

Assume:

- Words in the original title are separated by a single space character.
- There are no spaces before the first word or after the last word of the original title.
- The first word of the original title is less than 14 characters.

Write pseudocode for the function `Trim()`.

41. M/J 24/P22/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

- (b) The teacher has defined the first program module as follows:

Module	Description
DeleteSpaces()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: " IF X2 > 13 THEN" After: "IF X2 > 13 THEN"

Complete the pseudocode for module DeleteSpaces().

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
ENDFUNCTION
```

[6]

42. M/J 24/P23/Q1/a,b

Uses the insert (Appendix)

- 1** A program uses many complex algorithms.

One algorithm is repeated in several places. The code for the algorithm is the same wherever it is used, but the calculations within the algorithm may operate on different data.

The result of each calculation is used by the code that follows it.

It is decided to modify the program and implement the algorithm as a separate module.

- (a) (i)** State **two** benefits of this modification to the existing program.

1

2

[2]

- (ii)** Describe how the modification would be implemented.

[3]

- (b)** Four of the expressions used in the program are represented by pseudocode in the table.

Complete each pseudocode expression with a function or operator so that it evaluates to the value shown.

Any functions and operators used must be defined in the **insert**.

Pseudocode expression	Evaluates to
..... ("Random", 2, 3)	"and"
5 + (10/11/2023)	15
..... ("45000")	TRUE
(20 3) + 1	3

[4]

43. M/J 24/P23/Q4/a

- 4** A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:

- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

- (a)** Write pseudocode for the procedure `TwoParts()`.

[6]

44. M/J 24/P23/Q5/a,b

- 5** A program is being designed in pseudocode.

The program contains the following declaration:

```
DECLARE Data : ARRAY[1:1000] OF STRING
```

A procedure `ArrayInitialise()` is written to initialise the values in the array:

```
PROCEDURE ArrayInitialise(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 1
  WHILE Index ≤ 1000
    CASE OF (Index MOD 2)
      0 : Data[Index] ← FormatA(Label)
        Index ← Index + 1
      1 : Data[Index] ← FormatB(Label)
        Index ← Index + 1
    ENDCASE
  ENDWHILE
ENDPROCEDURE
```

Functions `FormatA()` and `FormatB()` apply fixed format case changes to the parameter string.

- (a)** The design of the procedure does **not** use the most appropriate loop construct.

Suggest a **more appropriate** construct that could be used **and** explain your choice.

Construct

Explanation

[2]

- (b)** The algorithm calls one of the functions `FormatA()` and `FormatB()` each time within the loop.

Explain why this is **not** efficient **and** suggest a more efficient solution.

[4]

45. M/J 24/P23/Q6/a,b(i)

- 6** A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

(a) Write pseudocode for procedure `Progress()`.

[6]

(b) The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

(i) Write pseudocode for the new module header.

[2]

46. M/J 24/P23/Q7/a(ii)

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (ii) Modules Mod-F and Mod-G are both called with Par3 as a parameter.
In the case of Mod-F, the parameter is passed by value.
In the case of Mod-G, the parameter is passed by reference.

Explain the effect of the **two** different ways of passing the parameter Par3.

[2]

47. M/J 24/P23/Q8/a

- 8 A teacher is designing a program to process pseudocode projects written by her students.

The program analyses a student project and extracts information about each module that is defined (each procedure or function). This information is stored in a global 2D array `ModInfo` of type string.

A module header is the first line of a module definition and starts with either of the keywords `PROCEDURE` or `FUNCTION`.

An example of part of the array is given below. Row 10 of the array shows that a procedure header occurs on line 27 and row 11 shows that a function header occurs on line 35. "P" represents a procedure and "F" represents a function:

	x = 1	x = 2	x = 3
<code>ModInfo[10, x]</code>	"27"	"P"	"MyProc(Z : CHAR)"
<code>ModInfo[11, x]</code>	"35"	"F"	"MyFun(Y : CHAR) RETURNS BOOLEAN"

The string stored in column 3 is called the module description. This is the module header **without** the keyword.

A valid module header will:

- be at least 13 characters long
- start with the keyword `PROCEDURE` or `FUNCTION`. The keyword may appear in either upper or lower case (or a mix of both) and **must** be followed by a space character.

The teacher has defined the first program module as follows:

Module	Description
<code>Header()</code>	• called with a parameter of type string representing a line of pseudocode • if the line is a valid procedure header, returns a string: "P<Module description>" • if the line is a valid function header, returns a string: "F<Module description>" • otherwise, returns an empty string For example, given the string: <pre>"FUNCTION Zap(X : INTEGER) RETURNS CHAR"</pre> <code>Header()</code> returns the string: <pre>"FZap(X : INTEGER) RETURNS CHAR"</pre>

- (a) Write pseudocode for module `Header()`.

48. O/N 23/P21/Q1/a,b,c

- 1 The following pseudocode represents part of the algorithm for a program:

```

CASE OF ThisValue
    < 30      : Level ← "Low"           // less than 30
               Check ← 1
    < 20      : Level ← "Very Low"      // less than 20
               Check ← ThisValue / 2
    30 TO 40  : Level ← "Medium"        // between 30 and 40
               Check ← ThisValue / 3
               Data[ThisValue] ← Data[ThisValue] + 1
    > 40      : Level ← "High"
ENDCASE

```

- (a) Complete the table by writing the answer for each row:

	Answer
The value assigned to <code>Level</code> when <code>ThisValue</code> is 40	
The value assigned to <code>Check</code> when <code>ThisValue</code> is 36	
The value assigned to <code>Level</code> when <code>ThisValue</code> is 18	
The number of elements in array <code>Data</code> that may be incremented	

[4]

- (b) The pseudocode contains four assignments to variable `Level`. One of these assignments will never be performed.

Identify this assignment **and** explain why this is the case.

[3]

- (c) The following line is added immediately before the `ENDCASE` statement:

```
OTHERWISE : Level ← "Undefined"
```

State why this assignment is never performed.

[1]

49. O/N 23/P21/Q6/a,b

6 The pseudocode `OUTPUT` command starts each output on a new line.

- (a)** A new procedure `MyOutput()` will take a string and a Boolean parameter. `MyOutput()` may be called repeatedly and will use concatenation to build a string using a global variable `MyString`, up to a maximum length of 255 characters.

`MyString` will be output in either of these two cases:

1. The Boolean parameter value is `TRUE`
2. The resulting string (after concatenation) would be longer than 255 characters.

If `MyString` is not output, the string is concatenated with `MyString`.

For example, the calls to `MyOutput()` given below would result in the output as shown:

```
MyOutput("Hello ", FALSE)
MyOutput("ginger ", FALSE)
MyOutput("cat", TRUE)
MyOutput("How are you?", TRUE)
```

Resulting output:

```
Hello ginger cat
How are you?
```

Notes:

- `MyString` is initialised to an empty string before `MyOutput()` is called for the first time.
- No string passed to `MyOutput()` will be longer than 255 characters.

Write pseudocode for `MyOutput()`.

[7]

- (b)** The design of the procedure given in **part (a)** is modified and `MyString` is changed from a global to a local variable declared in `MyOutput()`.

When the modified procedure is converted into program code, it does not work as expected.

Explain why it does not work as expected.

[2]

50. O/N 23/P21/Q8/d

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "*****"

(d) A new module has been defined:

Module	Description
GetField()	- takes two parameters: - a string containing a message - an integer containing a field number - If the field number is valid (in the range 1 to 3, inclusive), it returns a string containing the required field, otherwise it returns an empty string.

As a reminder, a message is defined as follows:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
Not applicable	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
Not applicable	ETX	a single character marking the end of the message (ASCII value 03)

Write pseudocode for module GetField().

[6]

51. O/N 23/P22/Q1/b

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(b) Other variables in the program have example values as shown:

Variable	Example value
Total	124.00
DepRate	2.00
Description	"AB12345:Cleaning Brush (small) "

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
<code>(Total * DepRate) + 1.5</code>	
<code>RIGHT (Description, 7)</code>	
<code>(LENGTH (Description) - 8) > 16</code>	
<code>NUM_TO_STR (INT (DepRate * 10)) & '%'</code>	

[4]

52. O/N 23/P22/Q2/b

- 2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(b) The solution to the algorithm includes iteration.

Give the name of a suitable loop structure that could be used.

Justify your answer.

Name

Justification

[2]

53. O/N 23/P22/Q4/a

4 A procedure `Count()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(a) Write pseudocode for the procedure `Count()`.

[6]

54. O/N 23/P22/Q6/b(i,ii)

6 (a) A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, `TestData.001`, `TestData.002` and `TestData.003`.

Each file will contain a single line. For example, file `TestData.002` would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

[6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(i) Identify the type of module that should be used for `CheckFiles()`.

[1]

(ii) Write the module header for `CheckFiles()`.

[1]

55. O/N 23/P22/Q8/b

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.

Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.

When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.

Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	Column 2	Column 3
Row 1	100	199	1
Row 2	200	259	2
Row 3	-1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to -1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
<code>GetPort()</code>	- takes a <code>DestinationID</code> as a parameter of type string - searches for the range corresponding to the <code>DestinationID</code> in the array - returns the port number, or returns <code>-1</code> if no corresponding range is found

- (b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
<code>StackMsg()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing the stack number - adds the message to the required stack - returns <code>TRUE</code> if the message is added to the required stack, otherwise returns <code>FALSE</code>
<code>ProcessMsg()</code>	- takes a message as a parameter of type string - ignores any message with a zero-length data field - extract the <code>DestinationID</code> from the message - checks whether the <code>DestinationID</code> is this computer or whether the message is to be forwarded - uses <code>StackMsg()</code> to add the message to the appropriate stack - outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

56. O/N 23/P23/Q1/b,c

- 1 A program is being developed in pseudocode before being converted into a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Active	TRUE
Fraction	0.2
Code	"Ab12345"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
Fraction >= 0.2 AND NOT Active	
INT((Fraction * 100) + 13.3)	
STR_TO_NUM(MID(Code, 4, 2)) + 5	
LENGTH("TRUE" & Code)	

[4]

- (c) The program makes use of complex statistical functions. The required functions are not built-in to the programming language and are too complicated for the programmer to write.

One solution would be to employ another programmer who has experience of writing these functions, as there is no time to train the existing programmer.

State **one other** way that these functions may be provided for inclusion in the program.

[1]

57. O/N 23/P23/Q4/a,b

- 4 A procedure `RandList()` will output a sequence of 25 random integers, where each integer is larger than the previous one.

(a) Write pseudocode for procedure `RandList()`.

[6]

- (b) Procedure `RandList()` is modified so that the random numbers are also written into a 1D array `Result`.

A new module is written to confirm that the numbers in the array are in ascending order.

This module contains an `IF` statement that performs a comparison between elements:

```
IF (Result[x + 1] = Result[x]) OR (Result[x] > Result[x + 1]) THEN
    Sequence ← FALSE
ENDIF
```

Write a simplified version of the conditional clause.

[1]

58. O/N 23/P23/Q6

- 6 A function `TestNum()` will take a six-digit string as a parameter.

The function will test whether the string meets certain conditions and will return an integer value as follows:

Return value	Condition	Example
1	The last three digits are the same but non-zero.	"253444"
2	The last three digits are zero.	"253000"
3	The first three and last three digits are the same.	"410410"

The function will return the highest possible value for the given string.

If the string does not meet any of the conditions, zero is returned.

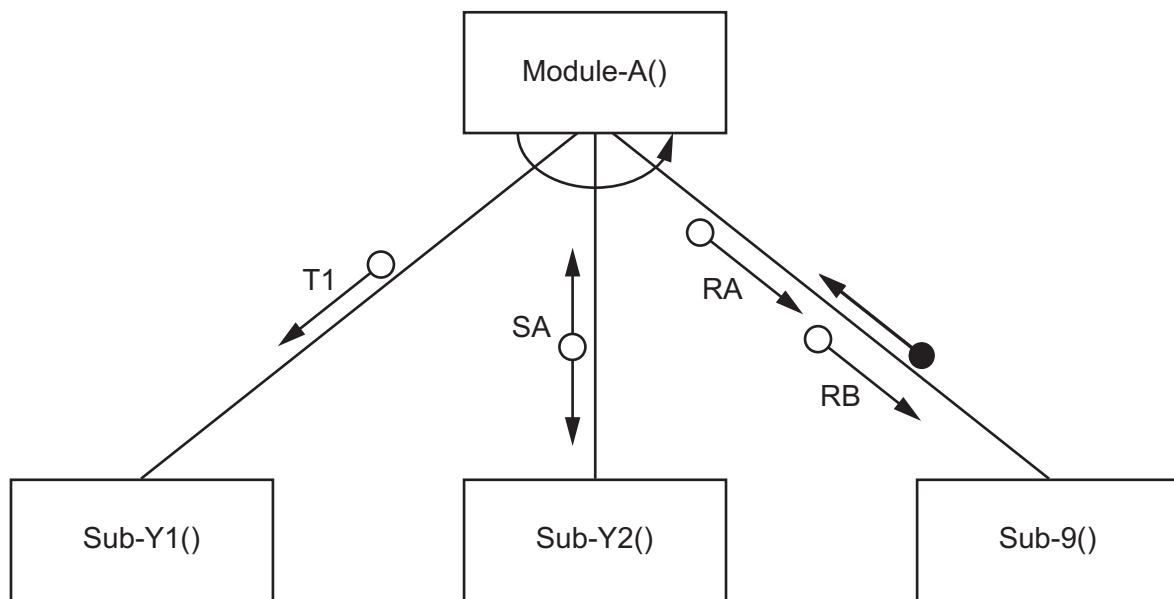
Write pseudocode for function `TestNum()`.

Assume that the parameter is valid.

[6]

59. O/N 23/P23/Q7/b(ii)

7 A structure chart shows the modular structure of a program:



(b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

(ii) The design is modified and Sub-9() is changed to a procedure.

The procedure will be called with a single parameter of type `MyType`.

Write the pseudocode header for procedure `Sub-9()`.

[2]

60. O/N 23/P23/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

<STX>"454202Hello Jack"<ETX>

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	- returns the data field from a message that has been received - If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	- takes a file name as a parameter of type string - creates a text file with the given file name (no checking required) - writes the data field returned by <code>GetData()</code> to the file - repeats until the data field is "****", which is not written to the file - outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing a port number - transmits the message using the given port
<code>Chat()</code>	- takes two parameters: - a string representing a Destination ID - an integer representing a port number - extracts data from a received message using <code>GetData()</code> and outputs it - forms a message using data input by the user and sends it using <code>Transmit()</code> - repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

61. M/J 23/P21/Q1/b,c

- 1 A programmer has written a program which includes the function `Calculate()`. When the program is run, the function returns an unexpected value.

(b) The algorithm for function `Calculate()` contains the three pseudocode statements shown.

Describe the error in each statement or write 'no error' if the statement contains no error.

Assume any variables used are of the correct type for the given function.

Statement 1: `Index ← STR_TO_NUM("27") + 2)`

Error

Statement 2: `Index ← STR_TO_NUM(MID("CPE1704TKS", 4, 2))`

Error

Statement 3: `IF MONTH(ThisDate) > '6' THEN`

Error

[3]

(c) The program contains variables with values as follows:

Variable	Value
Active	TRUE
Points	75
Exempt	FALSE

(i) Complete the table by evaluating each expression.

	Expression	Evaluation
1	<code>(Points > 99) OR Active</code>	
2	<code>(Points MOD 2 = 0) OR Exempt</code>	
3	<code>(Points <= 75) AND (Active OR Exempt)</code>	
4	<code>(Active OR NOT Active) AND NOT Exempt</code>	

[2]

(ii) Write expression 4 from the table in part (c)(i) in its simplest form.

[1]

62. M/J 23/P21/Q2/b

- 2 A program contains an algorithm to output a string of a specified length containing identical characters.
- (b) A different part of the program uses the variable `StartDate`.

Write pseudocode statements to declare `StartDate` **and** assign to it the date corresponding to 15/11/2005.

Declaration

Assignment

[3]

63. M/J 23/P21/Q4

- 4 Function `Replace()` will:
1. take three parameters:
 - a string (the original string)
 - a char (the original character)
 - a char (the new character)
 2. form a new string from the original string where all instances of the original character are replaced by the new character
 3. return the new string.

Write pseudocode for function `Replace()`.

[6]

64. M/J 23/P21/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `IsNewSupp()`.

Module `SuppExists()` has already been written and should be used as part of your solution. Module `SuppExists()` will generate a run-time error if the given parameter is not 5 characters in length.

65. M/J 23/P22/Q1/a

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (a) (i) Identify a more appropriate way of representing the postal costs.

[1]

- (ii) Describe the advantages of your answer to **part (a)(i)** with reference to this program.

[3]

66. M/J 23/P22/Q2/a,b

- 2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

- (a) Write a pseudocode statement, using a function from the **insert**, to assign the value corresponding to 17/11/2007 to `MyDOB`.

[1]

- (b) `MyDOB` has been assigned a valid value representing the user's date of birth.

Write a pseudocode statement to calculate the number of months from the month of the user's birth until the end of the year and to assign this to the variable `NumMonths`.

For example, if `MyDOB` contains a value representing 02/07/2008, the value 5 would be assigned to `NumMonths`.

[2]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and Character Functions

- A string of length 1 may be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

<code>LEFT(ThisString : STRING, x : INTEGER)</code> RETURNS STRING returns leftmost <code>x</code> characters from <code>ThisString</code> Example: <code>LEFT("ABCDEFGH", 3)</code> returns "ABC"
<code>RIGHT(ThisString : STRING, x : INTEGER)</code> RETURNS STRING returns rightmost <code>x</code> characters from <code>ThisString</code> Example: <code>RIGHT("ABCDEFGH", 3)</code> returns "FGH"
<code>MID(ThisString : STRING, x : INTEGER, y : INTEGER)</code> RETURNS STRING returns a string of length <code>y</code> starting at position <code>x</code> from <code>ThisString</code> Example: <code>MID("ABCDEFGH", 2, 3)</code> returns string "BCD"
<code>LENGTH(ThisString : STRING)</code> RETURNS INTEGER returns the integer value representing the length of <code>ThisString</code> Example: <code>LENGTH("Happy Days")</code> returns 10
<code>TO_UPPER(x : <datatype>)</code> RETURNS <datatype> <datatype> may be CHAR or STRING returns an object of type <datatype> formed by converting all characters of <code>x</code> to upper case. Examples: • <code>TO_UPPER("Error 803")</code> returns "ERROR 803" • <code>TO_UPPER('a')</code> returns 'A'
<code>TO_LOWER(x : <datatype>)</code> RETURNS <datatype> <datatype> may be CHAR or STRING returns an object of type <datatype> formed by converting all characters of <code>x</code> to lower case. Examples: • <code>TO_LOWER("JIM 803")</code> returns "jim 803" • <code>TO_LOWER('W')</code> returns 'w'
<code>NUM_TO_STR(x : <datatype1>)</code> RETURNS <datatype2> returns a string representation of a numeric value. <datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING Example: <code>NUM_TO_STR(87.5)</code> returns "87.5"
<code>STR_TO_NUM(x : <datatype1>)</code> RETURNS <datatype2> returns a numeric representation of a string. <datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER Example: <code>STR_TO_NUM("23.45")</code> returns 23.45
<code>IS_NUM(ThisString : <datatype>)</code> RETURNS BOOLEAN returns TRUE if <code>ThisString</code> represents a valid numeric value. <datatype> may be CHAR or STRING Example: <code>IS_NUM("-12.36")</code> returns TRUE

`ASC(ThisChar : CHAR)` RETURNS INTEGER

returns an integer value (the ASCII value) of character `ThisChar`

Example: `ASC('A')` returns 65, `ASC('B')` returns 66, etc.

`CHR(x : INTEGER)` RETURNS CHAR

returns the character whose integer value (the ASCII value) is `x`

Example: `CHR(65)` returns 'A', `CHR(66)` returns 'B', etc.

Numeric Functions

`INT(x : REAL)` RETURNS INTEGER

returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`RAND(x : INTEGER)` RETURNS REAL

returns a real number in the range 0 to `x` (**not** inclusive of `x`).

Example: `RAND(87)` may return 35.43

Date Functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

`DAY(ThisDate : DATE)` RETURNS INTEGER

returns the current day number from `ThisDate`

Example: `DAY(04/10/2003)` returns 4

`MONTH(ThisDate : DATE)` RETURNS INTEGER

returns the current month number from `ThisDate`

Example: `MONTH(04/10/2003)` returns 10

`YEAR(ThisDate : DATE)` RETURNS INTEGER

returns the current year number from `ThisDate`

Example: `YEAR(04/10/2003)` returns 2003

`DAYINDEX(ThisDate : DATE)` RETURNS INTEGER

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(09/05/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER)` RETURNS DATE

returns a value of type DATE with the value of <Day>/<Month>/<Year>

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY()` RETURNS DATE

returns a value of type DATE corresponding to the current date.

Text File Functions

`EOF(FileName : STRING)` RETURNS BOOLEAN

returns TRUE if there are no more lines to be read from file `FileName`

The function will generate an error if the file is not already open in READ mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" The operator may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison Operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes:
<	- may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR or STRING - cannot be used to compare two records
>=	
<=	Examples:
<>	"Program" = "program" evaluates to FALSE - Count = 4 evaluates to TRUE when variable Count contains the value 4

67. M/J 23/P22/Q4

4 A function `GetNum()` will:

- take two parameters: a string and a character
- count the number of times that the character occurs in the string
- return the count.

Any comparison between characters needs to be case sensitive. For example, character 'a' and character 'A' are not identical.

Write pseudocode for function `GetNum()`.

68. M/J 23/P22/Q6

- 6 A procedure `Square()` will take an integer value in the range 1 to 9 as a parameter and output a number square.

The boundary of a number square is made up of the character representing the parameter value. The inside of the number square is made up of the asterisk character (*).

Parameter value	1	2	3	4	...	9
Output	1	22 22	333 3*3 333	4444 4**4 4**4 4444	...	999999999 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 999999999

The pseudocode `OUTPUT` command starts each output on a new line. For example, the following three `OUTPUT` statements would result in the outputs as shown:

```
OUTPUT "Hello"
OUTPUT "ginger"
OUTPUT "cat"
```

Resulting output:

```
Hello
ginger
cat
```

Write pseudocode for procedure `Square()`.

Parameter validation is not required.

69. M/J 23/P22/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckInfo()`.

Module `OnlyAlpha()` should be used as part of your solution.

[7]

70. M/J 23/P23/Q2/a

- 2 A program stores a date of birth for a student using a variable, `MyDOB`, of type `DATE`.

- (a) `MyDOB` has been assigned a valid value corresponding to Kevin's date of birth.

Complete the pseudocode statement to test whether Kevin was born on a Thursday.

IF THEN [2]

71. M/J 23/P23/Q4

4 A function `MakeString()` will:

1. take two parameters:
 - a count as an integer
 - a character
2. generate a string of length equal to the count, made up of the character
3. return the string generated, or return "ERROR" if the count is less than 1.

For example, the function call:

`MakeString(3, 'Z')` will return the string "ZZZ"

Write pseudocode for function `MakeString()`.

[6]

72. M/J 23/P23/Q6/a,b

6 A procedure `Select()` will:

1. take two integer values as parameters representing start and end values where both values are greater than 9 and the end value is greater than the start value
2. output **each** integer value between the start and the end value (**not** including the start and end values), where the sum of the last two digits is 6, for example, 142.

(a) Write pseudocode for procedure `Select()`.

Parameter validation is **not** required.

[7]

(b) The check performed by procedure `Select()` on the last two digits is needed at several places in the program and will be implemented using a new function.

The new function `CheckNum()` will:

- allow the required sum to be specified (not just 6)
- check one number
- return an appropriate value.

Describe the function interface **and two** advantages of this modular approach.

Interface

Advantage 1

Advantage 2

[4]

73. O/N 22/P21/Q1/a,c

- 1 (a) An algorithm includes a number of complex calculations. A programmer is writing a program to implement the algorithm and decides to use library routines to provide part of the solution.

State **three** possible benefits of using library routines in the development of the program.

1

2

3

[3]

- (c) The pseudocode statements in the following table may contain errors.

State the error in each case or write 'NO ERROR' if the statement contains no error.

Assume that any variables used are of the correct type for the given function.

Statement	Error
IF EMPTY ← "" THEN	
Status ← IS_NUM(-23.4)	
X ← STR_TO_NUM("37") + 5	
Y ← STR_TO_NUM("37" + "5")	

[4]

74. O/N 22/P21/Q6/a

- 6** The following pseudocode algorithm attempts to check whether a string is a valid email address.

```

FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
            IF Ats > 1 THEN
              Valid ← FALSE
            ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF

  UNTIL Index > LENGTH(InString) OR Valid = FALSE

  IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
  ENDIF

  RETURN Valid

ENDFUNCTION

```

- (a)** Part of the validation is implemented by the line:

```
IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
```

State the values that would result in the condition evaluating to TRUE.

75. O/N 22/P21/Q7/a

7 A simple arithmetic expression is stored as a string in the format:

`<Value1><Operator><Value2>`

An operator character is one of the following: `'+'` `'-'` `'*'` `'/'`

Example arithmetic expression strings:

`"803+1904"`

`"34/7"`

(a) A procedure `Calculate()` will:

- take an arithmetic expression string as a parameter
- evaluate the expression
- output the result.

Assume:

- the string contains only numeric digits and a single operator character
- `Value1` and `Value2` represent integer values
- `Value1` and `Value2` are unsigned (they will not be preceded by `'+'` or `'-'`).

(i) Write pseudocode for the procedure `Calculate()`.

[7]

(ii) `Calculate()` is changed to a function that returns the value of the evaluated expression.

Write the header for the function in pseudocode.

[1]

76. O/N 22/P21/Q8/b

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	- takes the name of an existing project file as a parameter of type string - returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	- takes a line from a project file as a parameter of type string - returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	- takes two parameters: - the name of a project file as a string - the maximum number of errors as an integer - uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid - calls <code>CheckLine()</code> for each line in the project file - counts the number of errors - outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (b) `CountErrors()` is called to check the project file `Jim01Prog.txt` and to stop if more than 20 errors are found.

Write the pseudocode statement for this call.

[2]

77. O/N 22/P22/Q1/a,c

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc(Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers **Count** and **Message** **and** explain their use.

Term

Use

[2]

(c) Part of an identifier table is shown:

Variable	Type	Example value
FlagDay	DATE	23/04/2004
CharList	STRING	"ABCDEF"
Count	INTEGER	29

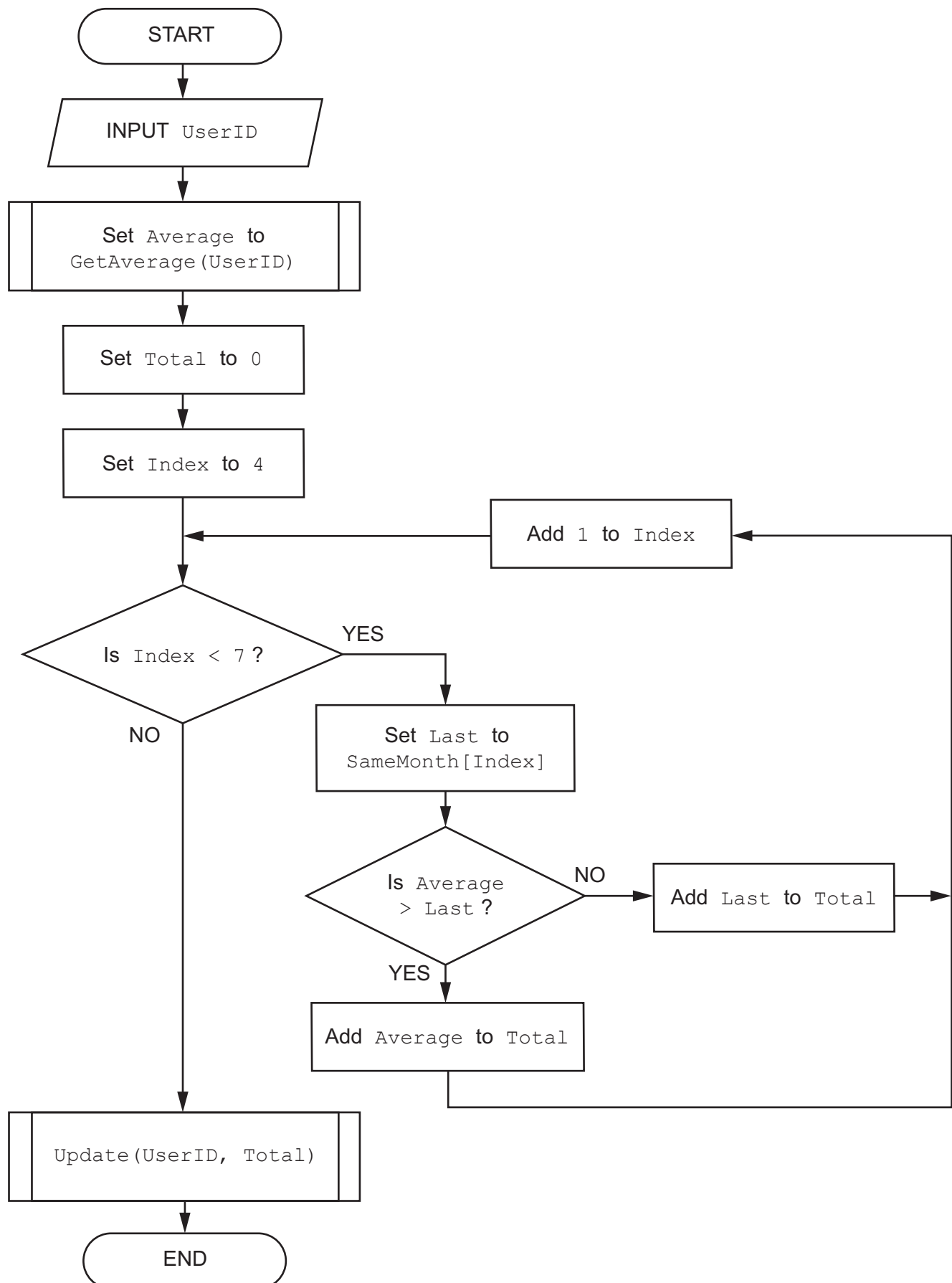
Complete the table by evaluating each expression using the example values.

Expression	Evaluation
MID(CharList, MONTH(FlagDay), 1)	
INT(Count / LENGTH(CharList))	
(Count >= 29) AND (DAY(FlagDay) > 23)	

[3]

78. O/N 22/P22/Q4/b

4 The program flowchart represents a simple algorithm.



(b) Give the name of the iterative construct in the flowchart.

[1]

79. O/N 22/P22/Q6/a

- 6 (a) The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter
- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

[7]

80. O/N 22/P23/Q5/a,b(ii)

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (i) Write pseudocode for the procedure `Unpack()`.

[6]

- (ii) Explain the term **procedure interface** with reference to procedure `Unpack()`.

[2]

- (b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
  DECLARE StockID : STRING
  DECLARE Description : STRING
  DECLARE Cost : REAL
ENDTYPE
```

- (ii) Procedure `Unpack()` is modified and converted to a function which takes the original text string as the only parameter.

Explain the other changes that need to be made to convert the procedure into a function.

[2]

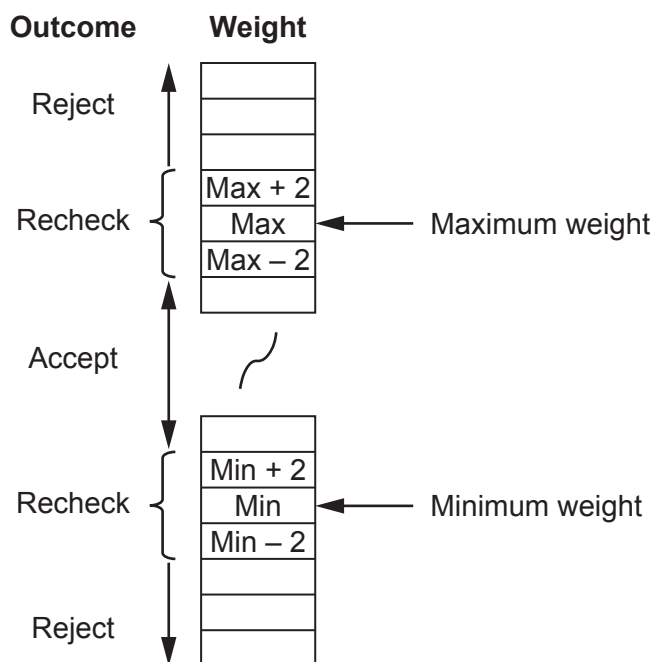
81. O/N 22/P23/Q6/b

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (b) Write pseudocode for `Status()`.

[6]

82. O/N 21/P21/Q6/c

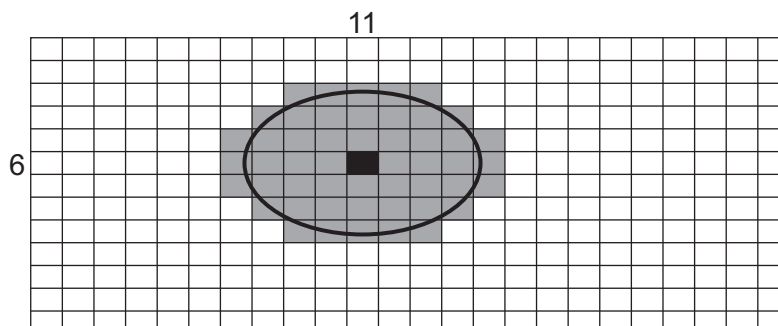
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
SetRow() (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
SearchInRow()	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
SearchInCol()	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
GetCentreCol()	- Called with a row number as an <code>INTEGER</code> - Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 - Returns the index of the column midway between the first and last columns - Returns <code>-1</code> if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

[6]

83. O/N 21/P22/Q1/c,d

(c) Complete the pseudocode expressions so that they evaluate to the values shown.

Any functions and operators used must be defined in the **insert**.

Expression	Evaluates to
..... ('C')	67
2 * ("27")	54
..... (27 /)	13
"Sub" & ("Abstraction" , ,)	"Subtract"

[4]

(d) Evaluate the expressions given in the following table. The variables have been assigned values as follows:

```
PumpOn ← TRUE
PressureOK ← TRUE
HiFlow ← FALSE
```

Expression	Evaluates to
PressureOK AND HiFlow	
PumpOn OR PressureOK	
NOT PumpOn OR (PressureOK AND NOT HiFlow)	
NOT (PumpOn OR PressureOK) AND NOT HiFlow	

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost x characters from ThisString

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Alphabetic characters that are not upper case are unchanged.

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Alphabetic characters that are not lower case are unchanged.

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case.

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case.

Example: `TO_LOWER("JIM 803")` returns "jim 803"

NUM_TO_STR(*x* : <data type1>) RETURNS <data type2>
returns a string representation of a numeric value.

Note: <data type1> may be REAL or INTEGER
<data type2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

STR_TO_NUM(*x* : <data type1>) RETURNS <data type2>
returns a numeric representation of a string.

Note: <data type1> may be CHAR or STRING
<data type2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

IS_NUM(ThisString : STRING) RETURNS BOOLEAN
returns the value TRUE if ThisString represents a valid numeric value.

Example 1: IS_NUM("12.36") returns TRUE

Example 2: IS_NUM("-12.36") returns TRUE

Example 3: IS_NUM("12.3a") returns FALSE

ASC(ThisChar : CHAR) RETURNS INTEGER
returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65

CHR(*x* : INTEGER) RETURNS CHAR
returns the character whose integer value (the ASCII value) is *x*

Example: CHR(87) returns 'W'

NUMERIC Functions

INT(*x* : REAL) RETURNS INTEGER
returns the integer part of *x*

Example: INT(27.5415) returns 27

RAND(*x* : INTEGER) RETURNS REAL
returns a real number in the range 0 to *x* (**not** inclusive of *x*).

Example: RAND(87) could return 35.43

OTHER Functions

EOF(FileName : STRING) RETURNS BOOLEAN
returns TRUE if there are no more lines to be read from file FileName

Note: The function will generate an error if the file is not already open in READ mode.

Note: An error occurs if an operator with a value of an incorrect type is used.

OPERATORS

&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE evaluates to FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE evaluates to TRUE
NOT	Performs a logical NOT on a Boolean value Example: NOT TRUE evaluates to FALSE
MOD	Finds the remainder when one number is divided by another Example: 10 MOD 3 evaluates to 1
DIV	Finds the quotient when one number is divided by another Example: 10 DIV 3 evaluates to 3

84. O/N 21/P22/Q4/a,c

4 The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```

10  PROCEDURE Check(InString : STRING)
11      DECLARE Odds, Evens, Index : INTEGER
12
13      Odds ← 0
14      Evens ← 0
15      Index ← 1
16
17      WHILE Index <= LENGTH(InString)
18          IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19              Odds ← Odds + 1
20          ELSE
21              Evens ← Evens + 1
22          ENDIF
23          Index ← Index + 1
24      ENDWHILE
25
26      CALL Result(Odds, Evens)
27  ENDPROCEDURE

```

(a) Complete the following table by giving the answers, using the given pseudocode.

Answer

A line number containing a variable being incremented	
The type of loop structure	
The number of functions used	
The number of parameters passed to STR_TO_NUM()	
The name of a procedure other than Check()	

[5]

(c) (i) The loop structure used in the pseudocode is not the most appropriate.

State a more appropriate loop structure **and** justify your choice.

Loop structure

Justification

[2]

(ii) The appropriate loop structure is now used. Two lines of pseudocode are changed and two lines are removed.

Write the line numbers of the two lines that are removed.

[1]

85. O/N 21/P22/Q6/b,c(ii),d

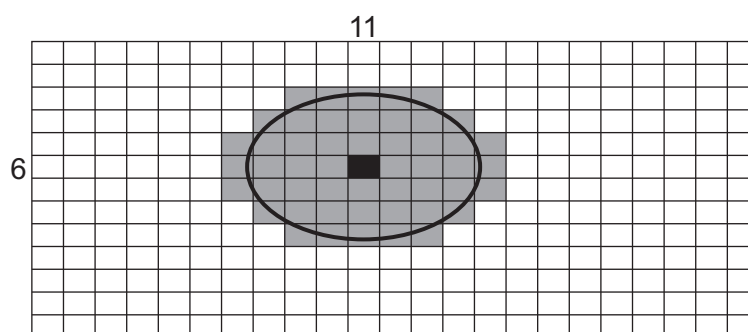
- 6** A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	- Searches for the first row that has an array element set to 1 - Returns the index of that row (1 is the first row) - Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	- Searches for the last row that has an array element set to 1 - Returns the index of that row - Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	- Searches for the first column that has an array element set to 1 - Returns the index of that column (1 is the first column) - Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	- Searches for the last column that has an array element set to 1 - Returns the index of that column - Returns -1 if there are no elements set to 1

- (a)** Write efficient pseudocode for the module `FirstRowSet()`.

- (b) Describe a feature of your solution to **part (a)** that indicates the pseudocode represents an efficient algorithm.

[2]

- (c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (ii) Give one possible advantage **and** one possible disadvantage of combining the four searches into a single module.

Advantage

Disadvantage

[2]

- (d) An additional module `GetCentre()` is defined as follows:

Module	Description
<code>GetCentre()</code>	- Calculates the coordinates of the centre point - Uses the four modules: <code>FirstRowSet()</code>, <code>LastRowSet()</code>, <code>FirstColSet()</code>, <code>LastColSet()</code> - Assigns values to <code>CentreRow</code> and <code>CentreCol</code> - Sets <code>CentreRow</code> to <code>-1</code> if no screen touch is detected

Write pseudocode for the module `GetCentre()`.

[6]

86. O/N 21/P23/Q6/c

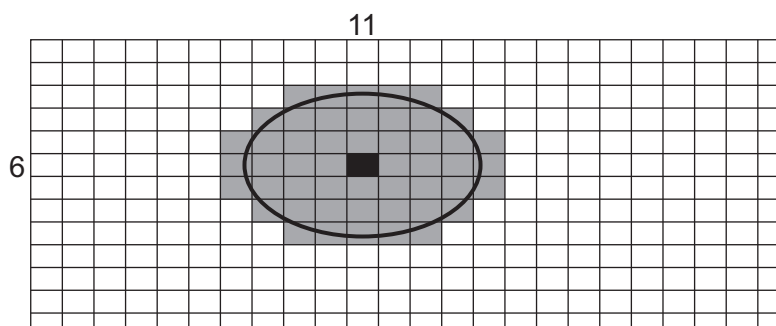
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
<code>SetRow()</code> (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
<code>SearchInCol()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
<code>GetCentreCol()</code>	• Called with a row number as an <code>INTEGER</code> • Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 • Returns the index of the column midway between the first and last columns • Returns -1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

87. M/J 21/P21/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

88. M/J 21/P21/Q2/b

- 2 (a)** Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either LoanExtend() or LoanReturn()
LoanExtend()	• Called with parameters LoanID and BookID • Calls CheckReserve() to see whether the book has been reserved for another library user • Returns TRUE if the loan has been extended, otherwise returns FALSE
CheckReserve()	• Called with BookID • Returns TRUE if the book has been reserved, otherwise returns FALSE
LoanReturn()	• Called with parameters LoanID and BookID • Returns a REAL (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

- (b)** The definition for module LoanReturn() is amended as follows:

Module name	Description
LoanReturn()	Called with parameters LoanID, BookID and Fine The module code checks whether the book has been returned on time and then assigns a new value to Fine

- LoanID and BookID are of type STRING
- Fine is of type REAL

Write the pseudocode header for the **amended** module LoanReturn().

[2]

89. M/J 21/P21/Q4/b

4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert (Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH (Name)
24     ThisChar ← MID (Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE (ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

90. M/J 21/P21/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

91. M/J 21/P22/Q5/b

(b) The following is a pseudocode function.

Line numbers are given for reference only.

```

01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03      DECLARE NextChar : CHAR
04      DECLARE OutString : STRING
05      DECLARE Counter : INTEGER
06
07      OutString ← ""
08
09      FOR Counter ← 1 TO LENGTH(InString)
10          NextChar ← MID(InString, Counter, 1)
11          NextChar ← LCASE(NextChar)
12          IF NOT((NextChar < 'a') OR (NextChar > 'z')) THEN
13              OutString ← OutString & NextChar
14          ENDIF
15      NEXT Counter
16
17      RETURN OutString
18
19  ENDFUNCTION

```

(i) Examine the pseudocode and complete the following table.

Answer

Give a line number containing an example of an initialisation statement.	
Give a line number containing the start of a repeating block of code.	
Give a line number containing a logic operation.	
Give the number of parameters to the function MID().	

[4]

(ii) Write a simplified version of the statement in line 12.

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

92. M/J 21/P22/Q8/b

- 8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as its parameter, representing the number of a word in <code>FNString</code> • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: <code>GetStart(3)</code> applied to "hot and cold" returns 9
<code>GetWord()</code>	• Called with the position of the first character of a word in <code>FNString</code> as its parameter • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"
<code>IgnoreWord()</code>	• Called with a <code>STRING</code> parameter representing a word • Searches for the word in the <code>IgnoreList</code> array • Returns <code>TRUE</code> if the word is found, otherwise returns <code>FALSE</code>
<code>GetInitials()</code>	• Processes the sequence of words in the full name one word at a time • Calls <code>GetStart()</code>, <code>GetWord()</code> and <code>IgnoreWord()</code> to process each word to form the new string • Outputs the new string

- (b) Write pseudocode for the module `GetInitials()`.

93. M/J 21/P23/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

94. M/J 21/P23/Q2/b

- 2 (a)** Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
LoanExtend()	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
CheckReserve()	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
LoanReturn()	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

- (b)** The definition for module `LoanReturn()` is amended as follows:

Module name	Description
LoanReturn()	Called with parameters <code>LoanID</code>, <code>BookID</code> and <code>Fine</code> The module code checks whether the book has been returned on time and then assigns a new value to <code>Fine</code>

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

[2]

95. M/J 21/P23/Q4/b

4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert (Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH (Name)
24     ThisChar ← MID (Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE (ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

96. M/J 21/P23/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

97. 9608 M/J 21/P21/Q1/c

- (c) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, **or** write 'NO ERROR' if the statement contains no error.

Refer to the **Appendix** on pages 22–23 for the list of built-in pseudocode functions and operators.

Statement	Error
Code $\leftarrow$ LEFT("Cat", 4)	
Status $\leftarrow$ MID("Aardvark", 0, 5)	
Size $\leftarrow$ LENGTH("Password)	
Stock[n] $\leftarrow$ Stock[n+1]	
Result $\leftarrow$ 3 OR 4	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
 returns the character value representing the lower case equivalent of `ThisChar`
 If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
 returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER)` RETURNS REAL
 returns a real number in the range 0 to `x` (not inclusive of `x`).

Example: `RAND(87)` could return 35.43

`NUM_TO_STRING(x : REAL)` RETURNS STRING
 returns a string representation of a numeric value.
 Note: This function will also work if `x` is of type `INTEGER`

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

98. 9608 M/J 21/P21/Q5/a(i)

- 5 Alice is developing an algorithm for a game. The game will count the number of attempts by a player to guess the secret number generated by the built-in random number function.

She will use pseudocode to develop a procedure, `GuessNum()`, that will:

1. generate a secret number that is a random integer in the range 1 to 20 inclusive
2. prompt and input the player's guess
3. output a message if the guess is not the same as the secret number and repeat from step 2
4. count the number of attempts it takes to guess the secret number
5. output a message when the secret number has been guessed correctly, giving the number of attempts.

- (a) (i) Write **pseudocode** for the procedure `GuessNum()`.

Refer to the **Appendix** on pages 22–23 for the list of built-in pseudocode functions and operators.

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER)` RETURNS REAL
returns a real number in the range 0 to `x` (not inclusive of `x`).

Example: `RAND(87)` could return 35.43

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

99. 9608 M/J 21/P22/Q1/d

- (d) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, **or** write 'NO ERROR' if the statement contains no error.

Refer to the **Appendix** on page 18 for the list of built-in pseudocode functions and operators.

Statement	Error
Code ← RIGHT("Cap" & "art", 4)	
Status ← MID("Computer", 7, 5)	
Size ← LENGTH("Password") * 2	
NextChar ← CHR('A')	
Index ← Index & 3	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString
Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString
Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString
Example: LEFT("ABCDEFGH", 3) returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`CHR(x : INTEGER)` RETURNS CHAR
returns the character whose ASCII value is `x`

Example: `CHR(87)` returns 'W'

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

100. 9608 M/J 21/P22/Q2/a

2 Study the following pseudocode.

```
DECLARE Overload : BOOLEAN
```



```
PROCEDURE LEM()
```

```
    DECLARE Status : BOOLEAN
```

```
    DECLARE Landed : INTEGER
```

```
    Overload ← FALSE
```

```
    Landed ← FALSE
```

```
    WHILE Landed = FALSE
```

```
        Status ← Sample()
```

```
        IF Status = TRUE
```

```
            THEN
```

```
                Landed ← SubA(42)
```

```
            ELSE
```

```
                Overload ← SubB(37)
```

```
                IF Overload = TRUE
```

```
                    THEN
```

```
                        CALL Display("Alarm 1202")
```

```
                    ENDIF
```

```
            ENDIF
```

```
    ENDWHILE
```

```
ENDPROCEDURE
```

(a) Examine the pseudocode **and** complete the following table:

Answer

The identifier name of a global variable	
The name of the loop structure	
The identifier involved in a data type mismatch	
The name of a procedure that takes a parameter	
The name of a function	

[5]

101. 9608 M/J 21/P22/Q4/a

4 **(a)** Using **pseudocode**, write a post-condition loop to output every odd number between 100 and 200.

[4]

102. 9608 M/J 21/P22/Q5/b

- (b) (i) A function, `Verify()`, is written in pseudocode.

Write the **two** missing lines to complete the pseudocode.

```
FUNCTION Verify(UserID : STRING) RETURNS BOOLEAN

    DECLARE Password : STRING
    OUTPUT "Please Input your password: "
    INPUT Password
    Response ← Validate(UserID, Password) AND Today()

ENDFUNCTION
```

[2]

- (ii) The function, `Verify()`, is to be amended as follows:

- if the `UserID` is "Guest", a password is not required and `TRUE` should be returned
- output a message to try again if the password entered is not valid
- return `FALSE` if the number of attempts to enter a valid password exceeds three.

Write **program code** for the **amended** function `Verify()`.

Visual Basic and Pascal: You should include the declaration statements for variables.
Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[6]

103. 9608 M/J 21/P22/Q6/c

- 6 A program stores data about stock items in four global 1D arrays as follows:

Array	Data type	Description	Example data value	Initial data value
StockID	STRING	the stock item ID (eight alpha-numeric characters)	"JBCD0002"	""
Description	STRING	a description of the item (alphabetic characters only)	"soap"	""
Quantity	INTEGER	the number in stock	9	0
Cost	REAL	the cost of the item	1.45	0.0

- Each array contains 10 000 elements.
- Elements with the same index relate to the same stock item. For example, `StockID[3]` contains the ID for the product whose description is in `Description[3]`.
- The `StockID` array is not sorted.

The program will be modified so that the data from the arrays can be stored in a text file for backup. You may assume that a backup file contains only valid stock data.

The programmer has started to define program modules as follows:

Module	Description
<code>Unpack()</code>	- called with two parameters: - an array index - a string value read from one line of the backup file - extracts the four data values from the string and assigns each to the appropriate array
<code>Restore()</code>	- called with a string representing the name of a backup file - returns <code>FALSE</code> if the file is empty - sets all elements of each array to the initial data value as given in the table - reads the backup file line by line - calls <code>Unpack()</code> to extract data from each line and assign values to the corresponding arrays - returns <code>FALSE</code> if the arrays are full but there are still lines in the file, otherwise returns <code>TRUE</code>
<code>StockSummary()</code>	For all items where <code>StockID</code> does not contain the initial value: - counts the number of stock entries in the <code>StockID</code> array - outputs the overall value of all items in stock (cost multiplied by the quantity) - outputs the number of stock entries

(c) The module description of `GetValidFilename()` is as follows:

Module	Description
<code>GetValidFilename()</code>	- prompts and inputs a filename - validates the filename by checking that it: - is between 4 and 10 characters in length (inclusive) - contains only alphanumeric characters - if the filename is invalid, outputs a warning message and asks the user to try again - otherwise returns the valid filename

Write **program code** for the module `GetValidFilename()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

104. 9608 M/J 21/P23/Q1/b,c

also Unit 10 (2 marks); this unit 5 marks

(b) An algorithm searches a 1D array to find the first index of an element that contains a given value. If the value is found, the index of that element is returned.

(i) State an appropriate loop structure for this algorithm. Justify your choice.

Loop structure

Justification

[2]

- (ii) Give **two** possible reasons why the search for the value in **part (b)(i)** would end.

You should assume there is no error.

1

2

[2]

- (c) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, **or** write 'NO ERROR' if the statement contains no error.

Refer to the **Appendix** on pages 22 and 23 for the list of built-in pseudocode functions and operators.

Statement	Error
Code $\leftarrow$ RIGHT("Cap" * 3, 2)	
Valid $\leftarrow$ IS_NUM(3.14159)	
NextChar $\leftarrow$ MID(ThisString, Index), 1	

[3]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if `x` is of type CHAR

Example: `STRING_TO_NUM("23.45")` returns 23.45

`IS_NUM(ThisString : STRING)` RETURNS BOOLEAN
returns the value TRUE if `ThisString` contains only numeric characters ('0' to '9').

Example: `IS_NUM("123a")` returns FALSE

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

105. 9608 M/J 21/P23/Q5/a

- 5 A program stores a contact list of telephone numbers. Each telephone number is stored as a string of six or more numeric characters.

Before they are displayed, number strings are formatted to make them easier to read. This involves forming the characters into groups, separated by the space character.

The maximum length of a number group is five characters.

Different numbers may have different groupings. A template string is used to define the grouping.

For example:

Number string	Template string	Formatted string
"01223553998"	"53"	"01223 553 998"
"509700101"	"222"	"50 97 00 101"
"4044496128"	"33"	"404 449 6128"

For the first row, template "53" results in a formatted string comprising:

- the first five characters in the first group
- a space character
- the next three characters in the second group
- a space character
- the remaining characters from the number string.

- (a) Write **pseudocode** for a function `GroupNum()`, which takes a telephone number and a template as parameter strings and returns a formatted string.

You may assume that the template and telephone number are valid.

Refer to the **Appendix** on pages 22 and 23 for the list of built-in pseudocode functions and operators.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type `INTEGER`

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if `x` is of type `CHAR`

Example: `STRING_TO_NUM("23.45")` returns 23.45

`IS_NUM(ThisString : STRING)` RETURNS BOOLEAN
returns the value `TRUE` if `ThisString` contains only numeric characters ('0' to '9').

Example: `IS_NUM("123a")` returns `FALSE`

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
OR	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

106. M/J 20/P21/Q1/b

(b) A program design includes the use of subroutines (functions and procedures).

Give **three** advantages of using subroutines in a program.

1

2

3

[3]

107. M/J 20/P21/Q6/c(i)

6 A company hires out rowing boats on a lake. The company has ten boats, numbered from 1 to 10.

The company is developing a program to help manage and record the hiring out process.

Hire information is stored in three global 1D arrays when a boat is hired out. Each array contains 10 elements representing each of the ten boats.

The three 1D arrays are summarised as follows:

Array	Data type	Description	Example data value
HireTime	STRING	The time the boat was hired out	"10:15"
Duration	INTEGER	The number of minutes of the hire	30
Cost	REAL	The cost of the hire	5.75

If an individual boat is not currently on hire, the corresponding element of the `HireTime` array will be set to "Available".

The programmer has started to define program modules as follows:

Module	Description
<code>AddTime()</code>	- Called with two parameters: - a <code>STRING</code> value representing a time - an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>
<code>ListAvailable()</code>	- Called with a <code>STRING</code> value representing the time the hire will start - Outputs the boat numbers that will be available for hire at the given start time. A boat will be available for hire if it is either: - currently not on hire, or - due back before the given hire start time - Outputs the number of each boat available - Outputs the total number of boats available or a suitable message if there are none
<code>RecordHire()</code>	- Called with four parameters: - an <code>INTEGER</code> value representing the boat number - a <code>STRING</code> value representing the hire start time - an <code>INTEGER</code> value representing the hire duration in minutes - a <code>REAL</code> value representing the cost of hire - Updates the appropriate element in each array - Adds the cost of hire to the global variable <code>DailyTakings</code> - Converts the four input parameters to strings, concatenated using commas as separators, and writes the resulting string to the end of the existing text file <code>HireLog.txt</code>

(c) The module description of `AddTime()` is repeated here for reference.

Module	Description
<code>AddTime()</code>	- Called with two parameters: - a <code>STRING</code> value representing a time - an <code>INTEGER</code> value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a <code>STRING</code>

- (i) Write **program code** for a statement that uses the `AddTime()` function to add a duration of 60 minutes to a start time contained in variable `BeginTime` and to assign the new time to variable `EndTime`.

Programming language

Program code

[2]

108. M/J 20/P22/Q1/c

- (c) Count-controlled and post-condition are two types of loop.

Describe the characteristics of each of these types of loop.

Count-controlled

Post-condition

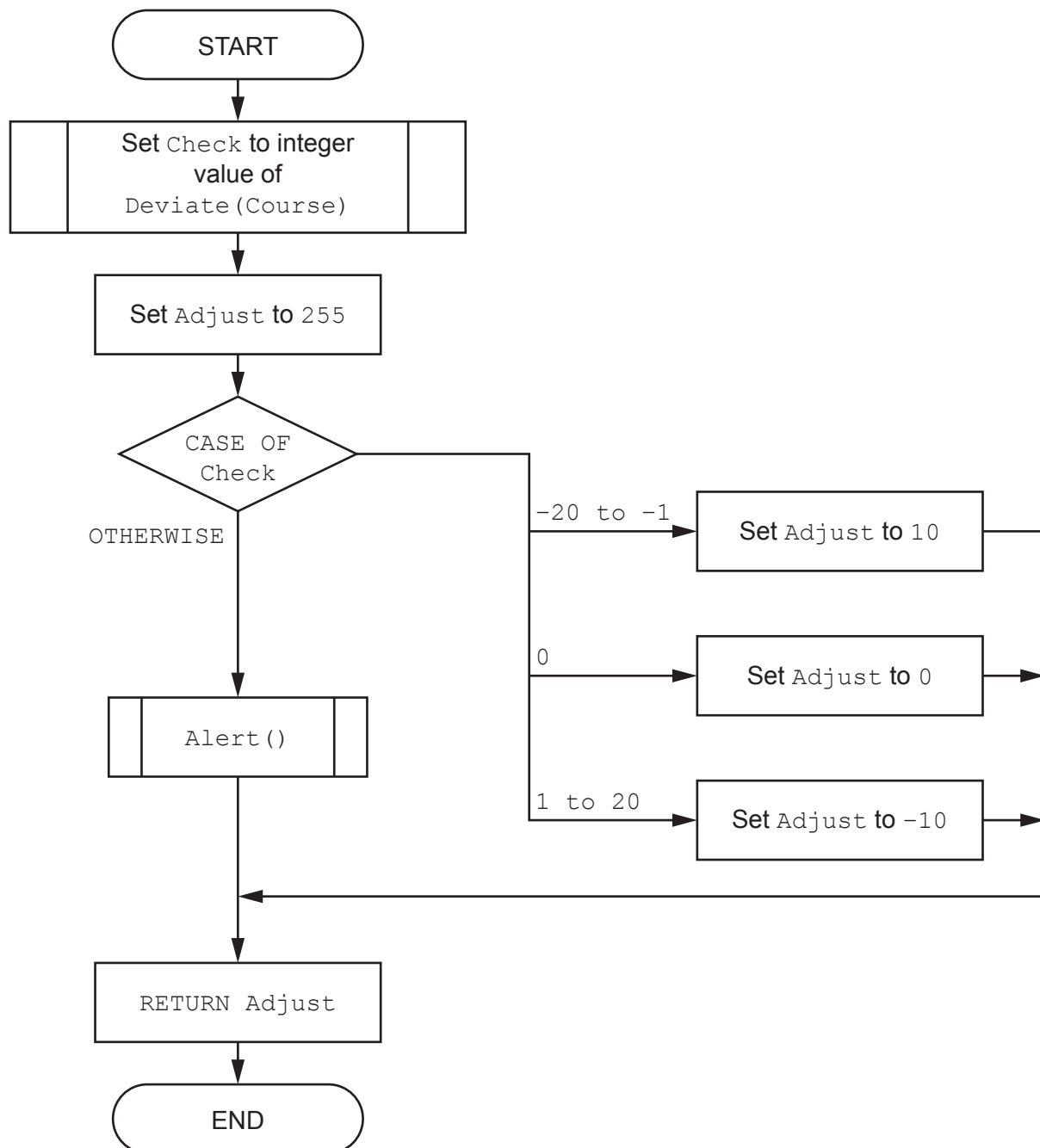
[2]

109. M/J 20/P22/Q3

- 3 A navigation program includes a function, `CheckCourse()`. This function is called with a real value, `Course`, and returns an integer value.

The identifier table and the program flowchart for the function are shown as follows:

Identifier	Type	Description
<code>Course</code>	REAL	The value passed to <code>CheckCourse()</code>
<code>Adjust</code>	INTEGER	The value returned by <code>CheckCourse()</code>
<code>Check</code>	INTEGER	A local variable
<code>Deviate()</code>	FUNCTION	A function that is passed a REAL value representing the course and returns a REAL value representing the current deviation
<code>Alert()</code>	PROCEDURE	A procedure that generates a warning



Write **pseudocode** to implement the function `CheckCourse()`. The pseudocode must follow the algorithm represented by the flowchart. Declare any local variables used.

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

[7]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost x characters from ThisString

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of x

Example: `INT(27.5415)` returns 27

`RAND(x : INTEGER)` RETURNS REAL
returns a real number in the range 0 to x (not inclusive of x)

Example: `RAND(87)` could return 35.43

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: `MOD(10, 3)` returns 1

`DIV(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the whole number part of the result when ThisNum is divided by ThisDiv

Example: `DIV(10, 3)` returns 3

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.

Example: If x has the value 87.5 then `NUM_TO_STRING(x)` returns "87.5"

Note: This function will also work if x is of type INTEGER

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.

Example: If x has the value "23.45" then `STRING_TO_NUM(x)` returns 23.45

Note: This function will also work if x is of type CHAR

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

110. M/J 20/P22/Q6/a(i)

6 A company hires out rowing boats on a lake. The company has 17 boats numbered from 1 to 17.

Boats may be hired between 9:00 and 18:00, with a maximum hire duration of 90 minutes.

The company is developing a program to help manage and record the boat hire process.

The programmer has decided to store all values relating to hire time as strings. The program will use a 24-hour clock format. For example:

Time (in words)	String value
Nine o'clock in the morning	"09:00"
Five minutes past ten o'clock in the morning	"10:05"
Ten minutes before three o'clock in the afternoon	"14:50"

The programmer has defined the first module as follows:

Module	Description
AddTime()	- Takes two parameters: - StartTime: a STRING value representing a time as described - Duration: an INTEGER value representing a duration in minutes - Adds the duration to the time to give a new time - Returns the new time as a STRING

(a) (i) Write **pseudocode** for the module AddTime(). Assume both input parameters are valid.

Refer to the **Appendix** on page 19 for a built-in list of pseudocode functions and operators.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

LENGTH(ThisString : **STRING**) RETURNS **INTEGER**
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : **STRING**, x : **INTEGER**) RETURNS **STRING**
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: **STRING**, x : **INTEGER**) RETURNS **STRING**
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : **REAL**) RETURNS **INTEGER**
returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : **INTEGER**) RETURNS **REAL**
returns a real number in the range 0 to x (not inclusive of x)

Example: RAND(87) could return 35.43

MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
 returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: MOD(10, 3) returns 1

DIV(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
 returns the integer value representing the whole number part of the result when ThisNum is divided by ThisDiv

Example: DIV(10, 3) returns 3

NUM_TO_STRING(x : REAL) RETURNS STRING
 returns a string representation of a numeric value.

Example: If x has the value 87.5 then NUM_TO_STRING(x) returns "87.5"

Note: This function will also work if x is of type INTEGER

STRING_TO_NUM(x : STRING) RETURNS REAL
 returns a numeric representation of a string.

Example: If x has the value "23.45" then STRING_TO_NUM(x) returns 23.45

Note: This function will also work if x is of type CHAR

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

111. M/J 20/P23/Q4/a,b

- 4 A global 1D array, `Contact`, of type `STRING` is used to store a list of names and email addresses. There are 1000 elements in the array. Each element stores one data item. The format of each data item is as follows:

`<Name>': '<EmailAddress>`

`Name` and `EmailAddress` are both variable-length strings.

For example:

`"Wan Zhu:zwan99@mymail.com"`

A function, `Extract()`, is part of the program that processes the array. A string data item is passed to the function as a parameter. The function will return the `Name` part. Validation is **not** necessary.

- (a) Write **program code** for the function `Extract()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[5]

- (b) The original function, `Extract()`, needs to be modified to separate the name from the email address. The calling program can then use **both** of these values.

Write, in **pseudocode**, the header for the modified subroutine. Explain the changes you have made.

Subroutine header

Explanation

[3]

112. M/J 20/P23/Q5/c

- 5 A company hires out rowing boats on a lake. The company has 20 boats, numbered from 1 to 20.

For safety reasons, the boats have to be serviced (checked and any damage repaired) regularly.

The company is developing a program to help manage the servicing of the boats.

Every time a boat is serviced, details are added at the end of the text file, `ServiceLog.txt`, as a single line of information. Each boat is serviced before it is hired out for the first time.

The format of each line is as follows:

`<BoatNumber><Date>`

`BoatNumber` and `Date` are as follows:

- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `Date` is an 8-digit numeric string in the format `YYYYMMDD`

The programmer has defined the first module as follows:

Module	Description
<code>GetLastService()</code>	• Called with a string parameter representing the <code>BoatNumber</code> • Searches through the file <code>ServiceLog.txt</code> • Returns the date of the last service in the form "YYYYMMDD"

- (b) (i) Every time a boat is hired out, details of the hire are added at the end of a text file, `Hirelog.txt`. Each line of the text file corresponds to information about the hire of one boat.

The format of each line of information is as follows:

`<Date><BoatNumber><HireDuration>`

- `Date` is an 8-digit numeric string in the format `YYYYMMDD`
- `BoatNumber` is a two-digit numeric string in the range "01" to "20"
- `HireDuration` is a variable-length string representing a numeric value in hours. For example, the string "1.5" would represent a hire duration of 1½ hours.

A module `GetHours()` is defined as follows:

Module	Description
<code>GetHours()</code>	• Takes two parameters: ◦ the <code>BoatNumber</code> as a string ◦ the date of the last service for that boat ("YYYYMMDD") as a string • Searches through file <code>Hirelog.txt</code> and calculates the sum of the hire durations for the given boat after the given date (hire durations on or before the given date are ignored) • Returns the total of the hire durations as a real

Note:

Standard comparison operators may be used with dates in this format.

For example:

`"20200813" > "20200812"` would evaluate to `TRUE`

Parameter validation is **not** required.

Write **pseudocode** for the module `GetHours()`.

Refer to the **Appendix** on page 19 for a list of built-in pseudocode functions and operators.

(c) A new module is described as follows:

Module	Description
ServiceList()	- Takes an integer as a parameter that represents the maximum number of hours before a boat must be serviced - Uses GetLastService() and GetHours() - Outputs: - a suitable heading - the BoatNumber of each boat hired for more than the maximum number of hours since its last service - the total hire duration for each of these boats An example output list is: <pre>Boat Service List 4: 123 17: 117</pre> If no boats are due to be serviced, the output is: <pre>Boat Service List No boats are due to be serviced</pre>

Write **program code** for the module ServiceList().

Python: You should show a comment statement for each variable used with its data type.
Visual Basic and Pascal: You should include the declaration statements for variables.

Programming language

Program code

[8]

113. O/N 19/P21/Q1/b(ii)

1 Study the following pseudocode.

```

FUNCTION Search() RETURNS INTEGER
    DECLARE N, C : INTEGER
    DECLARE V, L : REAL
    V ← GetLevel()
    L ← V * 1.34
    C ← 0
    FOR N ← 1 TO 10
        V ← GetLevel()
        IF V > L
            THEN
                C ← C + 1
            ENDIF
    ENDFOR
    OUTPUT "Process complete"
    RETURN C
ENDFUNCTION

```

(ii) Evaluate each expression in the following table.

If an expression is invalid then write 'ERROR'.

Refer to the **Appendix** on page 16–17 for the list of built-in functions and operators.

Expression	Evaluates to
LEFT("Start", 3) & RIGHT("Apple", 3)	
MID("sample", 3, 5)	
NUM_TO_STRING(12.3 * 2)	
INT(STRING_TO_NUM("53.4")) + 7	

[4]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if x is of type INTEGER

Example: NUM_TO_STRING(87.5) returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
 returns a numeric representation of a string.
 Note: This function will also work if `x` is of type `CHAR`

Example: `STRING_TO_NUM("23.45")` returns 23.45

`ASC(ThisChar : CHAR)` RETURNS INTEGER
 returns the ASCII value of `ThisChar`

Example: `ASC('A')` returns 65

`CHR(x : INTEGER)` RETURNS CHAR
 returns the character whose ASCII value is `x`

Example: `CHR(87)` returns 'W'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
 returns the character value representing the upper case equivalent of `ThisChar`
 If `ThisChar` is not a lower case alphabetic character, it is returned unchanged.

Example: `UCASE('a')` returns 'A'

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

114. O/N 19/P21/Q2/c

(c) **By value** is one method of passing a parameter to a subroutine.

Identify and describe the other method.

Method

Description

[2]

115. O/N 19/P21/Q3

3 The following is a function design in pseudocode.

Line numbers are given for reference only.

```

10 FUNCTION Check(InString : STRING) RETURNS BOOLEAN
11
12     DECLARE NumDots : INTEGER
13     DECLARE Index : INTEGER
14     DECLARE NumOthers : INTEGER
15
16     NumDots ← 0
17     NumOthers ← 0
18     Index ← 1
19
20     WHILE NumDots < 3 AND Index <= LENGTH(InString)
21
22         IF MID(InString, Index, 1) = '.'
23             THEN
24                 NumDots ← NumDots + 1
25             ELSE
26                 NumOthers ← NumOthers + 1
27             ENDIF
28         Index ← Index + 1
29
30     ENDWHILE
31
32     IF NumDots = NumOthers
33         THEN
34             RETURN TRUE
35         ELSE
36             RETURN FALSE
37         ENDIF
38
39 ENDFUNCTION

```

Study the pseudocode. Identify the relevant features in the following table.

Refer to the **Appendix** on pages 16–17 for the list of built-in functions and operators.

Feature	Answer
The number of the line containing a variable being incremented	
The range of line numbers containing a pre-condition loop	
The number of initialisation statements	
The number of the line containing a logical operator	
The range of line numbers containing a selection statement	
The name of a built-in function	
The name of a parameter	

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if x is of type INTEGER

Example: NUM_TO_STRING(87.5) returns "87.5"

STRING_TO_NUM(x : STRING) RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if x is of type CHAR

Example: STRING_TO_NUM("23.45") returns 23.45

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of ThisChar

Example: ASC('A') returns 65

CHR(x : INTEGER) RETURNS CHAR
returns the character whose ASCII value is x

Example: CHR(87) returns 'W'

UCASE(ThisChar : CHAR) RETURNS CHAR
 returns the character value representing the upper case equivalent of ThisChar
 If ThisChar is not a lower case alphabetic character, it is returned unchanged.

Example: UCASE('a') returns 'A'

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

116. O/N 19/P21/Q6/d(ii)

- 6 Account information for users of a library is held in one of two text files; UserListAtoM.txt and UserListNtoZ.txt

The format of the data held in the two files is identical. Each line of the file is stored as a string that contains an account number, name and telephone number separated by the asterisk character ('*') as follows:

<Account Number>*<Name>*<Telephone Number>

An example of one line from the file is:

"GB1234*Kevin Mapunga*07789123456"

The account number string may be **six** or **nine** characters in length and is **unique for each person**. It is made up of alphabetic and numeric characters only.

An error has occurred and the same account number has been given to different users in the two files. There is **no** duplication of account numbers **within each individual file**.

A program is to be written to search the two files and to identify duplicate entries. The account number of any duplicate found is to be written to an array, Duplicates, which is a 1D array of 100 elements of data type STRING.

The program is to be implemented as several modules. The outline description of three of these is as follows:

Module	Outline description
<code>ClearArray()</code>	Initialise the global array <code>Duplicates</code>. Set all elements to the empty string.
<code>FindDuplicates()</code>	- Read each line from the file <code>UserListAtoM.txt</code> - Check whether the account number appears in file <code>UserListNtoZ.txt</code> using <code>SearchFileNtoZ()</code> - If the account number does appear then add the account number to the array. - Output an error message and exit the module if there are more duplicates than can be written to the array.
<code>SearchFileNtoZ()</code>	- Search for a given account number in file <code>UserListNtoZ.txt</code> - If found, return <code>TRUE</code>, otherwise return <code>FALSE</code>

- (d) `ClearArray()` is to be modified to make it general purpose. It will be used to initialise any 1D array of data type `STRING` to any value.

It will now be called with three parameters as follows:

1. The array
2. The number of elements
3. The initialisation string

You should assume that the lower bound is 1.

- (ii) Write **program code** for a statement that calls the modified `ClearArray()` procedure to clear the array `Duplicates` to "Empty".

Programming language

Program code

[2]

117. O/N 19/P22/Q1/b(ii)**1** Study the following pseudocode.

```

PROCEDURE FillTank()

    DECLARE Tries : INTEGER
    DECLARE Full : BOOLEAN

    Tries ← 1

    Full ← ReadSensor("F1")

    IF NOT Full
        THEN
            WHILE NOT Full AND Tries < 4
                CALL TopUp()
                Full ← ReadSensor("F1")
                Tries ← Tries + 1
            ENDWHILE
            IF Tries > 3
                THEN
                    OUTPUT "Too many attempts"
                ELSE
                    OUTPUT "Tank now full"
                ENDIF
            ELSE
                OUTPUT "Already full"
            ENDIF
        ENDIF
    ENDPROCEDURE

```

(ii) Evaluate each expression in the following table.

If an expression is invalid then write 'ERROR'.

Refer to the **Appendix** on page 18 for the list of built-in functions and operators.

Expression	Evaluates to
RIGHT("Stop", 3) & LEFT("ich", 2)	
MID(NUM_TO_STRING(2019), 3, 1)	
INT(NUM_TO_STRING(-273.16))	
INT(13/2)	

[4]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

`STRING_TO_NUM(x : STRING)` RETURNS REAL
returns a numeric representation of a string.
Note: This function will also work if `x` is of type CHAR

Example: `STRING_TO_NUM("23.45")` returns 23.45

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

118. O/N 19/P22/Q4/a

- 4 A program is being written to control the operation of a portable music player. One part of the program controls the output volume.

The player has two buttons, one to increase the volume and one to decrease it. Whenever a button is pressed, a procedure `Button()` is called with a parameter value representing the button as follows:

Button	Parameter value
Volume increase	10
Volume decrease	20

For example, pressing the volume increase button three times followed by pressing the volume decrease button once would result in the calls:

```
CALL Button(10) // VolLevel increased by 1
CALL Button(10) // VolLevel increased by 1
CALL Button(10) // VolLevel increased by 1
CALL Button(20) // VolLevel decreased by 1
```

The program makes use of two global variables of type `INTEGER` as follows:

Variable	Description
<code>VolLevel</code>	The current volume setting. This must be in the range 0 to 49.
<code>MaxVol</code>	A value that can be set to limit the maximum value of <code>VolLevel</code> , in order to protect the user's hearing. A value in the range 1 to 49 indicates the volume limit. A value of zero indicates that no volume limit has been set.

The procedure `Button()` will modify the value of `VolLevel` depending on which button has been pressed and whether a maximum value has been set.

- (a) Write **pseudocode** for the procedure `Button()`. Declare any additional variables you use.

The value of `MaxVol` should **not** be changed within the procedure.

Parameter validation is **not** necessary.

[6]

119. O/N 19/P22/Q6/c

- 6 A text file, `StudentList.txt`, contains a list of information about students in a school.

Each line of the file contains a reference, name and date of birth for one student. All the information is held as strings and separated by the asterisk character (`' * '`) as follows:

```
<Reference>' * '<Name>' * '<Date Of Birth>
```

An example of one line from the file is:

```
"G1234*Aleza Hilton*05062001"
```

The reference string may be five or eight characters in length and is unique for each student. It is made up of alphabetic and numeric characters only.

A global 1D array, `Leavers`, contains the references of all students who have recently left the school. The array consists of 500 elements of data type `STRING`. Unused elements contain the empty string `""`.

A program is to be written to produce a new text file, `UpdatedList.txt`, containing information only for students who are still attending the school.

The program is to be implemented as several modules. The outline description of three of these is as follows:

Module	Outline description
<code>ProcessStudentList()</code>	- Read each line from the file <code>StudentList.txt</code> - Check whether the <code>Reference</code> appears in the array using <code>SearchLeavers()</code> - If the <code>Reference</code> does not appear then write the line to the file <code>UpdatedList.txt</code> - Return the number of lines not copied.
<code>SearchLeavers()</code>	- Search for a given <code>Reference</code> in the array <code>Leavers</code> - If the <code>Reference</code> is found, return <code>TRUE</code>, otherwise return <code>FALSE</code>
<code>CountTimes()</code>	- Take two parameters: the name of an array and a string. - Count the number of elements that are the same as the string. Return the count value.

- (c) `CountTimes()` is to be used to count how many unused elements there are in the `Leavers` array. An unused element is one that contains an empty string.

The module description is repeated here for reference.

Module	Outline description
<code>CountTimes()</code>	- Take two parameters: the name of an array and a string. - Count the number of elements that are the same as the string. Return the count value.

Write a statement in **program code** that uses `CountTimes()` to assign the count of unused elements to the variable `Result`.

Programming language

Program code

120. O/N 19/P23/Q1/a(ii)

- (ii) Identify the type of programming statement that assigns a data type to a variable.

[1]

121. O/N 19/P23/Q2/a(i)

- 2 (a) (i) Two types of loop that may be found in an algorithm are the 'pre-condition' and 'post-condition' loop.

Identify **one other** type of loop. Explain when it should be used.

Type

Explanation

[2]

122. O/N 19/P23/Q5

- 5 Abbreviations are often used in place of a full name. Concatenating the first letter of each word in the name makes an abbreviation.

For example:

Name	Abbreviation
United Nations	UN
World Wide Web	WWW
British Computer Society	BCS

A function, `Abbreviate()`, will take a string representing the full name and return a string containing the abbreviated form.

You should assume that:

- names only contain alphabetic characters and space characters
- names always start with an alphabetic character
- each word in the name always starts with an uppercase character
- only a single space separates words in the name.

Write **pseudocode** to implement the function `Abbreviate()`.

Refer to the **Appendix** on page 16 for the list of built-in functions and operators.

[8]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of `ThisChar`

Example: `ASC('A')` returns 65

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`

Example: `MOD(10, 3)` returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

123. M/J 19/P21/Q1/b(i)

(b) Program variables have values as follows:

Variable	Value
Title	"101 tricks with spaghetti"
Version	'C'
Author	"Eric Peapod"
PackSize	4
WeightEach	6.2
Paperback	TRUE

- (i) Evaluate each expression in the following table.
If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 16.

Expression	Evaluates to
MID(Title, 5, 3) & RIGHT(Author, 3)	
INT(WeightEach * PackSize)	
PackSize >= 4 AND WeightEach < 6.2	
LEFT(Author, ASC(Version) - 65)	
RIGHT(Title, (LENGTH(Author) - 6))	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL) RETURNS INTEGER`
 returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR) RETURNS INTEGER`
 returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER) RETURNS REAL`
 returns a real number in the range 0 to `x` (`x` not inclusive).

Example: `RAND(87)` could return 35.43

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

124. M/J 19/P21/Q2/a

- 2 (a) One type of loop that may be found in an algorithm is a count-controlled loop.

State **one other** type **and** explain when it should be used.

Type

Explanation

[2]

125. M/J 19/P21/Q3/b

- 3 (a)** A student is developing an algorithm to search through a 1D array of 100 elements. Each element of the array, `Result`, contains a `REAL` value.

The algorithm will output:

- the average value of all the elements
- the number of elements with a value of zero.

The structured English description of the algorithm is:

1. SET Total value to 0
2. SET Zero count to 0
3. SELECT the first element
4. ADD value of element to Total value
5. IF element value is 0 then INCREMENT Zero count
6. REPEAT from step 4 for next element, until element is last element
7. SET Average to Total / 100
8. OUTPUT a suitable message and Average
9. OUTPUT a suitable message and Zero count

Write **pseudocode** for this algorithm.

[7]

- (b)** The student decides to change the algorithm and implement it as a procedure, `ScanArray()`, which will be called with three parameters.

`ScanArray(AverageValue, ZeroCount, ArrayName)`

`ScanArray()` will modify the first two parameters so that the new values are available to the calling program or module.

Write the **pseudocode** procedure header for `ScanArray()`.

[4]

126. M/J 19/P21/Q5/a

- 5 (a)** Programming languages usually contain a range of built-in functions, such as a random number generator.

State **three** advantages of using built-in functions.

1

2

3

[3]

127. M/J 19/P21/Q6/a

- 6** A text file, `MyCDs.txt`, stores information relating to a Compact Disc (CD) collection. Information about each CD is stored on three separate lines in the file as follows:

```
Line 1:  <Artist Name>
Line 2:  <CD Title>
Line 3:  <Storage Location>
```

Information is stored as data strings.

A section of the file is shown:

File line	Data
100	"Green Floyd"
101	"Bowlful of Cereal"
102	"Shelf 4"
103	"Strolling Bones"
104	"Exile on Station Road"
105	"Box 12"

- (a)** A program, `CDOrganiser`, will be written to manage the stored information. The program will consist of three modules: `AddCD`, `FindCD` and `RemoveCD`.

Give **three** reasons why it is good practice to construct the program using modules.

- 1
- 2
- 3

[3]

128. M/J 19/P22/Q1/b(i)

- (b)** Program variables have values as follows:

Variable	Value
Married	03/04/1982
ID	"M1234"
MiddleInitial	'J'
Height	5.6
IsMarried	TRUE
Children	2

- (i)** Evaluate each expression in the following table.

If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 16.

Expression	Evaluates to
STRING_TO_NUM(RIGHT(ID, 3))	
INT(Height * Children)	
IsMarried AND Married < 31/12/1999	
LENGTH(ID & NUM_TO_STRING(Height))	
MID(ID, INT(Height) - Children, 2)	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length *y* starting at position *x* from *ThisString*

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of *ThisString*

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost *x* characters from *ThisString*

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost *x* characters from *ThisString*

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of *x*

Example: INT(27.5415) returns 27

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.

Example: NUM_TO_STRING(87.5) returns "87.5"

Note: This function will also work if *x* is of type INTEGER

STRING_TO_NUM(x : STRING) RETURNS REAL
returns a numeric representation of a string.

Example: STRING_TO_NUM("23.45") returns 23.45

Note: This function will also work if *x* is of type CHAR

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

129. M/J 19/P22/Q2/a

- 2 (a) (i) Procedures and functions are examples of subroutines.

State a reason for using subroutines in the construction of an algorithm.

[1]

- (ii) Give **three** advantages of using subroutines in a program.

1

2

3

[3]

- (iii) The following pseudocode uses the subroutine `DoSomething()`.

```
Answer ← 23 + DoSomething("Yellow")
```

State whether the subroutine is a function or a procedure. Justify your answer.

Type of subroutine

Justification

[2]

130. M/J 19/P22/Q6/a,b,c

- 6 Nadine is developing a program to store the ID and preferred name for each student in a school. For example, student Pradeep uses the preferred name "Prad".

The program will:

1. prompt and input a valid user ID and a preferred name
2. write the user ID and preferred name to one of two files
3. allow the user to end the program or repeat from step 1.

The program will consist of three separate modules. Each module will be implemented using either a procedure or a function.

Nadine has defined the modules as follows:

Module	Description
<code>TopLevel()</code>	• Call <code>GetInfo()</code> to obtain a string containing a valid user ID and a preferred name • Call <code>WriteInfo()</code> to write the string to either <code>File1.txt</code> or <code>File2.txt</code> depending on the first character of the user ID as follows: ◦ 'A' to 'M': writes to <code>File1.txt</code> ◦ 'N' to 'Z': writes to <code>File2.txt</code> For example, a string with a user ID of "G1234" writes to <code>File1.txt</code> • End the program if the file write was unsuccessful • Input (Y/N) to either repeat for the next user ID or to end the program
<code>GetInfo()</code>	• Input a user ID and repeat until the user ID is valid • Input a preferred name. This will be an empty string if no preferred name is input. • Concatenate the user ID and preferred name using a '*' character as a separator and return this string
<code>WriteInfo()</code>	• Open the file • Append the concatenated string to the file • Close the file • Return a Boolean value: ◦ <code>TRUE</code> if the file write was successful ◦ <code>FALSE</code> if the file write failed, for example, if the disk was full

A valid user ID:

- is five characters in length
- has a single upper case alphabetic character followed by four numeric characters, for example "G1234".

Nadine has decided that global variables and nested modules must not be used.

Nadine wants all inputs to have suitable prompts.

(a) Write **program code** for the module `GetInfo()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

(b) Write **program code** for the module `TopLevel()`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

(c) Write **pseudocode** for the module declaration of `WriteInfo()`.

[3]

131. M/J 19/P23/Q1/b

also Unit 10 (5 marks); this unit 5 marks

- 1 The following pseudocode searches for the longest run of identical characters in the array `Message`.

```
DECLARE Message : ARRAY[1:100] OF CHAR
```

```
PROCEDURE Search()
```

```
    DECLARE Index : INTEGER
    DECLARE ThisChar : CHAR
    DECLARE ThisRun : INTEGER
    DECLARE LongRun : INTEGER
```

```
    ThisChar ← Message[1]
    ThisRun ← 1
    LongRun ← 1
```

```
    FOR Index ← 2 TO 100
```

```
        IF Message[Index] = ThisChar
            THEN
                ThisRun ← ThisRun + 1
            ELSE
                ThisChar ← Message[Index]
                IF ThisRun > LongRun
                    THEN
                        LongRun ← ThisRun
                ENDIF
                ThisRun ← 1
            ENDIF
```

```
    ENDFOR
```

```
    OUTPUT "The longest run was " , LongRun
```

```
ENDPROCEDURE
```

(b) (i) Program variables have values as follows:

Variable	Value
MeltingPoint	180.5
Soluble	FALSE
Attempt	3
ProductName	"Mushroom Compost"
Version	'A'
ProductID	"BZ27-4"

Evaluate each expression in the following table.

If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 18.

Expression	Evaluates to
STRING_TO_NUM(MID(ProductID, 3, 2)) + 4	
INT(MeltingPoint / 2)	
Soluble AND Attempt > 3	
LENGTH(ProductID & NUM_TO_STRING(MeltingPoint))	
RIGHT(ProductName, 4) & MID(ProductName, 5, 4)	

[5]

(ii) Programming languages support different data types.

Give an appropriate data type for the following variables from **part (b)(i)**.

Variable	Data type
MeltingPoint	
Soluble	
Attempt	
Version	
ProductID	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.

Example: NUM_TO_STRING(x) returns "87.5" if x has the value 87.5

Note: This function will also work if x is of type INTEGER

STRING_TO_NUM(x : STRING) RETURNS REAL
returns a numeric representation of a string.

Example: STRING_TO_NUM(x) returns 23.45 if x has the value "23.45"

Note: This function will also work if x is of type CHAR

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

132. M/J 19/P23/Q6/c

- 6** A text file, `StudentContact.txt`, contains a list of names and telephone numbers of students in a school. Not all students in the school have provided a contact telephone number. In this case, their name will not be in the file.

Each line of the file is stored as a string that contains a name and telephone number, separated by the asterisk character (`'*'`) as follows:

`<Name>'*<TelNumber>`, for example:

`"Bill Smith*081234567"`

A 1D array, `ClassList`, contains the names of students in a particular class. The array consists of 40 elements of string data type. You can assume that student names are unique. Unused elements contain the empty string `""`.

A program is to be written to produce a **new** text file, `ClassContact.txt`, containing student names and numbers for all students in a particular class.

For each name contained in the `ClassList` array, the program will:

- search the `StudentContact.txt` file
- copy the matching string into `ClassContact.txt` if the name is found
- write the name together with `"*No number"` into `ClassContact.txt` if the name is not found.

The program will be implemented as three modules. The description of these is as follows:

Module	Description
<code>ProcessArray()</code>	• Check each element of the array: ○ Read the student name from the array ○ Ignore unused elements ○ Call <code>SearchFile()</code> with the student name ○ If the student name is found, call <code>AddToFile()</code> to write the student details to the class file ○ If the student name is not found, call <code>AddToFile()</code> to write a new string to the class file, formed as follows: <code><Name>"*No number"</code> • Return the number of students who have not provided a telephone number
<code>SearchFile()</code>	• Search for a given student name at the start of each line in the file <code>StudentContact.txt</code>: ○ If the search string is found, return the text line from <code>StudentContact.txt</code> ○ If the search string is not found, return an empty string
<code>AddToFile()</code>	• Append the given string to a specified file, for example, <code>AddToFile(StringName, FileName)</code>

(c) `ProcessArray()` is modified to make it general purpose. It will now be called with two parameters as follows:

- an array
- a string representing the name of a class contact file

It will still return the number of students who have not provided a contact telephone number.

Write **program code** for the header (declaration) of the modified `ProcessArray()`.

Programming language

Program code

[3]

133. O/N 18/P22/Q2/a(ii)

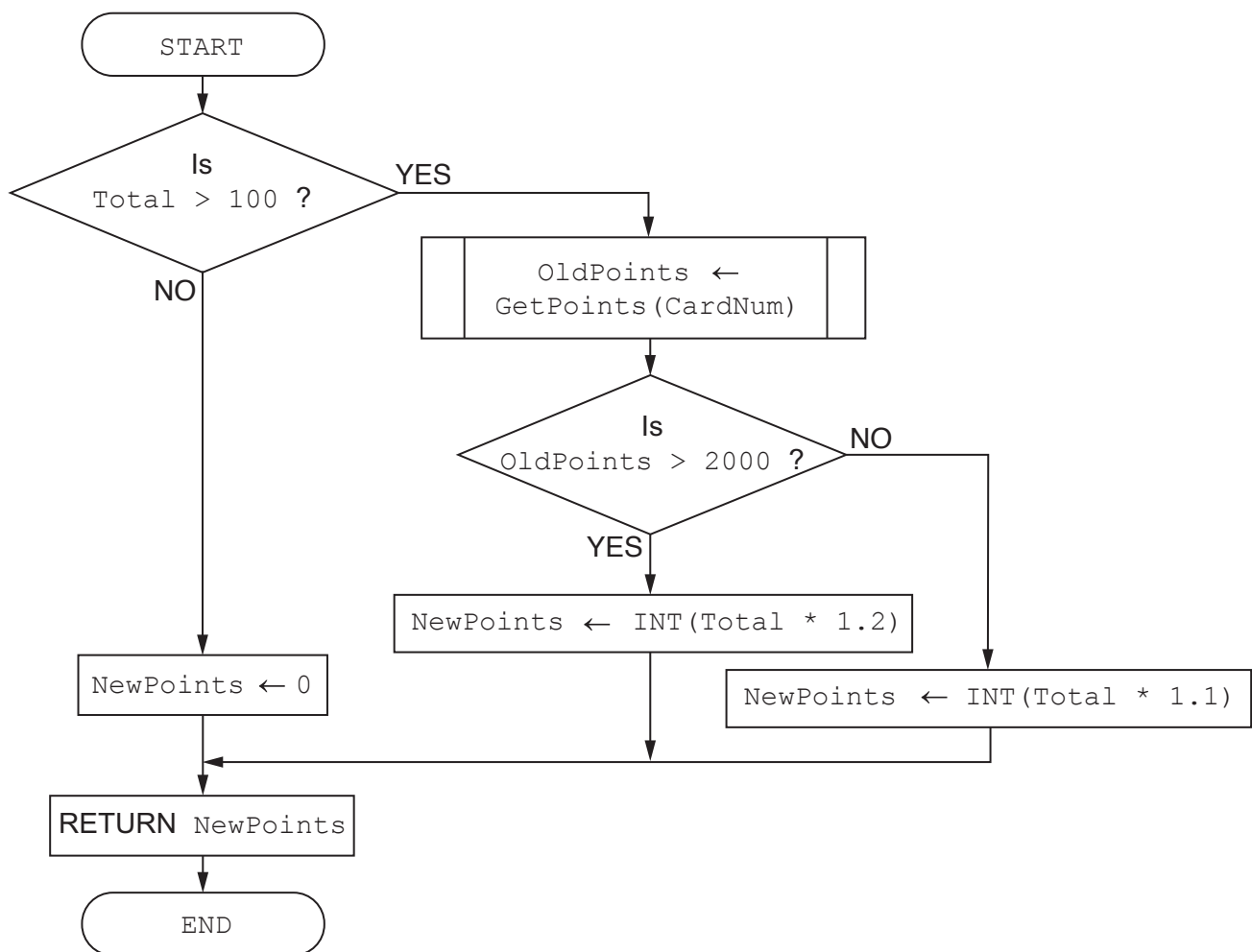
2 Shop customers have a discount card with a unique card number. Customers collect points each time they buy items. The number of points they collect depends on:

- the total amount they spend
- the number of points already collected.

The function `CalcPoints()` takes the card number and the total amount spent as parameters. It returns the number of new points collected. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
<code>CardNum</code>	STRING	A numeric string representing the unique card number
<code>OldPoints</code>	INTEGER	The number of points already collected
<code>NewPoints</code>	INTEGER	The number of new points collected
<code>Total</code>	REAL	The amount spent
<code>GetPoints()</code>	FUNCTION	Takes the card number as a parameter and returns the number of points already collected
<code>INT()</code>	FUNCTION	Refer to the Appendix on page 16



- (ii) The value of the total amount spent is calculated by an Electronic Point Of Sale (EPOS) system. This system does not have the prices of all items. For these items, a valid total amount has to be entered manually.

A function, `GetTotal()`, prompts the user to input this value.

If the user enters a valid value greater than 0 and less than 10000, the function returns the value. The function prompts the user to re-enter the value each time the user enters an invalid value.

Write **pseudocode** to complete the `GetTotal()` function.

```
FUNCTION GetTotal() RETURNS REAL
```

```
ENDFUNCTION
```

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.

Example: If `x` has the value 87.5 then `NUM_TO_STRING(x)` will return "87.5"

Note: This function will also work if `x` is of type integer.

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

134. O/N 18/P22/Q4/a(i),c

4 Part of a program written in pseudocode is shown.

```

010 DECLARE NextArrayElement : INTEGER
...
100 FUNCTION ScanFile(SearchString: STRING) RETURNS INTEGER
101
102     DECLARE FileData : STRING
103     DECLARE FileLine : INTEGER
104
105     NextArrayElement ← 1
106     FileLine ← 1
107     FileData ← ReadFileLine("DataFile.txt", FileLine)
108
109     WHILE FileData <> ""
110         IF LEFT(FileData, 7) = SearchString
111             THEN
112                 ResultArray[NextArrayElement] ← FileData
113                 NextArrayElement ← NextArrayElement + 1
114             ENDIF
115         FileLine ← FileLine + 1
116         FileData ← ReadFileLine("DataFile.txt", FileLine)
117     ENDWHILE
118
119     CALL ScanCompleted()
120     RETURN FileLine
121
122 ENDFUNCTION

```

(a) (i) Examine the pseudocode. Complete the following table.

Answer

The identifier name of a local variable	
The identifier name of a user-defined procedure	
The identifier name of a user-defined function	
The number of dimensions of ResultArray	
The scope of FileData	

[5]

(c) (i) State how structured programming languages support the implementation of sub-tasks.

[1]

(ii) State a benefit of using sub-tasks.

[1]

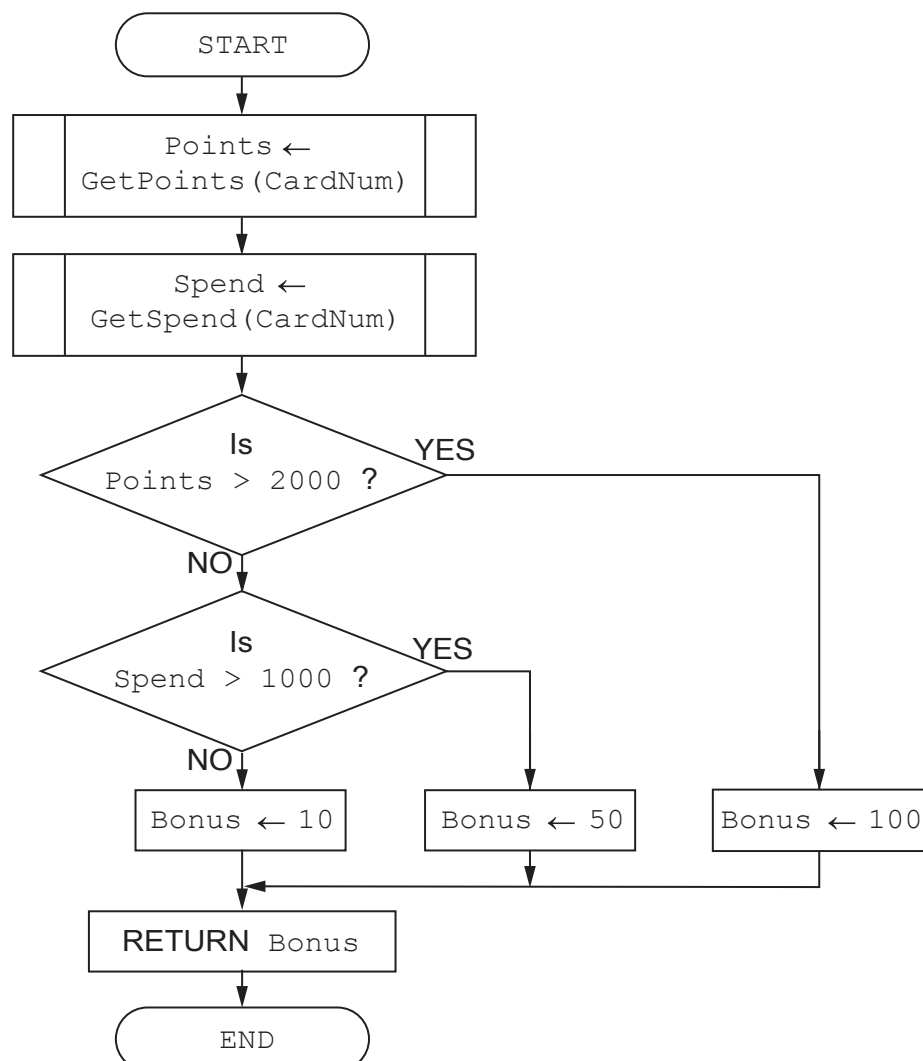
135. O/N 18/P23/Q2/a(ii)

- 2 Shop customers have a discount card with a unique card number. Customers collect points when they buy items. At the end of each year, customers are given bonus (extra) points related to the total amount they have spent during the year, and the number of points they have on their card.

The function `CalcBonus()` takes the card number as a parameter. It returns the bonus points for the customer. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
CardNum	STRING	A numeric string representing the unique card number
Points	INTEGER	The number of points collected
Spend	REAL	The total amount that customer has spent during the year
Bonus	INTEGER	The number of bonus points
GetPoints()	FUNCTION	Takes the card number as a parameter and returns the number of points already collected
GetSpend()	FUNCTION	Takes the card number as a parameter and returns the total amount that customer has spent during the year



- (ii) The function `GetCardNumber()` prompts the user to input a card number until the number input is valid.

A valid card number has 16 characters. Each character is a numeric character ('0' to '9').

Write **pseudocode** to complete the `GetCardNumber()` function.

You should refer to the function `IS_NUM()` in the **Appendix** on page 16.

```
FUNCTION GetCardNumber() RETURNS STRING  
  
ENDFUNCTION
```

[6]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

```
MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING  
returns a string of length y starting at position x from ThisString
```

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING  
returns leftmost x characters from ThisString
```

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

```
RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING  
returns rightmost x characters from ThisString
```

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

```
LENGTH(ThisString : STRING) RETURNS INTEGER  
returns the integer value representing the length of ThisString
```

Example: `LENGTH("Happy Days")` returns 10

```
INT(x : REAL) RETURNS INTEGER  
returns the integer part of x
```

Example: `INT(27.5415)` returns 27

```
ASC(ThisChar : CHAR) RETURNS INTEGER  
returns the ASCII value of ThisChar
```

Example: `ASC('A')` returns 65

IS_NUM(ThisString : STRING) RETURNS BOOLEAN
 returns the value TRUE if ThisString contains only numeric characters ('0' to '9').

Example: IS_NUM("1234X67") returns FALSE

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

136. O/N 18/P23/Q3/a

- 3 An array contains 100 integer values. An algorithm will find the maximum and minimum values stored in the array.

(a) A programmer has started to write this program using a conditional loop.

Name a more appropriate loop structure for this task **and** justify your choice.

Name

Justification

[2]

137. O/N 18/P23/Q4/a(i),b,c,d(ii)

- 4 Part of a program written in pseudocode is shown.

```
010 DECLARE ThisArray : ARRAY [1:100, 1:10] OF STRING
011 DECLARE LastElement : INTEGER
```

...

```

100 FUNCTION Update(NewData : STRING) RETURNS INTEGER
101     DECLARE ArrayIndex : INTEGER
102     DECLARE Found : BOOLEAN
103
104     ArrayIndex ← 1
105     Found ← FALSE
106
107     WHILE ArrayIndex ≤ LastElement AND Found = FALSE
108         IF ThisArray[ArrayIndex, 1] > NewData
109             THEN
110                 Found ← TRUE
111             ELSE
112                 ArrayIndex ← ArrayIndex + 1
113             ENDIF
114     ENDWHILE
115
116     IF Found = TRUE
117         THEN
118         CALL Insert(ArrayIndex, NewData)
119     ELSE
120         ArrayIndex ← 0
121     ENDIF
122
123     RETURN ArrayIndex
124 ENDFUNCTION

```

(a) (i) Examine the pseudocode **and** complete the following table.

Answer

The name of a global identifier	
The name of a user-defined procedure	
The scope of ArrayIndex	
The number of dimensions of ThisArray	
The scope of NewData	

[5]

(b) Line 118 of the function Update() calls the subroutine Insert(). A designer decides to convert Insert() from a procedure to a function. Insert() returns TRUE or FALSE.

A programmer must amend the function Update() as follows:

- If Insert() returns FALSE then the function Update() returns -1
- If Insert() returns TRUE then the function Update() returns the value as before.

Write **program code** to implement the **amended** Update() function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[8]

(c) The function Update() is an example of a module within a program.

Describe the mechanism that supports the transfer of values between modules.

[2]

- (ii) A programmer decides to declare `LastElement` as a constant instead of a variable.

Write a statement in **pseudocode** to declare `LastElement` as the value 200.

[1]

138. M/J 18/P21/Q1/b

also Unit 10 (5 marks); this unit 5 marks

- (b) (i) Program variables have values as follows:

Variable	Value
<code>MyGreeting</code>	"Happy Birthday"
<code>MyInitial</code>	'C'
<code>AgeInYears</code>	27
<code>Weight</code>	60.5
<code>Married</code>	TRUE
<code>Children</code>	TRUE

Evaluate each expression in the following table.

If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 15.

Expression	Evaluates to
"Mon" & MID(<code>MyGreeting</code> , 10, 2)	
<code>AgeInYears</code> + ASC(<code>MyInitial</code>)	
INT(<code>MyInitial</code>)	
MOD(<code>Weight</code> * 2, 10)	
<code>Married</code> AND (NOT <code>Children</code>)	

[5]

- (ii) Programming languages support different data types.

Give an appropriate data type for each of these variables from **part (b)(i)**.

Variable	Data type
<code>MyGreeting</code>	
<code>MyInitial</code>	
<code>AgeInYears</code>	
<code>Weight</code>	
<code>Married</code>	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`

If `ThisChar` is not an upper-case alphabetic character then it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`

Example: `MOD(10, 3)` returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

139. M/J 18/P21/Q2/b

2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```

01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03      DECLARE NextChar : CHAR
04      DECLARE OutString : STRING
05      DECLARE Counter : INTEGER
06      DECLARE MyString : STRING
07
08      OutString ← ""
09
10      FOR Counter ← 1 TO LENGTH(InString)
11
12          NextChar ← MID(InString,Counter,1)
13          NextChar ← LCASE(NextChar)
14
15          IF (NextChar >= 'a') AND (NextChar <= 'z')
16
17              THEN
18
19                  OutString ← OutString & NextChar
20
21          ENDIF
22
23      ENDFOR
24
25      RETURN OutString
26
27  ENDFUNCTION

```

(b) Study the function `StringClean()`. Identify the features of the function in the following table.

Feature	Answer
A line number containing an example of an assignment statement	
A line number containing the start of a repetition block	
A line number containing the end of a repetition block	
A line number containing the start of a selection statement	
The number of parameters of the MID function	
The Boolean operator used	
The number of local variables	
The number of function calls from within <code>StringClean()</code> resulting from the call: <code>NewString ← StringClean("Me")</code>	
The number of a line containing an unnecessary statement	

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of string ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar

If ThisChar is not an upper-case alphabetic character then it is returned unchanged.

Example: LCASE('W') returns 'w'

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: MOD(10, 3) returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

140. M/J 18/P21/Q7

- 7 A function, `ProcessMarks()`, is required to analyse test marks for a class of students.
- There are 20 students in the class.
 - A mark is between 0 and 100.
 - The marks for the class are stored in an array, `Mark`, which has 20 elements.
 - The array is passed to the function as a parameter.
 - The function will output a message stating the average and highest marks. For example:
"The average mark is 34 and the highest mark is 76"
 - The function returns the subscript of the highest mark.

Write **program code** to implement the `ProcessMarks()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[7]

141. M/J 18/P22/Q1/b

also Unit 10 (5 marks); this unit 5 marks

- (b) (i) Program variables have values as follows:

Variable	Value
Mark	60
Subject	"Computer Science"
Grade	'B'
CourseCompleted	TRUE
AverageMark	49.5

Evaluate each expression in the following table.

If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 16.

Expression	Evaluates to
"Fas" & MID(Subject, 6, 3)	
LEFT(Mark, 1)	
10 + ASC(Grade)	
MOD(AverageMark * 2, 3)	
CourseCompleted AND (Mark >= 60)	

[5]

- (ii) Programming languages support different data types.

Give an appropriate data type for each of these variables from **part (b)(i)**.

Variable	Data type
Mark	
Subject	
Grade	
CourseCompleted	
AverageMark	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`

Example: `MOD(10, 3)` returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

142. M/J 18/P22/Q2/b,c(ii)

2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```

01  FUNCTION CountDigits(InString : STRING) RETURNS CHAR
02
03  DECLARE nc : CHAR
04  DECLARE c : INTEGER
05  DECLARE n : INTEGER
06
07
08  c ← 0
09  n ← LENGTH(InString)    // get number of characters for loop
10
11  WHILE n > 0              // repeat until no more characters left
12
13  nc ← LEFT(InString,1)
14  n ← n - 1
15  InString ← RIGHT(InString,n) // remove first character
16
17  IF (nc < '0') OR (nc > '9')
18  THEN
19  // do nothing
20  ELSE
21  c ← c + 1
22  ENDIF
23
24  ENDWHILE
25
26  RETURN c
27
28  ENDFUNCTION

```

(b) Study the function `CountDigits()`. Identify the features of the function in the following table.

Feature	Answer
A line number containing an example of an assignment statement	
A line number containing the start of a 'pre-condition' loop	
A line number containing the end of a 'pre-condition' loop	
A line number containing the start of a selection statement	
The number of parameters passed to the <code>LEFT()</code> function	
The Boolean operator used	
The number of times the function <code>LEFT()</code> is called from within <code>CountDigits()</code> resulting from the call: <code>Result ← CountDigits("AB27C4")</code>	
The number of local variables	

[8]

(ii) Lines 17 to 22 of the pseudocode could be written in a simplified form.

Re-write the lines in a simplified form.

[4]

143. M/J 18/P22/Q5/c

- 5 A golf club holds information about its members. When a member completes a round of golf, their score is stored along with their membership number and the date of the round.

A program is to be written to store and process the score information. The information to be stored is formed into a string as follows:

`<MembershipNumber><Date><Score>`

- (c) The program needs a function, `GetNumber()`, to return a valid membership number. A valid membership number is a four-digit numeric string between “1111” and “9999”.

The structured English representing the algorithm for this function is as follows:

1. Prompt and input a membership number.
2. Validate the membership number.
3. Repeat from step 1 if the membership number is invalid.
4. Return the valid membership number as a string.

An example of the function call in pseudocode is:

`MembershipNumber ← GetNumber()`

Write **program code** for the `GetNumber()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[4]

144. M/J 18/P23/Q1/b

also Unit 10 (5 marks); this unit 5 marks

- (b) (i) Program variables have values as follows:

Variable	Value
Quality	'D'
DayNumber	20
MyName	"Stephen"
QualityConfirmed	TRUE
Factor	6.5

Evaluate each expression in the following table.

If an expression is invalid, write ERROR.

For the built-in functions list, refer to the **Appendix** on page 15.

Expression	Evaluates to
<code>MID(MyName, 4, 4) & "ol"</code>	
<code>QualityConfirmed AND (Factor >= 6.5)</code>	
<code>20 + ASC(Quality)</code>	
<code>QualityConfirmed + 3</code>	
<code>MOD(Factor * 2, 9)</code>	

[5]

(ii) Programming languages support different data types.

Give an appropriate data type for each of these variables from **part (b)(i)**.

Variable	Data type
QualityConfirmed	
DayNumber	
Factor	
Quality	
MyName	

[5]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER)` RETURNS INTEGER
returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`

Example: `MOD(10, 3)` returns 1

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND of two Boolean values. Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR of two Boolean values. Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

145. M/J 18/P23/Q2/b,c(ii)

- 2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```
01  FUNCTION GradeCalc(Mark : ARRAY[] OF INTEGER) RETURNS INTEGER
02
03      DECLARE DGradeCount : INTEGER
04      DECLARE n : INTEGER
05      DECLARE Grade : STRING
06      DECLARE ThisMark : INTEGER
07
08      DGradeCount ← 0      // initialise variables
09      n ← 1
10      ThisMark ← 1
11      REPEAT
12          ThisMark ← Mark[n]      // get next mark from Mark array
13
14          IF ThisMark > 74
15              THEN
16                  Grade ← "Distinction"
17                  DGradeCount ← DGradeCount + 1 // one more Distinction
18              ELSE
19                  IF ThisMark > 59
20                      THEN
21                          Grade ← "Merit"
22                      ELSE
23                          IF ThisMark > 39
24                              THEN
25                                  Grade ← "Pass"
26                              ELSE
27                                  Grade ← "Fail"
28                              ENDIF
29                          ENDIF
30          ENDIF
31
32          OUTPUT "Grade for Student " & n & " is " & Grade
33
34          n ← n + 1
35      UNTIL n = 101 // no more elements in the array
36
37      RETURN Grade // return the number of Distinction grades
38
39  ENDFUNCTION
40
41  DECLARE Mark : ARRAY[1:100] OF INTEGER
42  DECLARE DistinctionGrades : INTEGER
43
44  CALL GetMarks(Mark[]) // fill the array with student marks
45  DistinctionGrades ← GradeCalc(Mark[])
```


(b) Study the function `GradeCalc()`. Identify the features of the function in the following table.

Feature	Answer
A line number containing an example of an integer assignment statement	
A line number containing the start of a selection structure	
A line number containing the end of a selection structure	
The upper bound of the <code>Mark</code> array	
The number of dimensions of the <code>Mark</code> array	
The name for the type of loop structure used	
A line number containing an unnecessary assignment statement	
The number of times <code>OUTPUT</code> is called	
The number of local variables	

[9]

(ii) Lines 14 to 30 of the pseudocode on page 4 could be written using a `CASE` structure.

Write the equivalent `CASE` structure in **pseudocode**.

[4]

146. M/J 18/P23/Q3/a

3 A modular program design consists of four modules:

`Module1` has three sub-tasks. Each sub-task is implemented by a single subroutine (either a function or a procedure).

The subroutine headings are defined as follows:

```
FUNCTION Module2 (Weight : REAL) RETURNS BOOLEAN
PROCEDURE Module3 (Weight : REAL, Customer : STRING, Purchased : DATE)
FUNCTION Module4 (Purchased : DATE, Account : INTEGER) RETURNS INTEGER
```

(a) State the term given to values passed between modules.

[1]

147. M/J 18/P23/Q7

- 7 A function, `IsFactor()` takes two integer parameters and checks whether the second parameter is a factor of the first parameter.

For example `IsFactor(6, 3)` would return `TRUE` as six divided by three has a remainder of zero.

This function returns `FALSE` if there is a non-zero remainder, or if the value of the second parameter is zero.

Write **program code** to implement the `IsFactor()` function.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

[6]

148. O/N 17/P22/Q1/c,d

- (ii) The following is a declaration in a high-level language:

```
DEFINE MyGrade[1 to 100]
```

State the data structure of variable `MyGrade`.

[1]

- (c) Functions and procedures are subroutines.

Explain why parameters are used with subroutines.

[3]

- (d) The following pseudocode is an example of a `CASE` structure.

```
CASE OF MyMark
  75 to 100: MyGrade ← "Distinction"
  35 to 74: MyGrade ← "Pass"
  0 to 34: MyGrade ← "Fail"
  OTHERWISE: OUTPUT "Invalid value entered"
ENDCASE
```

- (i) Describe what will happen if the pseudocode is tested when `MyMark` has the following values:

27

101

[2]

- (ii) Use **pseudocode** to write an `IF` statement with the same functionality.

[5]

149. M/J 17/P21/Q1/b(ii,iii),c

- (ii) State the name given to the type of operators to which **AND** and **OR** belong.

[1]

- (iii) Evaluate the expressions given in the following table when the variable values are as follows:

FlagA ← TRUE
 FlagB ← FALSE
 FlagC ← TRUE

Expression	Evaluates to
FlagA AND (FlagB OR FlagC)	
FlagA AND (FlagB AND FlagC)	
(NOT FlagA) OR (NOT FlagC)	

[3]

- (c) A common construct found in many algorithms is a loop.

Using **pseudocode**, write a pre-condition loop to output all of the even numbers between 99 and 201.

[4]

150. M/J 17/P21/Q3

- 3** A string conversion function, **StringClean**, is to be written.

This function will form a new string, **OutString**, from a given string, **InString**, by:

- removing all non-alphabetic characters
- converting all alphabetic characters to lower case.

For example:

```
InString = "Good Morning, Dave"
OutString = "goodmorningdave"
```

The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode using relevant built-in functions.

For the built-in functions list, refer to the **Appendix** on page 14.

```
FUNCTION StringClean(.....) RETURNS .....
    DECLARE NextChar : .....
    DECLARE ..... : STRING
    ..... //initialise the return string
```

```

//loop through InString to produce OutString

FOR n ← 1 TO ..... //from first to last
    NextChar ← ..... //get next character and
    NextChar ← ..... //convert to lower case
    IF ..... //check if alphabetic
        THEN
            ..... //add to OutString
        ENDIF
    ENDFOR
    .....//return value
ENDFUNCTION

```

[11]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns string of length y starting at position x from ThisString.

Example: **MID** ("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of string ThisString.

Example: **LENGTH** ("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString.

Example: **LEFT** ("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString.

Example: **RIGHT** ("ABCDEFGH", 3) returns string "FGH"

LCASE(ThisChar : CHAR) RETURNS CHAR

returns the character value representing the lower case equivalent of ThisChar.

If ThisChar is not an upper-case alphabetic character then it is returned unchanged.

Example: **LCASE** ('W') returns 'w'

```
MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
```

returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`.

Example: `MOD(10, 3)` returns 1

```
DIV(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
```

returns the integer value representing the whole number part of the result when `ThisNum` is divided by `ThisDiv`.

Example: `DIV(10, 3)` returns 3

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND of two Boolean values. Example: <code>TRUE AND FALSE</code> produces FALSE
<code>OR</code>	Performs a logical OR of two Boolean values. Example: <code>TRUE OR FALSE</code> produces TRUE

151. M/J 17/P21/Q6/a

- 6 A multi-user computer system makes use of passwords.

To be valid, a password must comply with the following rules:

- at least two lower-case alphabetic characters
- at least two upper-case alphabetic characters
- at least three numeric characters
- alpha-numeric characters only

A function, `ValidatePassword`, is needed to check that a given password follows these rules. This function takes a string, `Pass`, as a parameter and returns a Boolean value:

- `TRUE` if `Pass` contains a valid password
- `FALSE` otherwise.

- (a) Write **program code** to implement the new function `ValidatePassword`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

152. M/J 17/P22/Q1/b,c

(b) (i) Complete the following two sentences.

A suitable operand type for an arithmetic operator is

A suitable operand type for a logical operator is

[2]

(ii) The following table shows the values of three variables.

Variable	Value
FlagA	TRUE
FlagB	FALSE
FlagC	TRUE

Evaluate these expressions.

Expression	Evaluates to
(FlagA AND FlagB) OR FlagC	
FlagA AND (FlagB OR FlagC)	
(NOT FlagA) OR (NOT FlagC)	

[3]

(c) The loop construct (also known as repetition or iteration) appears in many algorithms.

Use **pseudocode** to write a post-condition loop to output all the odd numbers between 100 and 200.

[4]

153. M/J 17/P22/Q3

3 A string conversion function, `ExCamel`, needs to be written.

This function forms a return string, `OutString`, from a given string, `InString`, by:

- 1 separating the original words (a word is assumed to start with a capital letter)
- 2 converting all characters to lower case.

The following shows a pair of example values for the string values `InString` and `OutString`.

```
InString : "MyUserInput"
OutString : "my user input"
```

You may assume that `InString` always starts with a capital letter.

The following is a first attempt at writing the pseudocode for this function.

Complete the **pseudocode** using appropriate built-in functions.

For the built-in functions list, refer to the **Appendix** on page 13.

```

FUNCTION ExCamel (.....) RETURNS .....

    DECLARE NextChar : .....

    DECLARE ..... : STRING

    DECLARE n: INTEGER

    ..... // initialise the return string
    // loop through InString to produce OutString
    FOR n ← 1 TO ..... // from first to last
        NextChar ← ..... // get next character
        IF ..... // check if upper case
            THEN
                IF n > 1 // if not first character
                    THEN
                        ..... // add space to OutString
                    ENDIF
                ..... // make NextChar lower case
            ENDIF
            ..... // add NextChar to OutString
        ENDFOR
        ..... // return value

    ENDFUNCTION

```

[11]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

```
RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`.

Example: **RIGHT("ABCDEFGH", 3)** returns string **"FGH"**

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of string `ThisString`.

Example: **LENGTH("Happy Days")** returns **10**

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns string of length `y` starting at position `x` from `ThisString`.

Example: `MID("ABCDEFGH", 2, 3)` returns string `"BCD"`

`LCASE(ThisChar : CHAR) RETURNS CHAR`

returns the character value representing the lower case equivalent of `ThisChar`.

If `ThisChar` is not an upper case alphabetic character then it is returned unchanged.

Example: `LCASE('W')` returns `'w'`

`UCASE(ThisChar : CHAR) RETURNS CHAR`

returns the character value representing the upper case equivalent of `ThisChar`.

If `ThisChar` is not a lower case alphabetic character then it is returned unchanged.

Example: `UCASE('h')` returns `'H'`

`MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER`

returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`.

Example: `MOD(10,3)` returns `1`

`DIV(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER`

returns the integer value representing the whole number part of the result when `ThisNum` is divided by `ThisDiv`.

Example: `DIV(10,3)` returns `3`

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
AND	Performs a logical AND of two Boolean values Example: <code>TRUE AND FALSE</code> produces FALSE
OR	Performs a logical OR of two Boolean values Example: <code>TRUE OR FALSE</code> produces TRUE

154. M/J 17/P22/Q4/a,b

- 4 (a) High-level programming languages have many features that support the modular approach. One such feature is the use of parameters.

State **two** other features.

1

2

[2]

- (b) Consider the following pseudocode.

```
PROCEDURE MyProc(x)
    x ← x + 1
ENDPROCEDURE
```

Intermediate lines of pseudocode not shown

```
x ← 4
CALL MyProc(x)
OUTPUT(x)
```

Parameter x is used to pass data to procedure `MyProc`.
There are two parameter passing methods that could be used.

Complete the following table for each of the two methods.

Name of parameter passing method	Value output	Explanation

[6]

155. M/J 17/P22/Q6/a

- 6** A computerised vehicle licensing system stores details about vehicles and their registration marks (number plates or license plates).

To be valid, a vehicle registration must comply with the following rules:

- It must be between six and nine characters long.
- Characters 1 to 3 are upper case alphabetic characters.
- Characters 4 to 5 are numeric characters.
- Remaining characters are upper case alphabetic.

A function, `ValidateRegistration` is needed to check that a given registration mark follows these rules. This function takes a string, `Registration` as a parameter and returns a Boolean value:

- `TRUE` if it is a valid registration
- `FALSE` otherwise.

(a) Write **program code** to implement the new function, `ValidateRegistration`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

156. M/J 17/P23/Q1/b(ii,iii),c

- (ii) State the name given to the type of operators to which **AND** and **OR** belong.

[1]

- (iii) Evaluate the expressions given in the following table when the variable values are as follows:

FlagA ← TRUE
 FlagB ← FALSE
 FlagC ← TRUE

Expression	Evaluates to
FlagA AND (FlagB OR FlagC)	
FlagA AND (FlagB AND FlagC)	
(NOT FlagA) OR (NOT FlagC)	

[3]

- (c) A common construct found in many algorithms is a loop.

Using **pseudocode**, write a pre-condition loop to output all of the even numbers between 99 and 201.

[4]

157. M/J 17/P23/Q3

- 3** A string conversion function, **StringClean**, is to be written.

This function will form a new string, **OutString**, from a given string, **InString**, by:

- removing all non-alphabetic characters
- converting all alphabetic characters to lower case.

For example:

```
InString = "Good Morning, Dave"
OutString = "goodmorningdave"
```

The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode using relevant built-in functions.

For the built-in functions list, refer to the **Appendix** on page 14.

```
FUNCTION StringClean(.....) RETURNS .....
    DECLARE NextChar : .....
    DECLARE ..... : STRING
    ..... //initialise the return string
```

```

//loop through InString to produce OutString

FOR n ← 1 TO ..... //from first to last

    NextChar ← ..... //get next character and

    NextChar ← ..... //convert to lower case

    IF ..... //check if alphabetic

        THEN

            ..... //add to OutString

        ENDIF

    ENDFOR

    .....//return value

ENDFUNCTION

```

[11]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns string of length y starting at position x from ThisString.

Example: **MID** ("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of string ThisString.

Example: **LENGTH** ("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString.

Example: **LEFT** ("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString.

Example: **RIGHT** ("ABCDEFGH", 3) returns string "FGH"

LCASE(ThisChar : CHAR) RETURNS CHAR

returns the character value representing the lower case equivalent of ThisChar.

If ThisChar is not an upper-case alphabetic character then it is returned unchanged.

Example: **LCASE** ('W') returns 'w'

```
MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
```

returns the integer value representing the remainder when `ThisNum` is divided by `ThisDiv`.

Example: `MOD(10, 3)` returns 1

```
DIV(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
```

returns the integer value representing the whole number part of the result when `ThisNum` is divided by `ThisDiv`.

Example: `DIV(10, 3)` returns 3

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND of two Boolean values. Example: <code>TRUE AND FALSE</code> produces FALSE
<code>OR</code>	Performs a logical OR of two Boolean values. Example: <code>TRUE OR FALSE</code> produces TRUE

158. M/J 17/P23/Q6/a

- 6 A multi-user computer system makes use of passwords.

To be valid, a password must comply with the following rules:

- at least two lower-case alphabetic characters
- at least two upper-case alphabetic characters
- at least three numeric characters
- alpha-numeric characters only

A function, `ValidatePassword`, is needed to check that a given password follows these rules. This function takes a string, `Pass`, as a parameter and returns a Boolean value:

- `TRUE` if `Pass` contains a valid password
- `FALSE` otherwise.

- (a) Write **program code** to implement the new function `ValidatePassword`.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

Program code

159. O/N 16/P21/Q3/a,b,c,d

3 You will need to refer to the list of pseudocode string-handling functions in the **Appendix**.

ASCII code table (part)					
Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

(a) For each statement, write the value assigned to the variable.

(i) `Term ← CHARACTERCOUNT("TSUNAMI")`

Term [1]

(ii) `Answer1 ← ASC('G') + ASC(<Space>)`

Answer1 [1]

(iii) `Answer2 ← CHR(CHARACTERCOUNT("HELLO") + 70)`

Answer2 [1]

(iv) `Word ← SUBSTR("Welcome home", 4, 7) )`

Word [1]

(b) A programmer wants to design a procedure to calculate a customer ID number from the customer's surname.

The procedure will:

- input the surname
- isolate each character in the surname and find the corresponding ASCII code
- calculate the total of all these ASCII codes
- this total is the customer ID

(i) Complete the pseudocode for this procedure.

You will need to refer to the list of pseudocode string-handling functions in the Appendix.

```

PROCEDURE CalculateCustomerID
    OUTPUT "Key in surname"
    INPUT Surname
    Length ← .....
    CustomerID ← 0
    FOR i ← 1 TO Length
        // NextChar is a single character from Surname
        NextChar ← .....
        NextCodeNumber ← ASC(NextChar)
        CustomerID ← CustomerID + .....
    ENDFOR
    OUTPUT "Customer ID is ", CustomerID

```

[3]

- (ii) Write **program code** for procedure `CalculateCustomerID`.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[6]

- (c) The programmer decides that it would be better to write the procedure as a function.
The user will now input the surname in the main program.

Write **program code** for the following:

State your programming language

- (i) The function header for this new function `CalculateCustomerID`

[3]

- (ii) The additional statement required within the function body to complete the change from a procedure to a function.

[1]

- (iii) The statement in the main program which:

- calls the function for surname Wilkes
- assigns the result to variable `ThisID`

[3]

- (d) (i) The new function `CalculateUserID` is an example of a 'user-defined function'.

State **two** differences between a built-in function and a user-defined function.

2

[2]

(ii) State **two** things that built-in and user-defined functions have in common.

1

2

[2]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR

returns the single character at position *Position* (counting from the start of the string with value 1) from the string *ThisString*.

For example: ONECHAR("New York", 5) returns 'Y'

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER

returns the number of characters in *ThisString*.

For example: CHARACTERCOUNT("New York") returns 8

SUBSTR(ThisString : STRING, Value1 : INTEGER, Value2 : INTEGER) RETURNS STRING

returns a sub-string from within *ThisString*.

Value1 is the start index position (counting from the left, starting with 1).

Value2 is the final index position.

For example: SUBSTR("art nouveau", 5, 11) returns "nouveau"

TONUM(ThisString : STRING) RETURNS INTEGER or REAL

returns the integer or real equivalent of the string *ThisString*.

For example: TONUM("502") returns the integer 502

TONUM("56.36") returns the real number 56.36

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`

returns an integer which is the ASCII character code for the character `ThisCharacter`.

For example: `ASC('A')` returns integer 65

`CHR(Value : INTEGER) RETURNS CHAR`

returns the character that ASCII code number `Value` represents.

For example: `CHR(65)` returns 'A'

`RND() RETURNS REAL`

returns a random number in the range 0 to 0.99999

For example: `RND()` returns 0.67351

`INT(ThisNumber : REAL) RETURNS INTEGER`

returns the integer part of `ThisNumber`.

For example: `INT(12.79)` returns 12

Errors

For any function, if the program calls the function incorrectly, the function returns an error.

Concatenation operator

`&` – Concatenates two expressions of `STRING` or `CHAR` data type.

For example: `"South" & " " & "Pole"` produces `"South Pole"`
`'B' & "000654"` produces `"B000654"`

160. O/N 16/P21/Q5

5 Study the following pseudocode statements.

```

CONST  Pi = 3.1           : REAL

DECLARE Triangle, Base, Height, Radius, Cone : REAL

DECLARE a, b, c, Answer2 : INTEGER

DECLARE Answer1           : BOOLEAN

Base ← 2.6

Height ← 10

Triangle ← (Base * Height) / 2

Radius ← 1

Height ← 2

Cone ← 2 * Pi * Radius * (Radius + Height)

a ← 13

b ← 7

c ← 3

Answer1 ← NOT((a + b + c) > 28)

Total ← 34

Total ← Total - 2

Answer2 ← a + c * c

```

Give the final value assigned to each variable.

- | | |
|---------------------|-----|
| (i) Triangle | [1] |
| (ii) Cone | [1] |
| (iii) Answer1 | [1] |
| (iv) Total | [1] |
| (v) Answer2 | [1] |

161. O/N 16/P22/Q2/a,b,c,d

also Unit 12 (7 marks); this unit 8 marks

2 You will need to refer to the list of pseudocode string-handling functions in the **Appendix**.

(a) Give the value of the variables *x*, *y* and *z* for the following sequence of statements.

- | | |
|--|--------------------|
| (i) <i>x</i> ← ONECHAR("Barcelona", 5) | <i>x</i> [1] |
| (ii) <i>y</i> ← ONECHAR("Cool", 1) & ONECHAR("Ball", 2) & "t-food" | <i>y</i> [1] |
| (iii) Temp1 ← "13" | |
| Temp2 ← ONECHAR("One-2-One", 5) | |
| <i>z</i> ← TONUM(Temp2 & Temp1) | <i>z</i> [1] |

A computer program is to simulate the reading and processing of a string of characters from an input device.

The character string consists of:

- a number of digit characters
- one or more <*> characters, each used as a separator
- a final <#> character.

A typical input character sequence, stored as `InputString` is:

13*156*9*86*1463*18*#

Study this pseudocode.

```

01 DECLARE Numbers ARRAY [1:100] OF INTEGER
02 DECLARE InputString      : STRING
03 DECLARE NextChar        : CHAR
04 DECLARE NextNumberString : STRING
05 DECLARE i                : INTEGER    // Numbers array index
06 DECLARE j                : INTEGER    // InputString index
07
08 OUTPUT "String ... "
09 INPUT InputString
10 j ← 1
11 NextChar ← ONECHAR(InputString, j)
12
13 i ← 1
14 WHILE NextChar <> '#'
15     NextNumberString = ""
16     WHILE NextChar <> '*'
17         NextNumberString ← NextNumberString & NextChar
18         j ← j + 1
19         NextChar ← ONECHAR(InputString, j)
20     ENDWHILE
21
22     // store the next integer to the array
23     Numbers[i] ← TONUM(NextNumberString)
24     i ← i + 1
25     j ← j + 1
26     NextChar ← ONECHAR(InputString, j)
27 ENDWHILE
28
29 CALL DisplayArray()
```

(b) Write the line number for:

- (i)** A statement which declares a global variable used to store a single character.

[1]

- (ii) A statement which runs code written as a procedure. [1]
 - (iii) A statement which indicates the start of a 'pre-condition' loop. [1]
 - (iv) A statement which increments a variable. [1]
- (c) Copy the condition which is used to control the inner loop. [1]
- (d) (i) Complete the trace table below for the given pseudocode as far as line 27.

Built-in functions (pseudocode)

`ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR`

returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.

For example: `ONECHAR("New York", 5)` returns 'Y'

`CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER`

returns the number of characters in `ThisString`.

For example: `CHARACTERCOUNT("New York")` returns 8

`SUBSTR(ThisString : STRING, Value1 : INTEGER, Value2 : INTEGER) RETURNS STRING`

returns a sub-string from within `ThisString`.

`Value1` is the start index position (counting from the left, starting with 1).

`Value2` is the final index position.

For example: `SUBSTR("art nouveau", 5, 11)` returns "nouveau"

`TONUM(ThisString : STRING) RETURNS INTEGER or REAL`

returns the integer or real equivalent of the string `ThisString`.

For example: `TONUM("502")` returns the integer 502

`TONUM("56.36")` returns the real number 56.36

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`

returns an integer which is the ASCII character code for the character `ThisCharacter`.

For example: `ASC('A')` returns integer 65

`CHR(Value : INTEGER) RETURNS CHAR`

returns the character that ASCII code number `Value` represents.

For example: `CHR(65)` returns 'A'

`RND() RETURNS REAL`

returns a random number in the range 0 to 0.99999

For example: `RND()` returns 0.67351

`INT(ThisNumber : REAL)` RETURNS INTEGER

returns the integer part of `ThisNumber`.

For example: `INT(12.79)` returns 12

Errors

For any function, if the program calls the function incorrectly, the function returns an error.

Concatenation operator

`&` operator – Concatenates two expressions of `STRING` or `CHAR` data type.

For example: `"South" & " " & "Pole"` produces `"South Pole"`
`'B' & "000654"` produces `"B000654"`

162. O/N 16/P22/Q4/a,b,c

- 4 A social club runs a weekly prize draw. A member can buy a ticket for \$2. Each week, the club sells up to 150 tickets, with consecutive numbers starting from 1. There is a main cash prize of \$120 and a further three prizes of \$10. The computer program will generate the winning numbers.

Throughout this question, you will need to refer to the list of pseudocode functions in the **Appendix**.

- (a) Write **pseudocode** to show how the `RND()` function can be used to generate a single integer in the range 1 to 150.

[3]

- (b) Write **program code** to generate and output four winning numbers.
 Ignore the issue that duplicate numbers may be generated.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

[4]

- (c) The prize draw is a success. After six months, the club decided to sell an unlimited number of tickets each week. In any week, the total number of prizes may vary.

The programmer modifies the code written in **part(b)**.

The revised program will use a function to generate a single winning number. Therefore, in a week when the club offers six prizes, the program will use the function six times to generate the six winning numbers.

The function has identifier `GenerateNumber` and:

- has a single parameter for the number of tickets sold that week
- returns a single winning number.

Write **program code** for the function header.

Programming language

[3]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

`ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR`

returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.

For example: `ONECHAR("New York", 5)` returns 'Y'

`CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER`

returns the number of characters in `ThisString`.

For example: `CHARACTERCOUNT("New York")` returns 8

`SUBSTR(ThisString : STRING, Value1 : INTEGER, Value2 : INTEGER) RETURNS STRING`

returns a sub-string from within `ThisString`.

`Value1` is the start index position (counting from the left, starting with 1).

`Value2` is the final index position.

For example: `SUBSTR("art nouveau", 5, 11)` returns "nouveau"

`TONUM(ThisString : STRING) RETURNS INTEGER or REAL`

returns the integer or real equivalent of the string `ThisString`.

For example: `TONUM("502")` returns the integer 502

`TONUM("56.36")` returns the real number 56.36

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`

returns an integer which is the ASCII character code for the character `ThisCharacter`.

For example: `ASC('A')` returns integer 65

`CHR(Value : INTEGER) RETURNS CHAR`

returns the character that ASCII code number `Value` represents.

For example: `CHR(65)` returns 'A'

`RND() RETURNS REAL`

returns a random number in the range 0 to 0.99999

For example: `RND()` returns 0.67351

`INT(ThisNumber : REAL)` RETURNS INTEGER

returns the integer part of `ThisNumber`.

For example: `INT(12.79)` returns 12

Errors

For any function, if the program calls the function incorrectly, the function returns an error.

Concatenation operator

`&` operator – Concatenates two expressions of `STRING` or `CHAR` data type.

For example: `"South" & " " & "Pole"` produces `"South Pole"`
`'B' & "000654"` produces `"B000654"`

163. O/N 16/P22/Q5/c(i,ii)

- 5 A team keeps a record of the scores made by each of their eight players in a number of games.

The data in the two tables below shows:

- the scores of the eight players after twenty games
- the eight player names.

	1	2	3		8
1	12	17	67		31
2	35	82	44		29
3	61	39	80		17
4	81	103	21		11
5	56	0	98		4
...					
19	45	6	81		77
20	12	11	3		6

1	Vorma
2	Ravi
3	Chada
4	Nigam
5	Bahri
6	Smith
7	Goyal
8	Lata

The team wants a computer program to input and record the player data.

- (a) A programmer designs the following pseudocode for the input of a player's score from one game.

```
01 INPUT GameNumber
02 INPUT PlayerNumber
03 INPUT PlayerGameScore
04 PlayerScore[GameNumber, PlayerNumber] ← PlayerGameScore
```

Describe the data structure the programmer has used for the storage of all player scores.

[2]

- (c) The team wants the program to produce a report, with the following specification.

The program outputs the total number of player scores that are:

- 50 and over but less than 100
- 100 or higher.

You can assume that before the section runs, the program has assigned all eight player scores to the `PlayerScore` data structure.

A first attempt at the pseudocode is shown below:

```

01 Total50 ← 0
02 Total100 ← 0
03 FOR PlayerIndex ← 1 TO 8
04   FOR GameIndex ← 1 TO 20
05     IF PlayerScore[GameIndex, PlayerIndex] > 100
06       THEN
07         Total100 ← Total100 + 1
08       ELSE
09         IF PlayerScore[GameIndex, PlayerIndex] > 50
10           THEN
11             Total50 ← Total50 + GameIndex
12           ENDIF
13         ENDIF
14       ENDFOR
15     ENDFOR
16 OUTPUT Total50
17 OUTPUT Total100

```

(i) Describe the control structure used in lines 03 and 04 and lines 14 and 15.

[2]

(ii) Consider the following two statements.

Write either TRUE or FALSE next to each statement.

Statement	TRUE or FALSE
The pseudocode considers all the scores for a player, before progressing to the next player.	
The pseudocode considers all scores in a game, before progressing to the next game.	

[1]

164. O/N 16/P22/Q6

6 Study the sequence of pseudocode statements.

```
CONST  a = 3.2      : REAL

DECLARE x, y, z, Answer1, Answer2, Answer3  : REAL

DECLARE p, q        : BOOLEAN

x ← 3

x ← x + 7

y ← 6

Answer1 ← 2 * (a + y)

z ← 6

Answer2 ← y ^ 2 + 5

p ← TRUE

q ← NOT (NOT (p) )

Answer3 ← y + a * 2
```

Give the final value assigned to each variable.

- | | |
|----------------------------|-----|
| (i) x | [1] |
| (ii) Answer1 | [1] |
| (iii) Answer2 | [1] |
| (iv) q | [1] |
| (v) Answer3 | [1] |

165. O/N 16/P23/Q3/a,b,c,d

3 You will need to refer to the list of pseudocode string-handling functions in the **Appendix**.

ASCII code table (part)					
Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

(a) For each statement, write the value assigned to the variable.

(i) `Term ← CHARACTERCOUNT("TSUNAMI")`

Term [1]

(ii) `Answer1 ← ASC('G') + ASC(<Space>)`

Answer1 [1]

(iii) `Answer2 ← CHR(CHARACTERCOUNT("HELLO") + 70)`

Answer2 [1]

(iv) `Word ← SUBSTR("Welcome home", 4, 7) )`

Word [1]

(b) A programmer wants to design a procedure to calculate a customer ID number from the customer's surname.

The procedure will:

- input the surname
- isolate each character in the surname and find the corresponding ASCII code
- calculate the total of all these ASCII codes
- this total is the customer ID

(i) Complete the pseudocode for this procedure.

You will need to refer to the list of pseudocode string-handling functions in the Appendix.

```

PROCEDURE CalculateCustomerID
    OUTPUT "Key in surname"
    INPUT Surname
    Length ← .....
    CustomerID ← 0
    FOR i ← 1 TO Length
        // NextChar is a single character from Surname
        NextChar ← .....
        NextCodeNumber ← ASC(NextChar)
        CustomerID ← CustomerID + .....
    ENDFOR
    OUTPUT "Customer ID is ", CustomerID

```

[3]

- (ii) Write **program code** for procedure `CalculateCustomerID`.

Visual Basic and Pascal: You should include declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[6]

- (c) The programmer decides that it would be better to write the procedure as a function.
The user will now input the surname in the main program.

Write **program code** for the following:

State your programming language

- (i) The function header for this new function `CalculateCustomerID`

[3]

- (ii) The additional statement required within the function body to complete the change from a procedure to a function.

[1]

- (iii) The statement in the main program which:

- calls the function for surname Wilkes
- assigns the result to variable `ThisID`

[3]

- (d) (i) The new function `CalculateUserID` is an example of a 'user-defined function'.

State **two** differences between a built-in function and a user-defined function.

2

[2]

(ii) State **two** things that built-in and user-defined functions have in common.

1

2

[2]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR

returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.

For example: `ONECHAR("New York", 5)` returns `'Y'`

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER

returns the number of characters in `ThisString`.

For example: `CHARACTERCOUNT("New York")` returns 8

SUBSTR(ThisString : STRING, Value1 : INTEGER, Value2 : INTEGER) RETURNS STRING

returns a sub-string from within `ThisString`.

`Value1` is the start index position (counting from the left, starting with 1).

`Value2` is the final index position.

For example: `SUBSTR("art nouveau", 5, 11)` returns `"nouveau"`

TONUM(ThisString : STRING) RETURNS INTEGER or REAL

returns the integer or real equivalent of the string `ThisString`.

For example: `TONUM("502")` returns the integer 502

`TONUM("56.36")` returns the real number 56.36

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`

returns an integer which is the ASCII character code for the character `ThisCharacter`.

For example: `ASC('A')` returns integer 65

`CHR(Value : INTEGER) RETURNS CHAR`

returns the character that ASCII code number `Value` represents.

For example: `CHR(65)` returns 'A'

`RND() RETURNS REAL`

returns a random number in the range 0 to 0.99999

For example: `RND()` returns 0.67351

`INT(ThisNumber : REAL) RETURNS INTEGER`

returns the integer part of `ThisNumber`.

For example: `INT(12.79)` returns 12

Errors

For any function, if the program calls the function incorrectly, the function returns an error.

Concatenation operator

`&` – Concatenates two expressions of `STRING` or `CHAR` data type.

For example: `"South" & " " & "Pole"` produces `"South Pole"`
`'B' & "000654"` produces `"B000654"`

166. O/N 16/P23/Q5

5 Study the following pseudocode statements.

```

CONST  Pi = 3.1           : REAL

DECLARE Triangle, Base, Height, Radius, Cone : REAL

DECLARE a, b, c, Answer2 : INTEGER

DECLARE Answer1           : BOOLEAN

Base ← 2.6

Height ← 10

Triangle ← (Base * Height) / 2

Radius ← 1

Height ← 2

Cone ← 2 * Pi * Radius * (Radius + Height)

a ← 13

b ← 7

c ← 3

Answer1 ← NOT((a + b + c) > 28)

Total ← 34

Total ← Total - 2

Answer2 ← a + c * c

```

Give the final value assigned to each variable.

- | | |
|---------------------|-----|
| (i) Triangle | [1] |
| (ii) Cone | [1] |
| (iii) Answer1 | [1] |
| (iv) Total | [1] |
| (v) Answer2 | [1] |

167. M/J 16/P21/Q1/a

also Unit 9 (6 marks); this unit 8 marks

1 The items in the table below are individual statements in a generic programming language.

For the built-in functions list, refer to the **Appendix** on page 18.

(a) (i) Show what type of programming construct each statement represents.

Complete the table by putting a tick (✓) in the appropriate column for each item.

Item	Statement	Selection	Iteration	Assignment
1	MyScore = 65			
2	FOR IndexVal = 0 TO 99			
3	MyArray[3] = MID(MyString,3,2)			
4	IF MyScore >= 70 THEN			
5	ENDWHILE			
6	ELSE Message = "Error"			

[6]

(ii) State the purpose of each statement in the table in **part (a)(i)**.

Do **not** use mathematical symbols in your descriptions.

Item	Purpose of statement
1	
2	
3	
4	
5	
6	

[6]

- (iii) Evaluate the following expressions when `MyString` has the value "Adaptive Maintenance".

Expression	Result
<code>'D' & RIGHT(MyString, 4)</code>	
<code>LEFT(RIGHT(MyString, 7), 3)</code>	

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` will return string "BCD"

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING

returns the leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` will return string "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` will return string "FGH"

`ASC(ThisChar : CHAR)` RETURNS INTEGER

returns the ASCII value of character `ThisChar`

Example: `ASC('w')` will return 87

`LENGTH(ThisString : STRING)` RETURNS INTEGER

returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` will return 10

String operator

& operator

concatenates (joins) two strings

Example: "Summer" & " " & "Pudding" produces "Summer Pudding"

168. M/J 16/P21/Q3/b(ii)

- 3 A string encryption function is needed. The encryption uses a simple character-substitution method.

In this method, a new character substitutes for each character in the original string. This will create the encrypted string.

The substitution uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, `Lookup`, which contains the substitute characters.

`Lookup` contains an entry for each of the ASCII characters. It may be assumed that the original string and the substitute characters are all printable.

For example:

- 'A' has ASCII value 65
- Array element with index 65 contains the character 'Y' (the substitute character)
- Therefore, 'Y' substitutes for 'A'
- There is a different substitute character for every ASCII value

The programmer writes a function, `EncryptString`, to return the encrypted string. This function will receive two parameters, the original, `PlainText` string and the 1D array.

- (a) The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION EncryptString(.....) RETURNS STRING
    DECLARE ..... : CHAR
    DECLARE OldCharValue : .....
    DECLARE n : INTEGER
    DECLARE OutString : STRING
    ..... //initialise the return string
    //loop through PlainText to produce OutString
    FOR n ← 1 TO ..... //from first to last character
        OldChar ← .....//get next character
        OldCharValue ← .....//find the ASCII value
        NewChar ← .....//look up substitute character
        .....//concatenate to OutString
    ENDFOR
ENDFUNCTION

```

- (b) Additional code needs to be written to allow the user to change some of the characters in the array `Lookup`.

The user will input:

- the array start position
- the number of elements to change
- each new substitute character

At the end, the program will finally output a confirmation message.

The first version of the algorithm is represented by the flowchart on the following page.

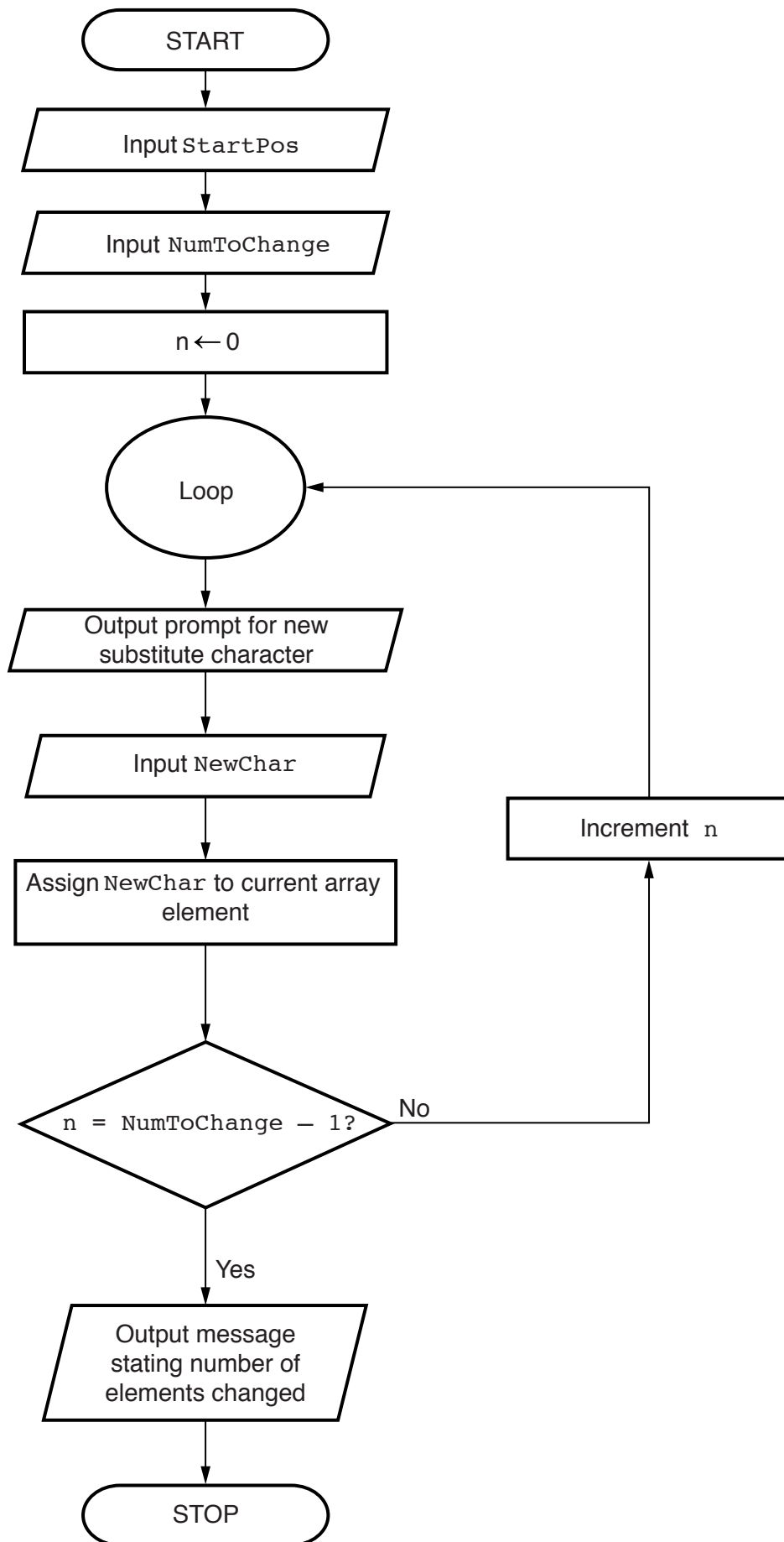
- (ii) Write **program code** to implement the flowchart design.

In addition to the `Lookup` array, assume that the following variables have been declared:

`StartPos`, `NumToChange`, `n`, `NewChar`

Programming language

[6]



169. M/J 16/P22/Q1/a

also Unit 9 (6 marks); this unit 8 marks

- 1** The items in the table below are individual statements in a generic programming language.

For the built-in functions list, refer to the **Appendix** on page 18.

- (a) (i)** Show what type of programming construct each statement represents.

Complete the table by putting a tick (✓) in the appropriate column for each item.

Item	Statement	Selection	Iteration	Assignment
1	MyScore = 65			
2	FOR IndexVal = 0 TO 99			
3	MyArray[3] = MID(MyString,3,2)			
4	IF MyScore >= 70 THEN			
5	ENDWHILE			
6	ELSE Message = "Error"			

[6]

- (ii)** State the purpose of each statement in the table in **part (a)(i)**.

Do **not** use mathematical symbols in your descriptions.

Item	Purpose of statement
1	
2	
3	
4	
5	
6	

[6]

- (iii) Evaluate the following expressions when `MyString` has the value "Adaptive Maintenance".

Expression	Result
<code>'D' & RIGHT(MyString, 4)</code>	
<code>LEFT(RIGHT(MyString, 7), 3)</code>	

[2]

Appendix of this paper:

Appendix

Built-in functions

In each function below, if the function call is not properly formed, the function returns an error.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING

returns the string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` will return string "BCD"

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING

returns the leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` will return string "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING

returns the rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` will return string "FGH"

`ASC(ThisChar : CHAR)` RETURNS INTEGER

returns the ASCII value of character `ThisChar`

Example: `ASC('w')` will return 87

`LENGTH(ThisString : STRING)` RETURNS INTEGER

returns the integer value representing the length of string `ThisString`

Example: `LENGTH("Happy Days")` will return 10

String operator

& operator

concatenates (joins) two strings

Example: "Summer" & " " & "Pudding" produces "Summer Pudding"

170. M/J 16/P22/Q3/b(ii)

- 3 A string encryption function is needed. The encryption uses a simple character-substitution method.

In this method, a new character substitutes for each character in the original string. This will create the encrypted string.

The substitution uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, `Lookup`, which contains the substitute characters.

`Lookup` contains an entry for each of the ASCII characters. It may be assumed that the original string and the substitute characters are all printable.

For example:

- 'A' has ASCII value 65
- Array element with index 65 contains the character 'Y' (the substitute character)
- Therefore, 'Y' substitutes for 'A'
- There is a different substitute character for every ASCII value

The programmer writes a function, `EncryptString`, to return the encrypted string. This function will receive two parameters, the original, `PlainText` string and the 1D array.

- (a) The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on page 18.

```

FUNCTION EncryptString(.....) RETURNS STRING
    DECLARE ..... : CHAR
    DECLARE OldCharValue : .....
    DECLARE n : INTEGER
    DECLARE OutString : STRING
    ..... //initialise the return string
    //loop through PlainText to produce OutString
    FOR n ← 1 TO ..... //from first to last character
        OldChar ← .....//get next character
        OldCharValue ← .....//find the ASCII value
        NewChar ← .....//look up substitute character
        .....//concatenate to OutString
    ENDFOR
ENDFUNCTION

```

- (b) Additional code needs to be written to allow the user to change some of the characters in the array `Lookup`.

The user will input:

- the array start position
- the number of elements to change
- each new substitute character

At the end, the program will finally output a confirmation message.

The first version of the algorithm is represented by the flowchart on the following page.

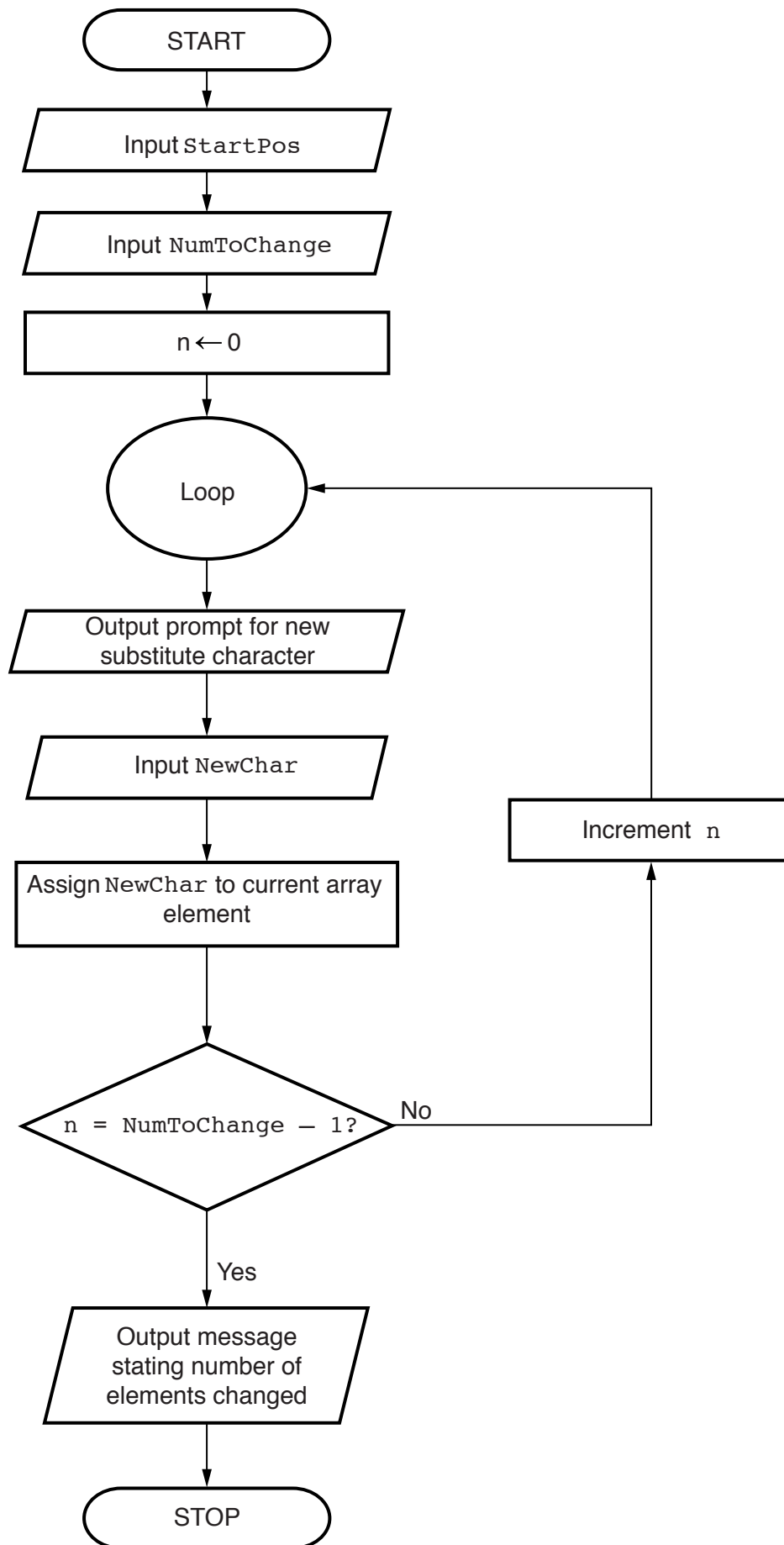
- (ii) Write **program code** to implement the flowchart design.

In addition to the `Lookup` array, assume that the following variables have been declared:

`StartPos`, `NumToChange`, `n`, `NewChar`

Programming language

[6]



171. M/J 16/P23/Q1/a

also Unit 9 (6 marks); this unit 8 marks

- 1** The items in the table below are statements from a program in a generic programming language.

For the built-in functions list, refer to the **Appendix** on the last page.

- (a) (i)** Show what type of programming construct each statement represents.

Complete the table by putting a tick (✓) in the appropriate column for each item.

Item	Statement	Selection	Iteration	Assignment
1	WHILE DegF > 37.5			
2	MyName = "Gordon"			
3	DegF = INT(DegF)			
4	ENDIF			
5	CASE OF MyFavourite			
6	UNTIL x = 5			

[6]

- (ii)** State the purpose of each statement in the table in **part (a)(i)**.

Do **not** use mathematical symbols in your descriptions.

Item	Purpose of statement
1	
2	
3	
4	
5	
6	

[6]

- (iii) Evaluate the following expressions when MyString has the value "Corrective Maintenance":

Expression	Evaluates to
'P' & MID(MyString, 13, 4)	
RIGHT(MID(MyString, 6, 10) ,4)	

[2]

Appendix of this paper:

Appendix

Built-in functions (Pseudocode)

In each function below, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns the string of length y starting at position x from ThisString

Example: **MID**("ABCDEFGH", 2, 3) will return string "BCD"

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns the leftmost x characters from ThisString

Example: **LEFT**("ABCDEFGH", 3) will return string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns the rightmost x characters from ThisString

Example: **RIGHT**("ABCDEFGH", 3) will return string "FGH"

CHR(x : INTEGER) RETURNS CHAR

returns the character whose ASCII value is x

Example: **CHR**(87) will return 'w'

ASC(x : CHAR) RETURNS INTEGER

returns the ASCII value of character x

Example: **ASC**('w') will return 87

LCASE(x : CHAR) RETURNS CHAR

returns the lower case equivalent character of x

Example: **LCASE**('W') will return 'w'

UCASE(x : CHAR) RETURNS CHAR

returns the upper case equivalent character of x

Example: **UCASE**('h') will return 'H'

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: **INT**(27.5415) will return 27

172. M/J 16/P23/Q3/b

- 3** String encryption was implemented using a simple character-substitution method. A function, **Decrypt**, is needed to reverse the encryption process and return the original character.

The encryption uses the 7-bit ASCII value for each character. This value is used as an index for a 1D array, **Lookup**, which contains the substitute characters. **Lookup** contains an entry for each of the ASCII characters.

This function, **Decrypt**, will accept two parameters, a single character, **CipherChar**, and the 1D array, **Lookup**.

The steps involved in Decrypt are follows:

- Search for the character in the array
- Note the index value where the character is found (the index value is the ASCII value of the original character)
- Use the index value to obtain the original character

(a) The first attempt at writing the pseudocode for this function is shown below.

Complete the pseudocode.

For the built-in functions list, refer to the **Appendix** on the last page.

```

FUNCTION Decrypt(..... , ..... ) RETURNS CHAR
    DECLARE Found      : .....
    DECLARE .....      : INTEGER
    DECLARE OriginalChar : CHAR
    Index ← 1          // .....
    Found ← FALSE
    //search for CipherChar in Lookup:
    WHILE .....
        //compare CipherChar with this array element:
        IF .....
            THEN
                .....//Set the flag
            ELSE
                Index .....//Move to next array element
            ENDIF
        ENDWHILE
        //dropped out of loop so must have found CipherChar:
        .....//convert Index to original character
    RETURN .....
ENDFUNCTION

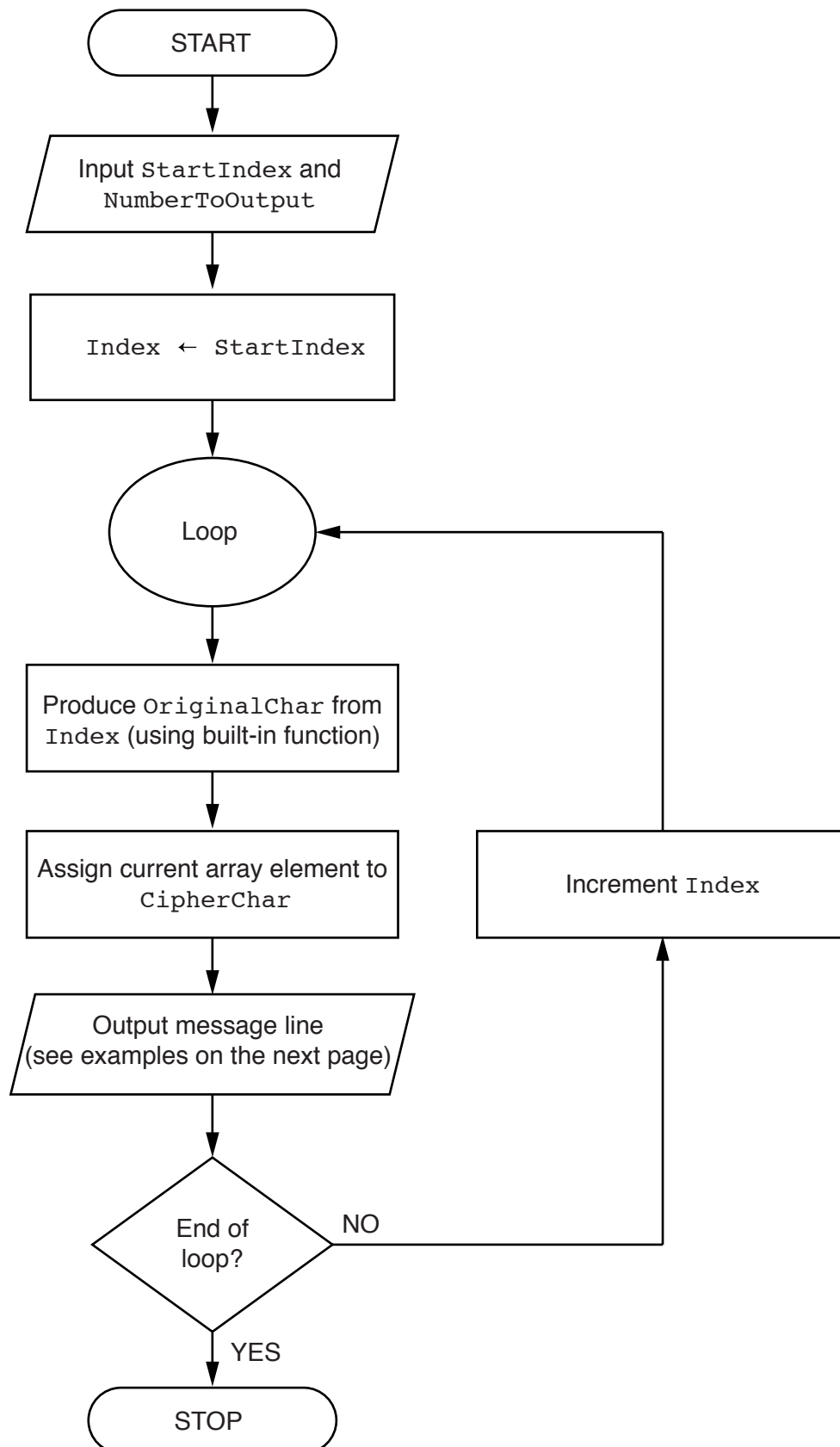
```

[11]

(b) A program is to be written to output part of the Lookup array.

The design of the algorithm is shown below.

It may be assumed that the characters output from `Lookup` are all printable.



For example, for the input of 65 and 3, the output will be:

Index 65: Character A has substitute character Y
Index 66: Character B has substitute character Q
Index 67: Character C has substitute character F

Write **program code** to implement the flowchart design.

In addition to the Lookup array, assume that the following variables have been declared:

StartIndex, NumberToOutput, Index

Programming language

[6]

173. M/J 16/P23/Q4/a

- 4 (a) Name **two** features of your chosen high-level programming language that support the implementation of a modular design.

1

2

[2]

174. O/N 15/P21/Q1

- 1 Computer programs have to evaluate expressions.

Study the sequence of pseudocode statements.

Write down the value assigned to each variable.

DECLARE h, w, r, Perimeter, Area : REAL	
DECLARE A, B, C, D, E : BOOLEAN	
h ← 13.6	
w ← 6.4	
Perimeter ← (h + w) * 2	(i) Perimeter [1]
r ← 10	
Area 3.142 * r^2	(ii) Area [1]
Z ← 11 + r / 5 + 3	(iii) Z [1]
A ← NOT(r > 10)	(iv) A [1]

175. O/N 15/P21/Q5/b

5 A driver buys a new car.

The value of the car reduces each year by a percentage of its current value.

The percentage reduction is:

- in the first year, 40%
- in each following year, 20%

The driver writes a program to predict the value of the car in future years.

The program requirements are:

- enter the cost of the new car (to nearest \$)
- calculate and output the value of the car at the end of each year
- the program will end when either the car is nine years old, or when the value is less than \$1000

(b) Complete the pseudocode for this design.

```

OUTPUT "Enter purchase price"
INPUT PurchasePrice

CurrentValue ← .....
YearCount ← 1

WHILE ..... AND .....

    IF .....
    THEN
        CurrentValue ← CurrentValue * (1 - 40 / 100)
    ELSE
        CurrentValue ← .....
    ENDIF

    OUTPUT YearCount, CurrentValue

ENDWHILE

```

[6]

176. O/N 15/P21/Q6/a,b,c,d

also Unit 10 (2 marks); this unit 15 marks

6 A firm employs five staff who take part in a training programme. Each member of staff must complete a set of twelve tasks which can be taken in any order. When a member of staff successfully completes a task, this is recorded.

A program is to be produced to record the completion of tasks for the five members of staff.

To test the code, the programmer makes the program generate test data.

The program generates pairs of random numbers:

- the first, in the range, 1 to 5 to represent the member of staff
- the second, in the range, 1 to 12 to represent the task

Each pair of numbers simulates the completion of one task by one member of staff.

- (a) Explain why the generation of 60 (5 staff x 12 tasks) pairs of random numbers will not simulate all tasks completed by all staff.

[2]

- (b) Data is currently recorded manually as shown.

Staff number	Task number											
	1	2	3	4	5	6	7	8	9	10	11	12
1												
2												
3				✓								
4												
5								✓				

The table shows that two members of staff have each successfully completed one task.

The program must use a suitable data structure to store, for all staff:

- tasks successfully completed
- tasks not yet successfully completed

The program will output the staff number and task number in the order in which tasks are completed.

The program design in pseudocode is produced as follows:

```

01 DECLARE StaffNum          : INTEGER
02 DECLARE TaskNum           : INTEGER
03 DECLARE .....
04 DECLARE NewStaffTask      : BOOLEAN
05
06 CALL InitialiseTaskGrid
07 Completed ← 0
08 WHILE Completed <> 60
09     NewStaffTask ← FALSE
10     WHILE NewStaffTask = FALSE
11         StaffNum ← RANDOM(1,5)           //generates a random number
12         TaskNum ← RANDOM(1,12)          //in the given range
13         IF TaskGrid[StaffNum, TaskNum] = FALSE
14             THEN
15                 TaskGrid[StaffNum, TaskNum] ← TRUE
16                 NewStaffTask ← TRUE
17                 OUTPUT StaffNum, TaskNum
18             ENDIF
19     ENDWHILE
20     Completed ← Completed + 1
21 ENDWHILE
22 OUTPUT "Staff Task Count", Completed
23
24 // end of main program
25
26 PROCEDURE InitialiseTaskGrid()
27     DECLARE i : INTEGER
28     DECLARE j : INTEGER
29     FOR i ← 1 TO 5
30         FOR j ← 1 TO 12
31             TaskGrid[i, j] ← FALSE
32         ENDFOR
33     ENDFOR
34 ENDPROCEDURE

```

Study the pseudocode and answer the questions below.

Give the line number for:

- (i) The declaration of a `BOOLEAN` global variable. [1]
- (ii) The declaration of a local variable. [1]
- (iii) The incrementing of a variable used as a counter, but not to control a 'count controlled' loop. [1]

(iv) A statement which uses a built-in function of the programming language.

[1]

(c) (i) State the number of parameters of the `InitialiseTaskGrid` procedure.

[1]

(ii) Copy the condition which is used to control a 'pre-condition' loop.

[1]

(iii) Explain the purpose of lines 13 – 18.

[3]

(iv) Give the global variable that needs to be declared at line 03.

[2]

(d) Line 17 in the pseudocode outputs the staff number and the task number.

A new requirement is to display the name of the member of staff given in the table.

Write a CASE structure using variable `StaffNum`.

Assign to a new variable `StaffName` the appropriate staff name.

Staff number	Staff name
1	Sadiq
2	Smith
3	Ho
4	Azmah
5	Papadopoulos

[4]

177. O/N 15/P21/Q7/a,b

7 ASCII character codes are used to represent a single character.

Part of the code table is shown below.

ASCII code table (part)

Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

Some pseudocode statements follow which use the built-in functions below:

`ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR`
 returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.
 For example: `ONECHAR("Barcelona", 3)` returns 'r'.

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER
 returns the number of characters in the string ThisString.
 For example: **CHARACTERCOUNT**("BRAZIL") returns 6.

CHR(ThisInteger : INTEGER) RETURNS CHAR
 returns the character with ASCII code ThisInteger.
 For example: **CHR**(65) returns character 'A'.

ASC(ThisCharacter : CHAR) RETURNS INTEGER
 returns the ASCII value for character ThisCharacter.
 For example: **ASC**('A') returns 65.

(a) Show the values stored by variables A, B, C and D.

The & operator is used to concatenate two strings.

Num1 ← 15	
A ← CHR(67) & CHR(65) & CHR(84)	(i) A [1]
B ← ASC('P') - ASC('F') + 3	(ii) B [1]
C ← ASC(ONECHAR("BISCUITS", 3))	(iii) C [1]
D ← CHARACTERCOUNT("New York City") + 2	(iv) D [1]

(b) A program is to be written which accepts a string and then calculates a numeric value from this string. The input string and the calculated value are then to be sent to a remote computer over a communications link.

Study the following pseudocode:

```

OUTPUT "Enter string"
INPUT MyString
StringTotal ← 0

FOR i ← 1 TO CHARACTERCOUNT(MyString)
    NextNum ← ASC(ONECHAR(MyString, i))
    StringTotal ← StringTotal + NextNum
ENDFOR

OUTPUT MyString, StringTotal
```

Write the above pseudocode algorithm as **program code**.

There is no need to show the declaration of variables or comment statements.

178. O/N 15/P22/Q1

- 1** Computer programs have to evaluate expressions.

Study the sequence of pseudocode statements.

Give the value assigned to each variable.

The statement may generate an error. If so, write ERROR.

The & operator is used to concatenate strings.

DECLARE N1	:	INTEGER
DECLARE N2	:	INTEGER
DECLARE Answer	:	REAL
DECLARE Found	:	BOOLEAN
DECLARE IsValid	:	BOOLEAN
N1 ← 3		
N2 ← 9		
Answer ← (N1 + N2) / 6		
Answer ← 3 * (N1 - 2) + N2 / 2		
IsValid ← (N1 > N2) AND (N2 = 9)		
Found ← FALSE		
IsValid ← (N1 > N2 / 2) OR (Found = FALSE)		
Answer ← "1034" & " + " & "65"		

- (i) Answer [1]
- (ii) Answer [1]
- (iii) IsValid [1]
- (iv) IsValid [1]
- (v) Answer [1]

179. O/N 15/P22/Q4/a,b,c,d,e,f,g

also Unit 10 (1 mark); this unit 14 marks

- 4** The standard pack of playing cards has four suits – called Clubs, Diamonds, Hearts and Spades. Each card has a value shown by its number or a name: 1 (Ace), 2, 3, ... 10, 11 (Jack), 12 (Queen), 13 (King). The pack of cards has one combination for each suit and value.

A program is to be written which simulates a magician dealing all 52 cards from the card pack.

The program generates pairs of random numbers:

- the first, in the range 1 to 4, to represent the suit
- the second, in the range 1 to 13, to represent the card value

- (a) Explain why the generation of 52 (4 suits x 13 card values) pairs of random numbers will not simulate the dealing of the complete pack.

[2]

- (b) A representation of dealing out the cards is shown below:

Suit number	Card value												
	1	2	3	4	5	6	7	8	9	10	11	12	13
1 (Clubs)	F	F	F	F	F	F	F	F	F	F	T	F	F
2 (Diamonds)	F	F	F	F	F	F	F	F	F	F	F	F	F
3 (Hearts)	F	F	T	F	F	F	F	F	F	F	F	F	F
4 (Spades)	F	F	F	F	F	F	F	F	F	F	F	F	F

The table shows two cards have been dealt so far; the 3 of Hearts and the Jack of Clubs.

When each card is dealt, the appropriate cell changes from F to T.

The program will output the suit and the card value in the order in which the cards are dealt.

The program design in pseudocode is produced as follows:

```
01 DECLARE SuitNum      : INTEGER
02 DECLARE CardValue    : INTEGER
03 DECLARE DealCount    : INTEGER
04 DECLARE NewCard      : BOOLEAN
05 DECLARE CardPack
06
07 CALL InitialiseCardPack
08 DealCount ← 0
09 WHILE DealCount <> 52
10     NewCard ← FALSE
11     WHILE NewCard = FALSE
12         SuitNum ← RANDOM(1,4)    // generates a random number
13         CardValue ← RANDOM(1,13) // in the range given
14         IF CardPack[SuitNum, CardValue] = FALSE
15             THEN
16                 CardPack[SuitNum, CardValue] ← TRUE
17                 NewCard ← TRUE
18                 OUTPUT SuitNum, CardValue
19             ENDIF
20     ENDWHILE
21     DealCount ← DealCount + 1
22 ENDWHILE
23
24 // end of main program
25
26 PROCEDURE InitialiseCardPack
27     DECLARE i : INTEGER
28     DECLARE j : INTEGER
29     FOR i ← 1 TO 4
30         FOR j ← 1 TO 13
31             CardPack[i, j] ← FALSE
32         ENDFOR
33     ENDFOR
34 ENDPROCEDURE
```

Study the pseudocode and answer the questions below:

Give the line number for:

- (i) A statement which marks the end of a count controlled loop.

[1]

- (ii) The declaration of a local variable.

[1]

- (iii) The initialisation of a variable used as a counter, but not to control a 'count controlled' loop.

[1]

- (iv) A statement which uses a built-in function of the programming language.

[1]

- (c) Give the number of procedures used by the pseudocode.

[1]

- (d) Copy the condition which is used to control a 'pre-condition' loop.

[1]

- (e) Explain the purpose of lines 14 – 19 in the design.

[2]

- (f) Complete the declaration of the global variable at line 05.

05 DECLARE CardPack

[1]

- (g) Line 18 in the design shows which new card is dealt each time.

When an Ace, Jack, Queen or King is dealt, the output displays the number for that card, not the name of the card.

Card value	Card name
1	Ace
11	Jack
12	Queen
13	King

A new requirement is to display the name of the card, where appropriate.

Write a CASE structure using variable `CardValue`.

Assign to a new variable `CardName` either:

- the card value (2, 3, 4, 5, 6, 7, 8, 9 or 10)
- or where appropriate, the card name Ace, Jack, Queen or King

[4]

180. O/N 15/P22/Q5/a(iii)

5 A program is to process a set of integers.

The integers are stored in an array, Num. The first N elements are to be processed.

The pseudocode for this program is shown below:

```
FOR i ← 1 TO (N - 1)
    j ← 1
    REPEAT
        IF Num[j] > Num[j + 1]
            THEN
                Temp ← Num[j]
                Num[j] ← Num[j + 1]
                Num[j + 1] ← Temp
            ENDIF
        j ← j + 1
    UNTIL j = (N - i + 1)
ENDFOR
```

(iii) Describe what evidence from the trace table suggests that the given pseudocode is inefficient.

[1]

181. O/N 15/P22/Q6/a,b(ii)

6 Some pseudocode statements follow which use the following built-in functions:

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR
 returns the single character at position Position (counting from the start of the string with value 1) from the string ThisString.
 For example: ONECHAR("Barcelona", 3) returns 'r'.

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER
 returns the number of characters in the string ThisString.
 For example: CHARACTERCOUNT("South Africa") returns 12.

(a) Study the following pseudocode statements.

Give the values assigned to variables x and y.

(i) $x \leftarrow \text{CHARACTERCOUNT}(\text{"New Delhi"}) + 3$ x [1]

(ii) $y \leftarrow \text{ONECHAR}(\text{"Sri Lanka"}, 5)$ y [1]

(b) A program is to be written as follows:

- the user enters a string
- the program will form a new string with all <Space> characters removed
- the new string is output

```

NewString ← ""
INPUT InputString

j ← CHARACTERCOUNT(InputString)
FOR i ← 1 TO j
    NextChar ← ONECHAR(InputString, i)
    IF NextChar <> " "
        THEN
            // the & character joins together two strings
            NewString ← NewString & NextChar
        ENDIF
    ENDFOR
OUTPUT NewString

```

- (ii) An experienced programmer suggests this pseudocode would be best designed as a function.

Complete the re-design of the pseudocode as follows:

The main program:

- the user enters `MyString`
- the function is called and the changed string is assigned to variable `ChangedString`

The function:

- has identifier `RemoveSpaces`
- has a single parameter
- will include the declaration for any local variables used by the function

```

// main program
INPUT MyString
ChangedString←RemoveSpaces(.....)
OUTPUT ChangedString

// function definition
FUNCTION RemoveSpaces(.....) RETURNS .....

    j ← CHARACTERCOUNT(InputString)
    FOR i ← 1 TO j
        NextChar ← ONECHAR(InputString, i)
        IF NextChar <> " "
            THEN
                // the & character joins together two strings
                NewString ← NewString & NextChar
            ENDIF
        ENDFOR
    ENDFUNCTION

```

182. O/N 15/P22/Q7/a,b(i)

7 ASCII character codes are used to represent a single character.

Part of the code table is shown below.

ASCII code table (part)

Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

Some pseudocode statements follow which use these built-in functions:

`CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER`
 returns the number of characters in the string `ThisString`.
 For example: `CHARACTERCOUNT("South Africa")` returns 12.

`CHR(ThisInteger : INTEGER) RETURNS CHAR`
 returns the character with ASCII value `ThisInteger`.
 For example: `CHR(66)` returns 'B'.

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`
 returns the ASCII value for character `ThisCharacter`.
 For example: `ASC('B')` returns 66.

(a) Give the values assigned to the variables A, B, C and D.

The & operator is used to concatenate two strings.

The expression could generate an error; if so, write ERROR.

<code>Num1 ← 5</code>
<code>A ← ASC('F') + Num1 + ASC('Z')</code>
<code>B ← CHR(89) & CHR(69) & CHR(83)</code>
<code>C ← CHARACTERCOUNT(B & "PLEASE")</code>
<code>D ← ASC(ONECHAR("CURRY SAUCE", 7))</code>

(i) A [1]

(ii) B [1]

(iii) C [1]

(iv) D [1]

(b) A program is to be written to input a message string and then encrypt the message.

Study the following pseudocode:

```

OUTPUT "Enter message"
INPUT MyMessage
EncryptString ← ""

FOR i ← 1 TO CHARACTERCOUNT(MyMessage)
    NextNum ← ASC(ONECHAR(MyMessage, i) + 3)
    EncryptString ← EncryptString & CHR(NextNum)
ENDFOR

OUTPUT EncryptString

```

(i) Write the above pseudocode algorithm as **program code**.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[8]

183. O/N 15/P23/Q1

1 Computer programs have to evaluate expressions.

Study the sequence of pseudocode statements.

Write down the value assigned to each variable.

DECLARE h, w, r, Perimeter, Area : REAL	
DECLARE A, B, C, D, E : BOOLEAN	
h ← 13.6	
w ← 6.4	
Perimeter ← (h + w) * 2	(i) Perimeter [1]
r ← 10	
Area 3.142 * r^2	(ii) Area [1]
Z ← 11 + r / 5 + 3	(iii) Z [1]
A ← NOT(r > 10)	(iv) A [1]

184. O/N 15/P23/Q5/b

5 A driver buys a new car.

The value of the car reduces each year by a percentage of its current value.

The percentage reduction is:

- in the first year, 40%
- in each following year, 20%

The driver writes a program to predict the value of the car in future years.

The program requirements are:

- enter the cost of the new car (to nearest \$)
- calculate and output the value of the car at the end of each year
- the program will end when either the car is nine years old, or when the value is less than \$1000

(b) Complete the pseudocode for this design.

```

OUTPUT "Enter purchase price"
INPUT PurchasePrice

CurrentValue ← .....
YearCount ← 1

WHILE ..... AND .....

    IF .....
    THEN
        CurrentValue ← CurrentValue * (1 - 40 / 100)
    ELSE
        CurrentValue ← .....
    ENDIF

    OUTPUT YearCount, CurrentValue

ENDWHILE

```

[6]

185. O/N 15/P23/Q6/a,b,c,d

also Unit 10 (2 marks); this unit 15 marks

6 A firm employs five staff who take part in a training programme. Each member of staff must complete a set of twelve tasks which can be taken in any order. When a member of staff successfully completes a task, this is recorded.

A program is to be produced to record the completion of tasks for the five members of staff.

To test the code, the programmer makes the program generate test data.

The program generates pairs of random numbers:

- the first, in the range, 1 to 5 to represent the member of staff
- the second, in the range, 1 to 12 to represent the task

Each pair of numbers simulates the completion of one task by one member of staff.

- (a) Explain why the generation of 60 (5 staff x 12 tasks) pairs of random numbers will not simulate all tasks completed by all staff.

[2]

- (b) Data is currently recorded manually as shown.

Staff number	Task number											
	1	2	3	4	5	6	7	8	9	10	11	12
1												
2												
3				✓								
4												
5								✓				

The table shows that two members of staff have each successfully completed one task.

The program must use a suitable data structure to store, for all staff:

- tasks successfully completed
- tasks not yet successfully completed

The program will output the staff number and task number in the order in which tasks are completed.

The program design in pseudocode is produced as follows:

```

01 DECLARE StaffNum          : INTEGER
02 DECLARE TaskNum           : INTEGER
03 DECLARE .....
04 DECLARE NewStaffTask      : BOOLEAN
05
06 CALL InitialiseTaskGrid
07 Completed ← 0
08 WHILE Completed <> 60
09     NewStaffTask ← FALSE
10     WHILE NewStaffTask = FALSE
11         StaffNum ← RANDOM(1,5)          //generates a random number
12         TaskNum ← RANDOM(1,12)         //in the given range
13         IF TaskGrid[StaffNum, TaskNum] = FALSE
14             THEN
15                 TaskGrid[StaffNum, TaskNum] ← TRUE
16                 NewStaffTask ← TRUE
17                 OUTPUT StaffNum, TaskNum
18             ENDIF
19     ENDWHILE
20     Completed ← Completed + 1
21 ENDWHILE
22 OUTPUT "Staff Task Count", Completed
23
24 // end of main program
25
26 PROCEDURE InitialiseTaskGrid()
27     DECLARE i : INTEGER
28     DECLARE j : INTEGER
29     FOR i ← 1 TO 5
30         FOR j ← 1 TO 12
31             TaskGrid[i, j] ← FALSE
32         ENDFOR
33     ENDFOR
34 ENDPROCEDURE

```

Study the pseudocode and answer the questions below.

Give the line number for:

- (i) The declaration of a `BOOLEAN` global variable. [1]
- (ii) The declaration of a local variable. [1]
- (iii) The incrementing of a variable used as a counter, but not to control a 'count controlled' loop.

[1]

(iv) A statement which uses a built-in function of the programming language.

[1]

(c) (i) State the number of parameters of the `InitialiseTaskGrid` procedure.

[1]

(ii) Copy the condition which is used to control a 'pre-condition' loop.

[1]

(iii) Explain the purpose of lines 13 – 18.

[3]

(iv) Give the global variable that needs to be declared at line 03.

[2]

(d) Line 17 in the pseudocode outputs the staff number and the task number.

A new requirement is to display the name of the member of staff given in the table.

Write a CASE structure using variable `StaffNum`.

Assign to a new variable `StaffName` the appropriate staff name.

Staff number	Staff name
1	Sadiq
2	Smith
3	Ho
4	Azmah
5	Papadopoulos

[4]

186. O/N 15/P23/Q7/a,b

7 ASCII character codes are used to represent a single character.

Part of the code table is shown below.

ASCII code table (part)

Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

Some pseudocode statements follow which use the built-in functions below:

`ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR`
 returns the single character at position `Position` (counting from the start of the string with value 1) from the string `ThisString`.
 For example: `ONECHAR("Barcelona", 3)` returns 'r'.

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER
 returns the number of characters in the string ThisString.
 For example: **CHARACTERCOUNT** ("BRAZIL") returns 6.

CHR(ThisInteger : INTEGER) RETURNS CHAR
 returns the character with ASCII code ThisInteger.
 For example: **CHR**(65) returns character 'A'.

ASC(ThisCharacter : CHAR) RETURNS INTEGER
 returns the ASCII value for character ThisCharacter.
 For example: **ASC**('A') returns 65.

(a) Show the values stored by variables A, B, C and D.

The & operator is used to concatenate two strings.

Num1 ← 15	
A ← CHR(67) & CHR(65) & CHR(84)	(i) A [1]
B ← ASC('P') - ASC('F') + 3	(ii) B [1]
C ← ASC(ONECHAR("BISCUITS", 3))	(iii) C [1]
D ← CHARACTERCOUNT("New York City") + 2	(iv) D [1]

(b) A program is to be written which accepts a string and then calculates a numeric value from this string. The input string and the calculated value are then to be sent to a remote computer over a communications link.

Study the following pseudocode:

```

OUTPUT "Enter string"
INPUT MyString
StringTotal ← 0

FOR i ← 1 TO CHARACTERCOUNT(MyString)
    NextNum ← ASC(ONECHAR(MyString, i))
    StringTotal ← StringTotal + NextNum
ENDFOR

OUTPUT MyString, StringTotal
  
```

Write the above pseudocode algorithm as **program code**.

There is no need to show the declaration of variables or comment statements.

187. M/J 15/P21/Q1/b(ii)

- 1 A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

```
INPUT race time as hours, minutes and seconds
CALCULATE race time in seconds
STORE race time in seconds
OUTPUT race time in seconds
```

- (b) Before the program is written, the design is amended.

The new design includes input of the runner's current personal best marathon time (in seconds).

The output will now also show one of the following messages:

- "Personal best time is unchanged"
- "New personal best time"
- "Equals personal best time"

- (ii) Write **program code** for the new design.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[7]

188. M/J 15/P21/Q2/a,b,c,d

- 2 A program displays a menu with choices 1 to 4. The code to display the menu is written as the procedure `DisplayMenu`.

- (a) Pseudocode which uses this procedure is:

```
CALL DisplayMenu
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
UNTIL Choice >= 1 AND Choice <= 4
```

- (i) Describe what this pseudocode will do.

[3]

- (ii) State why a loop is required.

[1]

(b) The following pseudocode is a revised design.

```

CONSTANT i ← 3
CALL DisplayMenu
NoOfAttempts ← 0
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
    NoOfAttempts ← NoOfAttempts + 1
UNTIL (Choice >= 1 AND Choice <= 4) OR NoOfAttempts = i

```

(i) Give the maximum number of inputs the user could be prompted to make.

[1]

(ii) State why this algorithm is an improvement on the one given in **part (a)**.

[1]

(c) The pseudocode is in its initial stage of development.

The table below shows the action currently taken by the pseudocode following each menu choice.

Menu choice	Description	Program response
1	Read data from the customer file	Calls a procedure <code>ReadFile</code> which for testing purposes outputs the message "Read file code"
2	Add a customer	Outputs message "Add customer code"
3	Search for a customer	Outputs message "Search customer code"
4	Terminates the program	Ends

Complete the pseudocode for the design in **part (b)**, shown again below, to respond to each menu choice.

```

CONSTANT i ← 3
CALL DisplayMenu
NoOfAttempts ← 0
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
    NoOfAttempts ← NoOfAttempts + 1
UNTIL (Choice >= 1 AND Choice <= 4) OR NoOfAttempts = i

```

[3]

(d) The algorithm in **part (c)** is to be amended. The program will:

- repeatedly display the menu and respond to the user's choice
- terminate when the user enters 4

Write **program code** for this final design which will be made up of:

- the main program
- procedure `ReadFile`
- procedure `DisplayMenu`

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[8]

189. M/J 15/P21/Q4/(i)**4** A game is played between two players:

- they take turns at rolling a six-sided die (numbered 1 to 6) and record their throw
- a player scores 1 point if their throw is higher than their opponent
- they each roll the die 20 times
- if the player's throw is the same as their opponent, the total points is unchanged
- the winner is the player with the larger number of points after 20 throws

The pseudocode will use variable `NoOfThrows` as shown.

Identifier	Data type	Description
<code>NoOfThrows</code>	INTEGER	Loop control variable

(i) Complete the pseudocode for the given algorithm.

```

FOR .....
    INPUT Player1Throw
    IF Player1Throw > Player2Throw
        THEN

    ENDIF
    IF Player2Throw > Player1Throw
        THEN
            Player2Total ← Player2Total + 1
        ENDIF
    IF Player1Total > Player2Total
        THEN
            OUTPUT "Player1 is the winner"
        ELSE
            OUTPUT "Player2 is the winner"
        ENDIF

```

[5]

190. M/J 15/P21/Q5/c**5** A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable `SalesX`) and Site Y (using variable `SalesY`).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

- (b) The programmer writes a program from the following pseudocode design.

```

x ← 0
FOR DayNumber ← 1 TO 7
    IF SalesX[DayNumber] + SalesY[DayNumber] >= 10
        THEN
            x ← x + 1
            OUTPUT SalesDate[DayNumber]
        ENDIF
    ENDFOR
OUTPUT x

```

- (c) The company wants a program to output the total monthly sales for one of the selected websites.

The programmer codes a function with the following function header:

```

FUNCTION MonthlyWebSiteSales(ThisMonth : INTEGER, ThisSite : CHAR)
    RETURNS INTEGER

```

The function returns the total number of bicycles sold for the given month and website.

The function will use the following:

Identifier	Data type	Description
ThisMonth	INTEGER	Represents the month number e.g. 4 represents April
ThisSite	CHAR	Coded as: • X for website X • Y for Website Y

- (i) Give the number of parameters of this function.

[1]

(ii) Some of the following function calls may be invalid.

Mark each call with:

- a tick (✓), for a valid call
- a cross (✗), for an invalid call

For any function calls which are invalid, explain why.

Function call	Tick (✓) /cross (✗)	Explanation (if invalid)
<code>MonthlyWebSiteSales(1, "Y")</code>		
<code>MonthlyWebSiteSales(11, 'X', 'Y')</code>		
<code>MonthlyWebSiteSales(12, 'X')</code>		

[3]

191. M/J 15/P22/Q1/b(ii)

- 1** A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

```
INPUT race time as hours, minutes and seconds
CALCULATE race time in seconds
STORE race time in seconds
OUTPUT race time in seconds
```

- (b)** Before the program is written, the design is amended.

The new design includes input of the runner's current personal best marathon time (in seconds).

The output will now also show one of the following messages:

- "Personal best time is unchanged"
- "New personal best time"
- "Equals personal best time"

- (ii)** Write **program code** for the new design.

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[7]

192. M/J 15/P22/Q2/a,b,c,d

- 2** A program displays a menu with choices 1 to 4. The code to display the menu is written as the procedure `DisplayMenu`.

- (a)** Pseudocode which uses this procedure is:

```
CALL DisplayMenu
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
UNTIL Choice >= 1 AND Choice <= 4
```

- (i)** Describe what this pseudocode will do.

[3]

- (ii)** State why a loop is required.

[1]

(b) The following pseudocode is a revised design.

```

CONSTANT i ← 3
CALL DisplayMenu
NoOfAttempts ← 0
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
    NoOfAttempts ← NoOfAttempts + 1
UNTIL (Choice >= 1 AND Choice <= 4) OR NoOfAttempts = i

```

(i) Give the maximum number of inputs the user could be prompted to make.

[1]

(ii) State why this algorithm is an improvement on the one given in **part (a)**.

[1]

(c) The pseudocode is in its initial stage of development.

The table below shows the action currently taken by the pseudocode following each menu choice.

Menu choice	Description	Program response
1	Read data from the customer file	Calls a procedure <code>ReadFile</code> which for testing purposes outputs the message "Read file code"
2	Add a customer	Outputs message "Add customer code"
3	Search for a customer	Outputs message "Search customer code"
4	Terminates the program	Ends

Complete the pseudocode for the design in **part (b)**, shown again below, to respond to each menu choice.

```

CONSTANT i ← 3
CALL DisplayMenu
NoOfAttempts ← 0
REPEAT
    OUTPUT "Enter choice (1..4)"
    INPUT Choice
    NoOfAttempts ← NoOfAttempts + 1
UNTIL (Choice >= 1 AND Choice <= 4) OR NoOfAttempts = i

```

[3]

(d) The algorithm in **part (c)** is to be amended. The program will:

- repeatedly display the menu and respond to the user's choice
- terminate when the user enters 4

Write **program code** for this final design which will be made up of:

- the main program
- procedure `ReadFile`
- procedure `DisplayMenu`

Visual Basic and Pascal: You should include the declaration statements for variables.

Python: You should show a comment statement for each variable used with its data type.

Programming language

[8]

193. M/J 15/P22/Q4/(i)

4 A game is played between two players:

- they take turns at rolling a six-sided die (numbered 1 to 6) and record their throw
- a player scores 1 point if their throw is higher than their opponent
- they each roll the die 20 times
- if the player's throw is the same as their opponent, the total points is unchanged
- the winner is the player with the larger number of points after 20 throws

The pseudocode will use variable `NoOfThrows` as shown.

Identifier	Data type	Description
<code>NoOfThrows</code>	INTEGER	Loop control variable

(i) Complete the pseudocode for the given algorithm.

```

FOR .....
    INPUT Player1Throw
    IF Player1Throw > Player2Throw
        THEN

    ENDIF
    IF Player2Throw > Player1Throw
        THEN
            Player2Total ← Player2Total + 1
        ENDIF
    IF Player1Total > Player2Total
        THEN
            OUTPUT "Player1 is the winner"
        ELSE
            OUTPUT "Player2 is the winner"
        ENDIF

```

[5]

194. M/J 15/P22/Q5/c

5 A company creates two new websites, Site X and Site Y, for selling bicycles.

Various programs are to be written to process the sales data.

These programs will use data about daily sales made from Site X (using variable `SalesX`) and Site Y (using variable `SalesY`).

Data for the first 28 days is shown below.

	SalesDate	SalesX	SalesY
1	03/06/2015	0	1
2	04/06/2015	1	2
3	05/06/2015	3	8
4	06/06/2015	0	0
5	07/06/2015	4	6
6	08/06/2015	4	4
7	09/06/2015	5	9
8	10/06/2015	11	9
9	11/06/2015	4	1
...			
28	01/07/2015	14	8

- (b) The programmer writes a program from the following pseudocode design.

```

x ← 0
FOR DayNumber ← 1 TO 7
    IF SalesX[DayNumber] + SalesY[DayNumber] >= 10
        THEN
            x ← x + 1
            OUTPUT SalesDate[DayNumber]
        ENDIF
    ENDFOR
OUTPUT x

```

- (c) The company wants a program to output the total monthly sales for one of the selected websites.

The programmer codes a function with the following function header:

```

FUNCTION MonthlyWebSiteSales(ThisMonth : INTEGER, ThisSite : CHAR)
    RETURNS INTEGER

```

The function returns the total number of bicycles sold for the given month and website.

The function will use the following:

Identifier	Data type	Description
ThisMonth	INTEGER	Represents the month number e.g. 4 represents April
ThisSite	CHAR	Coded as: • X for website X • Y for Website Y

- (i) Give the number of parameters of this function.

[1]

(ii) Some of the following function calls may be invalid.

Mark each call with:

- a tick (✓), for a valid call
- a cross (✗), for an invalid call

For any function calls which are invalid, explain why.

Function call	Tick (✓) /cross (✗)	Explanation (if invalid)
MonthlyWebSiteSales(1, "Y")		
MonthlyWebSiteSales(11, 'X', 'Y')		
MonthlyWebSiteSales(12, 'X')		

[3]

195. M/J 15/P23/Q2/a,b(ii)

- 2 (a) Two operators available in a programming language are DIV and MOD. They perform integer arithmetic as follows:

Expression

Explanation

X DIV Y Computes the number of times Y divides into X

X MOD Y Computes the remainder when X is divided by Y

Calculate the value of the variables shown for the following code fragments.

	Code	Variable	
(i)	NumberLeftOver ← 37 MOD 10	NumberLeftOver	[1]
(ii)	Quantity ← 208 BoxSize ← 100 NumberOfBoxes ← Quantity DIV BoxSize Temp ← (Quantity MOD BoxSize) + 1	NumberOfBoxes Temp	[2]

- (b) Bank customers withdraw money from their account at a cash dispenser machine using their bank card. The machine operates as follows:

- it can dispense the following notes:
 - \$50
 - \$20
 - \$10
- the maximum amount for a single withdrawal is \$500

When a customer withdraws money, they enter the amount to withdraw. (This must be a multiple of \$10).

The machine will always dispense the least possible number of notes.

A program is designed for the machine to process a withdrawal.

The following variables are used:

Identifier	Data type	Description
Amount	INTEGER	Amount to withdraw entered by the user
FiftyDollar	INTEGER	Number of \$50 notes to dispense
TwentyDollar	INTEGER	Number of \$20 notes to dispense
TenDollar	INTEGER	Number of \$10 notes to dispense
Temp	INTEGER	Used in the calculation of the number of each note required

(ii) Complete the pseudocode.

```

INPUT .....
IF Amount > 500
    THEN
        OUTPUT "Refused – amount too large"
    ELSE
        THEN
            OUTPUT "Refused – not a multiple of $10"
        ELSE
            FiftyDollar ← Amount DIV 50
            Temp ← .....
            TwentyDollar ← .....
            Temp ← .....
        ENDIF
    ENDIF
ENDIF

```

[5]

196. M/J 15/P23/Q3/(ii)

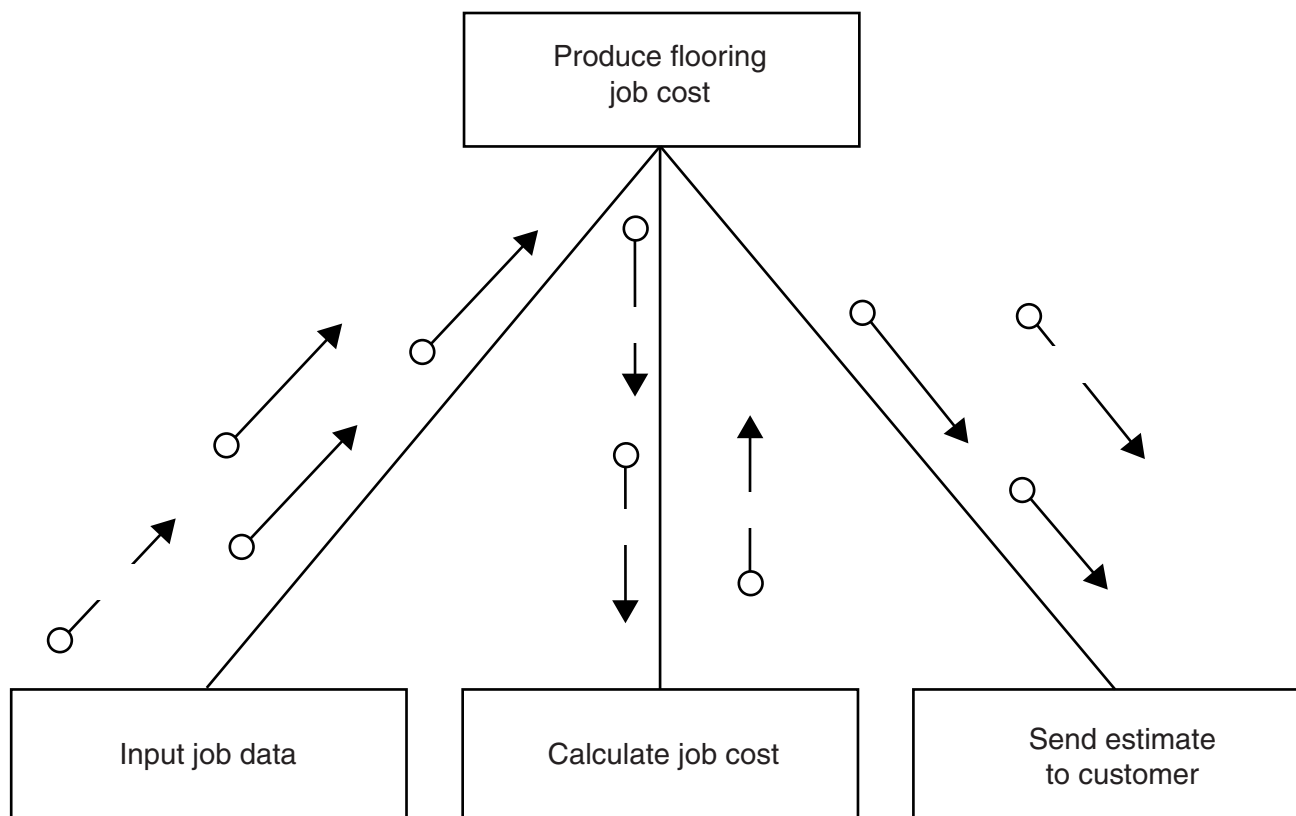
- 3** A flooring company provides for each customer an estimated price for a new job. Each job is given a Job ID.

The job cost is calculated from the length (nearest metre) and width (nearest metre) of the room.

The process for calculating the price is as follows:

- the floor area is calculated with 18% added to allow for wastage
- the job cost is calculated at \$50 per square metre

The structure chart shows the modular design for a program to produce a new job cost.



- (ii)** The procedure below is one of the modules shown on the structure chart.

Parameters can be passed 'by value' or 'by reference'.

Complete the procedure header below showing for each parameter:

- its parameter passing mechanism
- its identifier
- its data type

```

PROCEDURE CalculateJobCost( .....
                                )

    JobCost ← (Length * Width * 1.18) * 50
ENDPROCEDURE
  
```


197. M/J 15/P23/Q4/a

- 4** A programming language has the built-in function `CONCAT` defined as follows:

```
CONCAT(String1 : STRING, String2 : STRING [, String3 : STRING] ) RETURNS STRING
For example: CONCAT("San", "Francisco") returns "SanFrancisco"
              CONCAT("New", "York", "City") returns "NewYorkCity"
```

The use of the square brackets indicates that the parameter is optional.

- (a)** State the value returned by the following expressions.

If the expression is not properly formed, write `ERROR`.

- (i)** `CONCAT("Studio", 54)` [1]
(ii) `CONCAT("parity", "error", "check")` [1]
(iii) `CONCAT(CONCAT("Binary", "▼", "Coded"), "▼", "Decimal")`

▼ indicates a <Space> character

[2]

198. M/J 15/P23/Q5/c

- 5** A firm employs workers who assemble amplifiers. Each member of staff works an agreed number of hours each day.

The firm records the number of completed amplifiers made by each employee each day.

Management monitor the performance of all its workers.

Production data was collected for 3 workers over 4 days.

Daily hours worked		Production data				
Worker 1	5	Day 1	Worker 1	Worker 2	Worker 3	
Worker 2	10		Day 2	10	20	9
Worker 3	10		Day 3	11	16	11
			Day 4	10	24	13
			14	20	17	

A program is to be written to process the production data.

- (a)** The production data is to be stored in a 2-dimensional array `ProductionData`, declared as follows:

```
DECLARE ProductionData ARRAY[1:4, 1:3] : INTEGER
```

(b) Complete the trace table for the pseudocode algorithm below.

```

FOR WorkerNum ← 1 TO 3
    WorkerTotal[WorkerNum] ← 0
ENDFOR

FOR WorkerNum ← 1 TO 3
    FOR DayNum ← 1 TO 4
        WorkerTotal[WorkerNum] ← WorkerTotal[WorkerNum] +
                                ProductionData[DayNum, WorkerNum]
    ENDFOR
ENDFOR

FOR WorkerNum ← 1 TO 3
    WorkerAverage ← WorkerTotal[WorkerNum]/
                    (4 * DailyHoursWorked[WorkerNum])

    IF WorkerAverage < 2
        THEN
            OUTPUT "Investigate", WorkerNum
        ENDFOR
    ENDFOR

```

[illegible]

[8]

(c) An experienced programmer suggests that the pseudocode would be best implemented as a procedure `AnalyseProductionData`.

Assume that both arrays, `DailyHoursWorked` and `ProductionData`, are available to the procedure from the main program and they are of the appropriate size.

```

PROCEDURE AnalyseProductionData(NumDays : INTEGER, NumWorkers : INTEGER)
  DECLARE .....
  DECLARE .....
  DECLARE .....
  DECLARE .....
  FOR WorkerNum ← 1 TO 3
    WorkerTotal[WorkerNum] ← 0
  ENDFOR
  FOR WorkerNum ← 1 TO 3
    FOR DayNum ← 1 TO 4
      WorkerTotal[WorkerNum] ← WorkerTotal[WorkerNum] +
                               ProductionData[DayNum, WorkerNum]
    ENDFOR
  ENDFOR
  FOR WorkerNum ← 1 TO 3
    WorkerAverage ← WorkerTotal[WorkerNum]/
                    (4 * DailyHoursWorked [WorkerNum])
    IF WorkerAverage < 2
      THEN
        OUTPUT "Investigate", WorkerNum
      ENDFOR
    ENDFOR
  ENDFOR
ENDPROCEDURE

```

(i) Complete the declaration statements showing the local variables. [4]

(ii) The original pseudocode has been 'pasted' under the procedure header.

Circle all the places in the original pseudocode where changes will need to be made.

Write the changes which need to be made next to each circle. [3]

(iii) Write the statement for a procedure call which processes data for 7 days for 13 workers.

[1]

Answers Section

1. M/J 26/P21/Q1/a,b

1(a)	<u>Count-control(led) loop // Count-control(led) iteration</u>	1
1(b)	4	1

2. M/J 26/P21/Q5/a,b

5(a)	<i>Example solution:</i> <pre> FUNCTION BulbCost(Quantity : INTEGER) RETURNS REAL DECLARE Cost : REAL DECLARE Twenty : INTEGER DECLARE Other : INTEGER Twenty ← Quantity DIV 20 Other ← Quantity MOD 20 Cost ← (Twenty * 30) + (Other * 1.75) RETURN Cost ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and single parameter and returns REAL and end MP2 Declaration of required local variable(s) MP3 Correct calculation of 'multiples of 20' and 'other' amounts using MOD and DIV // alternative method MP4 Cost is returned	4
5(b)	TotalCost ← BulbCost (NumberPurchased)	1

3. M/J 26/P21/Q6/a,b

6(a)	MP1 LEFT(DataPacket, 3) // MID(DataPacket, 1, 3) MP2 MID(DataPacket, 5, 1)	2
------	--	----------

6(b)	<i>Example solution:</i> <pre> FUNCTION RejectDescription(ThisReject : STRING) RETURNS STRING DECLARE Description : STRING Description ← <u>MID(ThisReject, 7,</u> <i>Part 1</i> <u>LENGTH(ThisReject) - 6)</u> <i>Part 2</i> OR: Description ← RIGHT(ThisReject, LENGTH(ThisReject) - 6) <i>Part 1</i> <i>Part 2</i> RETURN Description ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and end function (and string parameter) MP2 Calculation for description expression (<i>part 1</i>) using MID/RIGHT MP3 Calculation for description expression (<i>part 2</i>) using the LENGTH function MP4 Return of a calculated value and RETURN STRING in the header	4
------	---	---

4. M/J 26/P22/Q1/a(ii,iii)

1(a)(ii)	<pre> IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN </pre> Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
----------	---	---	--

1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for BOOLEAN 2 LetterFlag ← <u>FALSE</u> 1 mark for FALSE 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for LENGTH 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for MID, Index and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for IS_NUM 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for NOT	7	
-----------	---	---	--

5. M/J 26/P22/Q5/c

5(c)(i)	1 mark: The parameters / Num1 and Num2 are passed by value / BYVAL // The parameters / Num1 and Num2 are not passed / should be passed by reference / BYREF 1 mark: (The swap made inside) the procedure does not change the arguments used /parameters (passed) / the values stored in the Numbers elements passed // The procedure does not swap / change the values stored in the array	2	
5(c)(ii)	Solution <pre>PROCEDURE Swap (BYREF Num1, Num2 : INTEGER) DECLARE Temp : INTEGER Temp ← Num1 Num1 ← Num2 Num2 ← Temp ENDPROCEDURE</pre> 2 marks: code as above 1 mark: just the procedure header as above	2	NE for header mark If ByRef is mentioned in the header but syntax is not correct but allow FT for second mark

6. M/J 26/P22/Q6/a

6(a)	Mark Scheme One – loop used to form return string (See MS Two for solutions using LEFT () /MID () to form return string) Example solution <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Space = ' ' DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 FOR Index ← LenOfPlaintext + 1 TO 25 Plaintext ← Plaintext & Space NEXT Index // Alternative way to append spaces using while loop WHILE LENGTH(Plaintext) <> 25 Plaintext ← Plaintext & Space ENDWHILE //Alternative way to append spaces DECLARE SpaceString : STRING SpaceString ← "" FOR Index ← LenOfPlaintext + 1 TO 25 SpaceString ← SpaceString & Space NEXT Index Plaintext ← Plaintext & SpaceString RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values 4 Conditional loop that correctly checks both boundaries of plaintext 5 ...Re-input of plaintext in a loop 6 Loop that iterates for number of spaces to add to end of plaintext 7 ...Spaces appended to plaintext // Spaces appended to an empty string 8 Correctly padded plaintext returned following a reasonable attempt at MP7	8	MP 4 Must be a conditional loop If no loop used then MP3, MP4 and MP5 can not be given
------	---	---	---

6(a)	Mark Scheme Two - No Loop used to form return string – use of LEFT () /MID () Function Example Solution: <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Fill = " " //24 Spaces DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 Plaintext ← LEFT(Plaintext & Fill, 25) RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values Conditional loop that correctly checks for length of plaintext 4 ...Re-input of plaintext in a loop 5 Use of string of at least 24 spaces Correct use of LEFT () / MID () function and & to append padding characters 6 Correctly padded plaintext returned following a reasonable attempt at MP 6 and MP 7		Do not accept use of language specific string manipulation which is not the compatible with the pseudocode used for the syllabus
------	---	--	---

7. M/J 26/P23/Q1/a,b

1(a)	<u>Post-condition(al) loop // post-condition(al) iteration</u>	1	
1(b)	5	1	

8. M/J 26/P23/Q5/a

5(a)	<pre> FUNCTION InterviewSummary(ThisApplicant : STRING) RETURNS STRING DECLARE Summary : STRING Summary ← MID(ThisApplicant, 8, <u>MP2</u> LENGTH(ThisApplicant) - 7) <u>MP3</u> Or: Summary ← RIGHT(ThisApplicant, LENGTH(ThisApplicant) - 7) <u>MP2</u> <u>MP3</u> RETURN Summary ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function identifier and function ending and parameter MP2 Calculation for summary expression (part 1) – using MID or RIGHT MP3 Calculation for summary expression (part 2) – using LENGTH MP4 Return of the calculated STRING value (earlier declaration optional) and RETURNS STRING in the header –	4	Do not accept a final missing close bracket (for the MID or RIGHT function).
------	---	----------	---

9. O/N 25/P21/Q1/a

1(a)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					</
------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

10. O/N 25/P21/Q5/b

5(b)(i)	The loop continues after the (first instance) of the search value / string / matching element has been found Comparisons/tests continue to be made even if the search value / string / matching element has been found	1
---------	--	----------

5(b)(ii)	One mark per point Construct: A conditional loop Justification: The loop / search can be terminated as soon as the (first occurrence of) SearchString / required value / required string / matching element is found	2
----------	---	----------

11. O/N 25/P21/Q6/b

6(b)	Example solution – conversion of Number to a string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR (Number) FOR Index ← 1 TO LENGTH (NumString) Total ← Total + STR_TO_NUM (MID (NumString, Index, 1)) NEXT Index CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit // NumString ← NumString & NUM_TO_STR (CheckDigit) RETURN Number // STR_TO_NUM (NumString) ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Convert Number to a string and loop for length of converted Number 4 attempt to form sum of digits in a loop 5 completely correct sum of digits in a loop 6 Calculate CheckDigit 7 Add CheckDigit to original number multiplied by 10 and return number // Convert CheckDigit to a character and appended to NumString and then convert to a number and return	7
------	---	----------

6(b)	Example Solution – no conversion to string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, CheckDigit, Remainder, Num: INTEGER Num ← Number Total ← 0 WHILE Num > 0 Remainder ← Num MOD 10 Num ← Num DIV 10 Total ← Total + Remainder ENDWHILE CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit RETURN Number ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Loop while Num > 0 4 attempt to use MOD and DIV to sum successive digits in Number 5 completely correct sum of digits 6 Calculate CheckDigit 7 Add CheckDigit to original number and return number	
------	--	--

12. O/N 25/P22/Q1/c

1(c)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>INT(Result) + 1 > 6</td><td>FALSE</td></tr><tr><td>NUM_TO_STR(LENGTH(MonthLetter))</td><td>"12"</td></tr><tr><td>NUM_TO_STR(Result + "3.2")</td><td>ERROR</td></tr><tr><td>MID(MonthLetter, MONTH(Birthday) - 2, 1)</td><td>"S"</td></tr></table> One mark per row	Expression	Evaluates to	INT(Result) + 1 > 6	FALSE	NUM_TO_STR(LENGTH(MonthLetter))	"12"	NUM_TO_STR(Result + "3.2")	ERROR	MID(MonthLetter, MONTH(Birthday) - 2, 1)	"S"	4
Expression	Evaluates to											
INT(Result) + 1 > 6	FALSE											
NUM_TO_STR(LENGTH(MonthLetter))	"12"											
NUM_TO_STR(Result + "3.2")	ERROR											
MID(MonthLetter, MONTH(Birthday) - 2, 1)	"S"											

1(b)	<table><tr><th>Variable</th><th>Example data value</th><th>Data type</th></tr><tr><td>Result</td><td>5.42</td><td>REAL</td></tr><tr><td>MonthLetter</td><td>"JFMAMJJASOND"</td><td>STRING</td></tr><tr><td>Birthday</td><td>15/11/2009</td><td>DATE</td></tr></table> One mark per row	Variable	Example data value	Data type	Result	5.42	REAL	MonthLetter	"JFMAMJJASOND"	STRING	Birthday	15/11/2009	DATE	3
Variable	Example data value	Data type												
Result	5.42	REAL												
MonthLetter	"JFMAMJJASOND"	STRING												
Birthday	15/11/2009	DATE												

13. O/N 25/P22/Q6

6	Example solution: <pre> FUNCTION Compare(String1, String2 : STRING, __ CaseMatters : BOOLEAN, Position : CHAR) RETURNS BOOLEAN DECLARE S1Length : INTEGER S1Length ← LENGTH(String1) IF LENGTH(String2) < S1Length THEN RETURN FALSE // String2 is shorter than String1 ENDIF IF CaseMatters = FALSE THEN String1 ← TO_UPPER(String1) String2 ← TO_UPPER(String2) ENDIF CASE Position 'e' : String2 ← RIGHT(String2, S1Length) 's' : String2 ← LEFT(String2, S1Length) ENDCASE IF String1 = String2 THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameters and ending and return type MP2 Calculate length of String1 MP3 if String2 is shorter than String1, return FALSE MP4 Compare (modified) strings MP5 Test CaseMatters - if FALSE and convert two string(s) to same case MP6 Test for the value of 's'/'e' and form modified string(s) in one or both cases MP7 Return appropriate BOOLEAN value in all cases	7
---	---	---

14. O/N 25/P23/Q1/a,b,c

1(a)	Answers include: 1 They are tried and tested // Free from errors 2 They are readily available // Speed up development time 3 They will be updated automatically when improvements are made // No maintenance of routine is needed (as automatically updated) 4 The algorithm will be compliant with the international standard One mark per point Max 3	3								
1(b)(i)	1 When a part of the algorithm performs a specific task 2 Part is repeated // performed in several places 3 Testing / debugging / maintenance is easier One mark for each point Max 2	2								
1(b)(ii)	<table><thead><tr><th>Term</th><th>Description</th></tr></thead><tbody><tr><td>Pass2</td><td>the name of the function</td></tr><tr><td>Count</td><td>The parameter / value passed to the function</td></tr><tr><td>BOOLEAN</td><td>The (data) type <u>returned</u> (by the function)</td></tr></tbody></table> One mark per description (excluding first row)	Term	Description	Pass2	the name of the function	Count	The parameter / value passed to the function	BOOLEAN	The (data) type <u>returned</u> (by the function)	2
Term	Description									
Pass2	the name of the function									
Count	The parameter / value passed to the function									
BOOLEAN	The (data) type <u>returned</u> (by the function)									
1(c)	<table><thead><tr><th>Expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>LENGTH(NUM_TO_STR(Multiplier))</td><td>3</td></tr><tr><td>MONTH(DoB) > 4</td><td>TRUE</td></tr><tr><td>15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))</td><td>20</td></tr></tbody></table> One mark per row	Expression	Evaluates to	LENGTH(NUM_TO_STR(Multiplier))	3	MONTH(DoB) > 4	TRUE	15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20	3
Expression	Evaluates to									
LENGTH(NUM_TO_STR(Multiplier))	3									
MONTH(DoB) > 4	TRUE									
15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20									

15. O/N 25/P23/Q5/b

5(b)(i)	1 and 2	1
---------	---------	----------

5(b)(ii)	<pre> 1 TO 2 : Num[Index] ← Num[Index] + Index Index ← Index + CaseVar Count ← Count + CaseVar </pre> One mark per point: • use of variable <code>CaseVar</code> rather than literal values • completely correct clause	2
----------	---	----------

16. O/N 25/P23/Q6/b

6(b)	Note 2 different solutions with different mark scheme Example solution – conversion to string and using a loop <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR(Number) CheckDigit ← STR_TO_NUM(RIGHT(NumString, 1)) // CheckDigit ← Number MOD 10 FOR Index ← 1 TO LENGTH(NumString) - 1 // 1 TO LENGTH(NUM_TO_STR(Number DIV 10)) Total ← Total + STR_TO_NUM(MID(NumString, Index, 1)) ENDFOR IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Function Header including Identifier, parameters and return type 2 Use of <code>Num_TO_STR</code> to convert parameter // Use of <code>Num_TO_STR</code> to convert parameter excluding least significant digit to a string 3 Extract <code>CheckDigit</code> from <code>NumString</code> // Extract least significant digit from parameter 4 Loop for length of <code>NumString</code> - 1 following correct conversion to string 5 attempt to form sum of digits in a loop 6 completely correct sum of digits in a loop and of sum initialised before loop 7 Calculate check digit from sum 8 Compare sum with final digit of parameter and return appropriate Boolean value Max 7	7
------	---	----------

6(b)	Example solution – numeric version <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Sum, CheckDigit, Remainder: INTEGER Remainder ← Number MOD 10 Number ← Number DIV 10 CheckDigit ← Remainder Sum ← 0 WHILE Number > 0 Remainder ← Number MOD 10 Number ← Number DIV 10 Total ← Total + Remainder ENDWHILE IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> 1 Function Header including Identifier, parameters and return type 2 Initialise Sum 3 Set CheckDigit to least significant digit in Number 4 Loop while Number > 0 5 attempt to use MOD and DIV to sum successive digits in Number in loop 6 completely correct sum of all digits in Number apart from least significant digit in loop 7 Calculate check digit from sum 8 Compare with CheckDigit and return appropriate Boolean value Max 7	
------	--	--

17. M/J 25/P21/Q4/a

4(a)	MP1 Count-controlled MP2 The number of iterations required is known	2
------	---	----------

18. M/J 25/P21/Q7/a

7(a)	Example solution: <pre> FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER DECLARE NewPoints, NumberFree: INTEGER NewPoints ← Points + Number NumberFree ← 0 WHILE NewPoints >= 11 AND Number > 0 NumberFree ← NumberFree + 1 NewPoints ← NewPoints -11 Number ← Number - 1 END WHILE OUTPUT "Number of free coffees is: ", NumberFree RETURN NewPoints ENDFUNCTION </pre> MP1 Function header and ending and parameters and return type MP2 Number of coffees ordered added to points MP3 Correct calculation of one free coffee MP4 Attempted calculation of multiple free coffees and reduced <code>NewPoints</code> MP5 Output number of free drinks with suitable message MP6 Return of <code>NewPoints</code>	6
------	---	----------

19. M/J 25/P23/Q1/a,b

1(a)(i)	MP1 When a task/module is repeated / reused / called // performed in several places MP2 A specific task can be identified /coded as a module / subroutine / procedure / function. MP3 Reduces complexity of <u>program / code</u> // Program / code is simplified MP4 Module / subroutine / procedure / function already available. MP5 <u>Testing</u> / <u>debugging</u> / <u>maintenance</u> is easier Max 2	2
1(a)(ii)	MP1 Local variables are used within a module // are accessible only within the module (in which they are declared) MP2 Global variables are accessible to all parts of the program / MP3 Local variables use memory only while the module is executing // Global variables use memory for the whole time the program is running Max 1	1

1(a)(iii)	MP1 The same variable name can be reused in other parts of the program MP2 Using local variables makes modules self-contained // cannot accidentally change the same identifier outside of the module MP3 Using local variables aids modularisation. MP4 Memory allocated to local variables can be reused when module not in use Max 2	2																		
1(b)	Mark as follows: <table border="1"> <thead> <tr> <th>Expression</th><th>Evaluates to</th><th>Mark</th></tr> </thead> <tbody> <tr> <td>CHR(68)</td><td>'D'</td><td>1</td></tr> <tr> <td>MONTH(04/02/2025)</td><td>2</td><td>1</td></tr> <tr> <td>// -2 + DAY(04/02/2025) // -2023 + YEAR(04/02/2025)</td><td></td><td></td></tr> <tr> <td>NOT TRUE // FALSE = TRUE</td><td>FALSE</td><td>1</td></tr> <tr> <td>STR_TO_NUM(MID("Court1.4Upper", 6, 3))</td><td>1.4</td><td>2</td></tr> </tbody> </table> Mark Row 4 as follows: STR_TO_NUM 1 mark MID("Court1.4Upper", 6, 3) 1 mark	Expression	Evaluates to	Mark	CHR (68)	'D'	1	MONTH (04/02/2025)	2	1	// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)			NOT TRUE // FALSE = TRUE	FALSE	1	STR_TO_NUM (MID ("Court1.4Upper", 6, 3))	1.4	2	5
Expression	Evaluates to	Mark																		
CHR (68)	'D'	1																		
MONTH (04/02/2025)	2	1																		
// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)																				
NOT TRUE // FALSE = TRUE	FALSE	1																		
STR_TO_NUM (MID ("Court1.4Upper", 6, 3))	1.4	2																		

20. M/J 25/P23/Q5

5	Example solution: <pre> FUNCTION Parity(BitString : STRING) RETURNS STRING DECLARE Index, Count: INTEGER Count ← 0 FOR Index ← 1 TO LENGTH(BitString) IF MID(BitString, Index, 1) = '1' THEN Count ← Count + 1 ENDIF NEXT Index IF Count MOD 2 = 1 THEN BitString ← BitString & '1' ELSE BitString ← BitString & '0' ENDIF RETURN BitString ENDFUNCTION </pre> MP1 Function heading and ending and parameter (STRING) and return type STRING MP2 Loop for length of BitString MP3 Extract current character in a loop MP4 Compare current character to '1' or '0' in a loop MP5 Increment count and was Initialised earlier in a loop MP6 Check if even/odd number of 1s MP7 Append '1' or '0' as appropriate MP8 Return amended BitString	8
---	--	---

21. M/J 25/P23/Q6/a

6(a)	MP1 A function returns a (single) value and a procedure does not MP2 A procedure can pass parameters by ByRef Max 1	1
------	--	---

22. M/J 25/P23/Q7/a,b(ii)

7(a)(i)	Example solution: <pre> PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE, __ BYREF LoyaltyPoints : INTEGER) IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN LoyaltyPoints ← LoyaltyPoints + 4 ELSE LoyaltyPoints ← LoyaltyPoints + 1 ENDIF LastVisitDate ← TODAY() ENDPROCEDURE </pre> MP1 Procedure heading and ending and two parameters of correct type MP2 ... both passed BYREF MP3 Use of TODAY () function to obtain current date MP4 Use MONTH function (×2) and one parameter is the header parameter and 2nd parameter is TODAY () / assigned variable MP5 If visit is same month, increase LoyaltyPoints by 4 otherwise increase by 1 MP6 LastVisitDate set to TODAY () / 'current date' variable (assigned or unassigned) Max 5	5
7(a)(ii)	DAYINDEX (LastVisitDate) = DAYINDEX (TODAY ())	1
7(b)(ii)	Example solution: <pre> DayInt ← STR_TO_NUM(LEFT(LastVisitDateString, 2)) MonthInt ← STR_TO_NUM(MID(LastVisitDateString, 3, 2)) YearInt ← STR_TO_NUM(RIGHT(LastVisitDateString, 4)) LastVisitDate ← SETDATE(DayInt, MonthInt, YearInt) </pre> Mark as follows: MP1 Use of one substring function MP2 DayInt, MonthInt and YearInt all correctly extracted MP3 Use of STR_TO_NUM x 3 MP4 SETDATE () function used to convert to a date type and assigned to LastVisitDate	4

23. O/N 24/P21/Q1/b,d

1(b)(i)	Use constants (to represent the tax rate values)	1
1(b)(ii)	One mark per bullet point (or equivalent to max 3): Tax rates are entered once only - Avoids / Minimise (input) error(s) / changing the Tax rates accidentally // avoids different values for tax rates at different points in the program - When required, the constant value (representing a tax rate) is changed (once) // Easier to maintain / update the program (when the tax rates change) - Makes the program / code easier to understand	3
1(d)	OTHERWISE	1

24. O/N 24/P21/Q2

2	Example solution: <pre> PROCEDURE Tick() SS ← SS + 1 IF SS = 60 THEN SS ← 0 MM ← MM + 1 IF MM = 60 THEN MM ← 0 HH ← HH + 1 IF HH = 24 THEN HH ← 0 CALL NewDay() ENDIF ENDIF CALL CheckAlarm() ENDIF ENDPROCEDURE </pre> Mark as follows: - Increment SS anywhere in the code - Call <code>CheckAlarm()</code> when <u>SS is set to zero</u> - Test if <code>SS = 60</code> anywhere in the code ... if so: - set SS to 0 - increment MM - When MM is set to 60 set MM to 0 and increment HH - When HH is set to 24 set HH to 0 and call <code>NewDay()</code> Max 6 marks	6
---	---	----------

25. O/N 24/P21/Q4/b,c

4(b)	Answers include: Reason: • This part of the algorithm performs a specific task // if the check or calculation is changed it is changed only once • A (similar) subroutine is already available // Library routine is already available • The program is simplified / easier to understand / easier to design / code / test / debug / maintain Max 1 marks for 'Reason' Avoided: • Unnecessary code duplication • Errors caused by differences where several copies of the check and calculation exist Max 1 marks for 'Avoided'	2
------	--	----------

4(c)	Example solution: <pre> FUNCTION Evaluate (NumStr1, NumStr2 : STRING) RETURNS Result DECLARE RetVal : Result DECLARE Num1, Num2 : REAL RetVal.Done ← FALSE IF IS_NUM(NumStr1) = TRUE AND IS_NUM(NumStr2) = TRUE THEN Num1 ← STR_TO_NUM(NumStr1) Num2 ← STR_TO_NUM(NumStr2) IF Num2 <> 0 THEN RetVal.Value ← (Num1 / Num2) RetVal.Done ← TRUE ENDIF ENDIF RETURN RetVal ENDFUNCTION </pre> Note that order of parameters is not specified so divisor could be either parameter. Mark as follows: 1 Function heading, parameters, ending 2 ... Correct return type of <code>Result</code> 3 Local declaration of type <code>Result</code> 4 Attempt at using <code>IS_NUM()</code> to check both parameters/strings 5 Attempt at using of <code>STR_TO_NUM()</code> to convert both parameters/strings 6 Check if the divisor, is non-zero ... 7 ... and if so correct calculation and assignment to <code>Value</code> field 8 Return variable of type <code>Result</code> having assigned <u>both</u> fields Max 6 marks	6
------	---	----------

26. O/N 24/P21/Q6/b

6(b)	Example Solution <pre> FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE DECLARE NewDate : DATE DECLARE DD, MM, YY, LastDay : INTEGER DD ← DAY(ProductionDate) MM ← MONTH(ProductionDate) YY ← YEAR(ProductionDate) DD ← DD + ShelfLife LastDay ← DaysInMonth[MM] IF DD > LastDay THEN MM ← MM + 1 DD ← DD - LastDay IF MM = 13 THEN MM ← 1 YY ← YY + 1 ENDIF ENDIF NewDate ← SETDATE(DD, MM, YY) RETURN NewDate ENDFUNCTION </pre> Mark as follows: 1 Declare three variables of TYPE INTEGER to store DD, MM and YY 2 Correct use of date functions from Insert to extract three integer values representing DD, MM and YY and use/assign to a variable 3 Add parameter ShelfLife to DD and use / assign result to a variable 4 Extract LastDay from DaysInMonth using MM as Index 5 Test for new DD after end of month / DaysInMonth[MM] ... and if DD > DaysInMonth[MM]: 6 MM is incremented by 1 and DD is set to DD - DaysInMonth[MM] 7 If MM = 13 / MM > 12 then increment YY and set MM to 1 8 Use of SETDATE to assign value to NewDate, NewDate must have been declared as a type DATE 9 Return date Max 7 marks	7
------	--	----------

27. O/N 24/P21/Q8/a

8(a)	Loop example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN DECLARE Index, Lower, Upper : INTEGER FOR Index ← 1 TO 5 Lower ← GradeBoundary[Index] - 2 Upper ← GradeBoundary[Index] + 2 IF Mark >= Lower AND Mark <= Upper THEN RETURN TRUE ENDIF NEXT Index RETURN FALSE ENDFUNCTION </pre> Mark as follows: 1 Loop through all elements in GradeBoundary array 2 Attempt to calculate range, both Lower and Upper, in a loop 3 Completely correct range calculation in a loop 4 Test if given mark / parameter, is within range in a loop 5 Set variable / immediate RETURN if recheck required in a loop 6 Return Boolean in both cases following a reasonable attempt Selection example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN CASE OF Mark GradeBoundary[1] - 2 TO GradeBoundary[1] + 2 : RETURN TRUE GradeBoundary[2] - 2 TO GradeBoundary[2] + 2 : RETURN TRUE GradeBoundary[3] - 2 TO GradeBoundary[3] + 2 : RETURN TRUE GradeBoundary[4] - 2 TO GradeBoundary[4] + 2 : RETURN TRUE GradeBoundary[5] - 2 TO GradeBoundary[5] + 2 : RETURN TRUE OTHERWISE : RETURN FALSE ENDCASE ENDFUNCTION </pre> Mark as follows: 1 Correct syntax used for selection structure(s) 2 Correct checks for two ranges 3 Correct checks for three ranges 4 Correct checks for four ranges 5 Correct checks for all five ranges 6 Return Boolean in both cases following a reasonable attempt	6
------	---	----------

28. O/N 24/P22/Q2/a,b(ii)

2(a)	Example 'loop solution': <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisk = '*' MaskedString ← RIGHT(CardNumber, 4) FOR Count ← 1 TO LENGTH(CardNumber) - 4 MaskedString ← Asterisk & MaskedString NEXT Count RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used - including the loop counter MP3 Calculate number of digits to mask / number of asterisks required MP4 use of a 'relevant' Loop MP5 Correct number of iterations MP6 Concatenate asterisk to start/end of MaskedString in a loop MP7 Assign last four characters to MaskedString // Concatenate the retained original last four digits MP8 Return masked string Max 6 marks ALTERNATIVE 'non loop' solution: <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisks = "*****" //20 asterisks Count ← LENGTH(CardNumber) - 4 MaskedString ← LEFT(Asterisks, Count) & RIGHT(CardNumber, 4) RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used MP3 Calculate number of digits to mask / number of asterisks required	6
------	--	----------

2(a)	MP4 Trim asterisk string to number calculated in MP3 MP5 Extract the last four characters MP6 Concatenate trimmed asterisk string with last four characters of CardNumber MP7 Return masked string Max 6 marks	
2(b)(ii)	Any reference to BYREF // 'by reference'	1

29. O/N 24/P22/Q4

4	Example solution: <pre> PROCEDURE Timer(Mins, Secs : INTEGER) DECLARE WarningTick, EndTick : INTEGER EndTick ← Tick + 1000 * ((Mins * 60) + Secs) WarningTick ← EndTick - (30 * 1000) REPEAT //do nothing UNTIL Tick = WarningTick OUTPUT "30 seconds to go" REPEAT //do nothing UNTIL Tick = EndTick OUTPUT "The time is up!" ENDPROCEDURE </pre> Mark as follows: MP1 Procedure heading and parameters and ending MP2 'Attempt' to calculate 'total time'/EndTick // 'elapsed time' // WarningTick MP3 Correct calculation of EndTick and WarningTick MP4 (Design mark) - Two separate loops – checking warning time then the final time, OR ... - Single loop checking the final time with an IF statement to check for warning time, OR ... - Single loop with two IF statements checking the warning time and final time MP5 Completely correct MP4 MP6 Output both messages (must be meaningful and follow successful MP4)	6
---	--	----------

30. O/N 24/P22/Q5/c

5(c)	Example solution: <pre>IF Index Mod 2 = 0 THEN ReturnValue ← TO_UPPER(RIGHT(Label, Count)) ELSE ReturnValue ← "*****" ENDIF</pre> Mark as follows: MP1 IF...THEN...ELSE...ENDIF MP2 Both correct assignments and the correct test/logic	2
------	--	----------

31. O/N 24/P23/Q1/c

1(c)	Expression		Evaluates to	3
	Available AND NOT(Index > 100)		TRUE	
	Index MOD 30		10	
	NUM_TO_STR(Index + "33")		ERROR	
	One mark per row			

1(b)	Variable	Example data value	Data type	3
	Available	TRUE	BOOLEAN	
	Received	"18/04/2021"	STRING	
	Index	100	INTEGER	
	One mark per row			

32. O/N 24/P23/Q6

6	Loop Solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE ThisDayNumber, SundayCount : INTEGER DECLARE ThisDate : DATE ThisDayNumber ← 0 SundayCount ← 0 REPEAT ThisDayNumber ← ThisDayNumber + 1 ThisDate ← SETDATE(ThisDayNumber, 3, ThisYear) IF DAYINDEX(ThisDate) = 1 THEN SundayCount ← SundayCount + 1 ENDIF UNTIL SundayCount = 3 RETURN ThisDayNumber ENDFUNCTION </pre> Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to create a date MP3 Loop until 3rd Sunday found MP4 Attempt to use both SETDATE() and DAYINDEX() in a loop MP5 Correctly generate value of type DATE using SETDATE() in a loop MP6 Test if value represents a Sunday using DAYINDEX() in a loop MP7 Increment Sunday count in a loop and initialised correctly before loop MP8 Return day number of third Sunday Max 7 marks Non-loop solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE FirstDayIndex: INTEGER DECLARE ThisDate : DATE ThisDate ← SETDATE(1, 3, ThisYear) FirstDayIndex ← DAYINDEX(ThisDate) IF FirstDayIndex = 1 THEN ThirdSunday ← FirstDayIndex + 14 //First day is Sunday ELSE ThirdSunday ← 23 - FirstDayIndex // Other days ENDIF RETURN ThirdSunday ENDFUNCTION </pre>	7
---	--	---

6	Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to hold a day number MP4 Attempt to use both <code>SETDATE()</code> and <code>DAYINDEX()</code> MP5 Correctly generate value of type <code>DATE</code> using <code>SETDATE()</code> for first of March / specific day in March MP6 Test if date represents a Sunday / specific day using <code>DAYINDEX()</code> and calculate third Sunday // Correctly calculate third Sunday for a value returned by <code>DAYINDEX()</code> for one day MP7 Calculate third Sunday for other six days MP8 Return day number of third Sunday Max 7 marks	
---	---	--

33. M/J 24/P21/Q1/b

1(b)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>Tries < 10 AND NOT Sorted</td><td>TRUE</td></tr><tr><td>Tries MOD 4</td><td>1</td></tr><tr><td>TO_LOWER(MID(ID, 3, 1))</td><td>'a' // "a"</td></tr><tr><td>LENGTH(ID & "xx") >= Tries</td><td>TRUE</td></tr></table>		Expression	Evaluates to	Tries < 10 AND NOT Sorted	TRUE	Tries MOD 4	1	TO_LOWER(MID(ID, 3, 1))	'a' // "a"	LENGTH(ID & "xx") >= Tries	TRUE	4
	Expression	Evaluates to											
	Tries < 10 AND NOT Sorted	TRUE											
	Tries MOD 4	1											
	TO_LOWER(MID(ID, 3, 1))	'a' // "a"											
	LENGTH(ID & "xx") >= Tries	TRUE											
One mark per row													

34. M/J 24/P21/Q2/b

2(b)	One mark per point: 1 A (variable of type) string will be input // by example e.g. "67,72" 2 A special / identified character would need to be used to separate each numeric value // all numbers are fixed length	2
------	---	---

35. M/J 24/P21/Q6/a

6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR (Len2 = Len1 + Len3) OR (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 One correct comparison of square of three lengths 4 All three comparisons... 5 combined using logical operators / nested IF // completely correct selection 6 Return result correctly in both cases	6
------	--	----------

36. M/J 24/P21/Q8/a

8(a)	<pre> FUNCTION DeleteComment(Line : STRING) RETURNS STRING DECLARE NewLine, TwoChars : STRING DECLARE Count, TrimTo : INTEGER CONSTANT Comment = "//" NewLine ← Line TrimTo ← 0 Count ← 1 WHILE Count < LENGTH(Line) AND TrimTo = 0 TwoChars ← MID(Line, Count, 2) // extract 2 chars IF TwoChars = Comment THEN TrimTo ← Count ENDIF Count ← Count + 1 ENDWHILE IF TrimTo <> 0 THEN NewLine ← LEFT(Line, TrimTo - 1) ENDIF RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of <code>Line</code> (parameter) // length of <code>Line</code> -1 2 Terminate loop on first double slash 3 Attempt to extract one/two characters in a loop 4 Check for <code>"//"</code> after attempt at extraction in a loop 5 Record start position of comment // Calculate amount of trim required 6 Attempt to trim <code>Line</code> from start of comment 7 Completely correct trimmed <code>Line</code> from start of comment 8 Return <code>Line</code> after a reasonable overall attempt	8
------	---	---

37. M/J 24/P22/Q1/b

1(b)			4
		Expression	
		MyInt ← INT (3.1415926)	
		MyChar ← MID ("Elwood", 3, 1)	
		Any of: • MyString ← NUM_TO_STR (INT (27.509)) • MyString ← CHR (INT (27.509)) • MyString ← TO_UPPER(NUM_TO_STR(27.509)) • MyString ← TO_LOWER(NUM_TO_STR(27.509))	
		Any of: • MyInt ← STR_TO_NUM (RIGHT ("ABC123", 3)) • MyInt ← LENGTH (RIGHT ("ABC123", 3)) • MyInt ← LENGTH (LEFT ("ABC123", 3))	
One mark per row			

38. M/J 24/P22/Q4

4	Example solution: <pre> PROCEDURE IsRA() DECLARE a, b, c : INTEGER OUTPUT "Input length of the first side" INPUT a OUTPUT "Input length of the second side" INPUT b OUTPUT "Input length of the third side" INPUT c IF (a * a = (b * b) + (c * c)) OR (b * b = (a * a) + (c * c)) OR (c * c = (a * a) + (b * b)) THEN OUTPUT "It is right-angled" ELSE OUTPUT "Not right-angled" ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending and declaration of all variables used 2. Appropriate prompt and input for each length 3. One correct length test 4. All three length tests // selection of which test is required 5. Output one of two messages following a reasonable attempt at MP3	5
---	---	---

39. M/J 24/P22/Q5/a(ii)

5(a)(ii)	One mark per point: Statement: • The OTHERWISE statement Explanation: • The result of MOD 2 can only be 0 or 1	2
----------	--	---

40. M/J 24/P22/Q6/a

6(a)	Example solution: <pre> Function Trim(Name : STRING) RETURNS STRING CONSTANT Dots = "... " CONSTANT Space = " " IF LENGTH(Name) <= 16 THEN RETURN Name ENDIF // Otherwise it has to be trimmed WHILE LENGTH(Name) > 13 REPEAT Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char UNTIL RIGHT(Name, 1) = Space // back to SPACE ENDWHILE Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space Name ← Name & Dots RETURN Name ENDFUNCTION </pre> Mark as follows: 1. Function heading, ending, parameter and return type 2. If length of original string <= 16 then return original string 3. Any Conditional loop... 4. until string is short enough 5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string 6. Attempt to strip characters back to space 7. Correct removal of word/words from end of string and remove final space 8. Concatenate <code>Dots</code> and return result Max 7 marks	7
------	---	----------

41. M/J 24/P22/Q8/b

8(b)	Example algorithm based on finding position of first non-space character and then using substring function: <pre> FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING DECLARE NewLine : STRING DECLARE EndOfLeading : BOOLEAN DECLARE Count, NumSpaces : INTEGER DECLARE NextChar : CHAR CONSTANT Space = " " NumSpaces ← 0 EndOfLeading ← FALSE FOR Count ← 1 TO LENGTH(Line) NextChar ← MID(Line, Count, 1) IF NextChar <> Space AND EndOfLeading = FALSE THEN NumSpaces ← Count - 1 // the number to trim EndOfLeading = TRUE ENDIF NEXT Count NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces) RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of parameter // Loop until first non-space character in Line 2 Extract a character in a loop 3 Identify <u>first</u> non-space character in a loop 4 Attempt at removing leading spaces in Line 5 Leading spaces removed from Line // Create new string without leading space 6 Return a string following a reasonable attempt at removing leading spaces in Line	6
------	--	----------

42. M/J 24/P23/Q1/a,b

1(a)(i)	Two marks for the benefits: 1. The code can be called when needed 2. Any subsequent change to the algorithm / calculation needs to be made once only // Easier to manage / maintain (the program) 3. Less code / no code duplication 4. The algorithm / code / calculation can be designed / coded / tested once Note: Max 2 marks	2
---------	---	----------

1(a)(ii)	One mark per point: 1. Create a module / function / procedure / subroutine using the (old) code / algorithm 2. Replace the old code / algorithm wherever it appears with a call to the module / function / procedure / subroutine 3. Pass the data as parameter(s) // Use of <u>global variable(s)</u> 4. <u>Return</u> the result (of the calculation) // Assign result to <u>global variable(s)</u> / <u>BYREF parameter(s)</u> 5. Create <u>local</u> variables as needed (to perform the calculation) Note: Max 3 marks	3										
1(b)	<table><thead><tr><th>Pseudocode expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>MID ("Random", 2, 3)</td><td>"and"</td></tr><tr><td>5 + DAY (10/11/2023)</td><td>15</td></tr><tr><td>IS_NUM ("45000")</td><td>TRUE</td></tr><tr><td>(20 MOD 3) + 1</td><td>3</td></tr></tbody></table> One mark per row	Pseudocode expression	Evaluates to	MID ("Random", 2, 3)	"and"	5 + DAY (10/11/2023)	15	IS_NUM ("45000")	TRUE	(20 MOD 3) + 1	3	4
Pseudocode expression	Evaluates to											
MID ("Random", 2, 3)	"and"											
5 + DAY (10/11/2023)	15											
IS_NUM ("45000")	TRUE											
(20 MOD 3) + 1	3											

43. M/J 24/P23/Q4/a

4(a)	Example solution: <pre> PROCEDURE TwoParts() DECLARE NextNum, Average : REAL TotalA ← 0.0 // 0 TotalB ← 0.0 // 0 REPEAT INPUT NextNum TotalA ← TotalA + NextNum UNTIL NextNum = 0 REPEAT INPUT NextNum TotalB ← TotalB + NextNum UNTIL NextNum = 0 Average ← (TotalA + TotalB) / 2 OUTPUT "The average is ", Average ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending 2. Declare all local variables 3. Initialise <code>TotalA</code> and <code>TotalB</code> 4. First conditional loop until zero entered, summing <code>TotalA</code> // Loop until both parts (sequences) have been entered 5. Second conditional loop until zero entered, summing <code>TotalB</code> // Loop summing appropriate Totals 6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message	6
------	--	----------

44. M/J 24/P23/Q5/a,b

5(a)	One mark per point • Count-controlled loop • the number of iterations is known	2
5(b)	Two mark for Statement of Problem: 1 The functions will return the same value every time they are called 2 ... because <code>Label</code> / the parameter value does not change within the loop Two marks for Solution: 3 Assign <code>FormatA(Label)</code> and <code>FormatB(Label)</code> to two (local) variables <u>before</u> the loop 4 Use the (new) variables in place of the function calls / in the loop // Replace the references to <code>FormatA()</code> and <code>FormatB()</code> in the CASE clauses with the new variable names	4

45. M/J 24/P23/Q6/a,b(i)

6(a)	Example Solutions: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING StepValue ← (Percent DIV 10) + 1 // INT(Percent / 10) + 1 FileName ← Root & "-" & NUM_TO_STR(StepValue) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Alternative: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING DECLARE Found : BOOLEAN StepValue ← 1 Found ← FALSE REPEAT IF Percent < StepValue * 10 THEN Found ← TRUE ENDIF StepValue ← StepValue + 1 UNTIL Found Filename ← Root & "-" & NUM_TO_STR(StepValue - 1) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for use of DIV/INT or loop solution: 1 Procedure heading, parameters and ending 2 Calculate StepValue as integer value // Attempt at calculating file number 3 Add 1 to obtain file number // Completely correct file number calculation 4 Use of NUM_TO_STR() to convert file number to string and use 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() with filename as parameter following a reasonable attempt	6
------	--	----------

6(a)	Example Selection Solution: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE Filename, StepValue : STRING CASE OF Percent < 10 : StepValue ← "1" < 20 : StepValue ← "2" < 30 : StepValue ← "3" < 40 : StepValue ← "4" < 50 : StepValue ← "5" < 60 : StepValue ← "6" < 70 : StepValue ← "7" < 80 : StepValue ← "8" < 90 : StepValue ← "9" < 100 : StepValue ← "10" OTHERWISE : StepValue ← "11" ENDCASE Filename ← Root & "-" & StepValue & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for loop solution: 1 Procedure heading, parameters and ending 2 Correct selection construct(s) structure 3 Use of selection statement to obtain two file numbers 4 Use of selection statement to obtain all the file numbers 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() once with filename as parameter following a reasonable attempt	
6(b)(i)	Example answer: <pre> FUNCTION Progress2(Percent, Steps : INTEGER, Root : STRING) RETURNS STRING </pre> One mark for each: • Keyword <code>FUNCTION Progress2</code> and three parameters of correct type • Keyword <code>RETURNS</code> and type string	2

46. M/J 24/P23/Q7/a(ii)

7(a)(ii)	Points required: 1 any change made to the parameter value / Par3 within Mod-G() is reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-G() terminates) 2 any change made to the parameter value / Par3 within Mod-F() is NOT reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-F() terminates) Mark as follows: 1 mark for a reasonable attempt to explain 2 marks for full explanation including context	2
----------	--	----------

47. M/J 24/P23/Q8/a

8(a)	Example: <pre> FUNCTION Header(Line : STRING) RETURNS STRING IF LENGTH(Line) >= 13 THEN IF TO_UPPER(LEFT(Line, 9)) = "FUNCTION " THEN RETURN "F" & RIGHT(Line, LENGTH(Line) - 9) ENDIF IF TO_UPPER(LEFT(Line, 10)) = "PROCEDURE " THEN RETURN "P" & RIGHT(Line, LENGTH(Line) - 10) ENDIF ENDIF RETURN "" ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameter, return type and ending 2 Check that the line is at least 13 characters long before attempting to extract and return empty string 3 Attempt at: Extract characters, 9 or 10 characters, corresponding to keyword plus space and compare with appropriate keyword plus space 4 Completely correct MP3 5 Use of type case conversion to allow for 'any case' 6 Calculation of 'rest of line' and concatenation with 'P or 'F' 7 Return string	7
------	---	----------

48. O/N 23/P21/Q1/a,b,c

1(a)	One mark per row:		4
		Answer	
	The value assigned to Level when ThisValue is 40	"Medium"	
	The value assigned to Check when ThisValue is 36	12	
	The value assigned to Level when ThisValue is 18	"Low"	
	The number of elements in array Data that may be incremented	11	

1(b)	One mark for identifying assignment: MP1 <code>Level ← "Very Low"</code> // the level is assigned value "very low" Explanation points: MP2 because CASE clauses are checked in sequence // because of the order of the clauses MP3 a value < 30 satisfies the first clause // Clause '<u>< 20</u>' will never be tested	3
1(c)	MP1 all of the possible values are addressed via all / four / three / the other clauses // there are no other possible values to map to OTHERWISE	1

49. O/N 23/P21/Q6/a,b

6(a)	<pre> PROCEDURE MyOutput(NewString : STRING, EOL : BOOLEAN) IF LENGTH(MyString) + LENGTH(NewString) > 255 THEN OUTPUT MyString // Resulting string would be too long MyString ← NewString ELSE MyString ← MyString & NewString // Concat with MyString IF EOL = TRUE THEN OUTPUT MyString MyString ← "" ENDIF ENDIF ENDPROCEDURE </pre> MP1 Procedure heading, including parameters, and ending MP2 Produce concatenated string MP3 ... Check whether resulting string would be too long MP4 If so, then output old MyString MP5 ... and assign NewString to MyString MP6 Else concatenate NewString to MyString MP7 (test for length < 255) Test EOL – If TRUE then Output MP8 ... and reset MyString to empty string	7
6(b)	MP1 A new (instance of) variable MyString is created each time the procedure is called / executed MP2 So the previous contents are lost	2

50. O/N 23/P21/Q8/d

8(d)	<pre> FUNCTION GetField(Msg : STRING, FieldNo : INTEGER) RETURNS STRING DECLARE RetString : STRING CASE OF FieldNo 1 : RetString ← MID(Msg, 2, 3) 2 : RetString ← MID(Msg, 5, 3) 3 : RetString ← MID(Msg, 8, LENGTH(Msg) - 8) OTHERWISE : RetString ← "" ENDCASE RETURN RetString ENDFUNCTION </pre> MP1 Use of CASE ... ENDCASE or IF ... THEN ... ENDIF MP2 Field 1 and Field 2 extracted correctly MP3 Calculate a length of field 3 MP4 Field 3 extracted <u>correctly</u> MP5 Return empty string in case of invalid parameter (via OTHERWISE or initialisation) MP6 Final RETURN, after a reasonable attempt	6
------	--	----------

51. O/N 23/P22/Q1/b

1(b)	One mark per row			4
		Expression	Evaluates to	
		(Total * DepRate) + 1.5	249.50	
		RIGHT(Description, 7)	"(small)"	
		(LENGTH(Description) - 8) > 16	TRUE	
		NUM_TO_STR(INT(DepRate * 10)) & '%'	"20%"	

52. O/N 23/P22/Q2/b

2(b)	One mark per point: - Name: (pre / post) conditional loop - Justification: the number of iterations is not known // loop ends following a specific input (in the loop)	2
------	---	----------

53. O/N 23/P22/Q4/a

4(a)	Example Solution <pre> PROCEDURE Count() DECLARE COdd, CEven, ThisNum : INTEGER COdd ← 0 CEven ← 0 INPUT ThisNum WHILE ThisNum <> 99 IF ThisNum MOD 2 = 1 THEN COdd ← COdd + 1 ELSE CEven ← CEven + 1 ENDIF INPUT ThisNum ENDWHILE OUTPUT "Count of odd and even numbers: ", COdd, CEven ENDPROCEDURE </pre> Mark as follows Max 6 marks: 1 Procedure heading and ending 2 Local COdd, CEven and ThisNum declared as integers 3 Conditional loop while ThisNum <> 99 4 Input ThisNum in a loop 5 Check ThisNum is odd or even in a loop 6 Increment appropriate count in a loop, both counts must have been initialised before loop 7 After the loop output COdd and CEven with a suitable message following a reasonable attempt at counting	6
------	---	----------

54. O/N 23/P22/Q6/b(i,ii)

6(b)(i)	Function	1
6(b)(ii)	FUNCTION CheckFiles (NameRoot : STRING) RETURNS INTEGER	1

6(a)	Example Solution <pre>PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER) DECLARE FileName, Suffix : STRING DECLARE Count : INTEGER FOR Count ← 1 TO NumFiles Suffix ← NUM_TO_STR(Count) WHILE LENGTH(Suffix) <> 3 Suffix ← '0' & Suffix ENDWHILE FileName ← NameRoot & '.' & Suffix OPENFILE FileName FOR WRITE WRITEFILE FileName, "This is File " & FileName CLOSEFILE FileName NEXT Count ENDPROCEDURE</pre> Mark as follows: 1 Procedure heading, including parameters, and ending 2 Loop for NumFiles iterations 3 Attempt to create filename suffix using NUM_TO_STR() in a loop 4 Completely correct filename 5 OPENFILE in WRITE mode and subsequent CLOSE in a loop 6 WRITE initial first line to the file in a loop	6
------	--	----------

55. O/N 23/P22/Q8/b

8(b)	Example solution <pre> PROCEDURE ProcessMsg(ThisMsg : STRING) DECLARE ThisDest : STRING DECLARE Response : BOOLEAN DECLARE StackNum : INTEGER IF LENGTH(ThisMsg) >= 4 THEN ThisDest ← LEFT(ThisMsg, 3) IF ThisDest = MyID THEN // It's for this computer StackNum ← 1 ELSE StackNum ← 2 ENDIF Response ← StackMsg(ThisMsg, StackNum) IF Response = FALSE THEN OUTPUT "Message discarded - no room on stack" ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Ignore message if data field is empty 2. Extract <code>ThisDest</code> from <code>ThisMsg</code> 3. Test if destination is this computer 4. Attempt to use <code>StackMsg()</code> 5. Fully correct use of <code>StackMsg()</code> for both cases / stacks 6. Test <code>StackMsg()</code> return value for both cases 7. Following a reasonable attempt at MP6 output warning if <code>StackMsg()</code> returns <code>FALSE</code>	7
------	---	----------

56. O/N 23/P23/Q1/b,c

1(b)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>Fraction >= 0.2 AND NOT Active</td><td>FALSE</td></tr><tr><td>INT((Fraction1 * 100) + 13.3)</td><td>33</td></tr><tr><td>STR_TO_NUM(MID(Code, 4, 2)) + 5</td><td>28</td></tr><tr><td>LENGTH("TRUE" & Code)</td><td>11</td></tr></table>		Expression	Evaluates to	Fraction >= 0.2 AND NOT Active	FALSE	INT((Fraction1 * 100) + 13.3)	33	STR_TO_NUM(MID(Code, 4, 2)) + 5	28	LENGTH("TRUE" & Code)	11	4
	Expression	Evaluates to											
	Fraction >= 0.2 AND NOT Active	FALSE											
	INT((Fraction1 * 100) + 13.3)	33											
	STR_TO_NUM(MID(Code, 4, 2)) + 5	28											
LENGTH("TRUE" & Code)	11												
1(c)	The use of a program <u>library</u> (routines)	1											

57. O/N 23/P23/Q4/a,b

4(a)	<pre> PROCEDURE RandList() DECLARE Count, BaseNum, ThisNum : INTEGER CONSTANT StepVal = 10 BaseNum ← 0 FOR Count ← 1 TO 25 ThisNum ← BaseNum + INT(RAND(StepVal)) OUTPUT ThisNum BaseNum ← BaseNum + StepVal NEXT Count ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 <u>Local</u> loop counter Count as integer MP3 Loop to iterate 25 times or more for each unique number MP4 'Attempt' to generate a random number including use of INT() in a loop MP5 Ensure that number generated is greater than previous and change 'previous' MP6 Output random number after an attempt at MP5 in a loop	6
4(b)	One mark for simplified logical expression: MP1 Result[x + 1] <= Result[x] ALTERNATIVE SOLUTION: Result[x] >= Result[x + 1]	1

58. O/N 23/P23/Q6

6	<pre> FUNCTION TestNum(ThisNum : STRING) RETURNS INTEGER IF LEFT(ThisNum,3) = RIGHT(ThisNum 3) THEN RETURN 3 ENDIF IF RIGHT(ThisNum, 3) = "000" THEN RETURN 2 ENDIF IF MID(ThisNum, 4, 1) = MID(ThisNum, 5, 1) AND MID(ThisNum, 5, 1) = MID(ThisNum, 6, 1) THEN RETURN 1 ENDIF RETURN 0 ENDFUNCTION </pre> MP1 Function heading and ending including parameter and return type MP2 Test for Condition 1 MP3 Test for Condition 2 MP4 Test for Condition 3 MP5 Return the highest value if more than one condition is satisfied MP6 Return zero if no condition matched	6
---	--	---

59. O/N 23/P23/Q7/b(ii)

7(b)(ii)	<pre> PROCEDURE Sub-9 (BYREF Param : MyType) </pre> MP 1 One mark for BYREF MP 2 One mark for the rest of the statement	2
----------	---	---

60. O/N 23/P23/Q8/c

8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 <code>OUTPUT</code> the data in a loop MP5 <code>INPUT</code> the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

61. M/J 23/P21/Q1/b,c

1(b)	One mark for correct description of: Error 1: Brackets mismatch // 2/value should be added after brackets/function // Addition between a string and a number is not valid // <code>STR_TO_NUM</code> / the function needs to be passed a string / not an integer Error 2: No Error Error 3: <code>MONTH()</code> returns an integer and this is being compared with a character/string // Integer cannot be compared to a string // 6 should not be in quotes	3
------	---	---

1(c)(i)			2
		Expression	Evaluation
	1	(Points > 99) OR Active	TRUE
	2	(Points MOD 2 = 0) OR Exempt	FALSE
	3	(Points <= 75) AND (Active OR Exempt)	TRUE
	4	(Active OR NOT Active) AND NOT Exempt	TRUE
One mark for any two rows correct Two marks for all rows correct			
1(c)(ii)	NOT Exempt		1

62. M/J 23/P21/Q2/b

2(b)	One mark for declaration: <u>DECLARE</u> <u>StartDate</u> : <u>DATE</u> One mark for each underlined part of assignment: <u>StartDate</u> ← <u>SETDATE</u> (15, 11, 2005)	3
------	---	----------

63. M/J 23/P21/Q4

4	<pre>Function Replace(OldString : STRING, Char1, Char2 : CHAR) __ RETURNS : STRING DECLARE NewString : STRING DECLARE ThisChar : CHAR DECLARE Index : INTEGER NewString ← "" FOR Index ← 1 TO LENGTH(OldString) ThisChar ← MID(OldString, Index, 1) IF ThisChar = Char1 THEN ThisChar ← Char2 ENDIF NewString ← NewString & ThisChar NEXT Index RETURN NewString ENDFUNCTION</pre> Mark as follows: - Function heading and ending, including parameters and return type - Declaration of local variables used including loop counter - Loop for length of OldString - Extract char and test in a loop - Use of concatenate to build NewString replace char if necessary, in a loop - Return NewString after reasonable attempt	6
---	---	----------

64. M/J 23/P21/Q8/a

8(a)	<pre> FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z' THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Check <code>ThisString</code> is exactly 5 characters in length 2 Use of <code>SuppExists()</code> with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character in a loop 5 Test if char is alphabetic in a loop 6 ...catering for both upper and lower case 7 Return Boolean following a reasonable attempt	7
------	---	----------

65. M/J 23/P22/Q1/a

1(a)(i)	Use of constants	1
1(a)(ii)	One mark per bullet point (or equivalent to max 3): 1 Postal rates are entered once only 2 Avoids input error / changing the cost accidentally // avoids different values for postal rates at different points in the program 3 When required, the constant representing the postal rate value is changed once only // easier to maintain the program when the postal rates change 4 Makes the program easier to understand Note: Max 3 marks	3

66. M/J 23/P22/Q2/a,b

2(a)	<code>MyDOB ← SETDATE(17, 11, 2007)</code>	1
------	--	----------

2(b)	<u>NumMonths</u> $\leftarrow$ <u>12 - MONTH(MyDOB)</u>	2
	One mark per underlined part	

67. M/J 23/P22/Q4

4	<pre> Function GetNum(ThisString : STRING, ThisChar : CHAR) RETURNS INTEGER DECLARE Index, Count : INTEGER Count $\leftarrow$ 0 FOR Index $\leftarrow$ 1 TO LENGTH(ThisString) IF MID(ThisString, Index, 1) = ThisChar THEN Count $\leftarrow$ Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Function heading and end, including parameters and return type 2 Declare local Integers for Index and Count 3 Loop for length of ThisString 4 Extract a character and compare with <u>parameter</u> in a loop 5 Increment Count if match in a loop 6 Return Count after loop	6
---	---	----------

68. M/J 23/P22/Q6

6	Example of iterative solution: <pre> PROCEDURE Square(Dim : INTEGER) DECLARE Count : INTEGER DECLARE ThisChar : CHAR DECLARE StringA, StringB : STRING CONSTANT FILLER = '*' StringA ← "" ThisChar ← NUM_TO_STR(Dim) FOR Count ← 1 TO Dim StringA ← StringA & ThisChar //build up first & last line NEXT Count StringB ← ThisChar FOR Count ← 1 TO Dim - 2 StringB ← StringB & FILLER //build up intermediate line NEXT Count StringB ← StringB & ThisChar // add final digit OUTPUT StringA FOR Count ← 1 TO Dim - 2 OUTPUT StringB NEXT Count IF Dim <> 1 THEN OUTPUT StringA ENDIF ENDPROCEDURE </pre> For loop-based solutions, mark as follows: 1 Procedure heading and ending including parameter 2 Loop using parameter, containing attempt to construct first line / last line 3 Construct first line / last line 4 Attempt at loop to construct intermediate line 5 Output first / last line of square when <code>Dim > 2</code> 6 Output all intermediate lines in a loop 7 Correct output of first two squares Note: Max 6 marks	6
---	--	---

6

Example of selection-based solution:

```

PROCEDURE Square(Dim : INTEGER)
  DECLARE Count : INTEGER

  CASE OF Dim
    1 : OUTPUT "1"
    2 : OUTPUT "22"
      OUTPUT "22"
    3 : OUTPUT "333"
      OUTPUT "3*3"
      OUTPUT "333"
    4 : OUTPUT "4444"
      FOR Count ← 1 TO 2
        OUTPUT "4**4"
      NEXT Count
      OUTPUT "4444"
    5 : OUTPUT "55555"
      FOR Count ← 1 TO 3
        OUTPUT "5***5"
      NEXT Count
      OUTPUT "55555"
    6 : OUTPUT "666666"
      FOR Count ← 1 TO 4
        OUTPUT "6****6"
      NEXT Count
      OUTPUT "666666"
    7 : OUTPUT "7777777"
      FOR Count ← 1 TO 5
        OUTPUT "7*****7"
      NEXT Count
      OUTPUT "7777777"
    8 : OUTPUT "88888888"
      FOR Count ← 1 TO 6
        OUTPUT "8*****8"
      NEXT Count
      OUTPUT "88888888"
    9 : OUTPUT "999999999"
      FOR Count ← 1 TO 7
        OUTPUT "9*****9"
      NEXT Count
      OUTPUT "999999999"

  ENDCASE

ENDPROCEDURE

```

For in-line / selection-based solutions, mark as follows:

- 1 Procedure heading and ending including parameter
- 2 Correct use of parameter to select all required squares
- 3 Correct output of first two squares
- 4 At least six squares correctly output
- 5 At least one loop to output lines with multiple asterisks
- 6 All number squares output correctly

69. M/J 23/P22/Q8/a

8(a)	<pre> FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are all numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	7
------	--	----------

70. M/J 23/P23/Q2/a

2(a)	One mark for each underlined part <pre> IF <u>DAYINDEX(MyDOB)</u> = <u>5</u> THEN </pre>	2
------	--	----------

71. M/J 23/P23/Q4

4	<pre> FUNCTION MakeString(Count : INTEGER, AChar : CHAR) RETURNS STRING DECLARE MyString : STRING DECLARE Index : INTEGER IF Count < 1 THEN MyString ← "ERROR" ELSE MyString ← "" FOR Index ← 1 TO Count MyString ← MyString & AChar NEXT Index ENDIF RETURN MyString ENDFUNCTION </pre> MP1 Function heading and end including parameters and return type MP2 Declaration of locals <code>Index</code> and <code>MyString</code> MP3 Test for <code>Count < 1</code> and if true, assign "ERROR" to <code>MyString</code> / Immediate RETURN MP4 Loop for <code>Count</code> iterations MP5 Use of concatenate – must have been initialised in a loop MP6 Return STRING (correctly in both cases)	6
---	--	---

72. M/J 23/P23/Q6/a,b

6(a)	Max 7 marks <pre> PROCEDURE Select(Start, End : INTEGER) DECLARE ThisNum, Total: INTEGER DECLARE ThisString : STRING DECLARE Char1, Char2 : CHAR FOR ThisNum ← Start+1 TO End-1 ThisString ← NUM_TO_STR(ThisNum) Char1 ← RIGHT(ThisString, 1) Char2 ← LEFT(RIGHT(ThisString, 2), 1) Total ← STR_TO_NUM(Char1) + STR_TO_NUM(Char2) IF Total = 6 THEN OUTPUT ThisString ENDIF NEXT ThisNum ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 (Count-controlled) Loop MP3 with correct range from Start+1 to End-1 MP4 Convert ThisNum (loop counter) to a string MP5 Extract the last two/first/second 'character digit(s)' required in a loop MP6 Extract the second individual 'character digit' required in a loop MP7 Calculate the sum of the last two digits MP8 If sum = 6 then OUTPUT the number (either string or integer) in a loop	7				
6(b)	Max 4 marks MP1 The function will take two integer parameters - the number and the (required) total MP2 ... and return a Boolean OR: <table> <tr> <td><u>CheckNum(Number, Total : INTEGER)</u></td> <td><u>RETURNS BOOLEAN</u></td> </tr> <tr> <td>MP1</td> <td>MP2</td> </tr> </table> Two marks for the advantages: MP3 CheckNum() can be called repeatedly as and when required MP4 CheckNum() is designed and tested once (then used repeatedly) MP5 Any subsequent change to <u>CheckNum()</u> needs to be made once only // is easier to maintain/modify	<u>CheckNum(Number, Total : INTEGER)</u>	<u>RETURNS BOOLEAN</u>	MP1	MP2	4
<u>CheckNum(Number, Total : INTEGER)</u>	<u>RETURNS BOOLEAN</u>					
MP1	MP2					

73. O/N 22/P21/Q1/a,c

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example encryption) 3 They are readily available / speed up development time	3
------	---	---

1(c)	One mark per row: <table><tr><th>Statement</th><th>Error</th></tr><tr><td>IF EMPTY ← "" THEN</td><td>Should be "=" not ←</td></tr><tr><td>Status ← IS_NUM(-23.4)</td><td>Parameter should be a string (or char) // should not be a real</td></tr><tr><td>X ← STR_TO_NUM("37") + 5</td><td>NO ERROR</td></tr><tr><td>Y ← STR_TO_NUM("37" + "5")</td><td>Wrong operator – should be & or Parameter is not a string</td></tr></table>	Statement	Error	IF EMPTY ← "" THEN	Should be "=" not ←	Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real	X ← STR_TO_NUM("37") + 5	NO ERROR	Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string	4
Statement	Error											
IF EMPTY ← "" THEN	Should be "=" not ←											
Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real											
X ← STR_TO_NUM("37") + 5	NO ERROR											
Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string											

74. O/N 22/P21/Q6/a

6(a)	One mark for any part correct (accept equivalent wording) (Max 1): • Condition evaluates to TRUE if bracket contents evaluate to FALSE: • Bracket contents evaluate to FALSE if: • Dots: zero / less than one or • Ats: not equal to one or • Others: less than nine	1
------	---	----------

75. O/N 22/P21/Q7/a

7(a)(i)	One mark per point (Max 7) as follows: 1 Declaration of local variables for Par1 Par2 and Par3 2 Loop to end of (parameter) string // until operator is found 3 Extract a character in a loop... 4 Attempt at extraction of three parts of expression using substring functions 5 Completely correct extraction of all three parts of expression 6. Convert string to Integer using STR_TO_NUM(<something sensible>) 7 Attempt to interpret at least two operators (Par2): + - * / 8 Corresponding correct calculation (all operators) and final Output of result <pre> PROCEDURE Calculate(Expression : STRING) DECLARE Val1, Val2, Index : INTEGER DECLARE Result : REAL DECLARE Par1, Par2, Par3 : STRING CONSTANT PLUS = '+' CONSTANT MINUS = '-' CONSTANT MULTIPLY = '*' CONSTANT DIVIDE = '/' FOR Index ← 1 TO LENGTH(Expression) //search for operator ThisChar ← MID(Expression, Index, 1) IF IS_NUM(ThisChar) = FALSE THEN Par1 ← LEFT(Expression, Index - 1) Par2 ← ThisChar Par3 ← RIGHT(Expression, LENGTH(Expression) - Index) ENDIF NEXT Index Val1 ← STR_TO_NUM(Par1) Val2 ← STR_TO_NUM(Par3) CASE OF Par2 PLUS : Result ← Val1 + Val2 MINUS : Result ← Val1 - Val2 MULTIPLY : Result ← Val1 * Val2 DIVIDE : Result ← Val1 / Val2 ENDCASE OUTPUT Result ENDPROCEDURE </pre>	7
7(a)(ii)	FUNCTION Calculate(Expression : STRING) RETURNS REAL	1

76. O/N 22/P21/Q8/b

8(b)	<code>CALL CountErrors("Jim01Prog.txt", 20)</code> One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	2
------	---	----------

77. O/N 22/P22/Q1/a,c

1(a)(i)	One mark for each point (Max 2): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	2
---------	---	----------

1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2
----------	---	----------

1(c)	One mark per row:		3
	Expression	Evaluation	
	MID(CharList, MONTH(FlagDay), 1)	'D'	
	INT(Count / LENGTH(CharList))	4	
	(Count >= 99) AND (DAY(FlagDay) > 23)	FALSE	

78. O/N 22/P22/Q4/b

4(b)	Pre-condition (loop) / count-controlled (loop)	1
------	--	----------

79. O/N 22/P22/Q6/a

6(a)	Example by repeated multiplication: Mark as follows (multiplication solution), (Max 7): 1 Function heading and ending including parameter and return type 2 Declaration and initialisation of local Num 3 Any conditional loop 4 Conditional loop until ThisValue found or Try out of range 5 Multiply Try by Num in a loop 6 Compare Try with ThisValue and set termination if the same in a loop 7 Increment Num and repeat in a loop 8 Attempt to Return Num if ThisValue is a factorial or -1 otherwise <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER DECLARE Found : BOOLEAN Num ← 0 Found ← FALSE Try ← 1 WHILE Try <= ThisValue AND Found = FALSE Num ← Num + 1 Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found Found ← TRUE ENDIF ENDWHILE IF Found = TRUE THEN RETURN Num ELSE RETURN -1 ENDIF ENDFUNCTION </pre>	7
------	---	----------

6(a)	Alternative FOR LOOP solution. Mark as follows: 1 Function heading and ending including parameter and return type 2 Declaration of local Integer value for Num and Try 3 Count-controlled Loop from 1 to ThisValue 4 Multiply Try by Num in a loop 5 Compare Try with ThisValue in a loop 6 ...Immediate return of Num if they are the same in a loop 7 Return -1 if ThisValue not found after loop <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER Try ← 1 FOR Num ← 1 TO ThisValue Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found RETURN Num ENDIF NEXT Num RETURN -1 ENDFUNCTION </pre>	
------	---	--

80. O/N 22/P23/Q5/a,b(ii)

5(a)(i)	One mark per point: 1 Procedure heading and ending including four parameters... 2 ...and use of BYREF for the three extracted values 3 Extract and assign SID 4 Extract and assign SDesc 5 Calculation of length of SCost (remainder of string) 6 Extract and assign SCost following an attempt at MP5 <pre> PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING) SID ← LEFT(TLine, 5) SDesc ← MID(TLine, 6, 32) SCost ← RIGHT(TLine, LENGTH(TLine) - 37) ENDPROCEDURE </pre>	6
5(a)(ii)	One mark each (Max 2): 1 Provides a mechanism to allow calling program to pass data 2 Defines the four parameters of Unpack() 3 ... giving their <u>data type and order</u>	2

5(b)(ii)	One mark per point (Max 2): • The new function will return an item of type <code>StockItem</code> • Need to declare/use a (local) variable of type <code>StockItem</code> • <code>Costfield</code> needs to be converted from a string to a real	2
----------	--	----------

81. O/N 22/P23/Q6/b

6(b)	One mark per point: 1 Function heading and ending including parameters 2 Declaration of local variable for Result / alt mechanism 3 Check for Reject 4 Check for Accept 5 Check for Recheck (or just default to third option) 6 Return Result following a reasonable attempt <pre> Function Status(Actual, Min, Max : INTEGER) RETURNS CHAR DECLARE Result : CHAR CONSTANT Accept = 'A' CONSTANT Reject = 'R' CONSTANT ReCheck = 'C' Result ← ReCheck //Check if reject IF Actual > Max + 2 OR Actual < Min - 2 THEN Result ← Reject ENDIF //Check if acceptable IF Actual < Max - 2 AND Actual > Min + 2 THEN Result ← Accept ENDIF RETURN Result ENDFUNCTION </pre>	6
------	--	----------

82. O/N 21/P21/Q6/c

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	----------

83. O/N 21/P22/Q1/c,d

1(c)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>ASC('C')</td><td>67</td></tr><tr><td>2 * STR_TO_NUM ("27")</td><td>54</td></tr><tr><td>INT(27 / 2)</td><td>13</td></tr><tr><td>"Sub" & MID("Abstraction" , 4 , 5)</td><td>"Subtract"</td></tr></table>		Expression	Evaluates to	ASC('C')	67	2 * STR_TO_NUM ("27")	54	INT(27 / 2)	13	"Sub" & MID("Abstraction" , 4 , 5)	"Subtract"	4
	Expression	Evaluates to											
	ASC('C')	67											
	2 * STR_TO_NUM ("27")	54											
	INT(27 / 2)	13											
	"Sub" & MID("Abstraction" , 4 , 5)	"Subtract"											
One mark per row													
Function names must be exactly as shown													

1(d)	Expression	Evaluates to	2
	PressureOK AND HiFlow	FALSE	
	PumpOn OR PressureOK	TRUE	
	NOT PumpOn OR (PressureOK AND NOT HiFlow)	TRUE	
	NOT (PumpOn OR PressureOK) AND NOT HiFlow	FALSE	
1 mark for any two rows correct 2 marks for all rows correct.			

84. O/N 21/P22/Q4/a,c

4(a)	Answer		5
	A line number containing a variable being incremented	19 / 21 / 23	
	The type of loop structure	pre-condition	
	The number of functions used	3	
	The number of parameters passed to function STR_TO_NUM()	1	
	The name of a procedure other than Check()	Result	
4(c)(i)	One mark per point: Structure: A count-controlled loop Justification: The number of iterations is known // repeats for the length of InString		2
4(c)(ii)	15, 23 One mark for both line numbers		1

85. O/N 21/P22/Q6/b,c(ii),d

6(b)	One mark for: (A flag is used to) exit the loops // iteration is terminated - as soon as a Screen element with value 1 is found	2
------	--	----------

6(c)(ii)	One mark for advantage and one for disadvantage: Advantage: (max 1) • Only have to change one module if specification changes • Less repetitive code / fewer lines of code • Aids re-usability Disadvantage: (max 1) • Single module more complex / more error prone / more difficult to debug ... • Single module cannot be split among programmers / teams Max 2	2
6(d)	<pre> PROCEDURE GetCentre () DECLARE StartRow, EndRow, StartCol, EndCol : INTEGER StartRow ← FirstRowSet() IF StartRow = -1 THEN CentreRow ← -1 // no 'touch' detected ELSE EndRow ← LastRowSet() StartCol ← FirstColSet() EndCol ← LastColSet() CentreRow ← INT((StartRow + EndRow)/2) CentreCol ← INT((StartCol + EndCol)/2) ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Call <any Set function> and check for -1 // check for no element set 2 ...and if so set CentreRow to -1 3 Call all 4 Set functions to get 'extremity' values 4 Calculate centre row and centre column 5 Use of INT() function or DIV operator on values from MP4 6 Assign calculated values to CentreRow and CentreCol Note: Max 5 if procedure heading and ending missing or incorrect (ignore array if passed as a parameter) or any local variables are undefined or of incorrect type	6

6(a)	<pre> FUNCTION FirstRowSet() RETURNS INTEGER DECLARE Row, Col : INTEGER DECLARE Found : BOOLEAN // array is 1280 x 800 Row ← 1 Found ← FALSE WHILE Row ≤ 800 AND Found = FALSE // top to bottom Col ← 1 WHILE Col ≤ 1280 AND Found = FALSE // left to right IF Screen[Row,Col] = 1 THEN Found ← TRUE // end function as soon as first // found ENDIF Col ← Col + 1 ENDWHILE Row ← Row + 1 ENDWHILE IF Found = FALSE THEN // nothing found Row ← 0 ENDIF RETURN Row - 1 ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending and return type 2 (Conditional) outer loop 1 to 800 (row) 3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column) 4 Reference <code>Screen</code> element and test for = 1 // <> 0 5 and if true save row number and exit loops 6 Increment index variables in both inner and outer loop 7 Return Row number or -1, following a reasonable attempt	7
------	---	---

86. O/N 21/P23/Q6/c

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	----------

87. M/J 21/P21/Q1/b

1(b)

Statement	Error
Result ← 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
SubString ← MID("pseudocode", 4, 1)	NO ERROR
IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect
Result ← Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator
Message ← "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number

One mark for each row

5

88. M/J 21/P21/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
	One mark for each underlined part	

89. M/J 21/P21/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	---	----------

90. M/J 21/P21/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	----------

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	----------

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	---	--

91. M/J 21/P22/Q5/b

5(b)(i)	<table><tr><th colspan="2">Answer</th></tr><tr><td>Give a line number containing an example of an initialisation statement.</td><td>07</td></tr><tr><td>Give a line number containing the start of a repeating block of code.</td><td>09 / 10</td></tr><tr><td>Give a line number containing a logic statement.</td><td>12</td></tr><tr><td>Give the number of parameters of function MID () .</td><td>3</td></tr></table> One mark for each row	Answer		Give a line number containing an example of an initialisation statement.	07	Give a line number containing the start of a repeating block of code.	09 / 10	Give a line number containing a logic statement.	12	Give the number of parameters of function MID () .	3	4
Answer												
Give a line number containing an example of an initialisation statement.	07											
Give a line number containing the start of a repeating block of code.	09 / 10											
Give a line number containing a logic statement.	12											
Give the number of parameters of function MID () .	3											
5(b)(ii)	IF (NextChar >= 'a') AND (NextChar <= 'z') THEN One mark for IF ... AND ... One mark for both conditions	2										

92. M/J 21/P22/Q8/b

8(b)	<pre> Procedure GetInitials() DECLARE NewString, NextWord : STRING DECLARE ThisWordNum, Index : INTEGER ThisWordNum ← 0 NewString ← "" REPEAT ThisWordNum ← ThisWordNum + 1 Index ← GetStart(ThisWordNum) IF Index <> -1 THEN //if there is ThisWordNum NextWord ← GetWord(Index) IF IgnoreWord(NextWord) = FALSE THEN NewString ← NewString & UCASE(LEFT(NextWord, 1)) ENDIF ENDIF UNTIL Index = -1 OUTPUT NewString ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declare NewString and initialise to empty string 2 Conditional loop to pick out all words from FNString 3 Evaluate result of GetStart() in a loop 4 Test result <> -1 and if not: 5 Assign result of GetWord() to a variable in a loop 6 Test result of IgnoreWord() in a loop 7 If not ignored, add the next initial letter to NewString in a loop 8 Increment ThisWordNum (must have been initialised) in a loop 9 Output NewString (must be all upper case) outside loop Note: Max 8 marks	8
------	---	----------

93. M/J 21/P23/Q1/b

1(b)	Statement	Error	5
	Result $\leftarrow$ 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING	
	SubString $\leftarrow$ MID("pseudocode", 4, 1)	NO ERROR	
	IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect	
	Result $\leftarrow$ Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator	
	Message $\leftarrow$ "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number	
One mark for each row			

94. M/J 21/P23/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
One mark for each underlined part		

95. M/J 21/P23/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	--	----------

96. M/J 21/P23/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	----------

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	----------

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	---	--

97. 9608 M/J 21/P21/Q1/c

1(c)

One mark for each error

Statement	Error
Code ← LEFT("Cat", 4)	Only 3 characters in string
Status ← MID("Aardvark", 0, 5)	Second parameter should start from 1
Size ← LENGTH("Password)	Missing closing quote / Opening quote should be removed
Stock[n] ← Stock[n+1]	NO ERROR / n may not be integer value / n out of bound
Result ← 3 OR 4	Not Boolean types

5

98. 9608 M/J 21/P21/Q5/a(i)

5(a)(i)	<pre> PROCEDURE GuessNum() DECLARE Count: INTEGER DECLARE RndNumber : INTEGER DECLARE MyGuess : INTEGER RndNumber $\leftarrow$ 1 + INT(RAND(20)) Count $\leftarrow$ 1 REPEAT OUTPUT "Input your guess" INPUT MyGuess IF MyGuess <> RndNumber THEN Count $\leftarrow$ Count + 1 OUTPUT "Incorrect - try again" ENDIF UNTIL MyGuess = RndNumber OUTPUT "You took ", Count, " guesses." ENDPROCEDURE </pre> 1 mark for each of the following: 1 Use of RAND () to generate an integer between 1 and 20 2 Conditional loop until random number is guessed 3 Prompt and input a guess ...in a loop 4 Comparison and increment Count and 'Try again' output message...in a loop 5 Final output message...not in a loop	5
---------	---	----------

99. 9608 M/J 21/P22/Q1/d

1(d)	Statement	Error	5
	Code ← RIGHT("Cap" & "art", 4)	NO ERROR	
	Status ← MID("Computer", 7, 5)	Not enough characters	
	Size ← LENGTH("Password") * 2	NO ERROR	
	NextChar ← CHR('A')	CHR() param should be integer // CHR() should be ASC()	
	Index ← Index & 3	3 is not character (should be '3') // and is not an arithmetic operator	
One mark for each line			

100. 9608 M/J 21/P22/Q2/a

2(a)	Answer		5
	The identifier name of a global variable	Overload	
	The name of the loop structure	Pre-condition loop	
	The identifier involved in a type mismatch	Landed	
	The name of a procedure that takes a parameter	Display()	
	The name of a function	Sample() // SubA() // SubB()	
One mark per row			

101. 9608 M/J 21/P22/Q4/a

4(a)	<pre> DECLARE Num : INTEGER Num ← 101 REPEAT OUTPUT Num Num ← Num + 2 UNTIL Num > 199 Mark as follows: 1 Counter declaration and initialisation to sensible value before the loop 2 REPEAT ... UNTIL <condition> 3 Correct selection of number to be output (use of MOD () or +2 or other) 4 Correct range of numbers output </pre>	4
------	--	----------

102. 9608 M/J 21/P22/Q5/b

5(b)(i)	<pre> DECLARE Response : BOOLEAN DECLARE Password : STRING OUTPUT "Please Input your password: " INPUT Password Response ← Validate(UserID, Password) AND Today() RETURN Response One mark per line in bold </pre>	2
---------	--	----------

5(b)(ii)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION Verify(UserID : STRING) RETURNS BOOLEAN DECLARE Count : INTEGER DECLARE Response : BOOLEAN DECLARE Password : STRING Count ← 1 Response ← FALSE IF UserID = "Guest" THEN Response ← TRUE ENDIF WHILE Response = FALSE AND Count < 4 OUTPUT "Please input your password: " INPUT Password Response ← Validate(UserID, Password) AND TODAY() IF Response = FALSE THEN IF Count < 3 THEN OUTPUT "Incorrect password - please try again" ENDIF Count ← Count + 1 ENDIF ENDWHILE RETURN Response ENDFUNCTION </pre> 1 mark for each of the following: 1 If <u>parameter</u> matches "Guest" then skip to Return (MP7) 2 Loop for max 3 attempts and terminate if valid password input 3 Prompt and Input first password attempt 4 Evaluate result of Validate() AND TODAY() in a loop... 5 Test result and increment count for incorrect password in a loop... 6 ...output 'try again' message and re-input password in a loop... 7 Return Boolean (correctly in all 3 cases) Note: Max 6 marks	6
----------	--	----------

Question 5(b)(ii): Visual Basic

```
Function Verify(UserID As String) As Boolean
    Dim Count As INTEGER
    Dim Response As BOOLEAN
    Dim Password As STRING

    Count = 1
    Response = FALSE

    If UserID = "Guest" Then
        Return TRUE
    End If

    While Response = FALSE And Count < 4
        Console.WriteLine("Please input your password: ")
        Password = Console.ReadLine()
        Response = Validate(UserID, Password) AND TODAY()
        If Response = FALSE Then
            Count = Count + 1
            If Count < 4 Then
                Console.WriteLine("Incorrect password - please try again")
            End If
        End If
    End While

    Return Response
End Function
```

Question 5(b)(ii): Pascal

```
function Verify(UserID : string) : boolean;
var
    Count : integer;
    Response : boolean;
    Password : string;

begin
    Count := 1;

    if UserID = "Guest" then
        Verify := TRUE // result := TRUE;

    while Response = FALSE And Count < 4
    begin
        writeln('Please input your password: ');
        readln(Password);
        Response := Validate(UserID, Password) and TODAY();
        if Response = FALSE then
            begin
                Count := Count + 1;
                if Count < 4 then
                    writeln('Incorrect password - please try again');
            end;
        end;

        Verify := Response // result := Response
    end;
```

Question 5(b)(ii): Python

```
def Verify(UserID):
    ## Count As INTEGER
    ## Response As BOOLEAN
    ## Password As STRING

    Count = 1

    if UserID == "Guest":
        return TRUE

    while Response == FALSE and Count < 4:
        Password = input("Please input your password: ")
        Response = Validate(UserID, Password) and TODAY()
        if Response == FALSE:
            Count = Count + 1
            if Count < 4:
                print("Incorrect password - please try again")

    return Response
```

103. 9608 M/J 21/P22/Q6/c

6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION GetValidFilename() RETURNS STRING DECLARE Filename : STRING DECLARE Valid : BOOLEAN DECLARE Index : INTEGER DECLARE NextChar : CHAR REPEAT Valid ← TRUE OUTPUT "Please input the name for the backup file " INPUT Filename IF LENGTH(Filename) < 4 OR LENGTH(Filename) > 10 THEN Valid ← FALSE ELSE Index ← 1 WHILE Index <= LENGTH(Filename) AND Valid = TRUE NextChar ← MID(Filename, Index, 1) IF NOT ((NextChar >='a' AND NextChar <='z') OR (NextChar >='A' AND NextChar <='Z') OR (NextChar >='0' AND NextChar <='9')) THEN Valid ← FALSE //not alphanumeric ENDIF Index ← Index + 1 ENDWHILE ENDIF IF Valid = FALSE THEN OUTPUT "Invalid filename - please try again" ENDIF UNTIL Valid = TRUE RETURN Filename ENDFUNCTION </pre> One mark for each of the following: 1 Conditional loop until valid filename input 2 Prompt and Input of filename in a loop 3 Test length is within range 4 Loop through filename: 5 Extract a single character 6 Test that character is numeric in a loop 7 Test that character is alphabetic in a loop 8 If filename invalid, output warning and repeat, if valid then return filename	8
------	--	----------

Question 6(c): Visual Basic

```
Function GetValidFilename() As String
    Dim Filename As String
    Dim Valid As Boolean
    Dim Index As Integer
    Dim NextChar As Char

    Valid = FALSE

    Do
        Valid = TRUE
        Console.WriteLine("Please input the name for the backup file ")
        Filename = Console.ReadLine()
        If Len(Filename) < 4 Or Len(Filename) > 10 Then
            Valid = FALSE
        Else
            Index = 1
            While Index <= Len(Filename) And Valid = TRUE
                NextChar = MID(Filename, Index, 1)
                If Not ((NextChar >='a' AND NextChar <='z') _
                    Or (NextChar >='A' AND NextChar <='Z') _
                    Or (NextChar >='0' AND NextChar <='9')) Then
                    Valid = FALSE //not alphanumeric
                End If
                Index = Index + 1
            End While
        End If

        If Valid = FALSE Then
            Console.WriteLine("Invalid filename - please try again")
        End If

        Loop Until Valid = TRUE
    Return Filename
End Function
```

Question 6(c): Pascal

```

function GetValidFilename() : String;
var
    Filename : String;
    Valid : Boolean;
    Index : Integer;
    NextChar : Char;

begin
    Valid := FALSE;

    repeat
        Valid := TRUE;
        writeln('Please input the name for the backup file ');
        readln(Filename);
        if length(Filename) < 4 Or length(Filename) > 10 then
            Valid := FALSE
        else
            begin
                Index := 1;
                while Index <= length(Filename) And Valid = TRUE do
                    begin
                        NextChar := MidStr(Filename, Index, 1);
                        if Not ((NextChar >='a' AND NextChar <='z')
                            Or (NextChar >='A' AND NextChar <='Z'
                                Or (NextChar >='0' AND NextChar <='9')) then
                            Valid := FALSE; //not alphanumeric
                        Index := Index + 1;
                    end;
                end;
            end;

        If Valid = FALSE then
            writeln('Invalid filename - please try again');

    until Valid = TRUE;
    result := Filename; // GetValidFilename := Filename
end;

```

Question 6(c): Python

```

def GetValidFilename():
    ## Filename As String
    ## Valid As Boolean
    ## Index As Integer
    ## NextChar As Char

    Valid = FALSE

    while not Valid:
        Valid = TRUE
        Filename = input("Please input the name for the backup file ")
        if Len(Filename) < 4 or Len(Filename) > 10:
            Valid = FALSE
        else:
            Index = 0
            while Index <= Len(Filename) and Valid = TRUE:
                NextChar = FileName[Index]
                if not ((NextChar >= '0' and NextChar <= '9'
                    or (NextChar >= 'a' and NextChar <= 'z')
                    or (NextChar >= 'A' and NextChar <= 'Z')):

```

```

Valid = FALSE

Index = Index + 1

if Valid == FALSE:
    print("Invalid filename - please try again")

return Filename

```

104. 9608 M/J 21/P23/Q1/b,c

1(b)(i)	One mark per bullet point. • conditional loop • the number of iterations is not known	2								
1(b)(ii)	One mark per bullet point. • the value is found • the end of the array is reached (and value not found)	2								
1(c)	<table><thead><tr><th>Statement</th><th>Error</th></tr></thead><tbody><tr><td>Code ← RIGHT("Cap" * 3, 2)</td><td>Cannot multiply a string (by 3)</td></tr><tr><td>Valid ← IS_NUM(3.14159)</td><td>Parameter should be a string</td></tr><tr><td>NextChar ← MID(ThisString, Index), 1</td><td>Closing bracket in wrong place</td></tr></tbody></table> One mark for each row	Statement	Error	Code ← RIGHT("Cap" * 3, 2)	Cannot multiply a string (by 3)	Valid ← IS_NUM(3.14159)	Parameter should be a string	NextChar ← MID(ThisString, Index), 1	Closing bracket in wrong place	3
Statement	Error									
Code ← RIGHT("Cap" * 3, 2)	Cannot multiply a string (by 3)									
Valid ← IS_NUM(3.14159)	Parameter should be a string									
NextChar ← MID(ThisString, Index), 1	Closing bracket in wrong place									

105. 9608 M/J 21/P23/Q5/a

5(a)	<pre> FUNCTION GroupNum(TelNum, Template : STRING) RETURNS STRING DECLARE FString : STRING DECLARE Index, ThisDigit, ThisGroup : INTEGER CONSTANT SPACE = ' ' FString ← "" ThisDigit ← 1 FOR Index ← 1 TO LENGTH(Template) ThisGroup ← STRING_TO_NUM(MID(Template, Index, 1)) FString ← FString & MID(TelNum, ThisDigit, ThisGroup) FString ← FString & SPACE ThisDigit ← ThisDigit + ThisGroup ENDFOR FString ← FString & RIGHT(TelNum, LENGTH(TelNum) - ThisDigit) RETURN FString ENDFUNCTION </pre> Mark as follows: 1 Function header and end including parameters and RETURN type 2 Local variable declaration and initialisation of FString (return string) 3 Loop to go through each char of Template (each group) 4 Extract character from template in a loop 5 Use of STRING_TO_NUM() on extracted character in a loop 6 Substring statement to pick up current group from TelNum and concatenate with FString in a loop 7 Concatenate SPACE separator in a loop 8 Concatenate final characters from TelNum after the loop (+ or – 1 characters) 9 Return the formatted string after reasonable attempt Max 8 Max 7 for not fully working solutions	8
------	---	----------

106. M/J 20/P21/Q1/b

1(b)	• Allows the subroutine code to be called from many/multiple places • Subroutine code may be (independently) tested and debugged • If the subroutine task changes the change needs to be made only once • Reduces unnecessary duplication / program lines • Enables sharing of development between programmers Or equivalent points that relate to a PROGRAM (not an algorithm) Max 3	3
------	--	----------

107. M/J 20/P21/Q6/c(i)

6(c)(i)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <u>EndTime</u> ← <u>Addtime</u> (<u>BeginTime</u>, 60) One mark per underlined section (Space before bracket for mark scheme clarification only)	2
---------	--	----------

Q6(c)(i): Visual Basic

```
EndTime = Addtime(BeginTime, 60)
```

Q6(c)(i): Pascal

```
EndTime := Addtime(BeginTime, 60)
```

Q6(c)(i): Python

```
EndTime = Addtime(BeginTime, 60)
```

108. M/J 20/P22/Q1/c

1(c)	One mark per bullet point: - Count controlled – the number of iterations is known / fixed - Post condition – the number of iterations depends on some condition being tested <u>at the end / before the loop is repeated // at least one iteration is always executed</u> For conditional: reject answer that also applies to pre-conditional	2
------	--	----------

109. M/J 20/P22/Q3

3	<pre> FUNCTION CheckCourse(Course : REAL) RETURNS INTEGER DECLARE Adjust, Check : INTEGER Check ← INT(Deviate(Course)) Adjust ← 255 CASE OF Check -20 to -1: Adjust ← 10 0 : Adjust ← 0 1 to 20 : Adjust ← -10 OTHERWISE CALL Alert() ENDCASE RETURN Adjust ENDFUNCTION </pre> 1 mark for each of the following: 1 FUNCTION heading and ending including parameter as given above 2 Assign value to Check using integer conversion and initialise Adjust to 255 3 CASE ... ENDCASE 4 Conditions -20 to -1 and 1 to 20 (and corresponding assignments) 5 Condition 0 (and corresponding assignment) 6 OTHERWISE 7 Return Adjust	7
---	---	---

110. M/J 20/P22/Q6/a(i)

6(a)(i)	<pre> FUNCTION AddTime(StartTime : STRING, Duration : INTEGER) _ RETURNS STRING DECLARE NewTime : STRING DECLARE StartMinutes, StartHours : INTEGER DECLARE Total, NewMinutes, NewHours : INTEGER StartHours ← STRING_TO_NUM(LEFT(StartTime,2)) StartMinutes ← STRING_TO_NUM(RIGHT(StartTime, 2)) Total ← (StartHours * 60) + StartMinutes + Duration NewHours ← DIV(Total, 60) NewMinutes ← MOD(Total, 60) NewTime ← "" IF NewHours < 10 THEN NewTime ← '0' // add leading zero to hours ENDIF NewTime ← NewTime & NUM_TO_STRING(NewHours) & ':' IF NewMinutes < 10 THEN NewTime ← NewTime & '0' // add leading zero ENDIF NewTime ← NewTime & NUM_TO_STRING(NewMinutes) RETURN NewTime ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameters 2 Extract StartHours and convert to integer 3 Extract StartMinutes and convert to integer 4 Add Duration to StartTime in minutes 5 Use DIV() to extract NewHours 6 Use MOD() to extract NewMinutes 7 Adding leading zeros when necessary to hours and minutes eg "09:05" 8 Return concatenated string Note: Accept alternative methods for calculation of NewHours and NewMinutes	8
---------	--	----------

111. M/J 20/P23/Q4/a,b

4(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION Extract(InString : STRING) RETURNS STRING DECLARE Name : STRING DECLARE NextChar : CHAR DECLARE Index : INTEGER CONSTANT COLON = ':' Index ← 1 Name ← "" NextChar ← LEFT(InString, 1) WHILE NextChar <> COLON Name ← Name & NextChar Index ← Index + 1 NextChar ← MID(InString, Index, 1) ENDWHILE RETURN Name ENDFUNCTION </pre> Alternative: <pre> FUNCTION Extract(InString : STRING) RETURNS STRING DECLARE Name : STRING DECLARE Index : INTEGER CONSTANT COLON = ':' Index ← 1 WHILE MID(InString, Index, 1) <> COLON Index ← Index + 1 ENDWHILE Name ← LEFT(InString, Index) RETURN Name ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending (where required) including parameters 2 Extract (next) character from InString 3 Conditional loop while character is not colon 4 Append character to Name and increment Index in a loop // calculate substring length and use LEFT() after loop 5 RETURN Name (may be combined with alternative mp 4)	5
------	---	----------

Q4(a): Visual Basic

```
Function Extract(InString As String) As String
    Dim Name As String
    Dim NextChar As Char
    Dim Index As Integer
    Const COLON = ':'

    Index = 1
    Name = ""
    NextChar = Left(InString, 1)

    While NextChar <> COLON
        Name = Name & NextChar
        Index = Index + 1
        NextChar = Mid(InString, Index, 1)
    Loop

    Return Name
End Function

Function Extract(InString As String) As String
    Return Mid(InString, 1, InString.IndexOf(":"))
End Function
```

Q4(a): Pascal

```
function extract(InString : string) : string;
var
    Name : string;
    NextChar : char;
    Index : integer;

const
    COLON = ':';

    Index := 1;
    Name := "";
    NextChar := copy(InString, 1, 1);

    While NextChar <> COLON do
    begin
        Name := Name & NextChar;
        Index := Index + 1;
        NextChar := copy(InString, Index, 1);
    end;

    extract := Name;

end;
```

Q4(a): Python

```
def Extract(InString)
    # Name : String
    # NextChar : Char
    # Index : Integer

    COLON = ':'

    Index = 1
    Name = ""
    NextChar = InString[1:2]

    While NextChar <> COLON:
        Name = Name + NextChar
        Index = Index + 1
        NextChar = InString[Index, Index + 1]

    Return Name
```

4(b)	Two alternatives: Alternative #1 Header: <u>PROCEDURE Extract (BYREF Name : STRING, BYREF Email : STRING, BYVALUE DataItem : STRING)</u> Explanation: Could additionally pass the name and email address to the procedure using BYREF. Procedure would extract name and email and assign values to BYREF parameters; these would then be available to calling program. Alternative #2 Header: <u>PROCEDURE Extract (DataItem : STRING)</u> Explanation: Declare new global variables for the name and email address. These could be assigned values within the new procedure and these values would be used by the calling program. Mark as follows: • Two marks for header – must be a Procedure not a Function (but see note below) • Max Two marks for explanation Note: Allow solution based on user-defined record type / record item returned from modified function. Max 3	3
------	---	----------

112. M/J 20/P23/Q5/c

5(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE ServiceList(MaxHours : INTEGER) DECLARE BoatNum, Hours : INTEGER DECLARE LastService : STRING DECLARE Due : BOOLEAN Due ← FALSE OUTPUT "Boat Service List" FOR BoatNum ← 1 TO 20 LastService ← GetLastService(NUM_TO_STRING(BoatNum)) Hours ← GetHours(NUM_TO_STRING(BoatNum), LastService) IF Hours > MaxHours THEN OUTPUT NUM_TO_STRING(Boatnum) & ": " ___ & NUM_TO_STRING(Hours) Due ← TRUE ENDIF ENDFOR IF Due = TRUE THEN OUTPUT "No boats are due to be serviced" ENDIF ENDPROCEDURE </pre> One mark for each of the following: 1 Procedure heading and ending (where appropriate) including parameter 2 Output "report header" line not in a loop 3 Loop from 1 to 20 4 Call GetLastService 5 Call GetHours 6 Check return value from GetHours > MaxHours 7 Suitable Output statement in a loop 8 Mechanism to initialise, count and produce final output if no boats to be serviced	8
------	---	----------

Q5(c): Visual Basic

```
Sub ServiceList(MaxHours As Integer)

    Dim BoatNum, Due As Integer
    Dim LastService As String

    Due = 0
    Console.WriteLine("Boat Service List")

    For BoatNum = 1 To 20
        LastService = GetLastService(CStr(BoatNum))
        Hours = GetHours(CStr(BoatNum), LastService)
        If Hours > MaxHours Then
            Console.WriteLine(CStr(Boatnum) & ": " & CStr(Hours))
            Due = Due + 1
        End If
    Next

    If Due = 0 Then
        Console.WriteLine("No boats are due to be serviced")
    End If

End Sub
```


Q5(c): Pascal

```
procedure ServiceList(MaxHours : Integer)

var
  BoatNum, Due : integer;
  LastService : string;

  Due := 0
  print("Boat Service List")

  for BoatNum := 1 to 20 do
  begin
    LastService := GetLastService(CInt(BoatNum));
    Hours := GetHours(BoatNum, LastService);
    If Hours > MaxHours then
      begin
        writeln(IntToStr(Boatnum) & ": " & IntToStr(Hours));
        Due := Due + 1;
      end;
    end;

  If Due = 0 then
    writeln ("No boats are due to be serviced")
  End If
End Sub
```

Q5(c): Python

```
def ServiceList(MaxHours)

  # BoatNum, Due : Integer
  # LastService : String

  Due = 0
  print("Boat Service List")

  For BoatNum in range(1, 21):
    LastService = GetLastService(str(BoatNum))
    Hours = GetHours(BoatNum, LastService)
    if Hours > MaxHours:
      print(str(Boatnum) & ": " & str(Hours))
      Due = Due + 1

  if Due == 0:
    print("No boats are due to be serviced")
```

113. O/N 19/P21/Q1/b(ii)

1(b)(ii)	One mark per row		4
	Expression	Evaluates to	
	LEFT("Start", 3) & RIGHT("Apple", 3)	"Staple"	
	MID("sample", 3, 5)	ERROR	
	NUM_TO_STRING(12.3 * 2)	"24.6"	
	INT(STRING_TO_NUM("53.4")) + 7	60	

114. O/N 19/P21/Q2/c

2(c)	One mark for identification, one mark for description: - By reference / ref - The <u>address of / pointer</u> to the parameter is passed to the subroutine // if the parameter value is changed in the subroutine this changes the original value	2
------	--	----------

115. O/N 19/P21/Q3

3	One mark per row:		7
		Answer	
	The number of the line containing a variable being	24 / 26 / 28	
	The range of line numbers containing a pre-condition loop	20 – 30	
	The number of initialisation statements	3	
	The number of the line containing a logical operator	20	
	The range of line numbers containing a selection statement	22 – 27 / 32 – 37	
	The name of a built-in function	MID / LENGTH	
	The name of a parameter	InString / Index	

116. O/N 19/P21/Q6/d(ii)

6(d)(ii)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <u>CALL ClearArray(Duplicates, 100, "Empty")</u> Mark as follows: - Procedure call - Parameter list (in brackets)	2
----------	--	----------

Question 6(d)(ii): Visual Basic

Call ClearArray(Duplicates, 100, "Empty") 'Call optional

Question 6(d)(ii): Pascal

ClearArray(Duplicates, 100, 'Empty');

Question 6(d)(ii): Python

ClearArray(Duplicates, 100, "Empty")

117. O/N 19/P22/Q1/b(ii)

1(b)(ii)	One mark per row		4
	Expression	Evaluates to	
	RIGHT("Stop", 3) & LEFT("ich",2)	"topic"	
	MID(NUM_TO_STRING(2019), 3, 1)	"1"	
	INT(NUM_TO_STRING(-273.16))	ERROR	
	INT(13/2)	6	

118. O/N 19/P22/Q4/a

4(a)	<pre> PROCEDURE Button(ButtonNum : INTEGER) DECLARE Limit : INTEGER IF ButtonNum = 10 // increase volume THEN IF MaxVol = 0 THEN Limit ← 49 ELSE Limit ← MaxVol ENDIF IF VolLevel < Limit THEN VolLevel ← VolLevel + 1 ENDIF ELSE // otherwise must be ButtonNum 20 - decrease IF VolLevel > 0 THEN VolLevel ← VolLevel - 1 ENDIF ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Check if parameter value = 10 2. Check if parameter value = 20 3. Check if MaxVol = 0 4. Decrement VolLevel and ensure still in range If attempting to increase volume (parameter value was 10): 5. Increment VolLevel 6. Ensure VolLevel still in range: for both cases. i.e: VolLevel ≤ 49 (for MaxVol = 0) VolLevel ≤ MaxVol (for MaxVol > 0)	6
------	---	----------

119. O/N 19/P22/Q6/c

6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. One mark per underlined section: <u>Result</u> ← <u>CountTimes</u> (<u>Leavers</u>, "") (Space before open bracket to highlight underlined section only)	3
------	--	----------

Q6 (c): Visual Basic

```
Result = CountTimes(Leavers, "")
```

Q6 (c): Pascal

```
Result := CountTimes(Leavers, "");
```

Q6 (c): Python

```
Result = CountTimes(Leavers, "")
```

120. O/N 19/P23/Q1/a(ii)

1(a)(ii)	Declaration	1
----------	-------------	----------

121. O/N 19/P23/Q2/a(i)

2(a)(i)	- Count-controlled // FOR loop - Used when the number of iterations is known / fixed	2
---------	--	----------

122. O/N 19/P23/Q5

5	<pre> FUNCTION Abbreviate(Name : STRING) RETURNS STRING DECLARE NewString : STRING DECLARE NextChar : CHAR DECLARE Index : INTEGER DECLARE Space : BOOLEAN CONSTANT SPACECHAR = ' ' Space ← TRUE NewString ← "" FOR Index ← 1 TO LENGTH(Name) NextChar ← MID(Name, Index, 1) IF Space = TRUE THEN NewString ← NewString & NextChar // first char of next word Space ← FALSE ELSE IF NextChar = SPACECHAR THEN Space ← TRUE ENDIF ENDIF ENDIF ENDFOR RETURN NewString ENDFUNCTION </pre> 1 mark for each of the following (max 8): 1 Function header, ending and return parameters 2 Declare and Initialise <code>NewString</code> to either "" or first character of name 3 FOR loop picking out all characters from <code>Name</code>: 4 extract an individual character in a loop 5 check for space character in a loop 6 concatenate the next character to <code>NewString</code> in a loop 7 Return <code>NewString</code> 8 Accommodate a string with trailing space	8
---	--	---

123. M/J 19/P21/Q1/b(i)

1(b)(i)	Expression	Evaluates to	5
	MID(Title, 5, 3) & RIGHT(Author, 3)	"tripod"	
	INT(WeightEach * PackSize)	24	
	PackSize >= 4 AND WeightEach < 6.2	FALSE	
	LEFT(Author, ASC(Version) - 65)	"Er"	
	RIGHT(Title, (LEN(Author) - 6))	"hetti"	
Quotes must be present Must be capital E in row 4			

124. M/J 19/P21/Q2/a

2(a)	Type: <u>Conditional</u> Explanation: The number of iterations is not known / dependent on a condition	2
------	---	----------

125. M/J 19/P21/Q3/b

3(b)	<pre>PROCEDURE ScanArray (<u>BYREF</u> AverageValue: REAL, <u>BYREF</u> ZeroCount: INTEGER, <u>ArrayName</u> : ARRAY)</pre> One mark for each underlined part Names unimportant but first two parameters must be BYREF	4
------	--	----------

126. M/J 19/P21/Q5/a

5(a)	- Saves development time / no need to write it / can't write it... - Pre-compiled and tested / Increased reliability / reduces chance of error - Is available to all programs	3
------	---	----------

127. M/J 19/P21/Q6/a

6(a)	One mark for each of: - To make a more manageable / understandable solution - Subroutine may be (independently) tested and debugged - Program is easier to maintain	3
------	---	----------

128. M/J 19/P22/Q1/b(i)

1(b)(i)	Expression	Evaluates to	5
	STRING_TO_NUM(RIGHT(ID, 3))	234.0 / 234	
	INT(Height * Children)	11	
	IsMarried AND Married < 31/12/1999	TRUE	
	LENGTH(ID & NUM_TO_STRING(Height))	8	
	MID((ID, INT(Height) - Children, 2)	"23"	
No quotes for row 1 Quotes (single or double) for row 5			

129. M/J 19/P22/Q2/a

2(a)(i)	- To make a more manageable / understandable solution - To support modular design	1
2(a)(ii)	- Allows the subroutine to be called from many / multiple places - Subroutine may be (independently) tested and debugged - If the task changes the change needs to be made only once - Reduces unnecessary duplication / program lines - Allows teams to work on different parts of the solution	3
2(a)(iii)	Type of subroutine: Function Justification: It returns a value // assigns a value to variable Answer One mark for type One mark for justification	2

130. M/J 19/P22/Q6/a,b,c

6(a)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION GetInfo() RETURNS STRING DECLARE ID : STRING DECLARE PreferredName : STRING DECLARE Valid : BOOLEAN Valid ← FALSE WHILE Valid = FALSE OUTPUT "Please Enter a valid ID" INPUT ID IF LENGTH(ID) = 5 AND LEFT(ID, 1) >= 'A' AND LEFT(ID, 1) <= 'Z' AND ISNUM(RIGHT(ID, 4)) THEN Valid ← TRUE ENDIF ENDWHILE OUTPUT "Please enter preferred name" INPUT PreferredName RETURN ID & '*' & PreferredName ENDFUNCTION </pre> One mark for each of the following: 1. Function header and end (where appropriate) 2. Local variables used are declared (commented in python) 3. Prompt and input for ID (until valid) and preferred name 4. Conditional loop repeating while ID is invalid 5. test length in a loop 6. test first character in a loop 7. test last four characters in a loop 8. Concatenate using correct separator character and return resulting string	8
------	---	----------

Q6 (a): Visual Basic

```

Function GetInfo() As String
    Dim ID As String = ""
    Dim PreferredName As String = ""
    Dim Valid As Boolean = False
    While Valid = False
        Console.WriteLine("Please enter a valid ID : ")
        ID = Console.ReadLine()
        If Len(ID) = 5 And Left(ID, 1) >= "A" And Left(ID, 1) <= "Z" ____
            And IsNumeric(Right(ID, 4)) Then
                Valid = True
            End If
        End While
        Console.WriteLine("Please enter preferred name : ")
        PreferredName = Console.ReadLine()
        Return ID & "*" & PreferredName
    End Function

```

Alternative:

```

Function GetInfo() As String

    Dim ID As String
    Dim PreferredName As String
    Dim Valid As Boolean
    Dim Number As String
    Dim Size As Integer
    Dim i As Integer

    Valid = False

    While Valid = False
        Console.WriteLine("Please Enter a valid ID")
        ID = Console.ReadLine()
        Size = Len(ID)

        If (Size = 5) And ((Left(ID, 1) >= "A") And (Left(ID, 1) <= "Z"))
Then
            Valid = True
            For i = 2 To 5
                Number = Mid(ID, i, 1)
                If (Number < "0") Or (Number > "9") Then
                    Valid = False
                End If
            Next
        End If
    End While

    Console.WriteLine("Please enter preferred name")
    PreferredName = Console.ReadLine()
    Return (ID & "*" & PreferredName)
End Function

```

Q6 (a): Pascal

```

function GetInfo() : String;
var
    ID : String;
    PreferredName : String;
    Valid : Boolean;
    Value, Code : Integer;
begin
    Valid := false;
    while not Valid do
    begin
        Write('Please enter a valid ID : ');
        Readln(ID);
        if (Length(ID) = 5) and (ID[1] >= 'A') and (ID[1] <= 'Z') then
            Valid := true;
        Val(Copy(ID, 2, 4), Value, Code);
        if Code <> 0 then
            Valid := false;
        end;
    end;
    Write('Please enter preferred name : ');
    Readln(PreferredName);
    GetInfo := ID + '*' + PreferredName;
end;

```

Free Pascal

```

function GetInfo() : String;
var
    ID : String;
    PreferredName : String;
    Valid : Boolean;
    Value, Code : Integer;
begin
    Valid := false;
    while not Valid do
    begin
        Write('Please enter a valid ID : ');
        Readln(ID);
        if (Length(ID) = 5) and (ID[1] >= 'A') and (ID[1] <= 'Z') and (IsNumber(SubStr(ID, 2, 4))) then
            Valid := true;
        end;
    end;
    Write('Please enter preferred name : ');
    Readln(PreferredName);
    result := ID + '*' + PreferredName;
end;

```

Q6 (a): Python

```

def GetInfo() :
    ID = "" # string variable
    PreferredName = "" # string variable
    Valid = False # Boolean variable
    while not Valid :
        ID = input("Please enter a valid ID : ")
        if len(ID) == 5 and ID[0] >= "A" and ID[0] <= "Z" and ID[1:].isnumeric() :
            Valid = True
        PreferredName = input("Please enter preferred name : ")
    return ID + "*" + PreferredName

```

6(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> PROCEDURE TopLevel() DECLARE Response : CHAR DECLARE InputData : STRING DECLARE Success : BOOLEAN Response ← 'Y' WHILE Response = 'Y' InputData ← GetInfo() IF LEFT(InputData,1) < 'N' THEN Success ← WriteInfo(InputData, "File1.txt") ELSE Success ← WriteInfo(InputData, "File2.txt") ENDIF IF NOT Success THEN Response ← 'N' ELSE OUTPUT "Enter details for another student? (Y/N)" INPUT Response ENDIF ENDWHILE ENDPROCEDURE </pre> One mark for each of the following: 1. Procedure header and end 2. Conditional loop terminated with user input 3. call to <code>GetInfo()</code> in a loop 4. check first character of returned <code>UserID</code> value in a loop 5. call(s) to <code>WriteInfo()</code> in both cases ... 6. ... with two <code>STRING</code> parameters in a loop 7. exit procedure if <code>WriteInfo()</code> unsuccessful in a loop 8. if <code>WriteInfo()</code> successful, prompt and check input to repeat / exit in a loop	8
------	--	---

Q6 (b): Visual Basic

```
Sub TopLevel()  
    Dim Response As String = "Y"  
    Dim InputData As String = ""  
    Dim Success As Boolean = True  
    While Response = "Y"  
        InputData = GetInfo()  
        If Left(InputData, 1) < "N" Then  
            Success = WriteInfo(InputData, "File1.txt")  
        Else  
            Success = WriteInfo(InputData, "file2.txt")  
        End If  
        If Not Success Then  
            Response = "N"  
        Else  
            Console.Write("Enter details for another student? Y/N ")  
            Response = Console.ReadLine()  
        End If  
    End While  
End Sub
```

Q6 (b): Pascal

```
procedure TopLevel();  
var  
    Response : Char;  
    InputData : String;  
    Success : Boolean;  
begin  
    Response := 'Y';  
    while Response = 'Y' do  
        begin  
            InputData := GetInfo();  
            if InputData[1] < 'N' then  
                Success := WriteInfo(InputData, 'File1.txt')  
            else  
                Success := WriteInfo(InputData, 'File2.txt');  
            if not Success then  
                Response := 'N'  
            else  
                begin  
                    Write('Enter details for another student? (Y/N) ');  
                    Readln(Response);  
                end;  
            end;  
        end;  
    end;  
end;
```

Q6 (b): Python

```
def TopLevel() :
    Response = "Y"           # string/character variable
    InputData = ""          # string variable
    Success = True           # Boolean variable
    while Response == "Y" :
        InputData = GetInfo()
        if InputData[0] < "N" :
            Success = WriteInfo(InputData, "File1.txt")
        else :
            Success = WriteInfo(InputData, "File2.txt")
        if not Success :
            Response = "Y"
        else :
            Response = input("Enter details for another student? (Y/N) ")
```

6(c)	<u>FUNCTION WriteInfo (FileData : STRING, Filename : STRING)</u> <u>RETURNS BOOLEAN</u>	3
	One mark per underlined section	

131. M/J 19/P23/Q1/b

1(b)(i)	Expression		Evaluates to	5
	STRING_TO_NUM(MID(ProductID, 3, 2)) + 4		31.0 / 31	
	INT(MeltingPoint / 2)		90	
	Soluble AND Attempt > 3		FALSE	
	LENGTH(ProductID & NUM_TO_STRING(MeltingPoint))		11	
	RIGHT(ProductName, 4) & MID(ProductName, 5, 4)		"postroom"	
	Quotes for row 5 only			

1(b)(ii)	Variable	Data type	5
	MeltingPoint	REAL	
	Soluble	BOOLEAN	
	Attempt	INTEGER	
	Version	CHAR	
	ProductID	STRING	
	One mark per data type		

132. M/J 19/P23/Q6/c

6(c)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <u>FUNCTION ProcessArray (ClassList : ARRAY, ClassContact : STRING)</u> <u>RETURNS : INTEGER</u> One mark per underlined section.	3
------	---	----------

Q6 (c): Visual Basic

Function ProcessArray (ClassList() As String, ClassContact As String) As Integer

OR

Function ProcessArray (ClassList As String(), ClassContact As String) As Integer

Q6 (c): Pascal

function ProcessArray (var ClassList:CList; ClassContact:string) :integer;

CList is user-defined type – could be any name that's not a keyword

Q6 (c): Python

def ProcessArray (ClassList, ClassContact) :

133. O/N 18/P22/Q2/a(ii)

2(a)(ii)	<pre> FUNCTION GetTotal() RETURNS REAL DECLARE Valid : BOOLEAN DECLARE Amount : REAL Valid ← FALSE REPEAT OUTPUT "Enter the amount" INPUT Amount IF Amount > 0 AND Amount < 10000 THEN Valid ← TRUE ENDIF UNTIL Valid = TRUE RETURN Amount ENDFUNCTION </pre> Note that the pseudocode shown is only an example. The use of an explicit flag and IF structure are not essential provided the functionality is provided. 1 mark for each of: 1 declaration of local variable(s) used 2 prompt followed by input 3 conditional loop 4 checking that input value > 0 and input value < 10000 5 returning the value	5
----------	---	----------

134. O/N 18/P22/Q4/a(i),c

4(a)(i)	The identifier name of a local variable	FileData / FileLine	5
	The identifier name of a user-defined procedure	ScanCompleted	
	The identifier name of a user-defined function	ReadFileLine / ScanFile	
	The number of dimensions of ResultArray	1	
	The scope of FileData	Local	
	1 mark for each correct answer		
4(c)(i)	Through the use of: Subroutines / Functions / Procedures / Parameters / Methods	1	
4(c)(ii)	Reduces program complexity // easier to develop / test / debug // tasks may be re-used // tasks can be allocated to different programmers/teams (with different skills) // limited scope of local variables	1	

135. O/N 18/P23/Q2/a(ii)

2(a)(ii)	The pseudocode shown here is only an example. The use of an explicit flag and IF structure are not essential provided the functionality is provided. <pre> FUNCTION GetCardNumber() RETURNS STRING DECLARE Valid : BOOLEAN DECLARE CardNum : STRING Valid ← FALSE REPEAT OUTPUT "Enter card number" INPUT CardNum IF LENGTH(CardNum) = 16 AND IS_NUM(CardNum) = TRUE THEN Valid ← TRUE ENDIF UNTIL Valid RETURN CardNum ENDFUNCTION </pre> 1 mark for each of the following: 1 Declaring local variable to store user input 2 Conditional Loop 3 Prompt and input of CardNum 4 Length check 5 Checking IS_NUM(CardNum) is TRUE 6 Return a value	6
----------	---	----------

136. O/N 18/P23/Q3/a

3(a)	Name: count controlled / FOR ... NEXT loop Justification: Known / fixed number of iterations // all elements of the array need to be checked 1 mark for name 1 mark for justification	2
------	---	----------

137. O/N 18/P23/Q4/a(i),b,c,d(ii)

4(a)(i)	The name of a global identifier	LastElement // ThisArray	5
	The name of a user-defined procedure	Insert	
	The scope of ArrayIndex	Local	
	The number of dimensions of ThisArray	2	
	The scope of NewData	Local	

4(b)	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear at the end of this mark scheme. <pre> FUNCTION Update(NewData: STRING) RETURNS INTEGER DECLARE ArrayIndex : INTEGER DECLARE Found : BOOLEAN DECLARE Validate : BOOLEAN ArrayIndex ← 1 Found ← FALSE WHILE ArrayIndex <= LastElement AND Found = FALSE IF ThisArray[ArrayIndex, 1] > NewData THEN Found ← TRUE ELSE ArrayIndex ← ArrayIndex + 1 ENDIF ENDWHILE IF Found = TRUE THEN Validate ← Insert(ArrayIndex, NewData) IF Validate = FALSE THEN ArrayIndex ← -1 ENDIF ELSE ArrayIndex ← 0 ENDIF RETURN ArrayIndex ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameters 2 Local variable declarations and Initialisation of <code>ArrayIndex</code> and <code>Found</code> 3 <code>WHILE</code> loop 4 First <code>IF-THEN-ELSE-ENDIF</code> clause 5 Second <code>IF</code> clause including function call to <code>Insert()</code> 6 Check Return value 7 Set / return -1 IF (Validate) FALSE 8 Return parameter value (-1 or 0 // <code>ArrayIndex</code>)	8
4(c)	Description to include: • mechanism involves using <u>parameters</u> .. to pass values from one procedure to another • parameters may be 'by reference' or 'by value'	2
4(d)(ii)	CONSTANT <code>LastElement</code> = 200	1

138. M/J 18/P21/Q1/b

1(b)(i)	Expression		Evaluates to	5
	"Mon" & MID(MyGreeting, 10, 2)		"Month"	
	AgeInYears + ASC(MyInitial)		94	
	INT(MyInitial)		ERROR	
	MOD(Weight * 2, 10)		1	
	Married AND (NOT Children)		FALSE	

1(b)(ii)	Variable	Data type	5
	MyGreeting	STRING	
	MyInitial	CHAR	
	AgeInYears	INTEGER	
	Weight	REAL	
	Married	BOOLEAN	
	One mark per answer		
Alternative appropriate data types acceptable			

139. M/J 18/P21/Q2/b

2(b)	Feature	Answer	9
	A line containing an example of an assignment statement	08,12,13,19	
	A line containing the start of a repetition block	10	
	A line containing the end of a repetition block	23	
	The line containing the start of a selection statement	15	
	The number of parameters of the MID function	3	
	The boolean operator used	AND	
	The number of local variables	4	
	The number of function calls from within StringClean() resulting from the call: NewString ← StringClean("Me")	5	
	The number of a line containing an unnecessary statement	06	

140. M/J 18/P21/Q7

7	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> FUNCTION ProcessMarks (Mark: ARRAY[1:20] OF INTEGER) RETURNS INTEGER DECLARE Highest : INTEGER DECLARE Average as REAL DECLARE Total as INTEGER DECLARE Position as INTEGER Total ← 0 Highest ← Mark[1] //The highest mark is the first one Position ← 1 FOR i ← 1 to 20 Total ← Total + Mark[i] IF Mark[i] > Highest THEN Highest ← Mark[i] Position ← i ENDIF ENDFOR Average ← Total/20 Output ("The average mark is " & Average & " and the highest mark is " & Highest) RETURN Position ENDFUNCTION </pre> 1 mark for each of the following: 1 Correct Function heading (including <code>Mark</code> as parameter) and ending 2 Declare local variable for <code>Highest</code> and initialise 3 Loop structure (1 to 20 or 0 to 19) 4 Comparison with current <code>Highest</code> 5 ...Assign new <code>Highest</code> 6 Calculate <code>Average</code> 7 Output message including both variables and explanatory text 8 Return value of index	Max7
---	--	-------------

Q7**Visual Basic**

```
Function ProcessMarks(ByVal Mark() As Integer) As Integer

    Dim Highest As Integer
    Dim Average As Single
    Dim Total As Integer
    Dim Position As Integer
    Dim i As Integer

    Total = 0
    Position = 1
    Highest = Mark(1)
    For i = 1 To 20
        Total = Total + Mark(i)
        If Mark(i) > Highest Then
            Highest = Mark(i)
            Position = i
        End If
    Next
    Average = Total / 20
    Console.WriteLine ("The average mark is " & Average & _
        " and the highest mark is " & Highest)

    Return Position
End Function
```

Python

```
def ProcessMarks (mark):
    #highest, i, position, total as integer
    #average as real

    highest = mark[0]
    total = 0
    position = 0
    for i in range(0,20):
        total = total + mark[i]
        if mark[i] > highest:
            highest = mark[i]
            position = i
    average = total/20
    print('The average mark is ' + str(average) + \
        ' and the highest mark is ' + str(highest))
    return position
```

Pascal

```

function ProcessMarks (mark:array of integer):integer;

var highest, total, position, i: integer;
    average: real;

begin
    highest := mark[1];
    total := 0;
    position := mark[1];
    for i := 1 to 20 do
        begin
            total := total + mark[i];
            if mark[i] > highest then
                begin
                    highest := mark[i];
                    position := i;
                end;
        end;
    average := total / 20;
    writeln ('The average mark is ', average, ' and the highest mark is ',
highest);
    ProcessMarks := position;
end;

```

141. M/J 18/P22/Q1/b

1(b)(i)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>"Fas" & MID(Subject, 6, 3)</td><td>"Faster"</td></tr><tr><td>LEFT(Mark, 1)</td><td>ERROR</td></tr><tr><td>10 + ASC(Grade)</td><td>76</td></tr><tr><td>MOD(AverageMark * 2, 3)</td><td>0</td></tr><tr><td>CourseCompleted AND (Mark >= 60)</td><td>TRUE</td></tr></table>	Expression	Evaluates to	"Fas" & MID(Subject, 6, 3)	"Faster"	LEFT(Mark, 1)	ERROR	10 + ASC(Grade)	76	MOD(AverageMark * 2, 3)	0	CourseCompleted AND (Mark >= 60)	TRUE	5
Expression	Evaluates to													
"Fas" & MID(Subject, 6, 3)	"Faster"													
LEFT(Mark, 1)	ERROR													
10 + ASC(Grade)	76													
MOD(AverageMark * 2, 3)	0													
CourseCompleted AND (Mark >= 60)	TRUE													
1(b)(ii)	<table><tr><th>Variable</th><th>Data type</th></tr><tr><td>Mark</td><td>INTEGER</td></tr><tr><td>Subject</td><td>STRING</td></tr><tr><td>Grade</td><td>CHAR</td></tr><tr><td>CourseCompleted</td><td>BOOLEAN</td></tr><tr><td>AverageMark</td><td>REAL</td></tr></table> One mark per answer	Variable	Data type	Mark	INTEGER	Subject	STRING	Grade	CHAR	CourseCompleted	BOOLEAN	AverageMark	REAL	5
Variable	Data type													
Mark	INTEGER													
Subject	STRING													
Grade	CHAR													
CourseCompleted	BOOLEAN													
AverageMark	REAL													

142. M/J 18/P22/Q2/b,c(ii)

2(b)	<table><tr><th>Feature</th><th>Answer</th></tr><tr><td>A line number containing an example of an assignment statement</td><td>08 / 09 / 13 / 14 / 15 / 21</td></tr><tr><td>A line number containing the start of a 'pre-condition' loop</td><td>11</td></tr><tr><td>A line number containing the end of a 'pre-condition' loop</td><td>24</td></tr><tr><td>A line number containing the start of a selection statement</td><td>17</td></tr><tr><td>The number of parameters of the LEFT() function</td><td>2</td></tr><tr><td>The Boolean operator used</td><td>OR</td></tr><tr><td>The number of times the function LEFT() is called from within CountDigits() resulting from the call: Result ← CountDigits("AB27C4")</td><td>6</td></tr><tr><td>The number of local variables</td><td>3</td></tr></table>	Feature	Answer	A line number containing an example of an assignment statement	08 / 09 / 13 / 14 / 15 / 21	A line number containing the start of a 'pre-condition' loop	11	A line number containing the end of a 'pre-condition' loop	24	A line number containing the start of a selection statement	17	The number of parameters of the LEFT() function	2	The Boolean operator used	OR	The number of times the function LEFT() is called from within CountDigits() resulting from the call: Result ← CountDigits("AB27C4")	6	The number of local variables	3	8
Feature	Answer																			
A line number containing an example of an assignment statement	08 / 09 / 13 / 14 / 15 / 21																			
A line number containing the start of a 'pre-condition' loop	11																			
A line number containing the end of a 'pre-condition' loop	24																			
A line number containing the start of a selection statement	17																			
The number of parameters of the LEFT() function	2																			
The Boolean operator used	OR																			
The number of times the function LEFT() is called from within CountDigits() resulting from the call: Result ← CountDigits("AB27C4")	6																			
The number of local variables	3																			
2(c)(ii)	<pre>IF (nc >= '0') AND (nc <= '9') THEN c ← c + 1 ENDIF</pre> One mark for each of: • Single IF THEN ENDIF statement (no ELSE) • Switching OR to AND • Lower value comparison • Upper value comparison ALTERNATIVE: <pre>IF NOT ((nc < '0') OR (nc > '9')) THEN c ← c + 1 ENDIF</pre> One mark for each of: • Single IF THEN ENDIF statement (no ELSE) • Inverting test using NOT... • ... correct use of brackets • ... both comparisons unchanged	4																		

143. M/J 18/P22/Q5/c

5(c)	Example 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre>FUNCTION GetNumber() AS STRING DECLARE Valid : BOOLEAN DECLARE MemberNumber : INTEGER REPEAT Valid ← TRUE OUTPUT "Please input a valid member number" INPUT MemberNumber IF (MemberNumber > 9999) OR (MemberNumber < 1111) THEN Valid ← FALSE ENDIF UNTIL Valid = TRUE RETURN STR(MemberNumber) ENDFUNCTION</pre> 1 mark for each of the following: 1 Function heading (as above) and ending 2 Conditional loop structure 3 Condition to check valid membership number 4 Returning string value	4
------	--	----------

Q5 (c): Visual Basic

```
Function GetNumber() As String
    Dim Valid As Boolean
    Dim MemberNumber As Integer
    Do
        Valid = True
        Console.WriteLine("Please enter a valid member number: ")
        MemberNumber = Console.ReadLine()
        If MemberNumber > 9999 Or MemberNumber < 1111 Then
            Valid = False
        End If
    Loop Until Valid = True
    Return MemberNumber.ToString()
End Function
```

Q5 (c): Pascal

```
function GetNumber() : string;
var
    Valid : boolean;
    MemberNumber : integer;
begin
    repeat
        Valid := true;
        write('Please enter a valid member number: ');
        readln(MemberNumber);
        if (MemberNumber > 9999) or (MemberNumber < 1111) then
            Valid := false;
        until Valid = true;
        GetNumber := IntToStr(MemberNumber);
    end;
```

Q5 (c): Python

```
def GetNumber() :
    # Valid : boolean
    # MemberNumber : integer
    Valid = False
    while not Valid :
        Valid = True
        MemberNumber = int(input("Please enter a valid member number: "))
        if MemberNumber > 9999 or MemberNumber < 1111 :
            Valid = False
    return str(MemberNumber)
```

144. M/J 18/P23/Q1/b

1(b)(i)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>MID(MyName, 4, 4) & "ol"</td><td>"phenol"</td></tr><tr><td>QualityConfirmed AND (Factor >= 6.5)</td><td>TRUE</td></tr><tr><td>20 + ASC(Quality)</td><td>88</td></tr><tr><td>QualityConfirmed + 3</td><td>ERROR</td></tr><tr><td>MOD(Factor * 2, 9)</td><td>4</td></tr></table>	Expression	Evaluates to	MID(MyName, 4, 4) & "ol"	"phenol"	QualityConfirmed AND (Factor >= 6.5)	TRUE	20 + ASC(Quality)	88	QualityConfirmed + 3	ERROR	MOD(Factor * 2, 9)	4	5
Expression	Evaluates to													
MID(MyName, 4, 4) & "ol"	"phenol"													
QualityConfirmed AND (Factor >= 6.5)	TRUE													
20 + ASC(Quality)	88													
QualityConfirmed + 3	ERROR													
MOD(Factor * 2, 9)	4													
1(b)(ii)	<table><tr><th>Variable</th><th>Data type</th></tr><tr><td>QualityConfirmed</td><td>BOOLEAN</td></tr><tr><td>DayNumber</td><td>INTEGER</td></tr><tr><td>Factor</td><td>REAL</td></tr><tr><td>Quality</td><td>CHAR</td></tr><tr><td>MyName</td><td>STRING</td></tr></table> One mark per answer	Variable	Data type	QualityConfirmed	BOOLEAN	DayNumber	INTEGER	Factor	REAL	Quality	CHAR	MyName	STRING	5
Variable	Data type													
QualityConfirmed	BOOLEAN													
DayNumber	INTEGER													
Factor	REAL													
Quality	CHAR													
MyName	STRING													

145. M/J 18/P23/Q2/b,c(ii)

2(b)

Feature	Answer
A line number containing an example of an integer assignment statement	8, 9, 10, 12, 17, 34, 45
A line number containing the start of a selection structure	14, 19, 23
A line number containing the end of a selection structure	28, 29, 30
The upper bound of the <code>Mark</code> array	100
The number of dimensions of the <code>Mark</code> array	1
The name for the type of loop structure used	'post condition'
A line number containing an unnecessary assignment statement	10
The number of times that <code>OUTPUT</code> is called	100
The number of local variables	4

9

2(c)(ii)	CASE OF ThisMark > 74: Grade ← "Distinction" DGradeCount ← DGradeCount + 1 60 TO 74: Grade ← "Merit" 40 TO 59: Grade ← "Pass" OTHERWISE Grade ← "Fail" ENDCASE One mark for each of: 1 CASE OF ThisMark ... ENDCASE 2 Three grade ranges with corresponding assignment of Grade 3 DGradeCount increment within CASE clause 4 OTHERWISE / fourth grade range with correct assignment of Grade	4
----------	---	---

146. M/J 18/P23/Q3/a

3(a)	Parameters Accept arguments	1
------	---	---

147. M/J 18/P23/Q7

7	'Pseudocode' solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> FUNCTION IsFactor (Num1: INTEGER, Num2: INTEGER) RETURNS BOOLEAN IF Num2 <> 0 THEN IF MOD (Num1, Num2) = 0 THEN RETURN TRUE ENDIF ENDIF RETURN FALSE ENDFUNCTION </pre> 1 mark for each of the following: 1 Correct Function heading (including parameters) and ending 2 Check that Num2 is not zero 3 Mechanism to ensure no call to MOD (or equivalent) if Num2 is zero 4 Use of MOD function or alternative 5 Check value of remainder 6 Return Boolean value	6
---	--	---

Q7: Visual Basic

```
Function IsFactor(Num1 As Integer, Num2 As Integer) As Boolean

    If Num2 <> 0 Then
        If Num1 Mod Num2 = 0 Then
            Return True
        End If
    End if
    Return False

End Function
```

Q7: Pascal

```
function IsFactor(Num1,Num2 : integer) : boolean;

begin
    if Num2 <> 0 then
        begin
            if Num1 MOD Num2 = 0 then
                Return True;
            end;
            Return False;
        end;
    end;
```

Q7: Python

```
def IsFactor (Num1, Num2):
    if Num2 != 0:
        if Num1 % Num2 == 0:
            Return True
    Return False
```

148. O/N 17/P22/Q1/c,d

1(c)	Explanation includes: • to pass values to/from the subroutine • to produce re-useable code • to avoid global variables • to allow recursion One mark per answer	Max 3
1(d)(i)	27: MyGrade assigned the value "Fail" 101: Output the text "Invalid Value Entered" Ignore minor spelling mistakes	2

1(d)(ii)	<pre> IF MyMark >= 75 AND MyMark <=100 THEN MyGrade ← "Distinction" ELSE IF MyMark >= 35 AND MyMark <=74 THEN MyGrade ← "Pass" ELSE IF MyMark >= 0 AND MyMark <=34 THEN MyGrade ← "Fail" ELSE OUTPUT "Invalid value entered" ENDIF ENDIF ENDIF ENDIF ENDIF ENDIF </pre> One mark for each of: • One correct range test • 'IF' equivalent (nested or not) to three CASE range tests... • ... with three corresponding assignments • Equivalent of CASE OTHERWISE with corresponding OUTPUT statement • Matching (three) ENDIFs (Or one if ELSIFs used) Max 4 if solution doesn't work under all circumstances // is not functionally equivalent to CASE	5
----------	---	----------

149. M/J 17/P21/Q1/b(ii,iii),c

1(b)(ii)	Logical / Boolean	1								
1(b)(iii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>FlagA AND (FlagB OR FlagC)</td><td>TRUE</td></tr><tr><td>FlagA AND (FlagB AND FlagC)</td><td>FALSE</td></tr><tr><td>(NOT FlagA) OR (NOT FlagC)</td><td>FALSE</td></tr></table> 1 mark per answer	Expression	Evaluates to	FlagA AND (FlagB OR FlagC)	TRUE	FlagA AND (FlagB AND FlagC)	FALSE	(NOT FlagA) OR (NOT FlagC)	FALSE	3
Expression	Evaluates to									
FlagA AND (FlagB OR FlagC)	TRUE									
FlagA AND (FlagB AND FlagC)	FALSE									
(NOT FlagA) OR (NOT FlagC)	FALSE									

1(c)	<pre> MyCount ← 100 WHILE MyCount < 201 Output MyCount MyCount ← MyCount + 2 ENDWHILE 1 mark for each of the following: • Counter initialisation • While ... End loop • Method for choosing (correct range of) even numbers • Output all even numbers in the range Note: Counter variable name must be consistent </pre>	4
------	--	---

150. M/J 17/P21/Q3

3	<pre> FUNCTION StringClean(<u>Instring STRING</u>) RETURNS <u>STRING</u> DECLARE NextChar : <u>CHAR</u> DECLARE <u>OutString</u> : STRING <u>OutString ← ""</u> // initialise the return string // loop through Instring to produce OutString FOR n ← 1 TO <u>LENGTH(InString)</u> // from first to last NextChar ← <u>MID(Instring, n, 1)</u> //get next character and NextChar ← <u>LCASE(NextChar)</u> //convert to lower case IF <u>NextChar >= 'a' AND NextChar <= 'z'</u> //check if alphabetic THEN <u>OutString ← OutString & NextChar</u> //add to OutString ENDIF ENDFOR <u>RETURN OutString</u> // return value ENDFUNCTION One mark per <u>underlined</u> word / expression </pre>	11
---	--	----

151. M/J 17/P21/Q6/a

6(a)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> FUNCTION ValidatePassword(InString : STRING) RETURNS BOOLEAN DECLARE LCaseChar, UCaseChar, NumChar, n : INTEGER DECLARE NextChar : CHAR DECLARE ReturnFlag : BOOLEAN ReturnFlag ← TRUE LCaseChar ← 0, UCaseChar ← 0, NumChar ← 0 FOR n ← 1 TO LENGTH(InString) NextChar ← MID(InString,n,1) IF NextChar >= 'a' AND NextChar <= 'z' THEN LCaseChar ← LCaseChar + 1 ELSE IF NextChar >= 'A' AND NextChar <= 'Z' THEN UCaseChar ← UCaseChar + 1 ELSE IF NextChar >= '0' AND NextChar <= '9' THEN NumChar ← NumChar + 1 ELSE ReturnFlag ← False //invalid character ENDIF ENDIF ENDIF ENDIF ENDFOR IF Not (LCaseChar>=2 AND UCaseChar>= 2 AND NumChar>= 3) THEN ReturnFlag ← FALSE ENDIF RETURN (ReturnFlag) ENDFUNCTION </pre> 1 mark for each of the following: 1. Correct Function heading and ending 2. Declaring three counter variables (upper, lower, numeric) 3. Initialising counters 4. Correct loop 5. Picking up NextChar from InString 6. Check and count number of lower case 7. Check and count number of upper case	10
------	--	-----------

Q6 (a): Visual Basic

```
Function ValidatePassword(InString As String) As Boolean
    Dim LCaseChar, UCaseChar, NumChar As Integer
    Dim NextChar As Char
    Dim ReturnFlag As Boolean
    Dim n As Integer
    ReturnFlag = TRUE
    LCaseChar = 0
    UCaseChar = 0
    NumChar = 0

    For n = 1 to Len(InString)
        NextChar = Mid(InString, n, 1)
        If NextChar >= 'a' And NextChar <= 'z' Then
            LCaseChar = LCaseChar + 1
        Else
            If NextChar >= 'A' And NextChar <= 'Z' Then
                UCaseChar = UCaseChar + 1
            Else
                If NextChar >= '0' And NextChar <= '9' Then
                    NumChar = NumChar + 1
                Else
                    ReturnFlag = False    //invalid character
                End If
            End If
        End If
    Next

    If NOT (LCaseChar >= 2 And UCaseChar >= 2 And NumChar >= 3) Then
        ReturnFlag = FALSE
    End If

    Return(ReturnFlag)

End Function
```


Q6 (a): Pascal

```
Function ValidatePassword(InString : String): Boolean;
  Var LCaseChar, UCaseChar, NumChar : Integer;
  Var NextChar : Char;
  Var ReturnFlag : Boolean;
  Var n : Integer;

  begin
    ReturnFlag := TRUE;
    LCaseChar := 0;
    UCaseChar := 0;
    NumChar := 0;

    For n := 1 to Length(InString) do
      begin
        NextChar := Copy(InString,n,1);
        If NextChar >= 'a' And NextChar <= 'z' Then
          LCaseChar := LCaseChar + 1
        Else If NextChar >= 'A' AND NextChar <= 'Z' Then
          UCaseChar := UCaseChar + 1
        Else If NextChar >= '0' AND NextChar <= '9' Then
          NumChar := NumChar + 1
        Else
          ReturnFlag := False      //invalid character
      end
    end

    If NOT(LCaseChar >= 2 And UCaseChar >= 2 And NumChar >=3)then
      ReturnFlag := False;

    ValidatePassword := ReturnFlag
  end;
```

Q6 (a): Python

```
def ValidatePassword(InString):
    # lCaseChar, uCaseChar, numChar : INTEGER
    # nextChar : CHAR
    # returnFlag : BOOLEAN
    # n : INTEGER
    returnFlag = TRUE
    lCaseChar = 0
    uCaseChar = 0
    numChar = 0

    for n in range (0, Len(InString))
        nextChar = InString[n]
        If nextChar >= 'a' and nextChar <= 'z':
            lCaseChar = lCaseChar + 1
        ELSE:
            IF nextChar >= 'A' and nextChar <= 'Z':
                uCaseChar = uCaseChar + 1
            ELSE:
                IF nextChar >= '0' and nextChar <= '9':
                    numChar = numChar + 1
                ELSE:
                    returnFlag = False    //invalid character

    IF Not (lCaseChar >= 2 and uCaseChar >= 2 and numChar >= 3):
        returnFlag = FALSE

    Return (returnFlag)

#next code block
```

6(a)	8. Check and count number of numeric 9. Check for invalid character 10. Combine all four tests into a single Boolean value 11. Returning correct Boolean value	
------	---	--

152. M/J 17/P22/Q1/b,c

1(b)(i)	- Integer / Real / Single / Double / Floating Point / Float - Boolean	2								
1(b)(ii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>(FlagA AND FlagB) OR FlagC</td><td>TRUE</td></tr><tr><td>FlagA AND (FlagB OR FlagC)</td><td>TRUE</td></tr><tr><td>(NOT FlagA) OR (NOT FlagC)</td><td>FALSE</td></tr></table> 1 mark per answer	Expression	Evaluates to	(FlagA AND FlagB) OR FlagC	TRUE	FlagA AND (FlagB OR FlagC)	TRUE	(NOT FlagA) OR (NOT FlagC)	FALSE	3
Expression	Evaluates to									
(FlagA AND FlagB) OR FlagC	TRUE									
FlagA AND (FlagB OR FlagC)	TRUE									
(NOT FlagA) OR (NOT FlagC)	FALSE									

1(c)	<pre>MyCount ← 101 REPEAT OUTPUT MyCount MyCount ← MyCount + 2 UNTIL MyCount > 199</pre> 1 mark for each of the following: • Counter initialisation • Repeat ... Until loop • Method for choosing (correct range of) odd numbers • Output all odd numbers in the range Note: Counter variable name must be consistent	4
------	--	----------

153. M/J 17/P22/Q3

3	<pre>FUNCTION ExCamel (InString: STRING) RETURNS STRING DECLARE NextChar : CHAR DECLARE OutString : STRING DECLARE n : INTEGER OutString ← "" // initialise the return string // loop through InString to produce OutString FOR n ← 1 TO LENGTH(InString) // from first to last NextChar ← MID(InString, n, 1) // get next character IF NextChar >= 'A' AND NextChar <= 'Z' // check if upper case // NextChar = UCASE(NextChar) THEN IF n > 1 // if not first character THEN OutString ← OutString & " " // add space to OutString ENDIF NextChar ← LCASE(NextChar) // make NextChar lower case ENDIF OutString ← OutString & NextChar // add Nextchar to OutString ENDFOR RETURN OutString // return value ENDFUNCTION</pre> 1 mark per underlined word / expression	Max 11
---	---	---------------

154. M/J 17/P22/Q4/a,b

4(a)	• Functions • Procedures • Global / Local variables 1 mark per item			Max 2									
4(b)	<table><tr><th>Name of parameter passing method</th><th>Value output</th><th>Explanation</th></tr><tr><td>(Call) by reference</td><td>5</td><td> • The <u>address of</u> the variable is passed. • <u>Original value is changed</u> when parameter changed in called module. </td></tr><tr><td>(Call) by value</td><td>4</td><td> • A <u>copy of</u> the variable itself is passed. • <u>Original value not changed</u> when parameter changed in called module. </td></tr></table> Mark as follows: • 1 mark for each name and value • 1 mark per bullet in explanation			Name of parameter passing method	Value output	Explanation	(Call) by reference	5	• The <u>address of</u> the variable is passed. • <u>Original value is changed</u> when parameter changed in called module.	(Call) by value	4	• A <u>copy of</u> the variable itself is passed. • <u>Original value not changed</u> when parameter changed in called module.	6
Name of parameter passing method	Value output	Explanation											
(Call) by reference	5	• The <u>address of</u> the variable is passed. • <u>Original value is changed</u> when parameter changed in called module.											
(Call) by value	4	• A <u>copy of</u> the variable itself is passed. • <u>Original value not changed</u> when parameter changed in called module.											

155. M/J 17/P22/Q6/a

6(a)	Pseudocode solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix. <pre> FUNCTION ValidateRegistration(Registration : STRING) RETURNS BOOLEAN DECLARE UCaseChar, NumChar : INTEGER DECLARE NextChar : CHAR DECLARE ReturnFlag : BOOLEAN DECLARE n : INTEGER ReturnFlag ← TRUE ValidateRegistration ← True IF LEN(Registration) < 6 OR LEN(Registration) > 9 //check length THEN ReturnFlag ← False ELSE FOR n ← 1 TO 3 //check for 3 upper case alpha NextChar ← MID(Registration, n, 1) IF NextChar < 'A' AND NextChar > 'Z' THEN ReturnFlag ← False ENDIF ENDFOR FOR n ← 4 TO 5 //check for 2 numeric NextChar ← MID(Registration, n, 1) IF NextChar < '0' AND NextChar > '9 THEN ReturnFlag ← False ENDIF ENDFOR FOR n ← 6 TO LEN(Registration) //check remaining characters NextChar ← MID(Registration, n, 1) IF NextChar < 'A' AND NextChar > 'Z' THEN ReturnFlag ← False ENDIF ENDFOR ENDIF RETURN (ReturnFlag) ENDFUNCTION </pre>	Max 9
------	--	--------------

6(a)	1 mark for each of the following: 1. Correct Function heading and ending 2. Check for correct length 3. Extract first three characters 4. Check first three characters are capitals 5. Extract characters four and five 6. Check characters four and five are numeric 7. Extract remaining characters 8. Check remaining characters are capitals 9. Combine all four tests results into a single Boolean value 10. Return a Boolean value	
------	---	--

156. M/J 17/P23/Q1/b(ii,iii),c

1(b)(ii)	Logical / Boolean	1								
1(b)(iii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>FlagA AND (FlagB OR FlagC)</td><td>TRUE</td></tr><tr><td>FlagA AND (FlagB AND FlagC)</td><td>FALSE</td></tr><tr><td>(NOT FlagA) OR (NOT FlagC)</td><td>FALSE</td></tr></table> 1 mark per answer	Expression	Evaluates to	FlagA AND (FlagB OR FlagC)	TRUE	FlagA AND (FlagB AND FlagC)	FALSE	(NOT FlagA) OR (NOT FlagC)	FALSE	3
Expression	Evaluates to									
FlagA AND (FlagB OR FlagC)	TRUE									
FlagA AND (FlagB AND FlagC)	FALSE									
(NOT FlagA) OR (NOT FlagC)	FALSE									
1(c)	MyCount ← 100 WHILE MyCount < 201 Output MyCount MyCount ← MyCount + 2 ENDWHILE 1 mark for each of the following: • Counter initialisation • While ... End loop • Method for choosing (correct range of) even numbers • Output all even numbers in the range Note: Counter variable name must be consistent	4								

157. M/J 17/P23/Q3

3	<pre> FUNCTION StringClean(<u>Instring</u> <u>STRING</u>) RETURNS <u>STRING</u> DECLARE NextChar : <u>CHAR</u> DECLARE <u>OutString</u> : STRING <u>OutString</u> ← "" // initialise the return string // loop through Instring to produce OutString FOR n ← 1 TO <u>LENGTH(InString)</u> // from first to last NextChar ← <u>MID(Instring, n, 1)</u> //get next character and NextChar ← <u>LCASE(NextChar)</u> //convert to lower case IF <u>NextChar >= 'a' AND NextChar <= 'z'</u> //check if alphabetic THEN <u>OutString ← OutString & NextChar</u> //add to OutString ENDIF ENDFOR <u>RETURN OutString</u> // return value ENDFUNCTION </pre> One mark per <u>underlined</u> word / expression	11
---	--	----

158. M/J 17/P23/Q6/a

6(a)	Pseudocode solution included here for development and clarification of mark scheme. Programming language solutions appear in the Appendix. <pre> FUNCTION ValidatePassword(InString : STRING) RETURNS BOOLEAN DECLARE LCaseChar, UCaseChar, NumChar, n : INTEGER DECLARE NextChar : CHAR DECLARE ReturnFlag : BOOLEAN ReturnFlag ← TRUE LCaseChar ← 0, UCaseChar ← 0, NumChar ← 0 FOR n ← 1 TO LENGTH(InString) NextChar ← MID(InString,n,1) IF NextChar >= 'a' AND NextChar <= 'z' THEN LCaseChar ← LCaseChar + 1 ELSE IF NextChar >= 'A' AND NextChar <= 'Z' THEN UCaseChar ← UCaseChar + 1 ELSE IF NextChar >= '0' AND NextChar <= '9' THEN NumChar ← NumChar + 1 ELSE ReturnFlag ← False //invalid character ENDIF ENDIF ENDIF ENDIF ENDFOR IF Not (LCaseChar>=2 AND UCaseChar>= 2 AND NumChar>= 3) THEN ReturnFlag ← FALSE ENDIF RETURN (ReturnFlag) ENDFUNCTION </pre> 1 mark for each of the following: 1. Correct Function heading and ending 2. Declaring three counter variables (upper, lower, numeric) 3. Initialising counters 4. Correct loop 5. Picking up NextChar from InString 6. Check and count number of lower case 7. Check and count number of upper case	10
------	--	-----------

Q6 (a): Visual Basic

```
Function ValidatePassword(InString As String) As Boolean
    Dim LCaseChar, UCaseChar, NumChar As Integer
    Dim NextChar As Char
    Dim ReturnFlag As Boolean
    Dim n As Integer
    ReturnFlag = TRUE
    LCaseChar = 0
    UCaseChar = 0
    NumChar = 0

    For n = 1 to Len(InString)
        NextChar = Mid(InString, n, 1)
        If NextChar >= 'a' And NextChar <= 'z' Then
            LCaseChar = LCaseChar + 1
        Else
            If NextChar >= 'A' And NextChar <= 'Z' Then
                UCaseChar = UCaseChar + 1
            Else
                If NextChar >= '0' And NextChar <= '9' Then
                    NumChar = NumChar + 1
                Else
                    ReturnFlag = False    //invalid character
                End If
            End If
        End If
    Next

    If NOT (LCaseChar >= 2 And UCaseChar >= 2 And NumChar >= 3) Then
        ReturnFlag = FALSE
    End If

    Return(ReturnFlag)

End Function
```

Q6 (a): Pascal

```
Function ValidatePassword(InString : String): Boolean;
  Var LCaseChar, UCaseChar, NumChar : Integer;
  Var NextChar : Char;
  Var ReturnFlag : Boolean;
  Var n : Integer;

  begin
    ReturnFlag := TRUE;
    LCaseChar := 0;
    UCaseChar := 0;
    NumChar := 0;

    For n := 1 to Length(InString) do
      begin
        NextChar := Copy(InString,n,1);
        If NextChar >= 'a' And NextChar <= 'z' Then
          LCaseChar := LCaseChar + 1
        Else If NextChar >= 'A' AND NextChar <= 'Z' Then
          UCaseChar := UCaseChar + 1
        Else If NextChar >= '0' AND NextChar <= '9' Then
          NumChar := NumChar + 1
        Else
          ReturnFlag := False      //invalid character
      end
    end

    If NOT(LCaseChar >= 2 And UCaseChar >= 2 And NumChar >=3)then
      ReturnFlag := False;

    ValidatePassword := ReturnFlag
  end;
```

Q6 (a): Python

```
def ValidatePassword(InString):
    # lCaseChar, uCaseChar, numChar : INTEGER
    # nextChar : CHAR
    # returnFlag : BOOLEAN
    # n : INTEGER
    returnFlag = TRUE
    lCaseChar = 0
    uCaseChar = 0
    numChar = 0

    for n in range (0, Len(InString))
        nextChar = InString[n]
        If nextChar >= 'a' and nextChar <= 'z':
            lCaseChar = lCaseChar + 1
        ELSE:
            IF nextChar >= 'A' and nextChar <= 'Z':
                uCaseChar = uCaseChar + 1
            ELSE:
                IF nextChar >= '0' and nextChar <= '9':
                    numChar = numChar + 1
                ELSE:
                    returnFlag = False    //invalid character

    IF Not (lCaseChar >= 2 and uCaseChar >= 2 and numChar >= 3):
        returnFlag = FALSE

    Return (returnFlag)

#next code block
```

6(a)	8. Check and count number of numeric 9. Check for invalid character 10. Combine all four tests into a single Boolean value 11. Returning correct Boolean value	
------	---	--

159. O/N 16/P21/Q3/a,b,c,d

3	(a)	(i)	7	[1]
		(ii)	103	[1]
		(iii)	'K'	[1]
		(iv)	"come"	[1]

```

(b) (i) PROCEDURE CalculateCustomerID
    OUTPUT "Key in surname"
    INPUT Surname
    Length ← CHARACTERCOUNT(Surname)
    CustomerID ← 0
    FOR i ← 1 TO Length
        //NextChar is a single character from Surname
        Nextchar ← 1 SUBSTR(Surname, i, 1) // ONECHAR(Surname, i)
        NextCodeNumber ← ASC(NextChar)
        CustomerID ← CustomerID + NextCodeNumber
    ENDFOR
    OUTPUT "Customer ID is ", CustomerID
ENDPROCEDURE

```

One mark per phrase in **bold**

[3]

- (ii)** 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```

PROCEDURE CalculateCustomerID
    DECLARE Surname : STRING
    DECLARE NextChar : CHAR
    DECLARE NextCodeNumber, i, CustomerID, SLength : INTEGER
    OUTPUT "Key in surname"
    INPUT Surname
    SLength ← LEN(Surname)
    CustomerID ← 0
    FOR i ← 1 TO SLength
        //NextChar is a single character from Surname
        Nextchar ← MID(Surname, i, 1)
        NextCodeNumber ← ASC(NextChar)
        CustomerID ← CustomerID + NextCodeNumber
    ENDFOR
    OUTPUT "Customer ID is ", CustomerID
ENDPROCEDURE

```

Mark as follows:

- Declaration of Surname as STRING and NextChar as CHAR and any three INTEGERS
- Prompt and Input
- Calculation of string length
- FOR Loop to process all characters in the string
- Assignment to NextChar in a loop
- Assignment to NextCodeNumber in a loop
- Totalling CustomerID in a loop
- Output following a loop

[6]

3(b)(ii): Visual Basic

```

Dim Surname As String
Dim NextChar As Char
Dim NextCodeNumber As Integer
Dim i As Integer
Dim CustomerID As Integer
Dim SLength As Integer

Console.Write("Key in surname ")
Surname = Console.ReadLine
SLength = Len(Surname)
CustomerID = 0
    For i = 1 To SLength
        \\ NextChar is a single character from surname
        NextChar = Mid(Surname, i, 1)
        NextCodeNumber = Asc(NextChar)
        CustomerID = CustomerID + NextCodeNumber
    Next

    Console.WriteLine("Customer ID is " & CustomerID)

```

3(b)(ii): Pascal

```

Var Surname : string;
    SLength, i, CustomerID, NextCodeNumber : integer;
    NextChar : char;
begin
    Writeln ('Enter the surname: ');
    Readln (Surname);
    SLength := Length(Surname);
    CustomerID := 0;
    For i := 1 to SLength do
        begin
            NextChar := SurName[i];
            NextCodeNumber := Ord(NextChar);
            CustomerID := CustomerID + NextCodeNumber;
        end;
    Writeln ('Customer ID is ', CustomerID);
    Readln;
end.

```

3(b)(ii): Python

```

# Surname String
# NextChar Char
# NextCodeNumber, I, CustomerID, SLength Integer

Surname = input("Key in Surname ")
SLength = len(Surname)
CustomerID = 0

for i in range(SLength):
    # NextChar is a single character from surname
    NextChar = Surname[i]
    NextCodeNumber = ord(NextChar)
    CustomerID = CustomerID + NextCodeNumber

print("Customer ID is " + str(CustomerID))

```

(c) (i) Visual Basic

Function CalculateCustomerID(ByVal AnyName AS STRING) As Integer

Pascal

FUNCTION CalculateCustomerID(AnyName : STRING) : INTEGER

Python

def CalculateCustomerID(AnyName) :

Mark as follows:

- Correct keyword + Function name
- Single input parameter of correct type
- Return parameter type

[3]**(ii) Visual Basic**

Return customerID // CalculateCustomerID = CustomerID

Pascal

Result := CustomerID // CalculateCustomerID := CustomerID

Python

Return CustomerID

[1]**(iii) Visual Basic**

ThisID = CalculateCustomerID ("Wilkes")

Pascal

ThisID := CalculateCustomerID ('Wilkes')

Python

ThisID = CalculateCustomerID ("Wilkes")

One mark per underlined element

[3]**(d) (i)**

- Built-in functions are made available by the programming language / already in the system
- Built-in functions are ready made and tested
- User-defined functions can be modified // built-in cannot be modified
- User defined functions can be designed to meet the user's requirements
- User-defined functions can only be used in that program / module

[Max 2]

- (ii)**
- They have an identifier name
 - They return a value
 - They have none, one or more arguments
 - Both perform a specific task
 - Both represent re-usable code
 - Both are 'called'

[Max 2]**160. O/N 16/P21/Q5**

- | | | |
|----------|---------------|------------|
| 5 | (i) 13 / 13.0 | [1] |
| | (ii) 18.6 | [1] |
| | (iii) TRUE | [1] |
| | (iv) 32 | [1] |
| | (v) 22 | [1] |

161. O/N 16/P22/Q2/a,b,c,d

- 2 (a) (i)** 'e' [1]
- (ii) "Cat-food" [1]
- (iii) 213 [1]
- (b) (i)** 03 // 3 [1]
- (ii) 29 [1]
- (iii) 14 // 16 [1]
- (iv) 18 // 24 // 25 [1]
- (c)** NextChar <> '*' [1]
- (d) (i)**

				Numbers			
i	j	NextChar	NextNumberString		1	2	3
1	1	'2'					
			" "				
			"2"				
	2	'3'	"23"				
	3	'*'			23		
2	4	'7'	" "				
			"7"				
	5	'3'	"73"				
	6	'1'					
	7	'*'	"731"			731	
3	8	'5'	" "				
	9	'*'	"5"				5
4	10	'#'					

One mark for each of columns 1 to 4

One mark for numbers 2 & 3 as shown in box

[5]

(ii) One mark for each of:

- Isolates / separates / splits up each numeric string / the numbers / data string separated by '*'
- Converts each numeric string / each number into an integer and
- Stores each integer in array (Numbers)

[Max. 2]

162. O/N 16/P22/Q4/a,b,c

4 (a) `INT (RND () * 150 ) + 1`

One mark for each part as follows:

- `RND () * 150`
- `+ 1`
- `INT ()`

[3]

- (b) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

Expected Loop-based solution:

```
DECLARE i , NextNumber : INTEGER
FOR i ← 1 TO 4
    NextNumber ← 1 + INT (Rnd () * 150)
    OUTPUT NextNumber
ENDFOR
```

Mark as follows:

- Declaration of all variables used including data types
- Loop
- Assignment / calculation of (four) different random numbers (0 to 150) in a loop
- Output of four values

ALTERNATIVE Non-Loop version

```
DECLARE Num1, Num2, Num3, Num4 : INTEGER

Num1 ← INT (RND () * 150) + 1
Num2 ← INT (RND () * 150) + 1
Num3 ← INT (RND () * 150) + 1
Num4 ← INT (RND () * 150) + 1

OUTPUT Num1, Num2, Num3, Num4
```

Mark as follows:

- Declaration of all variables used including data types
- Assignment of four different random numbers (0 to 150)
- Assignment to four separate variables
- Output of four values

[4]

Q4 (b): Visual Basic

```
Randomize()  
Dim i As Integer  
Dim NextNumber As Integer  
For i = 1 To 4  
    NextNumber = 1 + Int(Rnd() * 150)  
    Console.WriteLine(NextNumber)  
Next
```

OR

```
Randomize()  
Dim Num1, Num2, Num3, Num4 As Integer  
Num1 = 1 + Int(Rnd() * 150)  
Num2 = 1 + Int(Rnd() * 150)  
Num3 = 1 + Int(Rnd() * 150)  
Num4 = 1 + Int(Rnd() * 150)  
Console.WriteLine(Num1, Num2, Num3, Num4)
```

Q4 (b): Pascal

```
Var i : Integer;  
    NextNumber : Integer;  
Begin  
    Randomize;  
    For i := 1 To 4 Do  
        Begin  
            NextNumber := 1 + Random(150);  
            Writeln(NextNumber);  
        End;  
    Readln;  
End.
```

OR

```
Var Num1, Num2, Num3, Num4 : Integer;  
Begin  
    Randomize;  
    Num1 := 1 + Random(150);  
    Num2 := 1 + Random(150);  
    Num3 := 1 + Random(150);  
    Num4 := 1 + Random(150);  
    Writeln(Num1, Num2, Num3, Num4);  
    Readln;  
End.
```

Q4 (b): Python

```
import random
# i : Integer
# NextNumber : Integer

for i in range(1, 5) :
    NextNumber = 1 + int(150 * random.random())
    print(NextNumber)
```

Alternative:

```
import random
# i Integer
# NextNumber Integer
for i in range(1, 5) :
    NextNumber = random.randint(1, 150)
    print(NextNumber)
```

OR

```
import random
# i Integer
# Num1, Num2, Num3, Num4 Integer

Num1 = random.randint(1, 150)
Num2 = random.randint(1, 150)
Num3 = random.randint(1, 150)
Num4 = random.randint(1, 150)

print(Num1, Num2, Num3, Num4)
```

(c) Visual Basic

Function GenerateNumber (ByVal AnyName AS INTEGER) AS INTEGER

Pascal

FUNCTION GenerateNumber (AnyName : INTEGER) : INTEGER

Python

def GenerateNumber (AnyName):

- Mark as follows:
- Correct keyword + Function name
- Single input parameter of correct type
- Return parameter type

[3]

163. O/N 16/P22/Q5/c(i,ii)

- (c) (i) • a nested // an inner and an outer
 • count controlled // incremented loop(s)

[2]

(ii)

...	True
...	False

Both answers must be correct

[1]

164. O/N 16/P22/Q6

- | | | | |
|----------|--------------|-----------|-----|
| 6 | (i) | 10 / 10.0 | [1] |
| | (ii) | 18.4 | [1] |
| | (iii) | 41 | [1] |
| | (iv) | TRUE | [1] |
| | (v) | 12.4 | [1] |

165. O/N 16/P23/Q3/a,b,c,d

- | | | | | |
|----------|------------|--------------|--------|-----|
| 3 | (a) | (i) | 7 | [1] |
| | | (ii) | 103 | [1] |
| | | (iii) | 'K' | [1] |
| | | (iv) | "come" | [1] |

(b) (i) `PROCEDURE CalculateCustomerID`
 `OUTPUT "Key in surname"`
 `INPUT Surname`
 `Length ← CHARACTERCOUNT(Surname)`
 `CustomerID ← 0`
 `FOR i ← 1 TO Length`
 `//NextChar is a single character from Surname`
 `Nextchar ← 1 SUBSTR(Surname, i, 1) // ONECHAR(Surname, i)`
 `NextCodeNumber ← ASC(NextChar)`
 `CustomerID ← CustomerID + NextCodeNumber`
 `ENDFOR`
 `OUTPUT "Customer ID is ", CustomerID`
`ENDPROCEDURE`

One mark per phrase in **bold** [3]

- (ii) 'Pseudocode' solution included here for development and clarification of mark scheme. Programming language example solutions appear in the Appendix.

```
PROCEDURE CalculateCustomerID
  DECLARE Surname : STRING
  DECLARE NextChar : CHAR
  DECLARE NextCodeNumber, i, CustomerID, SLength : INTEGER
  OUTPUT "Key in surname"
  INPUT Surname
  SLength ← LEN(Surname)
  CustomerID ← 0
  FOR i ← 1 TO SLength
    //NextChar is a single character from Surname
    Nextchar ← MID(Surname, i, 1)
    NextCodeNumber ← ASC(NextChar)
    CustomerID ← CustomerID + NextCodeNumber
  ENDFOR
  OUTPUT "Customer ID is ", CustomerID
ENDPROCEDURE
```

Mark as follows:

- Declaration of Surname as STRING and NextChar as CHAR and any three INTEGERS
- Prompt and Input
- Calculation of string length
- FOR Loop to process all characters in the string
- Assignment to NextChar in a loop
- Assignment to NextCodeNumber in a loop
- Totalling CustomerID in a loop
- Output following a loop

[6]

3(b)(ii): Visual Basic

```
Dim Surname As String
Dim NextChar As Char
Dim NextCodeNumber As Integer
Dim i As Integer
Dim CustomerID As Integer
Dim SLength As Integer

Console.Write("Key in surname ")
Surname = Console.ReadLine
SLength = Len(Surname)
CustomerID = 0
    For i = 1 To SLength
        \\ NextChar is a single character from surname
        NextChar = Mid(Surname, i, 1)
        NextCodeNumber = Asc(NextChar)
        CustomerID = CustomerID + NextCodeNumber
    Next

    Console.WriteLine("Customer ID is " & CustomerID)
```

3(b)(ii): Pascal

```
Var Surname : string;
    SLength, i, CustomerID, NextCodeNumber : integer;
    NextChar : char;
begin
    Writeln ('Enter the surname: ');
    Readln (Surname);
    SLength := Length(Surname);
    CustomerID := 0;
    For i := 1 to SLength do
        begin
            NextChar := SurName[i];
            NextCodeNumber := Ord(NextChar);
            CustomerID := CustomerID + NextCodeNumber;
        end;
    Writeln ('Customer ID is ', CustomerID);
    Readln;
end.
```

3(b)(ii): Python

```
# Surname String
# NextChar Char
# NextCodeNumber, I, CustomerID, SLength Integer

Surname = input("Key in Surname ")
SLength = len(Surname)
CustomerID = 0

for i in range(SLength):
    # NextChar is a single character from surname
    NextChar = Surname[i]
    NextCodeNumber = ord(NextChar)
    CustomerID = CustomerID + NextCodeNumber

print("Customer ID is " + str(CustomerID))
```

(c) (i) Visual Basic

Function CalculateCustomerID(ByVal AnyName AS STRING) As Integer

Pascal

FUNCTION CalculateCustomerID(AnyName : STRING) : INTEGER

Python

def CalculateCustomerID(AnyName) :

Mark as follows:

- Correct keyword + Function name
- Single input parameter of correct type
- Return parameter type

[3]**(ii) Visual Basic**

Return customerID // CalculateCustomerID = CustomerID

Pascal

Result := CustomerID // CalculateCustomerID := CustomerID

Python

Return CustomerID

[1]**(iii) Visual Basic**

ThisID = CalculateCustomerID ("Wilkes")

Pascal

ThisID := CalculateCustomerID ('Wilkes')

Python

ThisID = CalculateCustomerID ("Wilkes")

One mark per underlined element

[3]**(d) (i)**

- Built-in functions are made available by the programming language / already in the system

- Built-in functions are ready made and tested
- User-defined functions can be modified // built-in cannot be modified
- User defined functions can be designed to meet the user's requirements
- User-defined functions can only be used in that program / module

[Max 2]

- (ii)**
- They have an identifier name
 - They return a value
 - They have none, one or more arguments
 - Both perform a specific task
 - Both represent re-usable code
 - Both are 'called'

[Max 2]**166. O/N 16/P23/Q5**

- | | | |
|----------|---------------|------------|
| 5 | (i) 13 / 13.0 | [1] |
| | (ii) 18.6 | [1] |
| | (iii) TRUE | [1] |
| | (iv) 32 | [1] |
| | (v) 22 | [1] |

167. M/J 16/P21/Q1/a

1 (a) (i)	<table><tr><th>Item</th><th>Statement</th><th>Selection</th><th>Iteration</th><th>Assignment</th></tr><tr><td>1</td><td>MyScore = 65</td><td></td><td></td><td>✓</td></tr><tr><td>2</td><td>FOR IndexVal = 0 TO 99</td><td></td><td>✓</td><td></td></tr><tr><td>3</td><td>MyArray[3] = ID(MyString,3,2)</td><td></td><td></td><td>✓</td></tr><tr><td>4</td><td>IF MyScore >= 70 THEN</td><td>✓</td><td></td><td></td></tr><tr><td>5</td><td>ENDWHILE</td><td></td><td>✓</td><td></td></tr><tr><td>6</td><td>ELSE Message = "Error"</td><td>✓</td><td></td><td>✓</td></tr></table>					Item	Statement	Selection	Iteration	Assignment	1	MyScore = 65			✓	2	FOR IndexVal = 0 TO 99		✓		3	MyArray[3] = ID(MyString,3,2)			✓	4	IF MyScore >= 70 THEN	✓			5	ENDWHILE		✓		6	ELSE Message = "Error"	✓		✓	6
	Item	Statement	Selection	Iteration	Assignment																																				
	1	MyScore = 65			✓																																				
	2	FOR IndexVal = 0 TO 99		✓																																					
	3	MyArray[3] = ID(MyString,3,2)			✓																																				
	4	IF MyScore >= 70 THEN	✓																																						
	5	ENDWHILE		✓																																					
	6	ELSE Message = "Error"	✓		✓																																				
One mark per row Additional ticks in any row cancels that row																																									
(ii)	<table><tr><th>Item</th><th>Purpose of statement</th></tr><tr><td>1</td><td>Assign <u>65</u> to <u>MyScore</u></td></tr><tr><td>2</td><td>(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times</td></tr><tr><td>3</td><td>Assign 2 chars from position 3/4 in MyString to MyArray element 3/4</td></tr><tr><td>4</td><td>Test if MyScore is <u>greater than or equal to</u> 70</td></tr><tr><td>5</td><td>Marks the end of WHILE / precondition loop //Return to top of loop to check condition</td></tr><tr><td>6</td><td>If a condition is FALSE, variable Message is assigned the value "ERROR"</td></tr></table>					Item	Purpose of statement	1	Assign <u>65</u> to <u>MyScore</u>	2	(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times	3	Assign 2 chars from position 3/4 in MyString to MyArray element 3/4	4	Test if MyScore is <u>greater than or equal to</u> 70	5	Marks the end of WHILE / precondition loop //Return to top of loop to check condition	6	If a condition is FALSE, variable Message is assigned the value "ERROR"	6																					
	Item	Purpose of statement																																							
	1	Assign <u>65</u> to <u>MyScore</u>																																							
	2	(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times																																							
	3	Assign 2 chars from position 3/4 in MyString to MyArray element 3/4																																							
	4	Test if MyScore is <u>greater than or equal to</u> 70																																							
	5	Marks the end of WHILE / precondition loop //Return to top of loop to check condition																																							
	6	If a condition is FALSE, variable Message is assigned the value "ERROR"																																							
Exact wording not important Explanation must refer to variables or values used in code (except for row 5)																																									
(iii)	<table><tr><th>Expression</th><th>Result</th></tr><tr><td>"D" & RIGHT(MyString, 4)</td><td>"Dance"</td></tr><tr><td>LEFT(RIGHT(MyString, 7), 3)</td><td>"ten"</td></tr></table>		Expression	Result	"D" & RIGHT(MyString, 4)	"Dance"	LEFT(RIGHT(MyString, 7), 3)	"ten"	2																																
	Expression	Result																																							
	"D" & RIGHT(MyString, 4)	"Dance"																																							
	LEFT(RIGHT(MyString, 7), 3)	"ten"																																							
Must have correct case Quotation marks optional																																									

168. M/J 16/P21/Q3/b(ii)**(ii)**

'Pseudocode' solution included here for development and clarification of mark scheme.

Programming language solutions appear in the Appendix.

6

```

INPUT StartPos } 1
INPUT NumToChange }
2 { FOR n ← 0 to NumToChange - 1
    OUTPUT " Input new value for position " 3
    INPUT NewChar 4
    Lookup[StartPos + n] ← NewChar 5
ENDFOR
    OUTPUT (NumToChange & " entries changed") 6

```

ALTERNATIVE:

```

INPUT StartPos } 1
INPUT NumToChange }
n ← 0
2 { REPEAT
    OUTPUT " Input new value for position " 3
    INPUT NewChar 4
    Lookup[StartPos + n] ← NewChar 5
    n ← n + 1
UNTIL n = NumToChange
    OUTPUT (NumToChange & " entries changed") 6

```

Mark points as circled, descriptions as below:

1. Two INPUT statements
2. Working loop (using values of n from flowchart)
3. OUTPUT prompt (exact text not specified)
4. INPUT NewChar
5. Assignment of NewChar to correct array element
6. OUTPUT final message after loop (exact text not specified but must include NumToChange or loop counter if value correct at that point)

3 (b)(ii): VB.NET

```
Console.Write("Enter start position: ")
StartPos = Console.ReadLine()
Console.Write("Enter number to change: ")
NumToChange = Console.ReadLine()
For n = 0 To NumToChange - 1
    Console.Write("Enter new value for position: ")
    NewChar = Console.ReadLine()
    Lookup(StartPos + n) = NewChar
Next
Console.WriteLine(NumToChange & " entries changed")
```

ALTERNATIVE:

```
Console.Write("Enter start position: ")
StartPos = Console.ReadLine()
Console.Write("Enter number to change: ")
NumToChange = Console.ReadLine()
n = 0
Do
    Console.Write("Enter new value for position: ")
    NewChar = Console.ReadLine()
    Lookup(StartPos + n) = NewChar
    n = n + 1
Loop Until n = NumToChange
Console.WriteLine(NumToChange & " entries changed")
```

3 (b)(ii): Pascal

```
write('Enter start position: ');
readln(StartPos);
write('Enter number to change: ');
readln(NumToChange);
for n := 0 to NumToChange - 1 do
begin
    write('Input new value for position: ');
    readln(NewChar);
    Lookup[Startpos + n] := NewChar;
end;
writeln(IntToStr(NumToChange) + ' entries changed');
```


(ii)	<table><tr><th>Item</th><th>Purpose of statement</th></tr><tr><td>1</td><td>Assign <u>65</u> to <u>MyScore</u></td></tr><tr><td>2</td><td>(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times</td></tr><tr><td>3</td><td>Assign 2 chars from position 3/4 in <u>MyString</u> to <u>MyArray</u> element 3/4</td></tr><tr><td>4</td><td>Test if <u>MyScore</u> is <u>greater than or equal to 70</u></td></tr><tr><td>5</td><td>Marks the end of WHILE / precondition loop //Return to top of loop to check condition</td></tr><tr><td>6</td><td>If a condition is FALSE, variable <u>Message</u> is assigned the value "ERROR"</td></tr></table>	Item	Purpose of statement	1	Assign <u>65</u> to <u>MyScore</u>	2	(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times	3	Assign 2 chars from position 3/4 in <u>MyString</u> to <u>MyArray</u> element 3/4	4	Test if <u>MyScore</u> is <u>greater than or equal to 70</u>	5	Marks the end of WHILE / precondition loop //Return to top of loop to check condition	6	If a condition is FALSE, variable <u>Message</u> is assigned the value "ERROR"	6
	Item	Purpose of statement														
	1	Assign <u>65</u> to <u>MyScore</u>														
	2	(Start of) loop with loop counter starting from zero & going to 99 / repeating 100 times														
	3	Assign 2 chars from position 3/4 in <u>MyString</u> to <u>MyArray</u> element 3/4														
	4	Test if <u>MyScore</u> is <u>greater than or equal to 70</u>														
	5	Marks the end of WHILE / precondition loop //Return to top of loop to check condition														
	6	If a condition is FALSE, variable <u>Message</u> is assigned the value "ERROR"														
Exact wording not important Explanation must refer to variables or values used in code (except for row 5)																

(iii)	<table><tr><th>Expression</th><th>Result</th></tr><tr><td>"D" & RIGHT(MyString, 4)</td><td>"Dance"</td></tr><tr><td>LEFT(RIGHT(MyString, 7), 3)</td><td>"ten"</td></tr></table>	Expression	Result	"D" & RIGHT(MyString, 4)	"Dance"	LEFT(RIGHT(MyString, 7), 3)	"ten"	2
	Expression	Result						
	"D" & RIGHT(MyString, 4)	"Dance"						
	LEFT(RIGHT(MyString, 7), 3)	"ten"						
Must have correct case Quotation marks optional								

170. M/J 16/P22/Q3/b(ii)

- (ii) 'Pseudocode' solution included here for development and clarification of mark scheme.
Programming language solutions appear in the Appendix.

6

```

INPUT StartPos } 1
INPUT NumToChange }
2 { FOR n ← 0 to NumToChange - 1
    OUTPUT " Input new value for position " 3
    INPUT NewChar 4
    Lookup[StartPos + n] ← NewChar 5
  ENDFOR
  OUPUT (NumToChange & " entries changed") 6

```

ALTERNATIVE:

```

INPUT StartPos } 1
INPUT NumToChange }
n ← 0
2 { REPEAT
    OUTPUT " Input new value for position " 3
    INPUT NewChar 4
    Lookup[StartPos + n] ← NewChar 5
    n ← n + 1
  UNTIL n = NumToChange
  OUTPUT (NumToChange & " entries changed") 6

```

Mark points as circled, descriptions as below:

1. Two INPUT statements
2. Working loop (using values of n from flowchart)
3. OUTPUT prompt (exact text not specified)
4. INPUT NewChar
5. Assignment of NewChar to correct array element
6. OUTPUT final message after loop (exact text not specified but must include NumToChange or loop counter if value correct at that point)

3 (b)(ii): VB.NET

```
Console.Write("Enter start position: ")
StartPos = Console.ReadLine()
Console.Write("Enter number to change: ")
NumToChange = Console.ReadLine()
For n = 0 To NumToChange - 1
    Console.Write("Enter new value for position: ")
    NewChar = Console.ReadLine()
    Lookup(StartPos + n) = NewChar
Next
Console.WriteLine(NumToChange & " entries changed")
```

ALTERNATIVE:

```
Console.Write("Enter start position: ")
StartPos = Console.ReadLine()
Console.Write("Enter number to change: ")
NumToChange = Console.ReadLine()
n = 0
Do
    Console.Write("Enter new value for position: ")
    NewChar = Console.ReadLine()
    Lookup(StartPos + n) = NewChar
    n = n + 1
Loop Until n = NumToChange
Console.WriteLine(NumToChange & " entries changed")
```

3 (b)(ii): Pascal

```
write('Enter start position: ');
readln(StartPos);
write('Enter number to change: ');
readln(NumToChange);
for n := 0 to NumToChange - 1 do
begin
    write('Input new value for position: ');
    readln(NewChar);
    Lookup[Startpos + n] := NewChar;
end;
writeln(IntToStr(NumToChange) + ' entries changed');
```

ALTERNATIVE:

```

write('Enter start position: ');
readln(StartPos);
write('Enter number to change: ');
readln(NumToChange);
n := 0;
repeat
    write('Input new value for position: ');
    readln(NewChar);
    LookUp[Startpos + n] := NewChar;
    n := n + 1;
until (n = NumToChange);
writeln(IntToStr(NumToChange) + ' entries changed');

```

3 (b)(ii): Python

```

StartPos = int(input("Enter start position: "))
NumToChange = int(input("Enter number to change: "))
for n in range(NumToChange) :
    NewChar = input("Input new value for position: ")
    LookUp[StartPos + n - 1] = NewChar
print(str(NumToChange) + " entries changed")

```

ALTERNATIVE:

```

StartPos = int(input("Enter start position: "))
NumToChange = int(input("Enter number to change: "))
n = 0
while n < NumToChange :
    NewChar = input("Input new value for position: ")
    LookUp[StartPos + n] = NewChar
    n = n + 1
print(str(NumToChange) + " entries changed")

```

171. M/J 16/P23/Q1/a**1 (a) (i)****[6]**

Item	Statement	Selection	Iteration	Assignment
1	WHILE DegF > 37.5		✓	
2	MyName = "Gordon"			✓
3	DegF = INT (DegF)			✓
4	ENDIF	✓		
5	CASE OF MyFavourite	✓		
6	UNTIL x = 5		✓	

One mark per row

Additional ticks in any row cancels that row

(ii)

[6]

Item	Purpose of statement
1	(Start of) loop – repeat while <u>DegF</u> greater than <u>37.5</u>
2	Assign (string) <u>"Gordon"</u> to <u>MyName</u>
3	Assign <u>integer</u> value / whole number part of DegF to DegF
4	End of an IF statement / selection statement
5	Head of CASE / selection statement based on variable MyFavourite
6	End of REPEAT / post-condition <u>loop</u> : repeated until x equals 5

Exact wording not important

Explanation must refer to variables or values used in code (except for row 4)

(iii)

[2]

Expression	Result
'P' & MID(MyString, 13, 4)	"Paint"
RIGHT(MID(MyString, 6, 10), 4)	"Main"

Must have correct case

Quotation marks optional

172. M/J 16/P23/Q3/b

(b) 'Pseudocode' solution included here for development and clarification of the mark scheme. Programming language example solutions appear in the Appendix. [6]

```
INPUT StartIndex  
INPUT NumberToOutput } ①  
  
② { FOR Index ← StartIndex to StartIndex + NumberToOutput - 1  
    OriginalChar ← CHR(Index) ③  
    CipherChar ← Lookup[Index] ④  
    OUTPUT ("Index " & Index & ": Character " & OriginalChar & ⑤  
           " has substitute character " & CipherChar) ⑥  
    ENDFOR
```

Mark points as circled, descriptions as below:

1. Two INPUT statements
2. Working loop using Index (allow alternative solutions including separate loop counter)
3. Assignment (using correct values of Index or other variable)
4. Assignment (using correct values of Index or other variable)
5. One mark for OUTPUT of a string combining text and variables...
6. ...a second mark if OUTPUT string is completely correct

Q3 (b): VB.NET

```

Console.WriteLine("Enter start position")
StartIndex = Console.ReadLine()
Console.WriteLine("Enter how many")
NumberToOutput = Console.ReadLine()
For Index = StartIndex To StartIndex + NumberToOutput - 1
    OriginalChar = Chr(Index)
    CipherChar = Lookup(Index)
    Console.WriteLine("Index " & Index & ": Character " & OriginalChar &
        " has substitute character " & CipherChar)
Next Index

```

Q3 (b): Pascal

```

Writeln('Enter start position');
Readln(StartIndex);
Writeln('Enter how many');
Readln(NumberToOutput);
For index := StartIndex To StartIndex + NumberToOutput - 1 Do
Begin
    OriginalChar := chr(index);
    CipherChar := Lookup[index];
    writeln("Index " + index + ": Character " + OriginalChar +
        " has substitute character " & CipherChar);
end;

```

Q3 (b): Python

```

startIndex = int(input("enter start position"))
numberToOutput = int(input("enter how many"))
for index in range(startIndex, (startIndex + numberToOutput)) :
    OriginalChar = chr(index)
    CipherChar = Lookup[index]
    print("Index " + (index) + ": Character " + OriginalChar +
        " has subst char " + CipherChar)

```

173. M/J 16/P23/Q4/a

- 4 (a)** • Functions / Procedures
 • Ability to pass parameters between modules
 • Use of local / global variables

[2]**174. O/N 15/P21/Q1**

- 1 (i)** 40
(ii) 314.2(0)
(iii) 16 // ERROR as identifier Z has not been declared
(iv) TRUE

[1]**[1]****[1]****[1]**

175. O/N 15/P21/Q5/b

```

(b) OUTPUT "Enter Purchase price"
INPUT PurchasePrice
CurrentValue ← PurchasePrice [1]
YearCount ← 1
WHILE YearCount < 9 AND CurrentValue >= 1000 [2]
    Note: Penalise: inclusion of $
    IF YearCount = 1 [1]
        THEN
            CurrentValue ← CurrentValue * (1 - 40/100)
        ELSE
            CurrentValue ← CurrentValue * 0.8 [1]
    ENDIF
    OUTPUT YearCount, CurrentValue
    YearCount ← YearCount + 1 [1]
ENDWHILE

```

176. O/N 15/P21/Q6/a,b,c,d

- 6 (a) Combination of staff and task number // the pair of numbers // the pair of random numbers [1]
 //there will be duplicates /repeats//some staff tasks will not be generated [1]
- (b) (i) 04 // 03 [1]
 (ii) 27 // 28 [1]
 (iii) 20 [1]
 (iv) 11 / 12 [1]
- (c) (i) Zero [1]
 (ii) Completed <> 60 // NewStaffTask = FALSE [1]
 Allow: Inclusion of the WHILE
- (iii) Determines whether this combination of StaffNum and TaskNum has been completed [1]
 Assigns value TRUE if not already generated [1]
 Flags that this is the first time this staff + task has been selected/to exit the loop [1]
 Outputs the new staff + task number [1]
- [MAX 3]
- (iv) TaskGrid : ARRAY[1:5, 1:12] OF BOOLEAN
 1 mark | 1 mark [2]

(d) Pseudocode ...

(SELECT) CASE (OF) + ENDCASE using StaffNo

1 mark	1 mark	
(CASE) 1: StaffName ← "Sadiq"		1
(CASE) 2: StaffName ← "Smith"		
(CASE) 3: StaffName ← "Ho"		
(CASE) 4: StaffName ← "Azmah"		
(CASE) 5: StaffName ← "Papadopolis"		
	(all four cases ...)	1
ENDCASE // ENDSELECT		[4]

Visual Basic

```

Select Case StaffNo
    Case 1
        StaffName = "Sadiq"
    Case 2
        StaffName = "Smith"
    Case 3
        StaffName = "Ho"
    Case 4
        StaffName = "Azmah"
    Case 5
        StaffName = "Papadopolis"
End Select

```

177. O/N 15/P21/Q7/a,b

- 7 (a) (i) CAT [1]
 Ignore any opening + closing quotes
- (ii) 13 [1]
- (iii) 83 [1]
- (iv) 15 [1]

- (b) *Input of string ...*
- | | | |
|--|---|---------|
| Correct syntax (for both prompt and assignment) | } | [1] |
| Uses MyString identifier | | |
| StringTotal set to 0 | | [1] |
| <i>FOR loop:</i> | | |
| FOR - NEXT keywords // (Python) correct indentation | | [1] |
| Correct start and /end boundaries | | [1] |
| Note: the end boundary must use the language length function/method // alternative Python syntax | | |
| Isolate single character number | | [1] |
| Followed by the use of Asc (VB) // Ord (Python) | } | [1] |
| Assigned to NextNum | | |
| Added to StringTotal | | [1] |
| Correct syntax for the output of the string and number | | [1] |
| | | [MAX 6] |

Python ...

```
MyString = input('key in string')

StringTotal = 0
for i in range (0, len(MyString)):
    NextNum = ord(MyString[i])
    StringTotal = StringTotal + NextNum

print(MyString, StringTotal)
```

Visual Basic...

```

Dim MyString As String
Dim StringTotal As Integer
Dim i As Integer
Dim NextNum As Integer

Console.Write("key in string")
MyString = Console.ReadLine
StringTotal = 0
For i = 1 To Len(MyString) // MyString.Length
    NextNum = Asc(Mid(MyString, i, 1))
    StringTotal = StringTotal + NextNum
Next

Console.WriteLine(MyString & " " & Str(StringTotal))

```

Pascal ...

```

VAR MyString      : String ;
VAR StringTotal   : Integer ;
VAR i             : Integer ;
VAR NextNum       : Integer ;
VAR SingleChar    : Char;

begin
    Writeln('key in string');
    readln(MyString) ;
    StringTotal := 0 ;

    For i := 1 To Length(MyString) do
        begin
            SingleChar := MyString[i] ;
            NextNum := Ord(SingleChar) ;
            StringTotal := StringTotal + NextNum ;
        end ;

    Writeln(MyString, StringTotal) ;

    ReadLn() ;
End.

```

178. O/N 15/P22/Q1

- | | | | |
|---|-------|--------------------|-----|
| 1 | (i) | 2 | [1] |
| | (ii) | 7.5
Accept: 7 ½ | [1] |
| | (iii) | FALSE | [1] |
| | (iv) | TRUE | [1] |
| | (v) | ERROR | [1] |

179. O/N 15/P22/Q4/a,b,c,d,e,f,g

- | | | | |
|---|-----|---|-----|
| 4 | (a) | The <u>combination</u> of suit and card number // the 'pair' of numbers // the <u>pair</u> of random numbers | [1] |
| | | There will be duplicates/repeats//not all cards will be drawn | [1] |
| | (b) | (i) 32 // 33 | [1] |

- (ii) 27 // 28 [1]
- (iii) 08 [1]
- (iv) 12 // 13 [1]
- (c) 1 [1]
- (d) DealCount <> 52 // NewCard = FALSE
Allow: Inclusion of the WHILE [1]
- (e) Test has the card has already been drawn? [1]
Set value TRUE for this card entry (in the array)/this card [1]
Flags that this is the first time this card has been drawn // decides if another card
must be generated [1]
Outputs the new card value [1]

[Max 2]

- (f) CardPack ARRAY[1:4 , 1:13] OF/:/AS BOOLEAN [1]
Allow: parentheses
- (g) **Pseudocode ...**
(SELECT) CASE (OF) CardValue + ENDCASE [1]
 (CASE) 1: CardName ← "Ace" 1 mark for any one correct [1]
 (CASE) 11: CardName ← "Jack"
 (CASE) 12: CardName ← "Queen"
 (CASE) 13: CardName ← "King" (final three cases ...) [1]
 OTHERWISE (/ELSE) CardName ← CardValue //
 (CASE) 2 TO 10: CardName ← CardValue [1]
ENDCASE // ENDSELECT

Note: Must be double quotes present and correct case

Visual Basic

```
Select Case CardValue
    Case 1
        CardName = "Ace"
    Case 11
        CardName = "Jack"
    Case 12
        CardName = "Queen"
    Case 13
        CardName = "King"
    Case Else // Case 2 to 10
        CardName = Str(CardValue) [4]
End Select
Allow: omission of Str
```

180. O/N 15/P22/Q5/a(iii)

- (iii) There were no swaps on the last pass / on pass 4 [1]

181. O/N 15/P22/Q6/a,b(ii)

- 6 (a) (i) 12 [1]
- (ii) 'L' [1]
Note: quotes are optional – must be upper case L

```

(ii) // main program
INPUT MyString
ChangedString ← RemoveSpaces(MyString) (1)
OUTPUT ChangedString

// function definition (1)
FUNCTION RemoveSpaces(InputString : STRING)
RETURNS STRING (1)
    DECLARE i :/AS INTEGER } (1)
    DECLARE j :/AS INTEGER
    DECLARE NextChar :/AS CHAR } (1)
    DECLARE NewString :/AS STRING
    NewString = "" (1)

    j ← CharacterCount(InputString)
    FOR i ← 1 TO j
        NextChar ← OneChar(InputString, i)
        IF NextChar <> " "
            THEN
                // the & character joins together two strings
                NewString ← NewString & NextChar
            ENDIF
    ENDFOR (1) (1) only awarded if follows the previous mark

    RETURN NewString //
    RemoveSpaces ← NewString
ENDFUNCTION

```

[Max 7]

182. O/N 15/P22/Q7/a,b(i)

- | | | | |
|---|---------|-----------------------|-----|
| 7 | (a) (i) | 165 | [1] |
| | (ii) | "YES" Quotes optional | [1] |
| | (iii) | 9 | [1] |
| | (iv) | 83 | [1] |

(b) (i) Use of correct identifiers only to score*Declaration/Commenting of variables*

```
MyMessage As String
EncryptString As String
i As Integer
NextNum As Integer
```

At least two variables correctly documented [1]

Input of string ...

Correct syntax (for both prompt and assignment) and ...
Uses the MyMessage identifier [1]

EncryptString set to 'empty string' [1]

Note: Must suggest 'empty' string

For loop ...

FOR - NEXT keywords // (Python) correct indentation [1]

Correct start/end boundaries [1]

Note: the end boundary must use the language length
function/method //alternative Python syntax

Isolate single character [1]

Use of language functions to calculate new number and

Assigned to NextNum [1]

Conversion of NextNum to a character and concatenated
to EncryptString [1]

Correct syntax for output of EncryptString [1]

[MAX 8]

SAMPLE CODE**PYTHON**

```
MyMessage = input("Enter message : ")
EncryptString = ""
for i in range(0, len(MyMessage)) :
    NextNum = ord(MyMessage[i]) + 3
    EncryptString = EncryptString + chr(NextNum)
print(EncryptString)
```

Alternative solution:

```
MyMessage = input("Enter message : ")
EncryptString = ""
for NextChar in MyMessage :
    NextNum = ord(NextChar) + 3
    EncryptString = EncryptString + chr(NextNum)
print(EncryptString)
```


VB

```

Dim MyMessage, EncryptString As String
Dim NextNum, i As Integer
Console.Write("Enter message : ")
MyMessage = Console.ReadLine()
EncryptString = ""
For i = 1 To Len(MyMessage)
    NextNum = Asc(Mid(MyMessage, i, 1)) + 3
    EncryptString = EncryptString + Chr(NextNum)
Next
Console.WriteLine(EncryptString)

```

Alternatives:
 GetChar(MyMessage, i)
 MyMessage.Substring(i, 1)

Allow: Use of InputBox and MsgBox

Alternative solution :

```

Dim MyMessage, EncryptString As String
Dim NextNum, i As Integer
Console.Write("Enter message : ")
MyMessage = Console.ReadLine()
EncryptString = ""
For i = 0 To Len(MyMessage) - 1
    NextNum = Asc(MyMessage.Chars(i)) + 3
    EncryptString = EncryptString + Chr(NextNum)
Next
Console.WriteLine(EncryptString)

```

PASCAL

```

var
    MyMessage, EncryptString : string;
    NextNum, i : integer;
begin
    write('Enter message : ');
    readln(MyMessage);
    EncryptString := '';
    for i := 1 to length(MyMessage) do
        begin
            NextNum := ord(MyMessage[i]) + 3;
            EncryptString := EncryptString + chr(NextNum);
        end;
    writeln(EncryptString);
end.

```

183. O/N 15/P23/Q1

- | | | |
|---|---|-----|
| 1 | (i) 40 | [1] |
| | (ii) 314.2(0) | [1] |
| | (iii) 16 // ERROR as identifier Z has not been declared | [1] |
| | (iv) TRUE | [1] |

184. O/N 15/P23/Q5/b

```

(b) OUTPUT "Enter Purchase price"
INPUT PurchasePrice
CurrentValue ← PurchasePrice [1]
YearCount ← 1
WHILE YearCount < 9 AND CurrentValue >= 1000 [2]
    Note: Penalise: inclusion of $
    IF YearCount = 1 [1]
        THEN
            CurrentValue ← CurrentValue * (1 - 40/100)
        ELSE
            CurrentValue ← CurrentValue * 0.8 [1]
    ENDIF
    OUTPUT YearCount, CurrentValue
    YearCount ← YearCount + 1 [1]
ENDWHILE

```

185. O/N 15/P23/Q6/a,b,c,d

- 6 (a) Combination of staff and task number // the pair of numbers // the pair of random numbers [1]
 //there will be duplicates /repeats//some staff tasks will not be generated [1]
- (b) (i) 04 // 03 [1]
 (ii) 27 // 28 [1]
 (iii) 20 [1]
 (iv) 11 / 12 [1]
- (c) (i) Zero [1]
 (ii) Completed <> 60 // NewStaffTask = FALSE [1]
 Allow: Inclusion of the WHILE
 (iii) Determines whether this combination of StaffNum and TaskNum has been completed [1]
 Assigns value TRUE if not already generated [1]
 Flags that this is the first time this staff + task has been selected/to exit the loop [1]
 Outputs the new staff + task number [1]
 [MAX 3]
- (iv) TaskGrid : ARRAY[1:5, 1:12] OF BOOLEAN
 1 mark | 1 mark [2]

(d) Pseudocode ...

(SELECT) CASE (OF) + ENDCASE using StaffNo

1 mark	1 mark	
(CASE) 1: StaffName ← "Sadiq"		1
(CASE) 2: StaffName ← "Smith"		
(CASE) 3: StaffName ← "Ho"		
(CASE) 4: StaffName ← "Azmah"		
(CASE) 5: StaffName ← "Papadopolis"		
	(all four cases ...)	1
ENDCASE // ENDSELECT		[4]

Visual Basic

```
Select Case StaffNo
    Case 1
        StaffName = "Sadiq"
    Case 2
        StaffName = "Smith"
    Case 3
        StaffName = "Ho"
    Case 4
        StaffName = "Azmah"
    Case 5
        StaffName = "Papadopolis"
End Select
```

186. O/N 15/P23/Q7/a,b

- 7 (a) (i) CAT [1]
 Ignore any opening + closing quotes
- (ii) 13 [1]
- (iii) 83 [1]
- (iv) 15 [1]

- (b) *Input of string ...*
- | | | |
|---|---|---------|
| Correct syntax (for both prompt and assignment) | } | [1] |
| Uses MyString identifier | | |
| StringTotal set to 0 | | [1] |
| <i>FOR loop:</i> | | |
| FOR - NEXT keywords // (Python) correct indentation | | [1] |
| Correct start and /end boundaries | | [1] |
| Note: the end boundary must use the language length
function/method // alternative Python syntax | | |
| Isolate single character number | | [1] |
| Followed by the use of Asc (VB) // Ord (Python) | } | [1] |
| Assigned to NextNum | | |
| Added to StringTotal | | [1] |
| Correct syntax for the output of the string and number | | [1] |
| | | [MAX 6] |

Python ...

```
MyString = input('key in string')
```

```
StringTotal = 0
```

```
for i in range (0, len(MyString)):
```

```
    NextNum = ord(MyString[i])
```

```
    StringTotal = StringTotal + NextNum
```

```
print(MyString, StringTotal)
```

Visual Basic...

```

Dim MyString As String
Dim StringTotal As Integer
Dim i As Integer
Dim NextNum As Integer

Console.Write("key in string")
MyString = Console.ReadLine
StringTotal = 0
For i = 1 To Len(MyString) // MyString.Length
    NextNum = Asc(Mid(MyString, i, 1))
    StringTotal = StringTotal + NextNum
Next

Console.WriteLine(MyString & " " & Str(StringTotal))

```

Pascal ...

```

VAR MyString      : String ;
VAR StringTotal   : Integer ;
VAR i             : Integer ;
VAR NextNum       : Integer ;
VAR SingleChar    : Char;

begin
    Writeln('key in string');
    readln(MyString) ;
    StringTotal := 0 ;

    For i := 1 To Length(MyString) do
        begin
            SingleChar := MyString[i] ;
            NextNum := Ord(SingleChar) ;
            StringTotal := StringTotal + NextNum ;
        end ;

    Writeln(MyString, StringTotal) ;

    ReadLn() ;
End.

```

187. M/J 15/P21/Q1/b(ii)**(ii) Mark as follows:**

- Declarations/comments for variables – at least 2
- Input (+ prompts) for hours, minutes, seconds
- Input (+ prompt) of personal best time
- Correct calculation of `RaceTimeInSeconds` (don't allow use of 'x' for '*')
- Output `RaceTimeInSeconds`
- Correct logic and output message for < personal best
- Correct logic and output message for > personal best
- Correct logic and output message for = personal best

[max 7]

188. M/J 15/P21/Q2/a,b,c,d

- 2 (a) (i) Displays the menu (choices)
 Repeats the prompt and input ...
 ...the input is a number between 1 and 4 // Checks number is between 1 and 4
"within range" is not enough [3]
- (ii) ...the input number is validated [1]
- (b) (i) 3 [1]
- (ii) Previous design repeated indefinitely // (new design) limits number of attempts
 Penalise "Program terminates/closes" [1]
- (c) IF Choice = 1 THEN (CALL) ReadFile (1)
 IF Choice = 2 THEN OUTPUT "Add Customer code" (1)
 IF Choice = 3 THEN OUTPUT "Search Customer code" (1)
 IF Choice = 4 THEN END (1)

alternative answer:**mark as follows:**

CASE OF <u>Choice</u> // Select CASE <u>Choice</u>	1 mark
1: (CALL) ReadFile	1 mark (allow CASE = 1)
2: OUTPUT " <u>Add Customer code</u> "	1 mark
3: OUTPUT " <u>Search Customer code</u> "	1 mark
4: END	
ENDCASE	

Output strings must match [max 3]

(d) Mark as follows:

- `Choice / NoOfAttempts` declared/commented as integer
Must appear within the 'main' program
Allow: different identifier names
- Constant `i` assigned a value 3
- There is an 'outer' loop to repeatedly display the menu
- Input 'choice' variable
- Three `IF` statements (or equivalent) for processing menu choices 1, 2 and 3
Note: they must be correctly formed as 'nested' or 'independent'
- Choice 1 calls procedure `ReadFile`
- Choice 2 outputs "Add Customer Code"
+ Choice 3 outputs "Search Customer Code"
- Outer loop terminates correctly with '`Choice = 4`' //or equivalent
- Procedure `DisplayMenu` shows the four menu options
- Procedure `ReadFile` is present ...
and contains a single output message 'Read file code'

[max 8]

189. M/J 15/P21/Q4/(i)

```

4 (i) FOR NoOfThrows ← 1 TO 20 / 0 TO 19
      1                1
      INPUT Player1Throw
      INPUT Player2Throw
      IF Player1Throw > Player2Throw
      THEN
          Player1Total ← Player1Total + 1
      ENDIF
      IF Player2Throw > Player1Throw
      THEN
          Player2Total ← Player2Total + 1
      ENDIF
      ENDFOR
      IF Player1Total > Player2Total
      THEN
          OUTPUT "Player1 is the winner"
      ELSE
          OUTPUT "Player2 is the winner"
      END

```

[5]

190. M/J 15/P21/Q5/c**(c) (i) 2**

[1]

(ii)

Tick Cross	Explanation (if invalid)
X // ✓	2 nd parameter should be CHAR // accept just tick
X	Three parameters/should be 2 parameters
✓	

[3]

191. M/J 15/P22/Q1/b(ii)

(ii) Mark as follows:

- Declarations/comments for variables – at least 2
- Input (+ prompts) for hours, minutes, seconds
- Input (+ prompt) of personal best time
- Correct calculation of `RaceTimeInSeconds` (don't allow use of 'x' for '*')
- Output `RaceTimeInSeconds`
- Correct logic and output message for < personal best
- Correct logic and output message for > personal best
- Correct logic and output message for = personal best

[max 7]

192. M/J 15/P22/Q2/a,b,c,d

2 (a) (i) Displays the menu (choices)

Repeats the prompt and input ...

...the input is a number between 1 and 4 // Checks number is between 1 and 4

"within range" is not enough

[3]

(ii) ...the input number is validated

[1]

(b) (i) 3

[1]

(ii) Previous design repeated indefinitely // (new design) limits number of attempts

Penalise "Program terminates/closes"

[1]

```

(c) IF Choice = 1 THEN (CALL) ReadFile           (1)
    IF Choice = 2 THEN OUTPUT "Add Customer code" (1)
    IF Choice = 3 THEN OUTPUT "Search Customer code" (1)
    IF Choice = 4 THEN END                       (1)

```

alternative answer:**mark as follows:**

```

CASE OF Choice // Select CASE Choice           1 mark
1: (CALL) ReadFile                               1 mark (allow CASE = 1)
2: OUTPUT "Add Customer code"                   1 mark
3: OUTPUT "Search Customer code"                 1 mark
4: END
ENDCASE

```

Output strings must match

[max 3]

(d) Mark as follows:

- `Choice / NoOfAttempts` declared/commented as integer
Must appear within the 'main' program
Allow: different identifier names
- Constant `i` assigned a value 3
- There is an 'outer' loop to repeatedly display the menu
- Input 'choice' variable
- Three `IF` statements (or equivalent) for processing menu choices 1, 2 and 3
Note: they must be correctly formed as 'nested' or 'independent'
- Choice 1 calls procedure `ReadFile`
- Choice 2 outputs "Add Customer Code"
+ Choice 3 outputs "Search Customer Code"
- Outer loop terminates correctly with '`Choice = 4`' //or equivalent
- Procedure `DisplayMenu` shows the four menu options
- Procedure `ReadFile` is present ...
and contains a single output message 'Read file code'

[max 8]

193. M/J 15/P22/Q4/(i)

```

4 (i) FOR NoOfThrows ← 1 TO 20 / 0 TO 19
      1                1
      INPUT Player1Throw
      INPUT Player2Throw
      IF Player1Throw > Player2Throw
      THEN
          Player1Total ← Player1Total + 1
      ENDIF
      IF Player2Throw > Player1Throw
      THEN
          Player2Total ← Player2Total + 1
      ENDIF
      ENDFOR
      IF Player1Total > Player2Total
      THEN
          OUTPUT "Player1 is the winner"
      ELSE
          OUTPUT "Player2 is the winner"
      END

```

[5]

194. M/J 15/P22/Q5/c

(c) (i) 2

[1]

(ii)

Tick Cross	Explanation (if invalid)
X // ✓	2 nd parameter should be CHAR // accept just tick
X	Three parameters/should be 2 parameters
✓	

[3]

195. M/J 15/P23/Q2/a,b(ii)**2 (a) (i) 7**

[1]

**(ii) 2
9**

[2]

(ii) **INPUT Amount**
IF Amount > 500
 THEN
 OUTPUT "Refused - amount too large"
 ELSE
 IF (Amount MOD 10) <> 0 / >0
 THEN
 OUTPUT "Refused - not a multiple of \$10"
 ELSE
 FiftyDollar ← Amount DIV 50
 Temp ← Amount MOD 50 //
 (Amount - 50 * FiftyDollar)
 TwentyDollar ← Temp DIV 20 //
 (Amount MOD 50) DIV 20
 Temp ← Temp MOD 20
 TenDollar ← Temp DIV 10
 ENDIF
 ENDIF

[max 5]

196. M/J 15/P23/Q3/(ii)

(ii) **PROCEDURE CalculateJobCost**
 (BYREF JobCost : INTEGER/CURRENCY/REAL,
 BYVALUE Length : INTEGER,
 BYVALUE Width : INTEGER)

mark as follows:

identifier + data type × 3 (3 marks)
 jobcost (only) BYREF (1 mark)
 length, width (only) BYVALUE/BYREF (1 mark)

[5]

197. M/J 15/P23/Q4/a**4 (a) (i) ERROR**

[1]

(ii) parityerrorcheck

[1]

(iii) Binary Coded Decimal // Binary ▼ Coded ▼ Decimal

[2]

198. M/J 15/P23/Q5/c

(c) (i) WorkerNum : INTEGER (1 mark)
 DayNum : INTEGER (1 mark)
 WorkerTotal : ARRAY OF INTEGER (1 mark) (1 mark)
 WorkerAverage : REAL (1 mark) [max 4]

(ii) PROCEDURE AnalyseProductionData (NumDays : INTEGER, NumWorkers : INTEGER)

```
FOR WorkerNum ← 1 TO 3
  WorkerTotal [WorkerNum] ← 0
ENDFOR
```

```
FOR WorkerNum ← 1 TO 3
  FOR DayNum ← 1 TO 4
    WorkerTotal [WorkerNum] ← WorkerTotal [WorkerNum] +
      ProductionData [WorkerNum, DayNum]
  ENDFOR
ENDFOR
```

```
FOR WorkerNum ← 1 TO 3
  WorkerAverage = WorkerTotal [WorkerNum] / 4 *
    DailyHoursWorked [WorkerNum]
  IF WorkerAverage < 2
    THEN
      OUTPUT "Investigate" WorkerNum
    ENDIF
ENDFOR
```

ENDPROCEDURE

Mark as follows:

All '3's changed to NumWorkers

All '4's changed to NumDays

WorkerAverage '4' changed to NumDays [3]

(iii) (CALL) AnalyseProductionData (7, 13) [1]